JavaScript Handbook — Vol 2

errors, async, DOM, browser APIs and security, Node.js, testing and TypeScript, interview prep, cheat sheet, question bank (Ch 11–19)

9 parts · one place to study: DesignGurus courses + classic books + the web, combined, beginner to intermediate

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 11 — Errors and Debugging**
   - 11.1 Errors and the built-in error types
   - 11.2 `throw`, `try`, `catch`, `finally`
   - 11.3 Custom errors
   - 11.4 Error chaining with `cause`
   - 11.5 Errors in asynchronous code (preview)
   - 11.6 Guaranteed cleanup: `finally` and `using`
   - 11.7 The console toolbox
   - 11.8 Reading stack traces and source maps
   - 11.9 Debugging with DevTools and the Node inspector
   - 11.10 Common runtime errors — a field guide
   - Chapter 11 TL;DR
2. **Chapter 12 — Asynchronous JavaScript**
   - 12.1 One thread, and why blocking hurts
   - 12.2 The event loop
   - 12.3 Ordering puzzles
   - 12.4 Node's event loop: `nextTick`, `setImmediate` and phases
   - 12.5 Timers
   - 12.6 Callbacks and callback hell
   - 12.7 Promises
   - 12.8 Promise combinators: `all`, `allSettled`, `race`, `any`
   - 12.9 `async` / `await`
   - 12.10 Error handling across async code — a checklist
   - 12.11 Cancellation: `AbortController` and `AbortSignal`
   - 12.12 Async iteration: `for await…of` and async generators
   - 12.13 Real-world async patterns
   - Chapter 12 TL;DR
3. **Chapter 13 — The DOM and Events**
   - 13.1 The DOM tree
   - 13.2 Selecting and traversing
   - 13.3 Creating, inserting and removing nodes
   - 13.4 Attributes, properties, classes, data and styles
   - 13.5 Events and listeners
   - 13.6 Propagation: capture, target, bubble
   - 13.7 Event delegation
   - 13.8 Default actions and `preventDefault`
   - 13.9 Forms
   - 13.10 Watching the DOM: `MutationObserver`
   - 13.11 Rendering performance: reflow, repaint and layout thrashing
   - 13.12 When scripts run: `DOMContentLoaded`, `load` and `defer`
   - Chapter 13 TL;DR
4. **Chapter 14 — Browser APIs and Security**
   - 14.1 `fetch` and HTTP
   - 14.2 The same-origin policy and CORS
   - 14.3 Web Storage: `localStorage` and `sessionStorage`
   - 14.4 Cookies and IndexedDB — choosing storage
   - 14.5 Timers and `requestAnimationFrame`
   - 14.6 `URL` and `URLSearchParams`
   - 14.7 The History API
   - 14.8 Web Workers
   - 14.9 `IntersectionObserver` and `ResizeObserver`
   - 14.10 Cross-site scripting (XSS) and sanitizing
   - 14.11 Cross-site request forgery (CSRF)
   - 14.12 Content Security Policy (CSP)
   - Chapter 14 TL;DR
5. **Chapter 15 — Node.js Basics**
   - 15.1 What Node.js is
   - 15.2 The event loop, the thread pool and blocking
   - 15.3 Modules in Node — the short version
   - 15.4 Files with `node:fs/promises`
   - 15.5 Paths and file URLs: `node:path`, `node:url`
   - 15.6 `process`: arguments, environment and exit codes
   - 15.7 `Buffer`: raw bytes
   - 15.8 Events: `EventEmitter`
   - 15.9 Streams
   - 15.10 Building an HTTP server with `node:http`
   - 15.11 Child processes and npm scripts
   - 15.12 Everyday Node flags, and Node vs Deno vs Bun
   - Chapter 15 TL;DR
6. **Chapter 16 — Testing and TypeScript**
   - 16.1 Why test, and what kinds of tests exist
   - 16.2 Assertions with `node:assert/strict`
   - 16.3 Writing and running tests with `node:test`
   - 16.4 Async tests and how tests fail
   - 16.5 Test doubles and mocking
   - 16.6 Snapshot tests and code coverage
   - 16.7 Good tests, and other runners
   - 16.8 TypeScript: what it is and how it's checked
   - 16.9 Running TypeScript: Node's type stripping and other tools
   - 16.10 Everyday types
   - 16.11 Narrowing
   - 16.12 Generics and utility types
   - 16.13 Classes, `as const`, `satisfies`, and enums
   - 16.14 Tests written in TypeScript
   - 16.15 JSDoc types: TypeScript checking for plain JavaScript
   - Chapter 16 TL;DR
7. **Chapter 17 — JavaScript for Interviews**

- 17.1 The JavaScript DSA toolkit
- 17.2 Coding patterns
- 17.3 Implement-from-scratch classics
- 17.4 Fifty-three tricky interview questions
- Chapter 17 TL;DR

8. Chapter 18 — Cheat Sheets and Glossary

- 18.1 Syntax, variables and types
- 18.2 Operators and the coercion table
- 18.3 Control flow and loops
- 18.4 Functions, scope, closures and `this`
- 18.5 Objects, Math, Date and JSON
- 18.6 Classes and prototypes
- 18.7 Strings and regular expressions
- 18.8 Arrays: mutating vs non-mutating
- 18.9 Map, Set, iterators and generators
- 18.10 Modules and tooling
- 18.11 Errors and debugging
- 18.12 Async and the event loop
- 18.13 DOM and events
- 18.14 Browser APIs and security
- 18.15 Node.js core modules
- 18.16 Testing and TypeScript
- 18.17 Which API for which job
- 18.18 ECMAScript versions ES2015 → ES2026
- 18.19 Where to go next
- 18.20 Interview question index
- 18.21 Glossary (A–Z)
- Chapter 18 TL;DR

9. Chapter 19 — Interview Question Bank

- How to use this bank
- 1. Types, values and coercion
- 2. Variables, hoisting, scope and closures
- 3. Functions and functional patterns
- 4. `this`, `call` / `apply` / `bind`, prototypes and classes
- 5. Objects, arrays, strings and collections
- 6. The event loop, promises and `async` / `await`
- 7. Errors, modules and tooling
- 8. The DOM and events
- 9. Browser APIs, storage, networking and security
- 10. Performance and memory
- 11. Node.js
- 12. TypeScript basics
- 13. Output-prediction puzzles ("What will this print?")
- 14. Coding tasks: implement-from-scratch classics
- 15. LeetCode "30 Days of JavaScript"
- Sources of the question lists

Chapter 11 — Errors and Debugging

JavaScript reports problems with **error objects** that are **thrown** and can be **caught**. This chapter covers how errors work, how to design your own, and how to find bugs fast with the console, stack traces and debuggers — in the browser and in Node.

What you'll learn

1. The built-in error types and what every error object contains.
2. `throw`, `try` / `catch` / `finally` — exact control flow, rethrowing, traps.
3. Custom error classes and a central error-handling layer.
4. Error chaining with `cause` (ES2022) and `AggregateError`.
5. Errors in asynchronous code (preview; full story in Chapter 12).
6. Guaranteed cleanup with `finally` and `using`.
7. The console toolbox beyond `console.log`.
8. Stack traces and source maps.
9. DevTools and the Node inspector; a systematic debugging method.
10. A gallery of common runtime errors with their exact messages.

Before you start. Custom errors use classes and `extends` (Chapter 9); type coercion, the source of many silent bugs, is in Chapter 2.

All runnable code is in `code/`. Outputs are real (Node 24, V8). Error **messages** are not standardized — Firefox and Safari word them differently — but error **types** are.

11.1 Errors and the built-in error types

What it is

An **error** (or **exception**) is an object describing something that went wrong. When code **throws**, normal execution stops and the engine unwinds the **call stack** (the functions currently running, newest on top — see Chapter 12) looking for a `catch`. If none is found the error is **uncaught**: the browser logs it and stops that script task; Node prints it and exits with a non-zero **exit code** (`0` = success, anything else = failure).

- **Runtime error** — thrown while code runs (`TypeError`, `RangeError`, …).
- **Parse-time (syntax) error** — the engine can't read the code; *nothing* in that script/module runs.

The built-in types

Type	Thrown when…	Typical example
`Error`	Generic base type — use for your own errors or subclass it	`throw new Error('config missing')`
`TypeError`	A value is the wrong type for an operation	`null.length`, calling a non-function, assigning to a `const`
`ReferenceError`	A name doesn't exist, or is used in its TDZ (before its `let`/`const` line)	typo in a variable name
`SyntaxError`	Code (or JSON) can't be parsed	`JSON.parse('{bad}')`, `new Function('1 +')`
`RangeError`	A number is outside the allowed range	`new Array(-1)`, `toFixed(101)`, stack overflow
`URIError`	Malformed URI component	`decodeURIComponent('%')`
`EvalError`	Not thrown by modern engines; kept for compatibility	—
`AggregateError` (ES2021)	Several errors reported together	`Promise.any` when every promise rejects
`SuppressedError` (ES2027)	An error happened while disposing after another error	`using` declarations (§11.6)

All inherit from `Error`, so `err instanceof Error` is `true` for every built-in. Environments add their own: browsers throw `DOMException` (e.g., `AbortError`, `NotAllowedError`); Node adds `code` properties like `'ENOENT'` ("Error NO ENTry" — file not found) to system errors.

`code/01_error_types.mjs` triggers each one for real:

```
const cases = {
  TypeError:      () => null.length,
  ReferenceError: () => notDeclaredAnywhere + 1,
  SyntaxError:    () => JSON.parse('{ bad json }'),
  RangeError:     () => new Array(-1),
  URIError:       () => decodeURIComponent('%'),
  EvalError:      () => { throw new EvalError('never thrown by modern engines — only kept for compatibility'); },
  AggregateError: () => { throw new AggregateError([new Error('a'), new TypeError('b')], 'several things failed');
},
};

for (const fn of Object.values(cases)) {
  try {
    fn();
  } catch (err) {
    console.log(`${err.name.padEnd(15)} | instanceof Error: ${err instanceof Error} | ${err.message}`);
  }
}
// ...the file also prints name/message/cause/stack, Promise.any's AggregateError,
// Error.isError and a parse-time SyntaxError from new Function('return 1 +')
```

Output (first part):

```
TypeError       | instanceof Error: true | Cannot read properties of null (reading 'length')
ReferenceError  | instanceof Error: true | notDeclaredAnywhere is not defined
SyntaxError     | instanceof Error: true | Expected property name or '}' in JSON at position 2 (line 1 column 3)
RangeError      | instanceof Error: true | Invalid array length
URIError        | instanceof Error: true | URI malformed
EvalError       | instanceof Error: true | never thrown by modern engines — only kept for compatibility
AggregateError  | instanceof Error: true | several things failed
```

Anatomy of an error object

Property	Standard?	Meaning
`name`	✓ (inherited from the prototype)	The type, e.g. `'TypeError'`
`message`	✓	Human-readable description
`cause`	✓ ES2022	The underlying reason: `new Error(msg, { cause })` (§11.4)
`errors`	✓ on `AggregateError`	Array of the individual errors
`stack`	de-facto (every engine; standard proposed)	Call stack text, captured when the error is **created** (`new`), not when thrown
`code`	Node / libraries	Machine-readable code, e.g. `'ENOENT'`, `'ERR_MODULE_NOT_FOUND'`

`String(err)` / `err.toString()` gives `"Name: message"`; `Promise.any` rejecting everywhere gives `AggregateError: All promises were rejected` with `.errors`. **`Error.isError(value)`** (ES2026; Node 24 and recent browsers) is the reliable "genuine error?" check — unlike `instanceof Error` it works across **realms** (separate global environments with their own built-ins, e.g. an iframe or a Node `vm` sandbox) and can't be fooled by a hand-made prototype.

Self-check

1. `undefinedFn()` where `undefinedFn` was never declared — which error? → `ReferenceError`. If declared but `undefined`, it's a `TypeError` (not a function).
2. Which property holds the underlying error in a wrapped error? → `cause`.
3. When is a `SyntaxError` in your source file reported? → At parse time, before any of it runs.

TL;DR

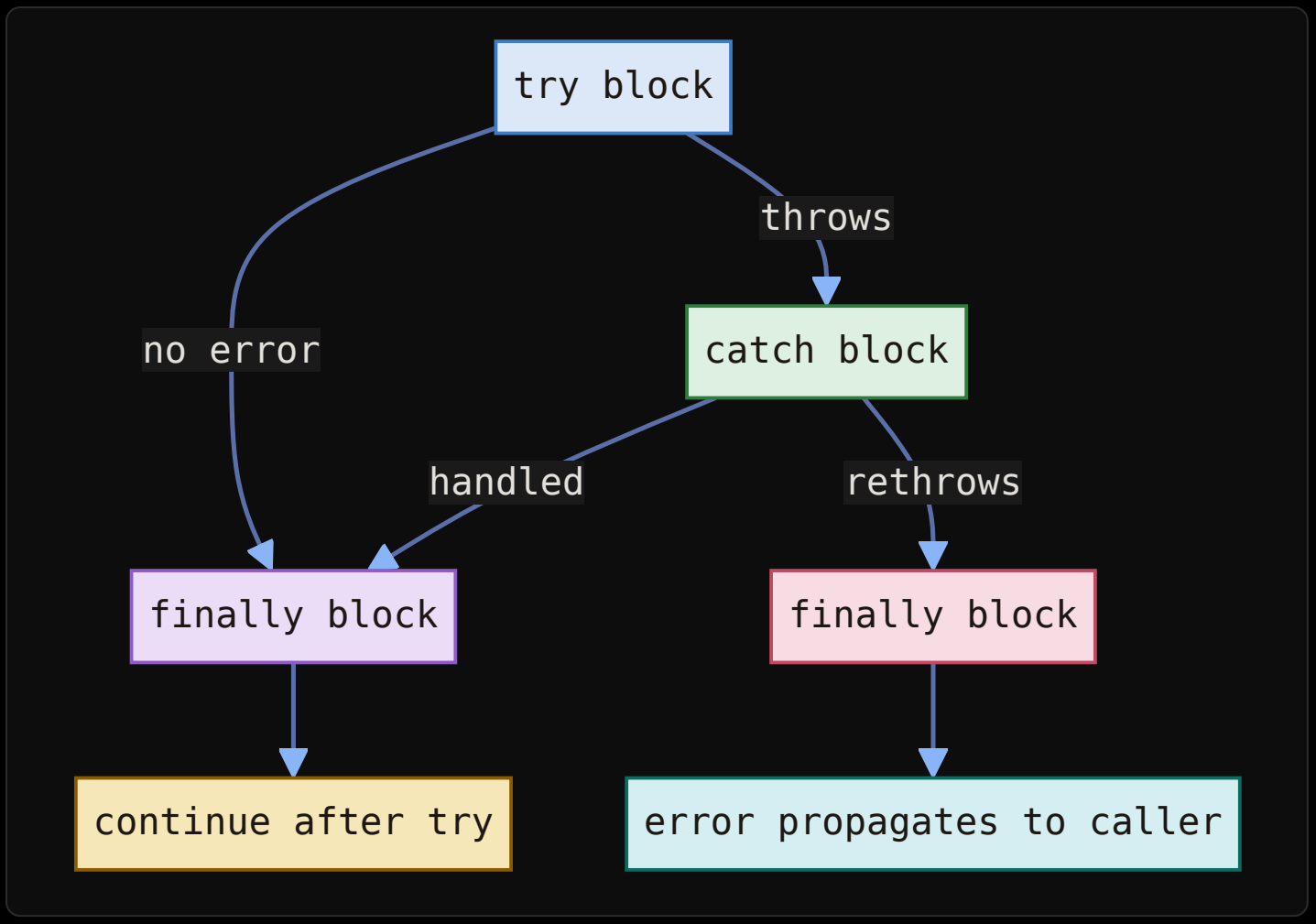
Errors are objects with `name`, `message`, `stack` (and optional `cause`). Core types: `TypeError` (wrong type), `ReferenceError` (unknown name/TDZ), `SyntaxError` (unparseable), `RangeError` (out of range), `AggregateError` (many).

11.2 `throw`, `try`, `catch`, `finally`

What it is

- `throw expression` — raises an error; execution jumps to the nearest enclosing `catch`.
- `try { … }` — code that might throw. Must be followed by `catch`, `finally`, or both.
- `catch (err) { … }` — runs only if `try` threw; `err` is the thrown value (block-scoped).
- `finally { … }` — always runs after `try`/`catch` — on success, error, or `return`. For cleanup.

How it works



If `catch` itself throws, `finally` still runs, then the new error propagates; if nothing up the stack catches it, it's uncaught.

`code/02_try_catch_finally.mjs`:

```
function parseAge(text) {
  const log = [];
  try {
    log.push('try start');
    const age = Number.parseInt(text, 10);
    if (Number.isNaN(age)) throw new TypeError(`not a number: "${text}"`);
    if (age < 0) throw new RangeError(`negative age: ${age}`);
    log.push('try end');
    return { age, log };
  } catch (err) {
    log.push(`catch ${err.name}`);
    return { age: null, log };
  } finally {
    log.push('finally');       // runs on success, on error, and even after return
  }
}
for (const input of ['42', 'abc', '-3']) {
  const { age, log } = parseAge(input);
  console.log(input.padEnd(4), '->', String(age).padEnd(5), log.join(' > '));
}

// finally's return OVERRIDES the try's return (and swallows errors) — avoid!
function sneaky() {
  try { throw new Error('lost forever'); }
  finally { return 'finally wins'; }  // eslint: no-unsafe-finally
}
console.log('\nsneaky():', sneaky());

// Optional catch binding (ES2019): omit (err) when you don't need it
function isValidJson(s) {
  try { JSON.parse(s); return true; } catch { return false; }
}
console.log('isValidJson:', isValidJson('{"a":1}'), isValidJson('{a:1}'));

// Catch only what you understand; rethrow the rest
function loadUser(raw) {
  try {
    return JSON.parse(raw).user.name;
  } catch (err) {
    if (err instanceof SyntaxError) return '(invalid JSON — using guest)';
    throw err;                     // a TypeError here means a real bug: don't hide it
  }
}
console.log('rethrow 1  :', loadUser('{oops'));
try { loadUser('{"nouser":true}'); } catch (err) { console.log('rethrow 2  :', err.name, '-', err.message); }

// (the file also throws a string, 42 and a plain object: only the Error has a .stack)

// try/catch is synchronous: it cannot catch an error thrown later by a callback
try {
  setTimeout(() => {
    try { throw new Error('from timer'); } catch (err) { console.log('\ntimer error caught INSIDE the callback:',
err.message); }
  }, 0);
} catch {
  console.log('never printed');
}
console.log('\n(outer try block already finished)');
```

Output:

```
42   -> 42    try start > try end > finally
abc  -> null  try start > catch TypeError > finally
-3   -> null  try start > catch RangeError > finally

sneaky(): finally wins
isValidJson: true false
rethrow 1  : (invalid JSON — using guest)
rethrow 2  : TypeError - Cannot read properties of undefined (reading 'name')

(outer try block already finished)

timer error caught INSIDE the callback: from timer
```

What to notice:

- `finally` ran in all three cases even though `try` and `catch` `return`: the return value is computed, `finally` runs, then the function returns.
- A `return` (or `throw`) *inside* `finally` replaces whatever `try`/`catch` was doing — `'lost forever'` vanished. ESLint's `no-unsafe-finally` flags it.
- **Catch narrowly, rethrow the rest.** `loadUser` recovers from bad JSON (`SyntaxError`); a `TypeError` means wrong data shape — a real bug that should surface.
- `catch` receives *whatever* was thrown. Strings/plain objects have no stack and fail `instanceof Error`. **Always throw `Error` objects.**
- `try/catch` only covers code that runs *synchronously inside it*. The timer callback ran later on a fresh call stack, after the outer `try` was gone.

When to use try/catch — and when not

Use it	Don't use it
At boundaries where failure is expected: parsing input, `JSON.parse`, network/file I/O, third-party code	To replace simple checks — `if (user?.name)` beats catching a `TypeError`
In a top-level handler that turns errors into a response/UI message and logs them	Wrapping every function "just in case" — hides bugs
To add context and rethrow (§11.4)	To silently swallow errors: `catch {}` with no handling

Performance: a `try` that doesn't throw costs essentially nothing; *throwing* is relatively expensive (stack capture), so don't use exceptions for normal control flow.

In the interview

30-second answer: "`throw` raises an error and unwinds the stack to the nearest `catch`. `finally` always runs — after success, error, or even `return` — so it's for cleanup; but a `return` inside `finally` overrides everything and swallows errors. `try/catch` is synchronous, so it can't catch errors from callbacks that run later — for promises you `await` inside `try`, or use `.catch()`. Always throw `Error` objects so you get a stack trace."

- *Q: What does `(() => { try { return 1 } finally { return 2 } })()` return?* A: `2`.
- *Q: Does `finally` run if `catch` rethrows?* A: Yes, then the error continues outward.
- *Q: Why is `catch (e) {}` dangerous?* A: It hides every error, including bugs, making failures silent.
- *Q: `try { } finally { }` without `catch`?* A: Legal; errors still propagate, cleanup runs.
- *Q: `err` after `throw 42`?* A: The number `42` (no stack).

TL;DR

`try` → `catch` on error → `finally` always. Throw `Error` objects, catch narrowly, rethrow what you can't handle, never `return` from `finally`; try/catch is synchronous.

11.3 Custom errors

What it is

A **custom error** is an `Error` subclass for a failure in *your* domain (`ValidationError`, `NotFoundError`). Callers react to *kinds* of failure with `instanceof` (or a `code`), and it carries extra data (invalid field, HTTP status to return — see System Design Fundamentals).

How it works

1. `class MyError extends Error`; call `super(message, options)` so `message`, `stack` and `cause` are set.
2. Set `this.name` (otherwise it prints as `Error`); `new.target.name` does it for every subclass.
3. Add fields (`code`, `status`, `field`…).
4. Handle errors in **one place** (top-level handler, Express error **middleware** — a function every request passes through — or a React error boundary) that maps types to responses and treats *unknown* errors as 500s without leaking details.

`code/03_custom_errors.mjs`:

```
class AppError extends Error {
  constructor(message, options = {}) {
    super(message, { cause: options.cause });   // pass cause up to Error
    this.name = new.target.name;                  // 'ValidationError', 'NotFoundError', ...
    this.code = options.code ?? 'APP_ERROR';
    this.status = options.status ?? 500;
  }
  toJSON() { return { name: this.name, code: this.code, message: this.message }; }
}

class ValidationError extends AppError {
  constructor(field, message) {
    super(message, { code: 'VALIDATION', status: 400 });
    this.field = field;
  }
}
class NotFoundError extends AppError {
  constructor(what, id) { super(`${what} ${id} not found`, { code: 'NOT_FOUND', status: 404 }); }
}

function placeOrder(order) {
  if (!Number.isInteger(order.qty) || order.qty < 1) throw new ValidationError('qty', 'qty must be a positive
integer');
  if (order.productId !== 'p1') throw new NotFoundError('Product', order.productId);
  return { ok: true };
}

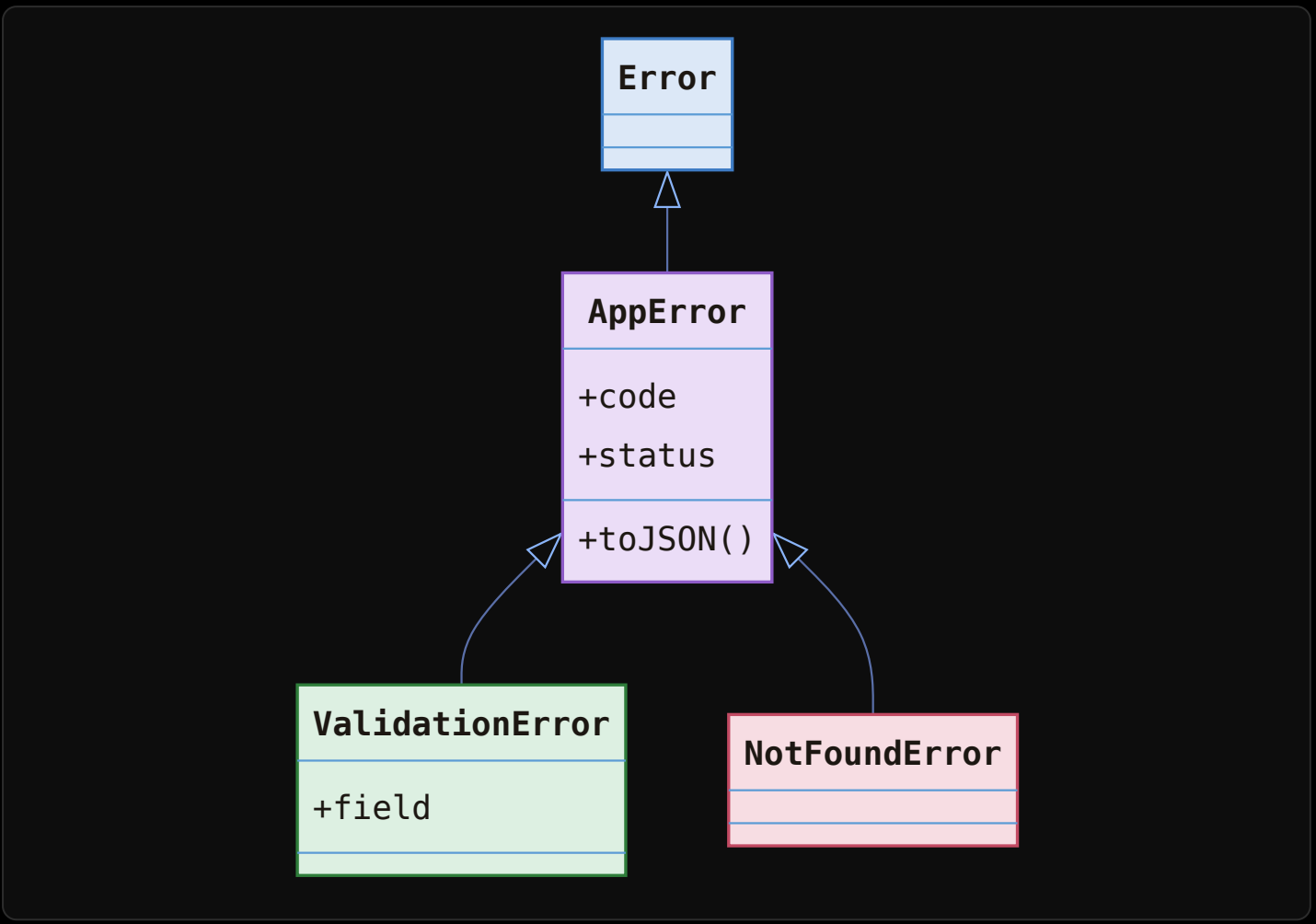
// A single place that turns errors into responses
function handle(order) {
  try {
    return { status: 200, body: placeOrder(order) };
  } catch (err) {
    if (err instanceof AppError) return { status: err.status, body: err.toJSON() };
    return { status: 500, body: { message: 'internal error' } };   // unknown -> don't leak details
  }
}
console.log(handle({ productId: 'p1', qty: 2 }));
console.log(handle({ productId: 'p1', qty: 0 }));
console.log(handle({ productId: 'p9', qty: 1 }));

const v = new ValidationError('email', 'bad email');
console.log('\ninstanceof  :', v instanceof ValidationError, v instanceof AppError, v instanceof Error);
console.log('name/field  :', v.name, v.field);
console.log('String(v)   :', String(v));
console.log('stack line 1:', v.stack.split('\n')[0]);
console.log('JSON        :', JSON.stringify(v));
console.log('plain Error JSON:', JSON.stringify(new Error('x')), '(message/stack are non-enumerable own props)');
```

Output:

```
{ status: 200, body: { ok: true } }
{
  status: 400,
  body: {
    name: 'ValidationError',
    code: 'VALIDATION',
    message: 'qty must be a positive integer'
  }
}
{
  status: 404,
  body: {
    name: 'NotFoundError',
    code: 'NOT_FOUND',
    message: 'Product p9 not found'
  }
}

instanceof  : true true true
name/field  : ValidationError email
String(v)   : ValidationError: bad email
stack line 1: ValidationError: bad email
JSON        : {"name":"ValidationError","code":"VALIDATION","message":"bad email"}
plain Error JSON: {} (message/stack are non-enumerable own props)
```



- The stack's first line says `ValidationError` because V8 builds `stack` lazily from `name` when first read. Other engines may not — always set `name` in the constructor.
- `JSON.stringify(new Error('x'))` is `{}`: `message` / `stack` are **non-enumerable** own properties (hidden from `for…in`, `Object.keys` and `JSON.stringify`) and `name` is inherited. Hence `toJSON()` (called automatically by `JSON.stringify`) — handy for logs and API bodies, but never send `stack` to end users.

Error codes vs classes

Approach	Pros	Cons
Subclasses + `instanceof`	Rich, hierarchical (`catch` a whole family)	Breaks across realms/duplicated packages (two copies of the class)
`code` string (`err.code === 'ENOENT'`)	Survives serialization, logs, process boundaries; Node's convention	No hierarchy; typos go unnoticed
Both (as above)	`instanceof` inside the app, `code` in logs and API bodies	A little more code

Common mistakes

- Forgetting `super(message)` → `ReferenceError` when you touch `this` (Chapter 9).
- Not setting `name` → logs say `Error: …` for everything.
- Leaking internals: sending `message`/`stack` of *unexpected* errors to clients can reveal SQL, file paths or secrets. Log server-side; send a generic message.
- Over-engineering: create a class only when some caller will handle it differently.

Real-world examples

Node system errors carry `code` (`ENOENT`, `EADDRINUSE` — port already taken, `ECONNREFUSED` — connection refused); `fetch` rejects with a `TypeError` on network failure and an `AbortError` `DOMException` when aborted; HTTP libraries throw errors with `status`/`response`; ORMs (libraries mapping database rows to objects) throw `UniqueConstraintError`-style subclasses.

Self-check

1. Why set `this.name`? → So `String(err)`, stacks and logs show the right type.
2. Why does `JSON.stringify(err)` print `{}`? → `message`/`stack` are non-enumerable; `name` is inherited.
3. Unknown error in a top-level handler? → Log full details internally; return a generic 500.

TL;DR

`class X extends Error`, `super(message, { cause })`, set `name`, add `code`/data, handle all errors in one place, never leak internals of unexpected errors.

11.4 Error chaining with `cause`

What it is

When you catch a low-level error and throw a higher-level one ("could not load config"), you **add context without losing the original**: since ES2022 every error constructor accepts `new Error(message, { cause })`, stored in `err.cause`. Before that, people lost the original or concatenated messages.

`code/04_error_cause.mjs`:

```
function readConfigText(name) {
  const err = new Error(`ENOENT: no such file or directory, open '${name}'`);   // pretend fs failed
  err.code = 'ENOENT';
  throw err;
}

function loadConfig(name) {
  try {
    return JSON.parse(readConfigText(name));
  } catch (err) {
    throw new Error(`Could not load config "${name}"`, { cause: err }); // add context, keep original
  }
}

function startServer() {
  try {
    loadConfig('server.json');
  } catch (err) {
    throw new Error('Server failed to start', { cause: err });
  }
}

try {
  startServer();
} catch (err) {
  let depth = 0;
  for (let e = err; e; e = e.cause) {                                    // walk the cause chain
    console.log(`${'  '.repeat(depth++)}${depth > 1 ? 'caused by: ' : ''}${e.message}${e.code ? ` [${e.code}]` :
''}`);
  }
  console.log('\nroot cause code:', rootCause(err).code);
}

function rootCause(e) { while (e.cause instanceof Error) e = e.cause; return e; }

import { inspect } from 'node:util';
const shown = inspect(new Error('outer', { cause: new Error('inner') }));
console.log('\nutil.inspect shows cause?', shown.includes('[cause]: Error: inner'));
```

Output:

```
Server failed to start
  caused by: Could not load config "server.json"
    caused by: ENOENT: no such file or directory, open 'server.json' [ENOENT]

root cause code: ENOENT

util.inspect shows cause? true
```

Each layer adds *what it was trying to do*; the root keeps *what actually failed*. Node's `console.log`/`console.error` and DevTools print the chain automatically.

`AggregateError` is the sibling for several independent failures: its `errors` array holds each one (`Promise.any` where every mirror failed; validation reporting all bad fields).

TL;DR

Wrap with `new Error('what I was doing', { cause: err })`; walk `.cause` to the root. `AggregateError` for many errors at once.

11.5 Errors in asynchronous code (preview)

The problem it solves

Async work finishes *after* the surrounding `try` has exited. Promises carry errors as **rejections** instead. The event loop and promises are explained in Chapter 12; here are the rules you need now.

`code/05_async_errors.mjs`:

```
const fetchUser = async (id) => {
  await null;                                    // simulate async work
  if (id !== 1) throw new Error(`user ${id} not found`);
  return { id, name: 'Ada' };
};

// 1) await inside try/catch — the normal way
try {
  await fetchUser(2);
} catch (err) {
  console.log('await + try/catch :', err.message);
}

// 2) .catch() on the promise
await fetchUser(3).catch((err) => console.log('.catch()          :', err.message));

// 3) Promise.allSettled — collect successes and failures together
const results = await Promise.allSettled([fetchUser(1), fetchUser(4)]);
console.log('allSettled        :', results.map((r) => (r.status === 'fulfilled' ? r.value.name :
r.reason.message)));

// 4) Forgetting to await: the try/catch finishes before the promise rejects
process.on('unhandledRejection', (reason) => {
  console.log('unhandledRejection:', reason.message, '(Node would crash here without this handler)');
});
try {
  fetchUser(5);                                  // no await!
} catch {
  console.log('never printed');
}
console.log('after the try     : the rejection has not happened yet');
await new Promise((r) => setTimeout(r, 10));      // let the event loop report it
```

Output:

```
await + try/catch : user 2 not found
.catch()          : user 3 not found
allSettled        : [ 'Ada', 'user 4 not found' ]
after the try     : the rejection has not happened yet
unhandledRejection: user 5 not found (Node would crash here without this handler)
```

Rules:

1. In an `async` function, `throw` becomes a rejected promise, and `await` of a rejected promise throws — so `try { await … } catch` works naturally.
2. **Always `await` (or `return`, or `.catch()`) every promise.** A **floating** promise's rejection escapes every `try`; `@typescript-eslint/no-floating-promises` flags them.
3. An **unhandled rejection** crashes Node by default (since Node 15) and logs "Uncaught (in promise)" in browsers.

Last-resort global handlers

Environment	Synchronous uncaught error	Unhandled promise rejection
Browser	`window.addEventListener('error', e => …)`	`window.addEventListener('unhandledrejection', e => …)`
Node	`process.on('uncaughtException', err => …)`	`process.on('unhandledRejection', reason => …)`

Use them to **log and report** (e.g., to an error tracker such as Sentry), not to carry on. After `uncaughtException` a Node process is in an unknown state — log, exit, and let a process manager (PM2, systemd, Kubernetes) restart it.

TL;DR

`await` inside `try`, or `.catch()` every promise; never leave a promise floating. Global handlers are for logging, not recovery.

11.6 Guaranteed cleanup: `finally` and `using`

What it is

Resources — files, DB connections, locks, timers, subscriptions — must be released even on failure. `finally` has always done this. **Explicit Resource Management** automates it: an object with a `[Symbol.dispose]()` method, declared with `using`, is disposed when the enclosing block exits, however it exits. It reached TC39 (the committee that designs JavaScript) stage 4 — finished — in 2025 and is slated for ES2027; supported in Node 24 and Chromium-based browsers (Chrome 134+) — check your targets for browsers. It mirrors C#'s `using`, Python's `with` and Java's try-with-resources.

`code/09_using_and_cleanup.mjs`:

```
function openFile(name) {
  console.log(`open  ${name}`);
  return {
    name,
    read() { if (name === 'bad.txt') throw new Error(`cannot read ${name}`); return `contents of ${name}`; },
    [Symbol.dispose]() { console.log(`close ${name}`); },   // called automatically by `using`
  };
}

// Classic pattern: try/finally
function classic(name) {
  const f = openFile(name);
  try { return f.read(); }
  finally { f[Symbol.dispose](); }
}
console.log('classic ->', classic('a.txt'));

// Same thing with `using`: disposed when the block exits, in reverse order, even on error
function modern(...names) {
  using first = openFile(names[0]);
  using second = openFile(names[1]);
  return [first.read(), second.read()];
}
console.log('modern  ->', modern('a.txt', 'b.txt'));
try {
  modern('c.txt', 'bad.txt');
} catch (err) {
  console.log('modern error ->', err.message, '(both files were still closed)');
}

// If disposal ALSO throws, you get a SuppressedError holding both errors
function doubleFailure() {
  using r = { [Symbol.dispose]() { throw new Error('dispose failed'); } };
  throw new Error('body failed');
}
try { doubleFailure(); } catch (err) {
  console.log(`${err.name}: error="${err.error.message}" suppressed="${err.suppressed.message}"`);
}
```

Output:

```
open  a.txt
close a.txt
classic -> contents of a.txt
open  a.txt
open  b.txt
close b.txt
close a.txt
modern  -> [ 'contents of a.txt', 'contents of b.txt' ]
open  c.txt
open  bad.txt
close bad.txt
close c.txt
modern error -> cannot read bad.txt (both files were still closed)
SuppressedError: error="dispose failed" suppressed="body failed"
```

- Disposal runs in **reverse** declaration order (last opened, first closed).
- `await using` does the same with `[Symbol.asyncDispose]()` for async cleanup.
- If body and disposal both throw you get a **`SuppressedError`**: `.error` = disposal error, `.suppressed` = the one it replaced — nothing is lost.

TL;DR

Release resources in `finally`, or with `using` + `[Symbol.dispose]` where supported. Disposal runs in reverse order, even on errors.

11.7 The console toolbox

What it is

`console` is the built-in logging object. In browsers it prints to the DevTools **Console** (filterable by level); in Node, `log/info/debug/table/dir/group/count/time` write to **stdout** and `warn/error/trace/assert` to **stderr** (a separate stream for problems, redirectable on its own).

`code/06_console_methods.mjs` runs every method below; the highlights:

```
const users = [
  { id: 1, name: 'Ada', role: 'admin' },
  { id: 2, name: 'Linus', role: 'dev' },
];
console.table(users);

const deep = { a: { b: { c: { d: { e: 'deep' } } } } };
console.log('default depth:', deep);

for (const k of ['a', 'b', 'a', 'a']) console.count(k);
console.countReset('a');
console.count('a');

console.assert(0.1 + 0.2 === 0.3, 'assertion failed: floating point strikes again'); // prints, does NOT throw
```

Output:

```
┌─────────┬────┬──────────┬──────────┐
│ (index) │ id │ name     │ role     │
├─────────┼────┼──────────┼──────────┤
│ 0       │ 1  │ 'Ada'    │ 'admin'  │
│ 1       │ 2  │ 'Linus'  │ 'dev'    │
└─────────┴────┴──────────┴──────────┘
default depth: { a: { b: { c: [Object] } } }
a: 1
b: 1
a: 2
a: 3
a: 1
Assertion failed: assertion failed: floating point strikes again
```

Method	Use it for
`log` / `info` / `debug`	General output; separate filter levels in browsers
`warn` / `error`	Problems — yellow/red in DevTools, stderr in Node, include a stack in browsers
`%s %d %i %f %o %O %c`	Format specifiers (string, integer, float, object; `%c` = CSS styling in browsers)
`table(data, cols?)`	Arrays of objects → grid
`dir(obj, { depth })`	Full inspection (Node's `console.log` stops at depth 2 → `[Object]`)
`group` / `groupCollapsed` / `groupEnd`	Indent related messages
`count(label)` / `countReset`	"How many times did this run?"
`assert(cond, msg)`	Logs only when `cond` is false — **does not throw**
`time` / `timeLog` / `timeEnd`	Quick timing (use `performance.now()` or a profiler for real measurement)
`trace(msg)`	Print the current stack without throwing

Common mistakes

- **Browser object logs are live references**: expanding shows the *current* state, not the state at log time. Log a snapshot: `structuredClone(obj)` or `JSON.stringify(obj)`.
- Leaving debug logs in production — use `no-console` or a leveled logger (debug/info/warn/error, filter by level).
- `console.log('user: ' + user)` → `user: [object Object]`. Pass objects as separate arguments.

TL;DR

Use `table`, `dir`, `group`, `count`, `assert`, `time`, `trace` — and remember browser object logs are live.

11.8 Reading stack traces and source maps

What it is

A **stack trace** lists the function calls active when the error was created — **newest first**. Each **frame** shows the function name and `file:line:column`. Top-down tells you *where* it failed; bottom-up tells you *how you got there*.

`code/07_stack_traces.mjs` (key parts):

```
function validate(cart) { if (cart.items.length === 0) throw new Error('empty cart'); }
function checkout(cart) { validate(cart); }
function handleClick() { checkout({ items: [] }); }

try {
  handleClick();
} catch (err) {
  // shorten absolute file paths so the output is the same on every machine
  const pretty = err.stack.replace(/file:\/\/\S+\//g, '').replace(/\(.*[\\/](\S+?:\d+:\d+)\)/g, '($1)');
  console.log(pretty.split('\n').slice(0, 5).join('\n'));
}
```

Output:

```
Error: empty cart
    at validate (07_stack_traces.mjs:4:62)
    at checkout (07_stack_traces.mjs:5:27)
    at handleClick (07_stack_traces.mjs:6:26)
    at 07_stack_traces.mjs:9:3
```

How to read it:

1. **Line 1** = `name: message` — *what* happened.
2. **First frame** = where it was thrown (`validate`, line 4, column 62).
3. **Following frames** = callers. Start at the first frame in *your* code (not a library or `node:internal`).
4. **Anonymous frames** (`at 07_stack_traces.mjs:9:3`) are top-level code or unnamed functions — name your functions.

V8-only helpers (Node, Chrome, Edge): `Error.stackTraceLimit` (default 10; raise it, or run `node --stack-trace-limit=50`), and `Error.captureStackTrace(obj, fn?)`, which attaches a stack to any object (optionally hiding frames above `fn`). V8 stitches **async stack traces** across `await` points (`at async functionName`), so `async/await` traces are far more useful than callback ones.

Source maps

Production code is bundled and minified (Chapter 10), so a raw trace reads `at e (app.3f9a1c.js:1:48213)`. A **source map** (`app.js.map`) maps each generated position back to the original file/line/column. DevTools applies it when it finds `//# sourceMappingURL=`; Node with `node --enable-source-maps`. Error trackers let you upload maps privately so traces are readable without publishing source.

TL;DR

First line = *what*, first frame in your code = *where*, the rest = *how*. Source maps for bundled code; name functions for clearer traces.

11.9 Debugging with DevTools and the Node inspector

What it is

A **debugger** lets you **pause** at a chosen line, **inspect** every variable in scope, and **step** line by line — far faster than add-a-log-and-rerun for "why is this value wrong here?". Browsers ship one in **DevTools** (F12 / Cmd+Option+I); Node exposes the same Chrome DevTools protocol through its **inspector**.

Core concepts

Concept	What it does
Breakpoint	Pauses before a line runs. Click a line number (Sources panel in Chrome, Debugger in Firefox) or write `debugger;` (ignored when no debugger is attached).
Conditional breakpoint	Pauses only when an expression is true (`item.qty > 100`) — essential in loops.
Logpoint	Logs a message without pausing or editing code.
Pause on exceptions	Stops at the throwing line (optionally caught ones too) — state at the moment of failure.
Step over (F10)	Run the current line, including calls, stop at the next.
Step into (F11)	Enter the function called on this line.
Step out (Shift+F11)	Finish the current function, stop in its caller.
Resume (F8)	Continue to the next breakpoint.
Call Stack pane	The frames — click one to inspect that caller's variables.
Scope pane	Local, closure and global variables for the frame (incl. closures from Chapter 4 and `#private` fields).
Watch expressions	Re-evaluated at every pause.
Console while paused	Evaluate expressions in the paused scope, even change variables.

(Shortcuts are Chrome's; VS Code uses F10/F11/Shift+F11, F5 to continue.)

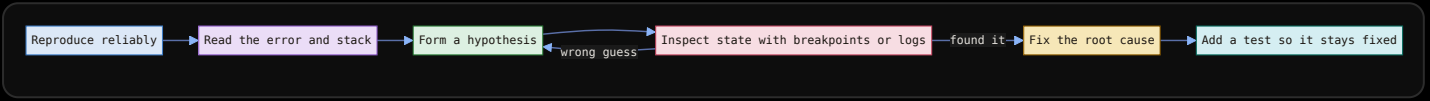
Browser-only extras: **DOM breakpoints** (pause when an element's subtree/attributes change), **event listener breakpoints** (pause on any `click`), **XHR/fetch breakpoints** (pause when a URL is requested; XHR = the older `XMLHttpRequest` API), the **Network** panel (every request, status and response — first stop for "the API call failed"), **Performance** and **Memory** panels (profiling and **heap snapshots** for finding **memory leaks** — memory never freed because something still references it), and an **ignore list** that hides library frames when stepping.

Debugging Node (not run — interactive)

```
node --inspect app.mjs          # start with the inspector on 127.0.0.1:9229
node --inspect-brk app.mjs      # same, but pause on the first line (so you can set breakpoints)
# then open chrome://inspect in Chrome -> "Open dedicated DevTools for Node"
node inspect app.mjs            # built-in terminal debugger: c(ont), n(ext), s(tep), o(ut), repl, bt
node --enable-source-maps dist/app.js   # readable traces for transpiled code
node --watch app.mjs            # restart automatically on save while you fix things
```

In **VS Code**, a "JavaScript Debug Terminal" makes editor breakpoints work for any Node script, test runner or dev server. Never expose `--inspect` on a public interface (anything but `127.0.0.1`): whoever connects can run code in your process.

A systematic method



1. **Reproduce** reliably with the smallest triggering input.
2. **Read** the actual message and stack.
3. **Hypothesize**, then **inspect** the values that confirm or refute it (breakpoints or labelled logs).

4. **Narrow down** by halving — is the value already wrong at the midpoint? (`git bisect` binary-searches commits for the change that introduced the bug.)
5. **Fix the root cause**, not the symptom.
6. **Add a test** that fails without the fix (Chapter 16).

Explaining the code line by line (even to a rubber duck) works because it forces you to state assumptions.

Worked example

`code/10_debugging_walkthrough.mjs`: a cart total that comes out wrong.

```
import assert from 'node:assert/strict';

// Data as it arrives from an HTML form or query string: everything is a string.
const cart = [
  { name: 'Book', price: '12.50', qty: '2' },
  { name: 'Pen', price: '1.25', qty: '4' },
];

// BUGGY version
function totalBuggy(items) {
  return items.reduce((sum, item) => sum + item.price * item.qty, '');
}
console.log('symptom       :', totalBuggy(cart), '(expected 30)');

// Inspect the intermediate values — labelled and structured
function totalTraced(items) {
  return items.reduce((sum, item, i) => {
    const next = sum + item.price * item.qty;
    console.log(`  step ${i}:`, { sum, typeofSum: typeof sum, line: item.price * item.qty, next });
    // debugger;   // <- with DevTools / `node inspect` attached, execution would pause here
    return next;
  }, '');
}
console.log('inspect       :');
totalTraced(cart);

// Fix: numeric initial value + explicit conversion at the boundary.
function total(items) {
  return items.reduce((sum, item) => sum + Number(item.price) * Number(item.qty), 0);
}
console.log('fixed         :', total(cart));

assert.equal(total(cart), 30);
assert.equal(total([]), 0);
assert.throws(() => assert.equal(totalBuggy(cart), 30), assert.AssertionError);
console.log('tests         : passed');
```

Output:

```
symptom       : 255 (expected 30)
inspect       :
  step 0: { sum: '', typeofSum: 'string', line: 25, next: '25' }
  step 1: { sum: '25', typeofSum: 'string', line: 5, next: '255' }
fixed         : 30
tests         : passed
```

`typeofSum: 'string'` from the first step: the initial `''` made `+` concatenate (`'25' + 5` → `'255'`), while `*` silently converted strings to numbers (Chapter 2). Nothing threw — **the most dangerous bugs are wrong values, not exceptions.** Converting at the boundary (`Number(...)`) plus tests prevents the whole class; TypeScript would also have flagged `string * string` (Chapter 16).

In the interview

> **30-second answer:** "I reproduce the bug first, read the stack trace, then use breakpoints — often conditional ones or 'pause on exceptions' — to inspect the actual state instead of guessing. I

> step over/into to follow the flow and check the Network panel for API issues. For Node I use `--inspect` with Chrome DevTools or VS Code's debug terminal. Once fixed, I add a test that reproduces the bug."

- *Q: What's a logpoint?* A: A breakpoint that logs an expression instead of pausing.
- *Q: How do you debug minified production errors?* A: Source maps, uploaded to the error tracker or loaded in DevTools.
- *Q: What does `debugger` do without DevTools open?* A: Nothing.

TL;DR

Pause, inspect, step: breakpoints (conditional, logpoints, pause-on-exceptions), Scope/Watch/Call Stack, Network panel. Node: `--inspect`/`--inspect-brk` + `chrome://inspect` or VS Code. Reproduce → hypothesize → inspect → fix root cause → add a test.

11.10 Common runtime errors — a field guide

`code/08_common_runtime_errors.mjs` triggers twenty everyday mistakes (e.g. `res.data.items` on `{}`, `const [a] = {}`, `[...123]`, `new Array(2 ** 32)`, `1n + 1`) and prints V8's exact messages (Node, Chrome, Edge):

```
call a non-function         TypeError: user.getName is not a function
property of undefined       TypeError: Cannot read properties of undefined (reading 'items')
property of null            TypeError: Cannot read properties of null (reading 'textContent')
set property on undefined   TypeError: Cannot set properties of undefined (setting 'debug')
undeclared variable         ReferenceError: totl is not defined
TDZ (use before let)        ReferenceError: Cannot access 'x' before initialization
reassign const              TypeError: Assignment to constant variable.
not iterable                TypeError: {} is not iterable
spread non-iterable         TypeError: 123 is not iterable
JSON.parse(undefined)       SyntaxError: "undefined" is not valid JSON
JSON.parse(HTML)            SyntaxError: Unexpected token '<', "<!DOCTYPE html>" is not valid JSON
class without new           TypeError: Class constructor Cart cannot be invoked without 'new'
reduce empty array          TypeError: Reduce of empty array with no initial value
BigInt + Number             TypeError: Cannot mix BigInt and other types, use explicit conversions
bad array length            RangeError: Invalid array length
toFixed digits              RangeError: toFixed() digits argument must be between 0 and 100
infinite recursion          RangeError: Maximum call stack size exceeded
circular JSON               TypeError: Converting circular structure to JSON
frozen object (strict)      TypeError: Cannot assign to read only property 'a' of object '#<Object>'
in on a primitive           TypeError: Cannot use 'in' operator to search for 'x' in hello
```

What each usually means in real code

Message (V8)	Usual real-world cause	Fix
`x is not a function`	Typo in a method name; wrong import (default vs named, Chapter 10); calling a data property; library version lacks the method	Check name and import; `console.log(typeof x)`
`Cannot read properties of undefined (reading 'y')`	Data not loaded yet (async), wrong API response shape, index out of range, missing `return`	Find *which* value is `undefined` (the part before `.y`); `?.` /defaults only if absence is legitimate
`Cannot set properties of undefined`	Writing into a nested object that doesn't exist	Create it first, or `obj.cfg ??= {}`
`x is not defined`	Typo, missing import, other scope, browser-only global in Node (`window`, `document`)	Declare/import it; check the environment
`Cannot access 'x' before initialization`	`let` / `const` / `class` used in its TDZ, often via circular imports	Reorder code; break import cycles
`Assignment to constant variable.`	Reassigning a `const` or an imported binding	Use `let`, or mutate instead
`… is not iterable`	`for…of` / spread / destructuring on a plain object, `undefined`, or a number	`Object.entries(obj)`; check for `undefined`
`"undefined" is not valid JSON` / `Unexpected token '<' … is not valid JSON`	Parsing a missing value; server returned an HTML error page	Check `response.ok` and `Content-Type` before `response.json()` (Chapter 14)
`Class constructor X cannot be invoked without 'new'`	Calling a class like a function	Add `new`
`Reduce of empty array with no initial value`	`reduce` on a possibly empty array	Always pass an initial value
`Cannot mix BigInt and other types`	`bigint + number`	`BigInt(n)` or `Number(b)`
`Maximum call stack size exceeded`	Infinite/too-deep recursion; a setter assigning to itself; mutual recursion	Fix the base case; iterate instead
`Converting circular structure to JSON`	Back-references (DOM nodes, parent links)	Pick fields, or use a replacer function
`Cannot assign to read only property`	Writing to a frozen object or getter-only accessor in strict code (always on in modules/classes)	Copy instead: `{ ...obj, a: 2 }`
`Cannot use 'in' operator to search for …`	`in` on a string/number (often un-parsed JSON)	Parse first; `in` works on objects only

Also worth recognizing: `Unexpected token` / `Unexpected end of input` (missing bracket or comma), `await is only valid in async functions and the top level bodies of modules`, `Cannot use import statement outside a module` (Chapter 10), and `Failed to fetch` (a `TypeError` for network/CORS failure — Chapter 14).

Self-check

1. `Cannot read properties of undefined (reading 'map')` — what is `undefined`? → The value *before* `.map`.
2. `JSON.parse` fails with `Unexpected token '<'` after a `fetch`? → The server returned HTML (an error page or `index.html`), not JSON.
3. Why does `[].reduce((a, b) => a + b)` throw but `[].reduce((a, b) => a + b, 0)` doesn't? → With no initial value, reduce needs an element to start from.

TL;DR

Most runtime errors are `TypeError`s from `undefined`/`null` or wrong types. Read the message literally — it names the value and property. Find where the bad value came from, not just where it exploded.

Chapter 11 TL;DR

- Errors are objects (`name`, `message`, `stack`, optional `cause`); core types `TypeError`, `ReferenceError`, `SyntaxError`, `RangeError`, `URIError`, `AggregateError`, plus `SuppressedError` for `using`. `Error.isError` (ES2026) is the robust check.
- `try` → `catch` → `finally`: finally always runs; never `return` from it. Catch narrowly, rethrow the rest, throw `Error` objects. try/catch is synchronous.
- **Custom errors**: `extends Error`, `super(msg, { cause })`, set `name`, add `code`/data, handle centrally, don't leak internals.
- `cause` chains context onto the original; walk `.cause` to the root.
- **Async**: `await` in `try` or `.catch()` every promise; global `error`/`unhandledrejection` (browser) and `uncaughtException`/`unhandledRejection` (Node) are for logging.
- **Cleanup**: `finally`, or `using` + `Symbol.dispose` (ES2027, Node 24).
- **Console**: `table`, `dir`, `group`, `count`, `assert`, `time`, `trace`; browser object logs are live.
- **Stack traces**: newest frame first; source maps for bundled code.
- **Debuggers**: breakpoints (conditional, logpoints, pause on exceptions), step over/into/out, Scope/Watch/Call Stack, Network; Node `--inspect` + `chrome://inspect` or VS Code.
- **Method**: reproduce → read → hypothesize → inspect → fix root cause → add a test.

Next: the event loop, promises and `async`/`await` — Chapter 12: Asynchronous JavaScript.

Chapter 12 — Asynchronous JavaScript

JavaScript runs your code on **one thread**, yet a page can download data, animate and respond to clicks at once, and a Node server can handle thousands of connections. The trick is **asynchrony**: slow work is handed to the host (browser or Node) and your callback runs later, when the result is ready. The **event loop** decides *when* each callback runs. This chapter explains it precisely enough to predict any ordering puzzle, then the three generations of async code built on it: callbacks, promises and `async`/`await`.

What you'll learn

1. Why JavaScript is single-threaded and what "blocking" means.
2. The event loop: call stack, task queue, microtask queue, rendering.
3. Seven ordering puzzles, run in Node **and** Chrome.
4. Node's additions: `process.nextTick`, `setImmediate`, loop phases.
5. Timers and their (non-)guarantees.
6. Callbacks, error-first callbacks, and callback hell.
7. Promises: states, chaining, error flow, `withResolvers`, `Promise.try`.
8. The combinators `all`, `allSettled`, `race`, `any`.
9. `async`/`await`: sequential vs parallel, loops, errors.
10. Cancellation with `AbortController` / `AbortSignal`.
11. Async iteration: `for await…of`, async generators, `Array.fromAsync`.
12. Production patterns: retry with backoff, concurrency limits, stale responses.

Before you start. Functions and closures (Chapter 4); generators/iterators (Chapter 8) for §12.12. §11.5 previewed async errors; this chapter completes the story. Top-level `await`: Chapter 10.

All runnable code is in `code/`. Outputs are real (Node 24); timing-dependent programs print rounded or yes/no values so output is stable.

12.1 One thread, and why blocking hurts

What it is

A **thread** is one sequence of instructions being executed; JavaScript gives your code exactly one. **Synchronous** code runs start to finish without pausing; **asynchronous** code starts an operation now and handles the result later via a callback.

- **Blocking** — code that keeps the thread busy, so nothing else (callbacks, clicks, rendering) can run.
- **Non-blocking I/O** — network/disk work done by the host in the background; JavaScript runs only when the result arrives.
- **Host / runtime** — the environment around the engine: the browser (Web APIs: timers, `fetch`, the page) or Node (libuv, its C library for async I/O). The engine (V8) runs JavaScript; the host does the waiting.

Why it matters

In a browser the same thread runs your JavaScript, handles input and (usually) renders. A 200 ms synchronous loop freezes scrolling, typing and animation for 200 ms. In Node, a blocking handler stalls *every* request.

`code/01_single_thread.mjs`:

```
const start = Date.now();
setTimeout(() => {
  const waited = Date.now() - start;
  console.log(`timer asked for 0 ms, ran after >= 200 ms: ${waited >= 200}`);
}, 0);

console.log('busy loop starts');
while (Date.now() - start < 200) { /* burn CPU: nothing else can run */ }
console.log('busy loop ends (the timer is still waiting)');

// Asynchronous work does NOT block: the callback is queued and we carry on.
import { readFile } from 'node:fs/promises';
readFile(new URL(import.meta.url)).then((buf) => console.log('file read finished,', buf.length > 0 ? 'got bytes' :
'empty'));
console.log('readFile started - main script keeps going');
```

Output:

```
busy loop starts
busy loop ends (the timer is still waiting)
readFile started - main script keeps going
timer asked for 0 ms, ran after >= 200 ms: true
file read finished, got bytes
```

The 0 ms timer could not interrupt the loop. Callbacks are never pre-empted: each runs to completion before anything else starts (**run-to-completion**). Benefit: no locks around your variables. Cost: keep each piece of work short.

When you really need parallel CPU work

Heavy computation belongs off the main thread: **Web Workers** in the browser (Chapter 14) or `worker_threads` in Node (Chapter 15) — separate threads with separate memory that talk by messages.

TL;DR

One thread runs your JavaScript; the host waits for I/O and timers. Long synchronous work blocks everything, including 0 ms timers and rendering.

12.2 The event loop

What it is

The **event loop** is the host's scheduler: it repeatedly picks waiting work and runs it on the single thread.

Part	What it holds	Examples
Call stack	Functions currently executing, newest on top	Main script, a callback in progress
Host APIs (Web APIs / libuv)	Operations in progress *outside* JavaScript	A timer counting down, an HTTP request, a file read
Task queue (macrotask / callback queue)	Callbacks ready to run, one per loop turn	Timers, I/O callbacks, `message` events, `click`
Microtask queue	Follow-ups that run "as soon as the current code finishes"	Promise `then` / `catch` / `finally`, code after `await`, `queueMicrotask`, `MutationObserver`
Rendering (browsers)	Style (which CSS applies), layout (sizes/positions), paint (pixels) — typically up to ~60×/s	`requestAnimationFrame` callbacks run just before paint

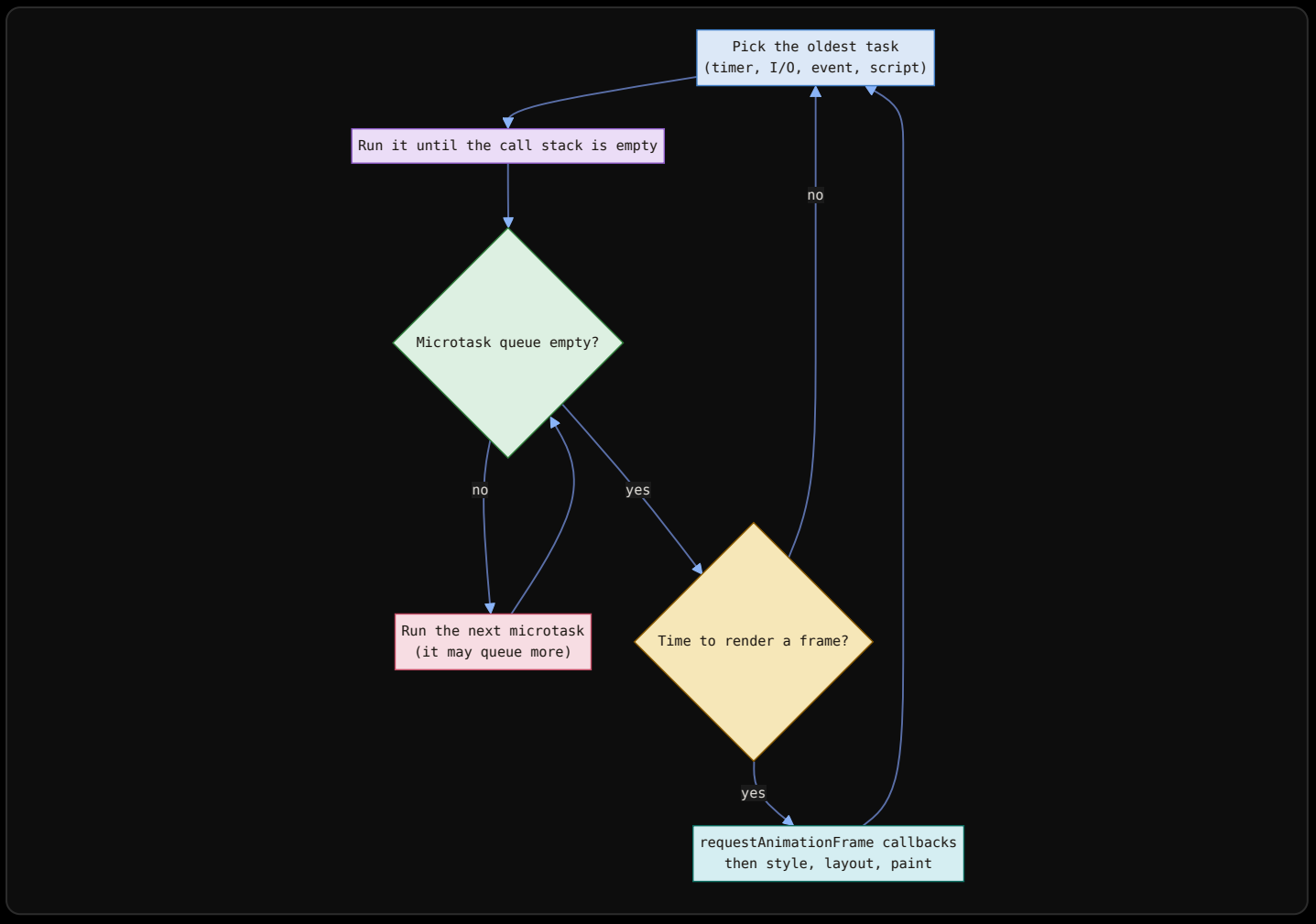
The HTML spec ("Event loops") defines the browser algorithm; ECMAScript only defines **jobs** (promise reactions) and leaves scheduling to the host. The spec allows several task queues (input may be prioritised over timers), but for predicting order treat tasks as one FIFO queue per source.

How it works

One turn, simplified from the HTML spec:

1. **Run one task** — the oldest runnable task, until the call stack is empty.
2. **Microtask checkpoint** — run microtasks until the queue is **empty**, including ones queued *by* microtasks.
3. **Maybe render** — if a new frame is due: `requestAnimationFrame` callbacks, style, layout, paint.
4. Repeat (or sleep until a task arrives).

The initial script is itself a task, so step 2 happens right after top-level code. Microtasks also run whenever the stack empties during a task — e.g. between two listeners on a real user click (but not on `el.click()` from code, since your code is still on the stack).



Why microtasks exist

A promise callback should run **after** the current synchronous code (callbacks are always async — consistent behaviour) but **before** anything else can observe inconsistent state (no other task or render in between). Microtasks give exactly that "end of the current job" timing.

Common mistakes

- `setTimeout(fn, 0)` **is not "immediately"** — it's a *later task*, after all pending microtasks and possibly a render.
- **Microtask starvation.** A microtask that always queues another (recursive `.then`) never lets the loop render or run tasks — the page freezes like `while(true)`. Yield with `setTimeout` (a task) when chunking long work.

- `await` **doesn't yield to rendering** — it only schedules a microtask. To let the page paint during a long job, split it across tasks (`setTimeout`, `scheduler.yield()` where supported, `requestAnimationFrame`).

> **TL;DR**

Loop: one task → **all** microtasks → maybe render → repeat. Promises and `await` continuations are microtasks; timers, I/O and events are tasks.

12.3 Ordering puzzles

With §12.2's rules you can answer any "what does this print?":

1. Run all synchronous code (including promise **executors** — the function passed to `new Promise` — and code before the first `await`).
2. Drain the microtask queue FIFO, appending new microtasks as they're created.
3. Take the next task (timers by due time; ties in creation order), run it, drain microtasks, repeat.

`code/02_ordering_puzzles.mjs` runs seven puzzles; each logs into an array and the order is printed after all queues drain. Predict before reading.

```
// P1 - the classic: sync code, then ALL microtasks, then the next task
puzzle('P1 classic', (log) => {
  log('A');
  setTimeout(() => log('B timeout'), 0);
  Promise.resolve().then(() => log('C then'));
  queueMicrotask(() => log('D microtask'));
  log('E');
});

// P2 - async/await: code before the first await runs synchronously
puzzle('P2 async/await', (log) => {
  async function f() {
    log('f start');
    await null;             // everything after this line is a microtask
    log('f after await');
  }
  log('1');
  f();
  log('2');
});

// P3 - microtasks queued BY microtasks still run before the next task
puzzle('P3 nested', (log) => {
  setTimeout(() => {
    log('T1');
    Promise.resolve().then(() => log('T1 -> micro'));
  }, 0);
  setTimeout(() => log('T2'), 0);
  Promise.resolve().then(() => {
    log('M1');
    setTimeout(() => log('M1 -> timeout'), 0);
    Promise.resolve().then(() => log('M1 -> micro'));
  });
});

// P4 - two promise chains interleave one step at a time
puzzle('P4 interleave', (log) => {
  Promise.resolve().then(() => log('a1')).then(() => log('a2')).then(() => log('a3'));
  Promise.resolve().then(() => log('b1')).then(() => log('b2')).then(() => log('b3'));
});

// P5 - the executor of new Promise runs synchronously; then() callbacks do not
puzzle('P5 executor', (log) => {
  log('start');
  const p = new Promise((resolve) => {
    log('executor');
    resolve('value');
    log('after resolve');   // resolve() does not stop the function
  });
  p.then((v) => log(`then: ${v}`));
  log('end');
});

// P6 - returning a promise from then() costs extra microtask ticks
puzzle('P6 return promise', (log) => {
  Promise.resolve()
    .then(() => { log('x1'); return Promise.resolve('inner'); }) // adopts: +2 ticks
    .then((v) => log(`x2 (${v})`));
  Promise.resolve()
    .then(() => log('y1'))
    .then(() => log('y2'))
    .then(() => log('y3'))
    .then(() => log('y4'));
});

// P7 - timers with different delays: order is by due time, then by creation
puzzle('P7 timer delays', (log) => {
  setTimeout(() => log('20ms'), 20);
  setTimeout(() => log('0ms first'), 0);
  setTimeout(() => log('1ms'), 1);
```

```
  setTimeout(() => log('0ms second'), 0);
});
// ... the harness runs each puzzle, waits 50 ms, prints the collected order
```

Output (Node 24):

```
P1 classic         -> A | E | C then | D microtask | B timeout
P2 async/await     -> 1 | f start | 2 | f after await
P3 nested          -> M1 | M1 -> micro | T1 | T1 -> micro | T2 | M1 -> timeout
P4 interleave      -> a1 | b1 | a2 | b2 | a3 | b3
P5 executor        -> start | executor | after resolve | end | then: value
P6 return promise  -> x1 | y1 | y2 | y3 | x2 (inner) | y4
P7 timer delays    -> 0ms first | 1ms | 0ms second | 20ms
```

The same puzzles in a page (`code/puzzles_browser.html`) run by headless Chrome give the identical order — promise and timer ordering is standardized, not a Node quirk:

Chapter 12 ordering puzzles — run by headless Chrome

Puzzle	Order the browser produced
P1 classic	A | E | C then | D microtask | B timeout
P2 async/await	1 | f start | 2 | f after await
P3 nested	M1 | M1 -> micro | T1 | T1 -> micro | T2 | M1 -> timeout
P4 interleave	a1 | b1 | a2 | b2 | a3 | b3
P5 executor	start | executor | after resolve | end | then: value
P6 return promise	x1 | y1 | y2 | y3 | x2 (inner) | y4
P7 timer delays	0ms first | 1ms | 0ms second | 20ms

HeadlessChrome/154.0.0.0

Figure — the ordering puzzles run in Chrome 154.

The explanations

Puzzle	Why
P1	`A`, `E` synchronous. `C`, `D` microtasks in queue order. `B` is a task, so it waits for the microtask queue to empty.
P2	An async function runs synchronously **up to the first** `await`; the rest is a microtask, so `2` prints first.
P3	Microtask `M1` runs first; it queues a timer (behind T1, T2) and a microtask that runs in the same checkpoint. Task T1's microtask runs **before** task T2. Finally the timer from M1.
P4	Each `.then` is queued only when the previous one settles, so two chains alternate.
P5	The executor is synchronous; `resolve()` doesn't return or throw. `then` callbacks are always async, even on a resolved promise.
P6	A `then` callback returning a promise (a **thenable**) makes the outer promise *adopt* it: one microtask to call `inner.then(...)`, another for that reaction, then the next `then` — so `x2` lands after `y3`. A plain value would print `x2` right after `y1`.
P7	Order by due time, ties by creation. Node (and current Chrome) treat `0` like `1` ms here — never depend on such fine differences.

P6 is why `return await p` was once discouraged "for performance"; the cost is a couple of microtasks, and inside `try` it is *required* to catch the rejection (§12.9).

In the interview

> **30-second answer:** "Synchronous code runs first. Then the engine drains the whole microtask queue — promise callbacks, code after `await`, `queueMicrotask` — including microtasks added along the way. Only then does the event loop take the next task, like a `setTimeout` callback or an event, and after each task it drains microtasks again. So promises always beat `setTimeout(0)`."

- *Q: Does `await` of a non-promise pause?* A: Yes — `await 5` still defers the rest to a microtask.
- *Q: Why does P6's `x2` come late?* A: Adopting a returned thenable takes two extra ticks.
- *Q: Can a microtask starve rendering?* A: Yes; nothing renders until the queue is empty.
- *Q: `setTimeout(a, 0); queueMicrotask(b); Promise.resolve().then(c);`?* A: `b`, `c`, `a`.
- *Q: A microtask queues a timer and another microtask — which runs first?* A: The microtask.

TL;DR

Sync → all microtasks (FIFO, incl. new ones) → one task → all microtasks → … Executors and code before the first `await` are synchronous.

12.4 Node's event loop: `nextTick`, `setImmediate` and phases

What it is

Node's loop (implemented by **libuv**) cycles through **phases**, each with its own queue:

Phase	Runs
timers	Expired `setTimeout` / `setInterval` callbacks
pending callbacks	Some deferred system callbacks (e.g. certain TCP errors)
poll	Waits for and runs I/O callbacks (files, sockets)
check	`setImmediate` callbacks
close callbacks	`'close'` events, e.g. `socket.on('close')`

Between every callback Node drains the **`process.nextTick` queue**, then the **promise microtask queue** — in CommonJS (Node's original `require`/`module.exports` system); ES modules (`import`/`export`, ESM) twist this (below).

`code/03_node_queues.mjs`:

```
import { readFile } from 'node:fs';

const log = [];
setTimeout(() => log.push('timeout'), 0);
setImmediate(() => log.push('immediate'));
Promise.resolve().then(() => log.push('promise'));
process.nextTick(() => log.push('nextTick'));
log.push('sync');
setTimeout(() => {
  console.log('main module :', log.join(' | '));
  console.log('  (timeout vs immediate at top level can swap between runs - see text)');

  // Inside an I/O callback the order of immediate vs timeout IS guaranteed.
  readFile(new URL(import.meta.url), () => {
    const io = [];
    setTimeout(() => io.push('timeout'), 0);
    setImmediate(() => io.push('immediate'));
    process.nextTick(() => io.push('nextTick'));
    Promise.resolve().then(() => io.push('promise'));
    setTimeout(() => console.log('in I/O cb  :', io.join(' | ')), 20);
  });
}, 30);
```

Output:

```
main module : sync | promise | nextTick | immediate | timeout
  (timeout vs immediate at top level can swap between runs - see text)
in I/O cb  : nextTick | promise | immediate | timeout
```

The CommonJS version (`code/03b_node_queues_cjs.js`) of the same top-level sync/promise/nextTick lines prints `CommonJS main: sync | nextTick | promise`.

Reading the results

1. **`nextTick` vs promises.** Normally `nextTick` runs first (I/O callback, CommonJS). But an ES module's top level is evaluated inside promise machinery, so its promise microtasks drain before the `nextTick` queue — hence `promise | nextTick` in the `.mjs`. Don't build logic on this.
2. **`setImmediate` vs `setTimeout(0)` at top level** is **non-deterministic**: it depends on whether 1 ms has passed when the loop first checks timers.
3. **Inside an I/O callback** it *is* deterministic: poll is followed by check, so `setImmediate` always wins.

When to use which

Want	Use
Run after current code, before anything else (portable)	`queueMicrotask(fn)` or `Promise.resolve().then(fn)`
Node-only "right after this operation", e.g. emit after the constructor returns	`process.nextTick(fn)` (Node docs recommend `queueMicrotask` for most new code)
Yield to I/O, then continue	`setImmediate(fn)`
Run after a delay	`setTimeout(fn, ms)`

> **Common mistake:** recursive `process.nextTick` starves I/O just like recursive microtasks. Node's architecture (libuv, thread pool) is in Chapter 15.

TL;DR

Phases: timers → pending → poll (I/O) → check (`setImmediate`) → close. Between callbacks: `nextTick`, then promises (ESM top level flips these). `setImmediate` vs `setTimeout(0)` is ordered only inside I/O callbacks.

12.5 Timers

What it is

Function	Does	Cancel with
`setTimeout(fn, ms, ...args)`	Run `fn(...args)` once, **at least** `ms` later	`clearTimeout(id)`
`setInterval(fn, ms, ...args)`	Run repeatedly, roughly every `ms`	`clearInterval(id)`
`queueMicrotask(fn)`	Run as a microtask	—
`requestAnimationFrame(fn)` (browser)	Run before the next paint (Ch. 14)	`cancelAnimationFrame(id)`
`setImmediate(fn)` (Node)	Run in the check phase	`clearImmediate(id)`

The **handle** is a number in browsers and a `Timeout` object in Node (with `.unref()` — let the process exit while pending — and `.refresh()`).

How it works

The timer registers with the host; when the delay elapses the callback joins the task queue and runs when the loop reaches it — possibly much later. The delay is a **minimum**. Also:

- Negative delays become `0`.
- After 5 levels of nested timers, browsers clamp delays below 4 ms to **4 ms**.
- Background tabs throttle timers (often ≤ 1/s) to save battery.
- Delays above $2^{31}-1$ ms (~24.8 days) overflow; Node warns and uses 1 ms.

`code/04_timers.mjs` (excerpt):

```
import { setTimeout as sleep } from 'node:timers/promises';

// The delay is a MINIMUM: a busy main thread delays the callback
const t0 = performance.now();
setTimeout(() => console.log(`asked 10 ms, got >= 40 ms: ${performance.now() - t0 >= 40}`), 10);
while (performance.now() - t0 < 40) {}
await sleep(20);

// Too-large delays overflow: > 2^31-1 ms (~24.8 days) becomes 1 ms
const big = setTimeout(() => console.log('huge delay fired immediately!'), 2 ** 31);
await sleep(20);
clearTimeout(big);
```

Output (the `(node:…)` lines are Node's stderr warning):

```
asked 10 ms, got >= 40 ms: true
(node:51319) TimeoutOverflowWarning: 2147483648 does not fit into a 32-bit signed integer.
Timeout duration was set to 1.
(Use `node --trace-warnings ...` to show where the warning was created)
huge delay fired immediately!
```

The full file also shows extra args + `clearTimeout`, `setInterval`/`clearInterval` after 3 ticks, recursive `setTimeout`, and the promise-based `sleep` from `node:timers/promises`.

`setInterval` vs recursive `setTimeout`

	`setInterval(fn, 100)`	recursive `setTimeout(fn, 100)`
Spacing	Start-to-start; if `fn` takes 80 ms only ~20 ms idle	End-to-start; always ≥ 100 ms idle
Slow or async work	Runs can bunch up or overlap (an async `fn` isn't awaited)	Next run scheduled only after this one finishes
Varying delay (backoff)	Awkward	Easy

For polling a server, prefer recursive `setTimeout` scheduled after the previous request completes.

Common mistakes

- `setTimeout(save(), 1000)` runs `save` now. Pass a function: `setTimeout(save, 1000)` or `() => save()`.
- Forgetting to clear intervals (e.g. when a UI component unmounts) → memory leaks and duplicate work.
- Using timers to "wait until ready" — await the actual event or promise.
- `setTimeout(obj.method, 10)` loses `this` — use an arrow or `bind` (Chapter 9).

TL;DR

Delays are minimums; the callback runs as a task when the thread is free. Clear what you schedule. Recursive `setTimeout` for polling; `node:timers/promises` gives an awaitable `setTimeout`.

12.6 Callbacks and callback hell

What it is

A **callback** is a function passed to another function to be called later with the result — the oldest async style (event handlers, `setTimeout`, Node's original APIs). Node standardized **error-first callbacks**: `callback(err, result)`, `err` is `null` on success.

`code/05_callbacks.mjs` — key parts:

```
// 1) Error-first callbacks (the Node convention): cb(err, result)
readFile(new URL('./missing.txt', import.meta.url), 'utf8', (err, text) => {
  if (err) { console.log('error-first:', err.code); return; }
  console.log(text);
});

// 2) Callback hell: each step needs the previous result
db.getUser(1, (err, user) => {
  if (err) return console.log(err.message);
  db.getOrders(user, (err, orders) => {
    if (err) return console.log(err.message);
    db.getItems(orders[0], (err, items) => {
      if (err) return console.log(err.message);
      console.log('pyramid result:', user.name, orders[0].id, items);
    });
  });
});

// 3) Inversion of control: a buggy library calls your callback twice
function buggyCharge(amount, cb) { cb(null, 'ok'); cb(null, 'ok'); }
let charges = 0;
buggyCharge(10, () => { charges += 1; });
console.log('callback-style charges:', charges);

// A promise can settle only once, so the second call is ignored
const charged = new Promise((resolve) => buggyCharge(10, (_e, r) => resolve(r)));
let promiseCharges = 0;
await charged.then(() => { promiseCharges += 1; });
console.log('promise-style charges :', promiseCharges);

// 4) Zalgo: a callback that is sometimes sync, sometimes async
function getValue(key, cb) {
  if (cache.has(key)) cb(cache.get(key));          // sync!
  else setTimeout(() => cb(42), 0);                // async
}
// ...
```

Output:

```
callback-style charges: 2
promise-style charges : 1
error-first: ENOENT
pyramid result: Ada 7 [ 'book', 'pen' ]
zalgo order: cached cb | after call (a) | after call (b) | fetched cb
```

The problems callbacks have

Problem	What goes wrong	How promises fix it
Callback hell ("pyramid of doom")	Each dependent step nests deeper; error checks repeat	Flat `.then` chains or linear `await`
Error handling	`try/catch` can't catch errors in a later callback; every level checks `err`	One `.catch` / `try` for any step
Inversion of control	You trust other code to call you once — not zero times or twice	A promise settles exactly once
"Releasing Zalgo"	Sometimes-sync, sometimes-async callbacks make ordering unpredictable (the cached callback ran *before* "after call")	`then` callbacks are always async
Composition	"Run in parallel, wait for all" is hand-written counting	`Promise.all` and friends

Callbacks remain right for **repeated events** (clicks, stream data) — a promise delivers one value. Named functions and early `return`s flatten a pyramid a little; promises are the real fix.

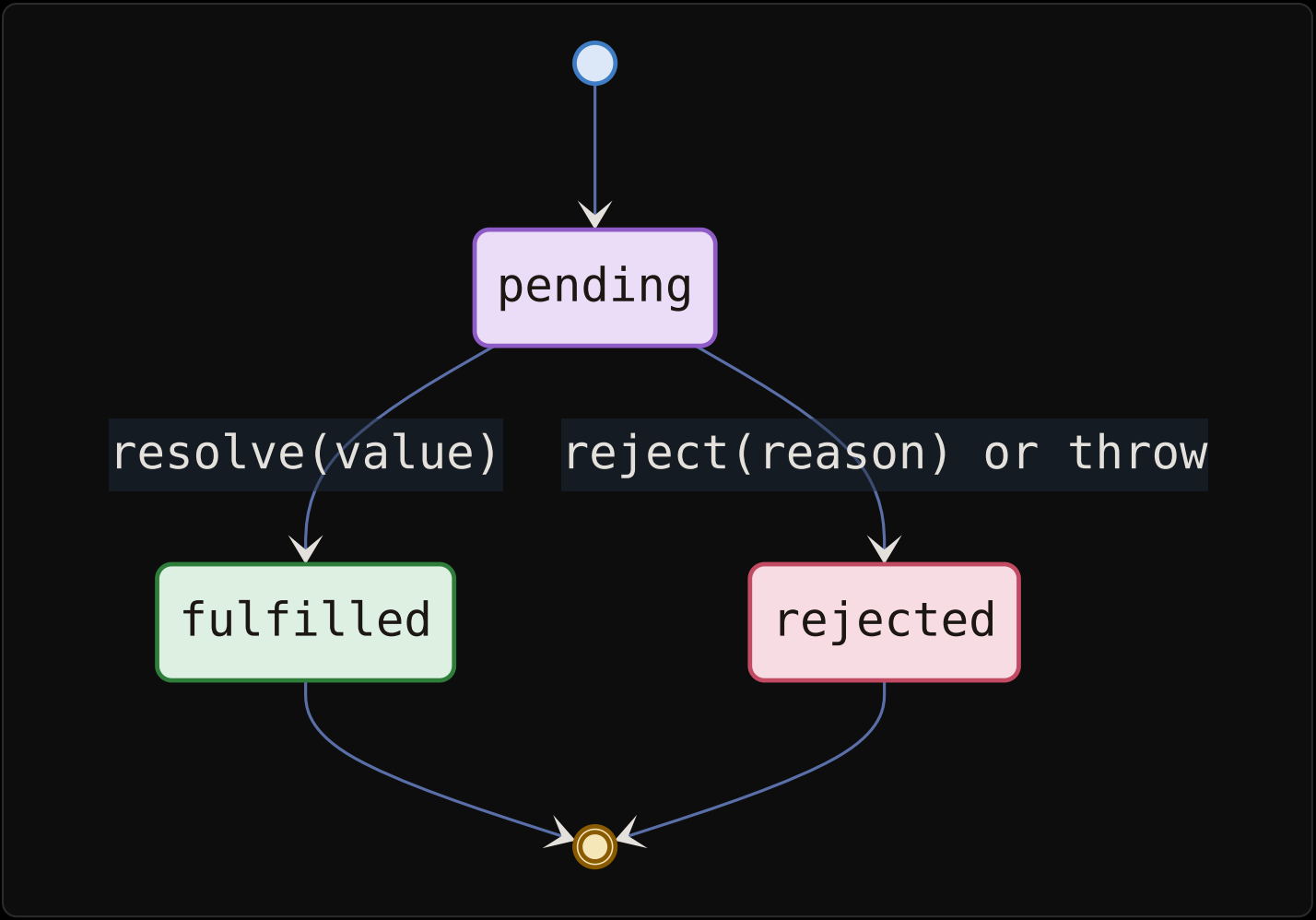
TL;DR

Callbacks: Node uses `cb(err, result)`. They nest badly, can't use `try/catch`, and require trusting the caller — promises solve all of these. Keep callbacks for multi-fire events.

12.7 Promises

What it is

A **promise** is an object representing a value available later — or an error explaining why not. It settles exactly once.



- **pending** — not finished. **fulfilled** — finished with a value. **rejected** — failed with a reason (normally an `Error`).
- **settled** — fulfilled or rejected; never changes again.
- **resolved** — "locked in": settled, or following another promise (a promise resolved with a pending promise is still pending).

How it works

1. `new Promise((resolve, reject) => { … })` — the **executor** runs immediately and synchronously; if it throws, the promise rejects.
2. `promise.then(onFulfilled, onRejected)` registers reactions and returns a **new** promise.
3. On settlement, reactions are queued as microtasks.
4. The new promise settles with what the reaction **returns** (value → fulfilled; promise → adopt its outcome) or **throws** (→ rejected).
5. A rejection skips `then(onFulfilled)` reactions until a `catch` (or `onRejected`); after `catch` returns normally the chain is fulfilled again.

6. `finally(fn)` calls `fn` with no argument and passes the original outcome through (unless `fn` throws or returns a rejected promise).

`code/06_promises.mjs`:

```
const later = (value, ms, fail = false) =>
  new Promise((resolve, reject) => setTimeout(() => (fail ? reject(new Error(value)) : resolve(value)), ms));

const p = later('done', 10);
console.log('right away :', p);      // Node shows the state
await p;
console.log('after await:', p);

// Chaining: each then() returns a NEW promise
const result = await Promise.resolve(2)
  .then((x) => x * 10)               // return a value -> next then gets it
  .then((x) => later(x + 1, 5))      // return a promise -> chain waits for it
  .then((x) => { throw new Error(`boom at ${x}`); })
  .then(() => console.log('skipped because the chain is rejected'))
  .catch((err) => { console.log('caught:', err.message); return 'recovered'; })
  .finally(() => console.log('finally: runs either way, sees no value'));
console.log('chain result:', result);

// Mistake: forgetting to return inside then() breaks the chain
await Promise.resolve(1)
  .then((x) => { later(x + 1, 5); })  // no return!
  .then((v) => console.log('forgot return ->', v));

// resolve/reject only count once
const once = new Promise((resolve, reject) => { resolve('first'); resolve('second'); reject(new Error('ignored')); });
console.log('settles once:', await once);

// Promise.resolve adopts thenables; a promise is passed through unchanged
const thenable = { then(onOk) { onOk('from a thenable'); } };
console.log('thenable   :', await Promise.resolve(thenable));
console.log('same obj   :', Promise.resolve(p) === p);

// ...promisify(readFile) — see the file

// Promise.withResolvers (ES2024): resolve from outside the executor
const { promise, resolve } = Promise.withResolvers();
setTimeout(() => resolve('resolved from outside'), 5);
console.log('withResolvers:', await promise);

// Promise.try (ES2025): run a function that may throw synchronously OR return a promise
const risky = (x) => { if (x < 0) throw new RangeError('negative'); return later(x * 2, 1); };
for (const x of [3, -1]) {
  await Promise.try(risky, x).then((v) => console.log('Promise.try ok  :', v), (e) => console.log('Promise.try fail:', e.message));
}
```

Output:

```
right away : Promise { <pending> }
after await: Promise { 'done' }
caught: boom at 21
finally: runs either way, sees no value
chain result: recovered
forgot return -> undefined
settles once: first
thenable   : from a thenable
same obj   : true
withResolvers: resolved from outside
Promise.try ok  : 6
Promise.try fail: negative
```

Key API

API	Meaning
`new Promise(executor)`	Wrap a callback API. Wrapping something that already returns a promise is the **explicit-construction anti-pattern**.
`Promise.resolve(v)` / `Promise.reject(e)`	Already-settled promise (`resolve` adopts thenables; returns `v` itself if it's a native promise)
`.then(ok, fail)` / `.catch(fail)` / `.finally(fn)`	Reactions; each returns a new promise
`Promise.withResolvers()` (ES2024)	`{ promise, resolve, reject }` — when resolve is triggered elsewhere
`Promise.try(fn, ...args)` (ES2025)	Call `fn`; a sync throw becomes a rejection, a value/promise is adopted
`util.promisify(fn)` (Node)	Error-first callback function → promise-returning. Most Node APIs also have promise versions: `node:fs/promises`, `node:timers/promises`.

A **thenable** is any object with a `then` method; promise libraries interoperate through this duck-typed protocol (Chapter 9 — duck typing).

Common mistakes

- **Forgetting** `return` in a `then` callback → `undefined`, and the inner promise's errors become unhandled.
- **Nesting** `then` inside `then` instead of returning — callback hell again.
- `.then(ok, fail)` **vs** `.then(ok).catch(fail)`**:** the first does **not** catch errors thrown inside `ok`.
- **No final** `catch` → unhandled rejection (§11.5).
- **Thinking promises are cancellable or lazy.** A promise represents work *already started*; cancel the work with `AbortController` (§12.11).

Self-check

1. `.then(() => 5)` returns? → A new promise fulfilled with 5.
2. `Promise.resolve().then(() => { throw new Error('x') }).then(() => 1).catch(e => 2)` fulfils with? → `2`.
3. What does `finally`'s callback receive? → Nothing.

TL;DR

A one-time, eventually-settled value: pending → fulfilled/rejected. `then` returns a new promise settled by the callback's return value or thrown error. Always return inside `then`; end chains with `catch`.

12.8 Promise combinators: `all`, `allSettled`, `race`, `any`

What it is

Static methods that take an iterable of promises (or plain values) and return one promise.

Method	Fulfils when	Rejects when	Result	Added
`Promise.all`	**all** fulfil	**first** rejection (fail-fast)	values in **input order**	ES2015
`Promise.allSettled`	all settle	never	`{status, value}` / `{status, reason}`	ES2020
`Promise.race`	first to **settle** is fulfilled	first to settle is rejected	that value/reason	ES2015
`Promise.any`	first to **fulfil**	**all** reject	value, or `AggregateError` with `.errors`	ES2021

`code/07_combinators.mjs`:

```
console.log('all       :', await Promise.all([later('slow', 30), later('fast', 10), 42]), at());
t0 = performance.now();
await Promise.all([later('ok', 30), later('db down', 10, true)])
  .catch((e) => console.log('all fails :', e.message, 'after', at()));

const settled = await Promise.allSettled([later('ok', 5), later('bad', 5, true)]);
console.log('allSettled:', settled.map((r) => (r.status === 'fulfilled' ? `ok:${r.value}` :
`err:${r.reason.message}`)));

console.log('race      :', await Promise.race([later('A', 20), later('B', 10)]));
await Promise.race([later('A', 20), later('B failed', 10, true)]).catch((e) => console.log('race err  :',
e.message));

console.log('any       :', await Promise.any([later('mirror1 down', 5, true), later('mirror2', 15)]));
await Promise.any([later('x', 5, true), later('y', 5, true)])
  .catch((e) => console.log('any fails :', e.name, e.errors.map((x) => x.message)));

console.log('empty     :', await Promise.all([]), (await Promise.allSettled([])).length);
await Promise.any([]).catch((e) => console.log('any([])   :', e.name));
console.log('race([]) stays pending forever - never await it');

const withTimeout = (promise, ms) =>
  Promise.race([promise, new Promise((_, rej) => setTimeout(() => rej(new Error(`timed out after ${ms} ms`)),
ms))]);
await withTimeout(later('slow api', 100), 20).catch((e) => console.log('timeout   :', e.message));
```

Output:

```
all       : [ 'slow', 'fast', 42 ] 30 ms
all fails : db down after 10 ms
allSettled: [ 'ok:ok', 'err:bad' ]
race      : B
race err  : B failed
any       : mirror2
any fails : AggregateError [ 'x', 'y' ]
empty     : [] 0
any([])   : AggregateError
race([]) stays pending forever - never await it
timeout   : timed out after 20 ms
```

- `all` took 30 ms (the slowest), not 40 — the waits overlapped. Results are in input order though `fast` finished first; plain values are allowed.
- On failure `all` rejects at 10 ms but the other operation **keeps running** — combinators cancel nothing; pair with an `AbortSignal` (§12.11).
- `withTimeout` via `race` is a classic interview answer; in real code `AbortSignal.timeout(ms)` also stops the underlying work.

Choosing

Situation	Use
Page needs user, settings and feed	`all`
Send 20 notifications; report which failed	`allSettled`
Fastest of several mirrors / CDNs (content delivery networks)	`any`
Result or timeout, whichever first	`race` (or `AbortSignal.timeout`)

In the interview

30-second answer: "`all` waits for everything and fails fast on the first rejection; `allSettled` waits for everything and never rejects; `race` settles with the first promise to settle, success or

failure; `any` takes the first success and only rejects — with an `AggregateError` — if all fail. None of them cancel the losers."

- *Q: Implement `Promise.all`.* A: New promise; results array + counter; for each input `Promise.resolve(item).then(v => { results[i] = v; if (++done === n) resolve(results) }, reject)`; resolve immediately for empty input. Runnable polyfills: Chapter 17.
- *Q: One rejection in `Promise.all` — do the others stop?* A: No; their results are ignored.

TL;DR

`all` (all or first failure), `allSettled` (everything, never rejects), `race` (first settled), `any` (first success, else `AggregateError`). None cancel.

12.9 `async` / `await`

What it is

Syntax (ES2017) for writing promise code as if it were synchronous.

- An `async` **`function`** always returns a promise: `return v` fulfils it; `throw` rejects it.
- `await expr` suspends the function until the promise settles, then yields its value — or **`throws`** its reason. The thread isn't blocked: the rest resumes later as a microtask while the caller continues.
- `await` is allowed only in async functions and at the **`top level of modules`**.

Under the hood an async function is a generator yielding promises, driven by a runner that resumes it as each settles.

`code/08_async_await.mjs`:

```
// (1: answer() instanceof Promise -> true; a throwing async fn -> rejected promise)
// 2) Sequential vs parallel
let t = performance.now();
const a = await later('a', 30);
const b = await later('b', 30);
console.log('sequential:', a + b, round(performance.now() - t), 'ms');

t = performance.now();
const [c, d] = await Promise.all([later('c', 30), later('d', 30)]);
console.log('parallel  :', c + d, round(performance.now() - t), 'ms');

// 3) try/catch/finally with await
async function load(id) {
  try {
    return await later(`user ${id}`, 5, id < 0);   // "return await" keeps the error inside this try
  } catch (err) {
    return `fallback for ${err.message}`;
  } finally {
    console.log('  cleanup for', id);
  }
}
console.log('load(1)   :', await load(1));
console.log('load(-1)  :', await load(-1));

// 4) await in loops: for...of is sequential; forEach does NOT wait
const ids = [1, 2, 3];
const seen = [];
ids.forEach(async (id) => { await later(id, 12 - id * 3); seen.push(id); });
console.log('forEach done? seen =', seen, '(nothing yet!)');
await later(null, 20);
console.log('...later   seen =', seen, '(finish order, not input order)');
const inOrder = [];
for (const id of ids) inOrder.push(await later(id * 10, 2));
console.log('for...of  :', inOrder);
console.log('map+all   :', await Promise.all(ids.map((id) => later(id * 100, 5))));
// (5: async arrows and async methods work too; `await 'plain value'` just gives the value)
```

Output:

```
sequential: ab 60 ms
parallel  : cd 30 ms
  cleanup for 1
load(1)   : user 1
  cleanup for -1
load(-1)  : fallback for user -1
forEach done? seen = [] (nothing yet!)
...later   seen = [ 3, 2, 1 ] (finish order, not input order)
for...of  : [ 10, 20, 30 ]
map+all   : [ 100, 200, 300 ]
```

Sequential vs parallel — the most common performance bug

Two independent `await`s in a row take the **sum** (60 ms); `Promise.all` takes the **max** (30 ms). `await` one after another only when step 2 needs step 1's result.

```
// Also parallel: start first, await later
const userP = getUser(id), postsP = getPosts(id);   // both requests in flight
const user = await userP, posts = await postsP;    // careful: if postsP rejects before
                                                    // userP settles, it's briefly "unhandled"
```

`Promise.all` is safer: it attaches handlers to every promise immediately.

Loops

Loop	Behaviour
`for…of` / `for` / `while` with `await` inside	One at a time, in order
`array.forEach(async …)`	Fires all, **doesn't wait**, ignores results and errors — avoid
`await Promise.all(array.map(async …))`	All at once, results in order
Concurrency-limited pool (§12.13)	At most N at a time

Error handling

- Wrap `await` in `try` / `catch`; the rejection reason is the caught value.
- `return await p` inside `try`: without `await` the promise is handed back *before* it rejects, so `catch` never sees it and `finally` runs while work is in progress. Outside `try`, `return p` ≡ `return await p`.
- Catch where you can act (message, retry, fallback); let the rest propagate (custom errors and `cause`).

Trade-offs: `async/await` vs `.then` chains

	`async` / `await`	`.then` chains
Readability	Reads like sync code; normal `if`, loops, `try`	Fine for short linear pipelines
Errors	`try` / `catch`	`.catch`
Where it works	Async functions and module top level	Anywhere

Same machinery (both still need `Promise.all` for parallelism); mix freely: `await fetch(url).then(r => r.json())`.

In the interview

30-second answer: "`async` functions always return promises; `await` suspends just that function until the promise settles, resuming it as a microtask, so the thread stays free. Rejections become exceptions, so `try/catch` works. The classic mistake is awaiting independent calls one by one — use `Promise.all` — and using `forEach` with an async callback, which doesn't wait."

- *Q: `async function f() {}; f()` returns?* A: A promise fulfilled with `undefined`.
- *Q: Code after `await` of an already-resolved promise — synchronous?* A: No, always a microtask.
- *Q: Why `return await` inside `try`?* A: So that `try`'s `catch` sees the rejection.
- *Q: `await` in a regular function?* A: SyntaxError.
- *Q: Three independent 100 ms requests awaited in sequence?* A: ~300 ms; with `Promise.all` ~100 ms.

TL;DR

`async` → returns a promise; `await` → unwraps or throws. Independent work → `Promise.all`. Sequential loop → `for…of`; never `forEach(async)`. `try { return await … } catch`.

12.10 Error handling across async code — a checklist

§11.5 gave the rules; with the event loop in hand, here's where each error goes.

Situation	Where the error goes	Handle with
`throw` inside an `async` function	Rejects its returned promise	`try/catch` around `await f()`, or `f().catch()`
`throw` inside a `then` callback	Rejects the promise `then` returned	A later `.catch`
`throw` inside a `setTimeout` / event callback	Uncaught in *that task* — no enclosing `try` exists any more	`try/catch` **inside** the callback; global `error` handler for logging
Rejection nobody handles	`unhandledrejection` (browser) / crash (Node ≥ 15)	Always `await` / `return` / `.catch`
One of many parallel operations fails	`Promise.all` rejects with the first; others ignored	`allSettled` for every outcome
Error in an error-first callback	First argument	`if (err) …` first

Rules of thumb:

1. Every promise has an owner who awaits it, returns it, or attaches `.catch`.
2. Catch where you can **act**; otherwise propagate, adding context with `new Error(msg, { cause })`.
3. Promisify callback APIs inside async functions, or their errors escape your `try`.
4. Global handlers (`window.onunhandledrejection`, `process.on('unhandledRejection')`) are for logging and reporting, not recovery.

TL;DR

try/catch works across `await` because rejections are re-thrown on resumption. Errors in plain callbacks belong to their own task. Leave no promise floating.

12.11 Cancellation: `AbortController` and `AbortSignal`

What it is

Promises can't be cancelled, but the *work* behind them can. The standard mechanism (DOM spec; also in Node):

- **`AbortController`** — held by whoever decides to cancel; `controller.abort(reason?)`.
- **`AbortSignal`** — `controller.signal`, handed to the operation; has `.aborted`, `.reason`, an `abort` event and `throwIfAborted()`.

API	Purpose	Support
`new AbortController()` / `.abort(reason)`	Manual cancel	All modern browsers, Node ≥ 15
`AbortSignal.timeout(ms)`	Aborts with a `TimeoutError` `DOMException` after `ms`	Baseline since 2022 (Node ≥ 17.3)
`AbortSignal.any([s1, s2])`	Aborts when any input aborts	Baseline since 2024 (Node ≥ 20.3)
`AbortSignal.abort(reason)`	Already-aborted signal	Modern browsers, Node
`{ signal }` option	Accepted by `fetch`, `addEventListener`, Node `timers/promises`, `fs.readFile`, streams, `events.on` …	—

`code/09_abort.mjs` (excerpt):

```
// 1) Making your own function cancellable with an AbortSignal
function delay(ms, { signal } = {}) {
  return new Promise((resolve, reject) => {
    signal?.throwIfAborted();                       // already aborted? fail fast
    const id = setTimeout(() => { signal?.removeEventListener('abort', onAbort); resolve(`waited ${ms} ms`); },
ms);
    function onAbort() { clearTimeout(id); reject(signal.reason); }
    signal?.addEventListener('abort', onAbort, { once: true });
  });
}

const controller = new AbortController();
setTimeout(() => controller.abort(), 20);
try {
  await delay(1000, { signal: controller.signal });
} catch (err) {
  console.log('manual abort  :', err.name, '-', err.message);
}
console.log('signal state  :', controller.signal.aborted);

// 2) Custom reason
const c2 = new AbortController();
c2.abort(new Error('user left the page'));
await delay(10, { signal: c2.signal }).catch((e) => console.log('custom reason :', e.message));

// 3) AbortSignal.timeout(ms) - aborts with a TimeoutError
await delay(1000, { signal: AbortSignal.timeout(15) }).catch((e) => console.log('timeout       :', e.name));

// 4) AbortSignal.any - abort if ANY of several signals aborts
const userCancel = new AbortController();
const combined = AbortSignal.any([userCancel.signal, AbortSignal.timeout(5000)]);
setTimeout(() => userCancel.abort(), 10);
await delay(1000, { signal: combined }).catch((e) => console.log('any()         :', e.name));

// 5) One abort() removes a listener added with { signal }
const target = new EventTarget();
const listeners = new AbortController();
target.addEventListener('ping', () => console.log('ping heard'), { signal: listeners.signal });
target.dispatchEvent(new Event('ping'));
listeners.abort();                                  // removes the listener
target.dispatchEvent(new Event('ping'));
console.log('listener removed by abort()');
```

Output:

```
manual abort  : AbortError - This operation was aborted
signal state  : true
custom reason : user left the page
timeout       : TimeoutError
any()         : AbortError
ping heard
listener removed by abort()
```

(The file also aborts Node's promise `sleep` from `timers/promises`, which rejects with `AbortError`.)

How to write abortable functions

1. Accept `{ signal }` in an options object.
2. `signal?.throwIfAborted()` at the start (and between steps of long work).
3. Listen for `abort` once; on abort, stop the work and reject with `signal.reason`.
4. Remove the listener on normal completion (avoid leaks on long-lived signals).
5. Pass the signal to every API you call (`fetch(url, { signal })`).

Callers tell a deliberate cancel from a failure by `err.name === 'AbortError'` (or `'TimeoutError'`) and usually ignore it. Fetch with abort/timeouts: Chapter 14.

Real-world examples

- Search-as-you-type: abort the previous request on each keystroke.
- React effects: abort the fetch in the effect's cleanup when the component unmounts.
- Removing a group of listeners: one controller, `{ signal }` on each `addEventListener`, one `abort()`.

TL;DR

`AbortController` cancels; `AbortSignal` carries the cancel to the work. `AbortSignal.timeout` / `AbortSignal.any` cover timeouts and combined cancel. Rejections are named `AbortError` / `TimeoutError`.

12.12 Async iteration: `for await…of` and async generators

What it is

The **async iterable protocol** (ES2018) is Chapter 8's iterable protocol (`[Symbol.iterator]()` → iterator whose `next()` returns `{ value, done }`) with `[Symbol.asyncIterator]()` and a `next()` that returns a **promise** of `{ value, done }`. Consume with **`for await (const x of source)`**; produce with an **async generator** (`async function*`), which can both `await` and `yield`. Use it for data arriving in pieces: paginated APIs, file/network streams (Node streams and web `ReadableStream`s are async iterable), event streams.

`code/10_async_iteration.mjs` (excerpt):

```
// Paginated API -> one flat async stream
const pages = { 1: { items: ['a', 'b'], next: 2 }, 2: { items: ['c'], next: 3 }, 3: { items: ['d', 'e'], next:
null } };
const fetchPage = async (n) => { await null; return pages[n]; };
async function* allItems() {
  let page = 1;
  while (page) {
    const { items, next } = await fetchPage(page);
    yield* items;                  // delegate to the array's iterator
    page = next;
  }
}
const collected = [];
for await (const item of allItems()) {
  collected.push(item);
  if (collected.length === 4) break;   // break calls return() -> generator stops fetching
}
console.log('first 4 items :', collected);

// Array.fromAsync (ES2026) collects an async iterable into an array
console.log('fromAsync     :', await Array.fromAsync(allItems()));

// Events as an async stream (Node events.on)
const bus = new EventEmitter();
const ac = new AbortController();
setTimeout(() => { bus.emit('msg', 'hello'); bus.emit('msg', 'world'); ac.abort(); }, 5);
try {
  for await (const [text] of on(bus, 'msg', { signal: ac.signal })) console.log('event:', text);
} catch (e) {
  console.log('stream ended  :', e.name);
}
```

Output:

```
first 4 items : [ 'a', 'b', 'c', 'd' ]
fromAsync     : [ 'a', 'b', 'c', 'd', 'e' ]
event: hello
event: world
stream ended  : AbortError
```

The file also has a timed `countdown` generator, `for await` over an array of promises, and a hand-written `[Symbol.asyncIterator]` object.

Notes

- `for await` is allowed wherever `await` is.
- `break`, `return` or a throw inside the loop calls the iterator's `return()`, so an async generator's `finally` blocks run — closes connections.
- Items are processed **one at a time**. For independent promises `Promise.all` is better; `for await` over such an array can't report a rejection until the loop reaches it.
- `Array.fromAsync` is ES2026, in Node 22+ and current browsers.

> **TL;DR**
>
> Async iterables deliver `{value, done}` promises; `for await…of` consumes; `async function*` produces. Great for pagination and streams; `break` cleans up.

12.13 Real-world async patterns

`code/11_patterns.mjs` implements three helpers found in almost every production codebase.

```
// 1) Retry with exponential backoff
async function retry(fn, { tries = 4, base = 5 } = {}) {
  for (let attempt = 1; ; attempt++) {
    try {
      return await fn(attempt);
    } catch (err) {
      if (attempt >= tries) throw new Error(`gave up after ${attempt} tries`, { cause: err });
      const wait = base * 2 ** (attempt - 1);   // 5, 10, 20 ... (add random jitter in production)
      console.log(`  attempt ${attempt} failed (${err.message}); retrying in ${wait} ms`);
      await sleep(wait);
    }
  }
}

// 2) Run many tasks but at most `limit` at once
async function mapLimit(items, limit, fn) {
  const results = new Array(items.length);
  let next = 0, running = 0, peak = 0;
  async function worker() {
    while (next < items.length) {
      const i = next++;                       // safe: no other code runs between read and write
      running++; peak = Math.max(peak, running);
      results[i] = await fn(items[i]);
      running--;
    }
  }
  await Promise.all(Array.from({ length: limit }, worker));
  return { results, peak };
}

// 3) Race condition: an older, slower response arrives last and overwrites newer data
let latest = 0;
await Promise.all(['ja', 'jav'].map(async (q) => {
  const ticket = ++latest;
  const r = await search(q);
  if (ticket === latest) shown = r;               // ignore answers to old requests
}));
```

Output:

```
  attempt 1 failed (503); retrying in 5 ms
  attempt 2 failed (503); retrying in 10 ms
retry result: payload
mapLimit    : [
   1,  4,  9, 16,
  25, 36, 49
] peak concurrency = 3
naive shows : results for "ja" <- stale!
guarded     : results for "jav"
```

Pattern	Problem it solves	Production notes
Retry + exponential backoff (double the wait each failure)	Transient failures (HTTP 503, network blips)	Random **jitter** so clients don't retry in sync; retry only **idempotent** operations (safe to repeat — System Design Fundamentals); cap attempts and total time; respect `Retry-After`
Concurrency limit	10,000 requests at once overload the server or hit rate limits / file-descriptor limits	`p-limit`, `p-map`; the worker pool above suffices for scripts
Last-request-wins	Out-of-order responses (search-as-you-type) — a **race condition** where stale data overwrites fresh	Better: abort the old request (§12.11); React Query and similar do this
Timeout	A request that never answers	`AbortSignal.timeout(ms)`
Debounce / throttle (wait until events stop / at most once per interval)	Too many events (keystrokes, scroll)	Implemented in Chapter 17

`mapLimit`'s `next++` needs no lock: run-to-completion guarantees no other worker runs between the read and the increment.

> **TL;DR**
>
> Retry with backoff + jitter for transient errors; cap concurrency with a worker pool; guard stale responses with request IDs or abort; time out everything that talks to the network.

Chapter 12 TL;DR

- **One thread**, run-to-completion; the host does the waiting. Long sync work blocks timers, input and rendering.
- **Event loop:** one task → drain **all** microtasks → maybe render (rAF → style → layout → paint) → repeat. Promise reactions, code after `await`, `queueMicrotask` = microtasks; timers, I/O, events = tasks.
- **Puzzles:** sync first (incl. executors and code before the first `await`), then microtasks FIFO, then tasks. Returning a promise from `then` costs 2 extra ticks.
- **Node:** timers → poll → check (`setImmediate`) → close; `nextTick` before promises (except at ESM top level); `setImmediate` vs `setTimeout(0)` ordered only inside I/O callbacks.
- **Timers** are minimums; nested clamp 4 ms; overflow above $2^{31}-1$ ms; recursive `setTimeout` for polling.
- **Callbacks** → callback hell, no `try/catch`, inversion of control, Zalgo. Keep for repeated events.
- **Promises:** settle once; `then` returns a new promise; always `return`, end with `catch`. `withResolvers` (ES2024), `Promise.try` (ES2025).
- **Combinators:** `all`, `allSettled`, `race`, `any` — none cancel.
- **async/await:** returns a promise; `await` throws rejections; `Promise.all` for independent work; no `forEach(async)`; `return await` inside `try`.
- **Cancel** with `AbortController` / `AbortSignal.timeout` / `AbortSignal.any`.
- **Async iteration:** `for await…of`, `async function*`, `Array.fromAsync` (ES2026).
- **Patterns:** retry with backoff + jitter, concurrency limits, last-request-wins.

Next: the page itself — Chapter 13: DOM and Events.

Chapter 13 — The DOM and Events

To JavaScript, a web page is a tree of objects called the **DOM**. Every interactive feature — a dropdown opening, a todo ticked off, a form rejecting an invalid email — is JavaScript finding nodes in that tree, changing them, and reacting to **events** such as clicks and key presses. This chapter covers the DOM API (the objects and functions for reading and changing the page), the full event system including the capture/bubble path, forms, and the performance rules that keep pages smooth.

What you'll learn

1. The DOM: nodes, elements, the tree, and how HTML becomes a DOM.
2. Selecting elements and walking the tree.
3. Creating, inserting, moving and removing nodes; `textContent` vs `innerHTML`; templates and fragments.
4. Attributes vs properties; `classList`, `dataset`, styles.
5. Events: listeners, the event object, listener options, custom events.
6. Propagation: capture, target and bubble phases; stopping it.
7. Event delegation — one listener for many elements.
8. Default actions, `preventDefault`, passive listeners.
9. Forms: `submit`, `FormData`, the Constraint Validation API, `input` vs `change`.
10. `MutationObserver`.
11. Rendering performance: reflow, repaint, layout thrashing (measured in Chrome).
12. When scripts run: `DOMContentLoaded`, `load`, `defer`.

Before you start. Closures (Chapter 4), `this` in handlers (Chapter 9 §9.1); listeners run as tasks on the event loop (Chapter 12). HTML and CSS — semantic elements, forms markup, selectors (patterns like `.title` or `#app`) — are in the HTML & CSS Handbook (`../../../html-css/`).

How the code runs. Node has no DOM, so the `.mjs` programs in `code/` start with `import '../../../../tools_dom.mjs';`, which creates a **jsdom** document (a JavaScript implementation of the DOM standard used by test tools) and exposes `document`, `window` etc. as globals. jsdom implements DOM and events faithfully but has **no layout engine** (no sizes, no painting); for those parts, small `.html` pages were run in headless Chrome and screenshotted after a scripted interaction. All outputs are real.

13.1 The DOM tree

What it is

The **Document Object Model (DOM)** is the browser's live, in-memory representation of a page: a **tree** of **nodes** with an API (WHATWG — the browser makers' standards group — DOM and HTML standards) to read and change it. Change the tree and the screen updates. "View Source" shows the original HTML; DevTools' Elements panel shows the current DOM.

- **Node** — any item in the tree. Types: **element** (`<li>`, nodeType 1), **text** (3), **comment** (8), **document** (9), **document fragment** (11).
- **Element** — a node created from a tag; has attributes and can have children.
- **Parent / child / sibling / ancestor / descendant** — tree relations.
- `document` — the root; `document.documentElement` is `<html>`, plus `document.head` and `document.body`.

How it works

1. The HTML **parser** builds the tree, fixing errors in standard ways (unclosed tags, missing `<tbody>`).
2. Whitespace between tags becomes **text nodes** — present though invisible.
3. Scripts change the tree through the DOM API; changes are live everywhere.

`code/01_dom_tree.mjs` — key parts:

```
document.body.innerHTML = `
<main id="app">
  <h1 class="title">Groceries</h1>
  <!-- a comment node -->
  <ul id="list">
    <li data-id="1">Milk</li>
    <li data-id="2" class="done">Bread</li>
  </ul>
</main>`;

function show(node, depth = 0) {           // recursive walk over childNodes
  // prints #text / #comment / <tag> ELEMENT with indentation
  for (const child of node.childNodes) show(child, depth + 1);
}
show(document.querySelector('#app'));

const main = document.getElementById('app');
console.log('\nchildNodes (all nodes):', main.childNodes.length, '| children (elements only):',
main.children.length);
// ...
const items = document.getElementById('list').children;   // live HTMLCollection
console.log('items before:', items.length);
document.getElementById('list').insertAdjacentHTML('beforeend', '<li data-id="3">Eggs</li>');
console.log('items after :', items.length);

const parsed = new DOMParser().parseFromString('<table><tr><td>cell</table><p>unclosed', 'text/html');
console.log('\nparser auto-inserted <tbody>:', parsed.querySelector('table > tbody > tr') !== null);
console.log('parsed body:', parsed.body.innerHTML);
```

Output:

```
<main> ELEMENT
  #text (whitespace)
  <h1> ELEMENT
    #text "Groceries"
  #text (whitespace)
  #comment "a comment node"
  #text (whitespace)
  <ul> ELEMENT
    #text (whitespace)
    <li> ELEMENT
      #text "Milk"
    #text (whitespace)
    <li> ELEMENT
      #text "Bread"
    #text (whitespace)
  #text (whitespace)

childNodes (all nodes): 7 | children (elements only): 2
document.documentElement: HTML | head: HEAD | body: BODY
nodeType of document: 9 === Node.DOCUMENT_NODE: true
items before: 2
items after : 3

parser auto-inserted <tbody>: true
parsed body: <table><tbody><tr><td>cell</td></tr></tbody></table><p>unclosed</p>
```

Common mistakes

- `childNodes` / `firstChild` / `nextSibling` return whitespace text nodes. Prefer **element-only** versions: `children`, `firstElementChild`, `nextElementSibling`.
- Assuming the DOM equals the source — the parser inserts, moves and closes elements.
- A `<head>` script querying body elements before they're parsed (§13.12).

TL;DR

The DOM is a live tree of nodes (elements, text, comments) built by the parser. Use element-only navigation to skip whitespace text nodes.

13.2 Selecting and traversing

What it is

Method	Returns	Notes
`document.getElementById(id)`	Element or `null`	Fastest; id without `#`
`el.querySelector(css)`	First match or `null`	Any CSS selector; searches descendants of `el`
`el.querySelectorAll(css)`	**Static** `NodeList` (snapshot)	Has `forEach`; spread to an array for `map`/`filter`
`el.getElementsByClassName(c)` / `getElementsByTagName(t)`	**Live** `HTMLCollection`	Updates automatically; no `forEach`
`document.forms.name.elements.field`	Form controls by `name`	§13.9
`el.closest(css)`	Nearest ancestor **or itself** matching, or `null`	Key to event delegation
`el.matches(css)`	`true`/`false`	"Does this element match?"
`parent.contains(node)`	`true`/`false`	Includes `parent` itself

Traversal: `parentElement`, `children`, `firstElementChild`, `lastElementChild`, `previousElementSibling`, `nextElementSibling` (node-level: `parentNode`, `childNodes`, `firstChild`, `nextSibling`…).

`code/02_select_traverse.mjs` — key parts:

```
console.log('querySelector       :', document.querySelector('.links a').textContent);        // first match
console.log('querySelectorAll    :', [...document.querySelectorAll('.links a')].map((a) => a.textContent));
console.log('attribute selector  :', document.querySelector('[data-track]').textContent);
console.log(':not + :nth-child   :', [...document.querySelectorAll('li:not(:first-child) a')].map((a) =>
a.textContent));
console.log('no match           :', document.querySelector('.nope'), document.querySelectorAll('.nope').length);

const staticList = document.querySelectorAll('li');          // snapshot
const liveList = document.getElementsByTagName('li');         // live
document.querySelector('ul').append(document.createElement('li'));
console.log('\nafter adding an <li>: static', staticList.length, '| live', liveList.length);

const blog = document.querySelector('a[href="/blog"]');
console.log('closest("nav")      :', blog.closest('nav').id);
console.log('matches(":not(.x)") :', blog.matches('a:not(.active)'));
const li = blog.parentElement;
console.log('nextSibling (a node):', JSON.stringify(li.nextSibling.textContent), '<- whitespace text node');

try { document.querySelector('a[href=/]'); } catch (e) { console.log('\nbad selector        :', e.name); }
try { document.querySelector('#123'); } catch (e) { console.log('id starting w/ digit:', e.name, '-> use
getElementById("123") or CSS.escape'); }
```

Output (lines for the code shown):

```
querySelector       : Home
querySelectorAll    : [ 'Home', 'Blog', 'About' ]
attribute selector  : About
:not + :nth-child   : [ 'Blog', 'About' ]
no match            : null 0

after adding an <li>: static 3 | live 4
closest("nav")      : menu
matches(":not(.x)") : true
nextSibling (a node): "\n    " <- whitespace text node

bad selector        : SyntaxError
id starting w/ digit: SyntaxError -> use getElementById("123") or CSS.escape
```

Common mistakes

- `document.querySelector('.item').classList.add(…)` with no match → `TypeError: Cannot read properties of null`. Check for `null`, or use `?.` when absence is fine.
- `.map` on a `NodeList` — it has `forEach` only; use `[...list]` or `Array.from(list)`.
- Removing items while looping a **live** collection — indexes shift, items are skipped. Loop a static copy.
- Unquoted attribute values containing `/` or ids starting with digits → `SyntaxError`. Quote: `a[href="/"]`.

Self-check

1. Which returns a live collection? → `getElementsBy…`; `querySelectorAll` is static.
2. `el.closest('li')` when `el` is an `<li>`? → `el`.

TL;DR

`querySelector(All)` for almost everything; `getElementById` for ids; `closest` / `matches` / `contains` for relationships. `querySelectorAll` = static snapshot; `getElementsBy*` = live.

13.3 Creating, inserting and removing nodes

What it is

Task	API
Create	`document.createElement('li')`, `document.createTextNode('hi')`, `document.createDocumentFragment()`
Insert inside	`parent.append(...nodesOrStrings)`, `parent.prepend(...)`, `parent.replaceChildren(...)` (also empties)
Insert beside	`el.before(...)`, `el.after(...)`
Remove / replace	`el.remove()`, `el.replaceWith(...)`
Insert HTML string	`el.insertAdjacentHTML('beforebegin' / 'afterbegin' / 'beforeend' / 'afterend', html)`
Copy	`el.cloneNode(true)` (deep = with descendants)
Text	`el.textContent = '...'` (safe, never parsed)
Markup	`el.innerHTML = '...'` / `el.outerHTML` (parsed as HTML)

Legacy equivalents: `appendChild`, `insertBefore`, `removeChild`, `replaceChild` (nodes only, no strings).

`code/03_create_modify.mjs` — key parts:

```
const li = document.createElement('li');
li.textContent = 'appended';
list.append(li, 'a text node too');          // append accepts nodes AND strings
// ... prepend, after
list.lastChild.remove();                       // the text node
list.querySelector('li:nth-child(3)').replaceWith(/* new li "replaced" */);
document.getElementById('out').append(list.firstElementChild);   // appending an existing node MOVES it

const userInput = '<img src=x onerror="alert(1)">';
box.textContent = userInput;                   // shown as text, safe
console.log('3) textContent ->', box.innerHTML);
box.innerHTML = '<b>bold</b> and <i>italic</i>'; // parsed as HTML (only for trusted strings!)

p.insertAdjacentHTML('beforebegin', '<span>bb</span>');  // + afterbegin, beforeend, afterend

const frag = document.createDocumentFragment();
for (const name of ['Ada', 'Grace', 'Linus']) {
  const item = document.createElement('li');
  item.textContent = name;
  frag.append(item);
}
list.replaceChildren(frag);                    // one insertion

const tpl = document.getElementById('card');    // <template id="card"><article>...</article></template>
for (const pr of products) {
  const node = tpl.content.cloneNode(true);     // deep copy of the template's content
  node.querySelector('h2').textContent = pr.name;
  node.querySelector('.price').textContent = `$${pr.price}`;
  grid.append(node);
}
```

Output:

```
1) <li>prepended</li><li>after first</li><li>first</li><li>appended</li>a text node too
2) <li>after first</li><li>replaced</li><li>appended</li> | out: <li>prepended</li>
3) textContent -> &lt;img src=x onerror="alert(1)"&gt;
   innerHTML   -> 2 elements; textContent = "bold and italic"
   outerHTML   -> <div id="out"><b>bold</b> and <i>italic</i></div>
4) <span>bb</span><p><span>ab</span>P<span>be</span></p><span>ae</span>
5) <li>Ada</li><li>Grace</li><li>Linus</li> | fragment is now empty: 0
6) <article class="card"><h2>Lamp</h2><p class="price">$25</p></article><article class="card"><h2>Desk</h2><p
class="price">$180</p></article>
7) innerText in jsdom: not implemented (use textContent)
```

`textContent` vs `innerText` vs `innerHTML`

Property	Reads	Writes	Cost / safety
`textContent`	All descendant text, incl. hidden elements and `<script>` text	Replaces children with one text node	Fast; **safe** for user data
`innerText`	Only *rendered* text (respects `display:none`, line breaks)	Like textContent but `\n` → ` `	Needs layout → slower; not in jsdom
`innerHTML`	Serialized HTML of the children	**Parses** the string, replaces children	**XSS** risk (cross-site scripting — attacker code injected into your page) with untrusted strings; destroys existing children and their listeners

Line 3: with `textContent` the "image" became harmless text (`<img …`). Via `innerHTML` it would be a real `<img>` whose `onerror` runs attacker code — see Chapter 14 §14.10.

Why fragments and templates

- A `DocumentFragment` is a lightweight, parentless container. Appending it moves all its children in one operation (it ends up empty — line 5): one page update instead of N.
- A `<template>` holds inert markup (not rendered, scripts don't run, images don't load). `template.content.cloneNode(true)` stamps copies — how framework-free apps and Web Components (the browser's built-in custom HTML tags) render lists. Fill clones with `textContent`.

Common mistakes

- `el.innerHTML += '<li>…</li>'` in a loop re-parses everything and wipes listeners and input state. Use `append` or `insertAdjacentHTML('beforeend', …)`.
- Appending an existing node **moves** it (line 2); clone for a copy.
- `cloneNode` doesn't copy `addEventListener` listeners.

TL;DR

`createElement` + `textContent` + `append`/`before`/`after`/`remove`/`replaceWith`. `textContent` for data, `innerHTML` only for trusted markup. Batch inserts with a fragment; stamp repeated markup from a `<template>`.

13.4 Attributes, properties, classes, data and styles

What it is

- An **attribute** is what's written in the HTML (`<input value="1">`) — always a string; `getAttribute`, `setAttribute`, `hasAttribute`, `removeAttribute`, `toggleAttribute`.
- A **property** is a field on the element object (`input.value`). Most standard attributes are **reflected** as same-named properties, but properties can be typed (booleans, numbers), normalized (absolute URLs) and hold **live state** the attribute doesn't.

`code/04_attributes_properties.mjs` — key parts:

```
const qty = document.getElementById('qty');      // <input id="qty" type="number" value="1">
qty.value = '5';                                 // what the user "typed"
console.log('value property       :', qty.value, '| value attribute:', qty.getAttribute('value'), '|
defaultValue:', qty.defaultValue);
console.log('valueAsNumber        :', qty.valueAsNumber, typeof qty.valueAsNumber);
console.log('href property        :', link.href, '(resolved, absolute)');
console.log('href attribute       :', link.getAttribute('href'));

agree.checked = false;                           // <input type="checkbox" checked>
console.log('checked prop / attr :', agree.checked, '/', agree.hasAttribute('checked'));
buy.setAttribute('disabled', 'false');           // still disabled!
console.log('disabled="false"    :', buy.disabled);

buy.classList.add('loading');
buy.classList.remove('primary');
buy.classList.toggle('active');                  // add if missing
buy.classList.toggle('active', false);           // force off
buy.classList.replace('btn', 'button');

console.log('dataset             :', { ...buy.dataset });     // data-product-id="42" data-in-stock="true"
buy.dataset.variantColor = 'blue';               // -> data-variant-color="blue"

buy.style.backgroundColor = 'yellow';            // camelCase property
buy.style.setProperty('--accent', 'teal');       // custom properties need setProperty
console.log('computed color      :', getComputedStyle(buy).color);
```

Output (lines for the code shown):

```
value property       : 5 | value attribute: 1 | defaultValue: 1
valueAsNumber        : 5 number
href property        : http://localhost/docs?page=2 (resolved, absolute)
href attribute       : /docs?page=2
checked prop / attr : false / true
disabled="false"     : true
dataset              : { productId: '42', inStock: 'true' }
computed color       : rgb(255, 0, 0)
```

Key rules

Rule	Example
Form controls: **attribute = initial/default value**, **property = current value**	`value` / `defaultValue`, `checked` / `defaultChecked`
Boolean attributes: present = true, whatever the value	`disabled="false"` is still disabled; use `el.disabled = false` or `removeAttribute`
Some names differ	`class` → `className` / `classList`; `for` → `htmlFor`
URL properties are absolute	`a.href`, `img.src`
`data-*` → `el.dataset.camelCase`, always strings	`data-in-stock` → `dataset.inStock === 'true'`; read numbers with `Number(el.dataset.userId)`
`el.style` = **inline** style only; `getComputedStyle(el)` = final values from all CSS	Computed values are resolved (`rgb(255, 0, 0)`)

Prefer toggling **classes** over many inline styles: the look stays in CSS and one class change is one style recalculation. Inline `style` suits truly dynamic values, ideally via CSS custom properties (`--name` variables): `el.style.setProperty('--progress', '40%')`.

Self-check

1. User types "7" into `<input value="1">`. `getAttribute('value')`? → `"1"`; `input.value` is `"7"`.
2. Un-disable a button? → `btn.disabled = false` (or `removeAttribute('disabled')`), not `setAttribute('disabled', 'false')`.

TL;DR

Attributes = HTML strings (initial values); properties = live typed state. `classList` for classes, `dataset` for `data-*`, `style` for inline, `getComputedStyle` to read final styles.

13.5 Events and listeners

What it is

An **event** is a notification that something happened — click, key press, form submission, resource loaded. An **event listener** (handler) is a function registered to run when an event type reaches a **target** (element, `document`, `window`, or any `EventTarget` — the base type of everything that can receive events). Multiple listeners run in registration order.

How it works

1. `target.addEventListener(type, listener, options?)` registers.
2. The event occurs (user action, network, or `target.dispatchEvent(event)`).
3. The browser builds an **event object** and sends it along a path through the tree (§13.6), calling matching listeners.
4. User-generated events arrive as **tasks**; `dispatchEvent` calls listeners **synchronously**, before it returns.

Member	Meaning
`type`	`'click'`, `'keydown'`, …
`target`	Where the event originated (deepest element clicked)
`currentTarget`	Element whose listener is running now (= `this` in a non-arrow listener)
`eventPhase`	1 capture, 2 at target, 3 bubble
`preventDefault()` / `defaultPrevented`	Cancel the default action (§13.8)
`stopPropagation()` / `stopImmediatePropagation()`	Stop the journey (§13.6)
`isTrusted`	`true` only for real user/browser events
`key`, `code`, `ctrlKey`, `shiftKey`, `altKey`, `metaKey`	Keyboard (`key` = character/meaning, `code` = physical key)
`clientX/Y`, `button`	Pointer position and button
`detail`	Data on a `CustomEvent`

`code/05_events_basics.mjs` — key parts:

```
function onClick(event) {
  console.log('1) type:', event.type, '| target:', event.target.id, '| currentTarget:', event.currentTarget.id,
    '| this === currentTarget:', this === event.currentTarget, '| isTrusted:', event.isTrusted);
}
btn.addEventListener('click', onClick);
btn.click();                                    // synthetic click (isTrusted false)

btn.addEventListener('click', () => console.log('2) second listener'));
btn.addEventListener('click', onClick);         // duplicate, ignored
btn.click();

btn.removeEventListener('click', () => {});     // different function: does nothing
btn.removeEventListener('click', onClick);
console.log('3) after remove:'); btn.click();

b2.addEventListener('click', () => console.log('4) once fired'), { once: true });
b2.click(); b2.click();
const ac = new AbortController();
b2.addEventListener('click', () => console.log('   A'), { signal: ac.signal });
b2.addEventListener('click', () => console.log('   B'), { signal: ac.signal });
b2.click(); ac.abort(); b2.click();

b2.onclick = () => console.log('5) handler 1');
b2.onclick = () => console.log('5) handler 2 replaced 1');
b2.click();

input.addEventListener('keydown', (e) => console.log('6) key:', e.key, '| code:', e.code, '| ctrl:', e.ctrlKey));
input.dispatchEvent(new KeyboardEvent('keydown', { key: 's', code: 'KeyS', ctrlKey: true, bubbles: true }));

document.addEventListener('cart:add', (e) => console.log('7) custom event detail:', e.detail));
btn.dispatchEvent(new CustomEvent('cart:add', { detail: { sku: 'A-1', qty: 2 }, bubbles: true }));

console.log('8) before dispatch');
btn.addEventListener('ping', () => console.log('   listener'));
btn.dispatchEvent(new Event('ping'));
console.log('   after dispatch');
```

Output:

```
1) type: click | target: save | currentTarget: save | this === currentTarget: true | isTrusted: false
1) type: click | target: save | currentTarget: save | this === currentTarget: true | isTrusted: false
2) second listener
3) after remove:
2) second listener
4) once fired
   A
   B
   (both removed by one abort)
5) handler 2 replaced 1
6) key: s | code: KeyS | ctrl: true
   input event, value = Ada
7) custom event detail: { sku: 'A-1', qty: 2 }
8) before dispatch
   listener
   after dispatch
```

Listener options

Option	Effect
`capture: true`	Run during the capture phase (§13.6). Must match when removing.
`once: true`	Auto-remove after the first call
`passive: true`	Promise not to call `preventDefault()` — the browser can scroll without waiting (§13.8)
`signal: abortSignal`	Remove when the signal aborts — removes many at once (Ch. 12 §12.11)

`addEventListener` vs `on…` properties vs inline attributes

Style	Multiple handlers?	Notes
`el.addEventListener('click', fn)`	Yes	Options, removable — the standard choice
`el.onclick = fn`	No — assigning replaces (line 5)	Fine for quick scripts
`<button onclick="doIt()">`	—	JS in HTML, code from a string, blocked by a strict CSP (Content Security Policy — Ch. 14) — avoid

Common mistakes

- `addEventListener('click', handle())` calls `handle` now and registers its return value. Pass the function.
- Removing with a *new* arrow function (line 3's no-op): keep a reference, or use `{ signal }`.
- Arrow listeners don't get `this = element`; `event.currentTarget` works for both styles.
- The event type has no `on` prefix (`'click'`, not `'onclick'`).

TL;DR

`addEventListener(type, fn, { once, passive, signal, capture })`. `target` = origin, `currentTarget` = listener owner; cancel defaults or stop propagation from the event object. `dispatchEvent` + `CustomEvent` for your own events.

13.6 Propagation: capture, target, bubble

What it is

Clicking a button inside nested boxes also clicks each box, the body, the document and the window. The event travels this **propagation path** in three phases:

1. **Capture phase** — from `window` down through each ancestor; only `{ capture: true }` listeners run.
2. **Target phase** — at the target; listeners in registration order (capture ones first in modern browsers).
3. **Bubble phase** — back up from the target's parent to `window`; normal listeners run. Most events bubble; some don't.

`code/propagation.html` registers a capture and a bubble listener on `window`, `document`, `#outer`, `#inner` and the button, then clicks once:



Figure — one click, ten listener calls: capture (blue) down, target (green), bubble (orange) up. Headless Chrome.

`code/06_propagation.mjs` shows the same path in jsdom plus the ways to stop it:

```
for (const el of [window, document, ...els]) {
  const name = el === window ? 'window' : el === document ? 'document' : el.id;
  el.addEventListener('click', (e) => log.push(`${name}:${PHASE[e.eventPhase]}`), { capture: true });
  el.addEventListener('click', (e) => log.push(`${name}:${PHASE[e.eventPhase]}`));
}
document.getElementById('btn').click();

const stop = (e) => e.stopPropagation();
els[1].addEventListener('click', stop);          // on #inner, bubble phase
els[2].click();

btn.addEventListener('click', (e) => { log.push('btn:first-extra'); e.stopImmediatePropagation(); });
btn.addEventListener('click', () => log.push('btn:NEVER'));
btn.click();

document.body.addEventListener('focus', () => seen.push('focus (bubble listener)'));
document.body.addEventListener('focusin', () => seen.push('focusin'));
input.focus();
```

Output:

```
1) full path:
   window:CAPTURE > document:CAPTURE > outer:CAPTURE > inner:CAPTURE > btn:TARGET > btn:TARGET > inner:BUBBLE >
outer:BUBBLE > document:BUBBLE > window:BUBBLE
2) stopPropagation at inner:
   window:CAPTURE > document:CAPTURE > outer:CAPTURE > inner:CAPTURE > btn:TARGET > btn:TARGET > inner:BUBBLE
3) stopImmediatePropagation:
   window:CAPTURE > document:CAPTURE > outer:CAPTURE > inner:CAPTURE > btn:TARGET > btn:TARGET > btn:first-extra
4) new Event() defaults to x: bubbles=false
   body heard: [ 'focusin' ] <- focus does not bubble, focusin does
```

Stopping

Call	Effect
`stopPropagation()`	Finish listeners on the current element, then stop travelling
`stopImmediatePropagation()`	Also skip the remaining listeners on the current element
`preventDefault()`	Unrelated to propagation: cancels the default action only

Events that don't bubble

`focus` / `blur` (element gaining/losing keyboard **focus**; delegate with `focusin` / `focusout`), `mouseenter` / `mouseleave` (use `mouseover` / `mouseout`), `load` / `error` on elements like `<img>`, `scroll` on elements. `new Event(type)` **doesn't bubble unless you pass** `{ bubbles: true }` (line 4) — a common reason a parent "doesn't hear" a custom event.

When not to stop propagation

`stopPropagation` also blocks analytics listeners, "click outside to close" handlers on `document`, and framework root listeners (React attaches its listeners at the root container). Prefer checking `event.target` in the outer handler, or `defaultPrevented` as a signal.

In the interview

30-second answer: "Events travel in three phases: capture from window down to the target, the target phase, then bubbling back up. `addEventListener` listens in the bubble phase by default; `{capture: true}` switches to capture. `stopPropagation` halts the journey, `stopImmediatePropagation` also skips other listeners on the same element, and `preventDefault` is separate — it only cancels the browser's default action."

- *Q: `target` vs `currentTarget`?* A: `target` is where the event started; `currentTarget` is the element whose listener is running.
- *Q: When use capture?* A: To see an event before children can stop it — global "close menus on any click", focus tracking.

TL;DR

Capture (down, `{capture:true}`) → target → bubble (up, default). `focus`, `mouseenter`, element `load` don't bubble. Stop sparingly.

13.7 Event delegation

What it is

Event delegation: **one** listener on a common ancestor instead of one per child, using `event.target` (usually with `closest`) to find which child was acted on. It works because events bubble.

The problem it solves

1,000 rows with 1,000 listeners cost memory and setup time, and items added later (from `fetch` or the user) need listeners attached — and removed ones cleaned up. With delegation, new items are handled automatically.

How it works

1. Attach a listener to a stable container (`ul`, `table`, a component root, or `document`).
2. In the handler, `const item = event.target.closest('selector')`.
3. Ignore `null` (click elsewhere) or items outside the container (`container.contains(item)`).
4. Read what you need (`item.dataset.id`) and act.

`code/07_delegation.mjs`:

```
list.addEventListener('click', (event) => {
  const del = event.target.closest('button.del');
  if (del && list.contains(del)) {
    const li = del.closest('li');
    console.log('delete item', li.dataset.id);
    li.remove();
    return;
  }
  const li = event.target.closest('li');
  if (li) {
    li.classList.toggle('done');
    console.log('toggle item', li.dataset.id, '-> done =', li.classList.contains('done'));
  }
});

let nextId = 3;
document.getElementById('add').addEventListener('click', () => {
  list.insertAdjacentHTML('beforeend', `<li data-id="${nextId}"><span>New task ${nextId}</span> <button
class="del">x</button></li>`);
  nextId += 1;
});

list.querySelector('li[data-id="1"] span').click();      // click on the text (a child of li)
document.getElementById('add').click();                   // adds item 3 AFTER the listener was attached
list.querySelector('li[data-id="3"] span').click();      // still handled - no new listener needed
list.querySelector('li[data-id="2"] .del').click();
```

Output:

```
toggle item 1 -> done = true
toggle item 3 -> done = true
delete item 2
remaining: [ '1(done)', '3(done)' ]
listeners attached to items: 0, to the list: 1
```

As a real page, `code/todo_delegation.html`: one `click` listener on the `<ul>`, one `submit` listener on the form (with `preventDefault`), new items built with `textContent` — so a task typed as `Call <b>mom</b>` shows literally. After adding two tasks, ticking two and deleting one:

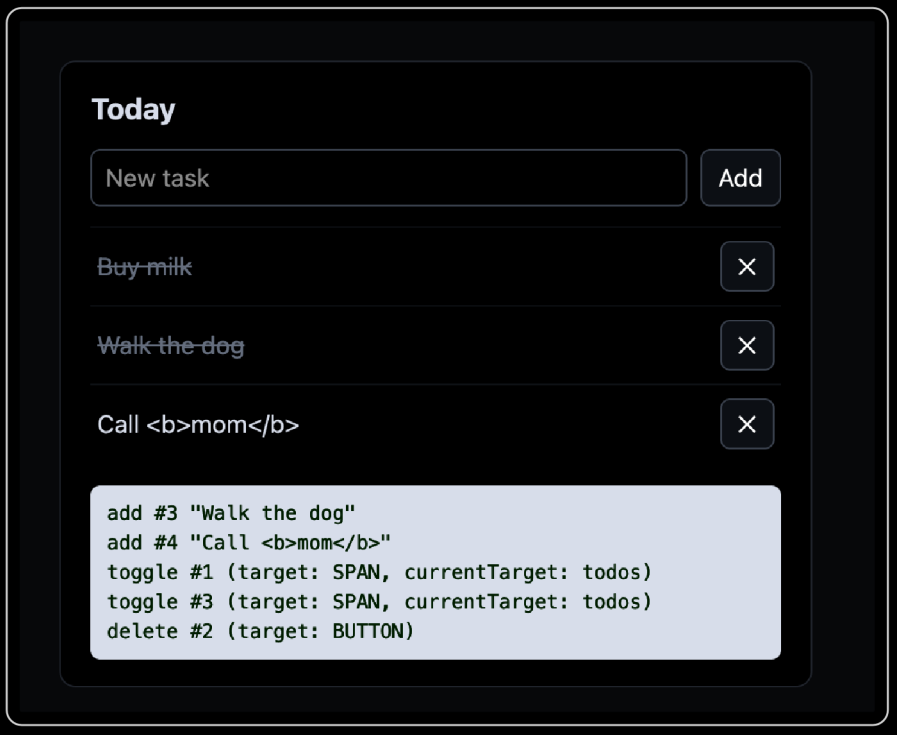


Figure — `target` is the inner `SPAN` / `BUTTON`; `currentTarget` is always the list.

Why `closest` and not `event.target` directly

`event.target` is the **deepest** element clicked — the `<span>` in the `<li>`, an `<svg>` icon in a button. `closest('li')` climbs to the element you care about.

Trade-offs

	Delegation	Listener per element
Listeners	One	N
Dynamic items	Handled automatically	Must attach/detach
Non-bubbling events	Needs the bubbling twin (`focusin`) or capture	Works directly
Handler complexity	Must route by selector	Simple

TL;DR

One listener on the container + `event.target.closest(selector)` handles current and future children. Use for lists, tables, menus.

13.8 Default actions and `preventDefault`

What it is

Many events have a **default action** the browser performs after listeners run: a link navigates, a form submits (reloading the page), a checkbox toggles, a key types, a wheel scrolls, a right-click opens the context menu. `event.preventDefault()` cancels it if the event is **cancelable** (most user-input events are). `dispatchEvent` returns `false` when a listener cancelled it.

Event	Default action	Typical reason to prevent
`submit` on a form	Send the form and navigate	Validate and send with `fetch`
`click` on `<a href>`	Navigate	Client-side routing (change view and URL without reload; History API, Ch. 14)
`keydown`	Insert the character / browser shortcut	Custom shortcut like Ctrl+S
`contextmenu`	Show the browser menu	Custom context menu
`dragover`	Refuse drops	Allow dropping files

(`preventDefault` output lines are in the forms program, §13.9.)

Passive listeners

For `touchstart`, `touchmove` and `wheel`, the browser must wait for your listener before scrolling in case you call `preventDefault()` — janky scrolling. `{ passive: true }` promises you won't, so scrolling starts immediately. Browsers other than Safari already default `passive` to `true` for these on `window`, `document` or `body`; `preventDefault()` in a passive listener is ignored (with a console warning).

Common mistakes

- `return false` in an `addEventListener` handler does nothing (works only in `on…` properties).
- Preventing link/button defaults and forgetting keyboard and screen-reader users — use real `<button>`s for actions and `<a>` for navigation.
- `preventDefault()` does **not** stop propagation; `stopPropagation()` does **not** prevent the default.

TL;DR

`preventDefault()` cancels the browser's built-in response (navigation, submit, typing, scrolling). Passive listeners for scroll/touch.

13.9 Forms

What it is

- `form.elements` — controls by `name` or index (`form.elements.email`).
- **`submit` event** on the form (fires for Enter in a field and for submit buttons) — listen here, not on the button's `click`.
- **`FormData`** — reads every successful control (named, enabled, checked where relevant) into key/value entries; usable directly as a `fetch` body.
- The **Constraint Validation API** — `required`, `type="email"`, `min`, `max`, `minlength`, `pattern` produce a `validity` object; `checkValidity()`, `reportValidity()`, `setCustomValidity(msg)`.
- **`input`** fires on every value change; **`change`** when the change is committed (on blur for text, immediately for checkboxes/selects).

`code/08_forms.mjs` — key parts:

```
age.value = '7';
console.log('1) form valid?', form.checkValidity());
for (const el of [email, age]) {
  const v = el.validity;
  console.log(`   ${el.name}: valid=${v.valid} valueMissing=${v.valueMissing} rangeUnderflow=${v.rangeUnderflow}
msg="${el.validationMessage}"`);
}
email.value = 'not-an-email';
console.log('   typeMismatch for "not-an-email":', email.validity.typeMismatch);

email.value = 'ada@example.com';
email.setCustomValidity(email.value.endsWith('@example.com') ? 'Please use your work email' : '');
console.log('2) customError:', email.validity.customError, '|', email.validationMessage);
email.setCustomValidity('');                 // '' clears the custom error

form.addEventListener('submit', (event) => {
  event.preventDefault();                     // stay on the page, send with fetch instead
  const data = new FormData(form);
  console.log('3) submit prevented:', event.defaultPrevented);
  console.log('   entries   :', JSON.stringify([...data.entries()]));
  console.log('   getAll    :', data.getAll('topics'));
  console.log('   as object :', JSON.stringify(Object.fromEntries(data)), '<- keeps only the LAST topic');
  console.log('   submitter :', event.submitter?.textContent);
});
form.elements.topics[1].checked = true;
form.requestSubmit(form.querySelector('button'));  // like clicking the button: validates, fires submit

age.addEventListener('input', () => log.push(`input(${age.value})`));
age.addEventListener('change', () => log.push(`change(${age.value})`));
// ... two input events, then one change event

const a = document.createElement('a'); a.href = '/leave';
a.addEventListener('click', (e) => e.preventDefault());
console.log('5) link click not cancelled?', a.dispatchEvent(new MouseEvent('click', { cancelable: true })));
cb.addEventListener('click', (e) => e.preventDefault());
cb.click();
console.log('   checkbox still checked after prevented click:', cb.checked);
```

Output:

```
1) form valid? false
   email: valid=false valueMissing=true rangeUnderflow=false msg="Constraints not satisfied"
   age: valid=false valueMissing=false rangeUnderflow=true msg="Constraints not satisfied"
   typeMismatch for "not-an-email": true
2) customError: true | Please use your work email
   now valid? true
3) submit prevented: true
   entries   : [["email","ada@example.com"],["age","36"],["topics","js"],["topics","css"],["plan","pro"],
["bio","Hello"]]
   getAll    : [ 'js', 'css' ]
   as object : {"email":"ada@example.com","age":"36","topics":"css","plan":"pro","bio":"Hello"} <- keeps only the
LAST topic
   submitter : Sign up
4) input(4) input(42) change(42)
5) link click not cancelled? false
   checkbox still checked after prevented click: true
```

(jsdom's `validationMessage` is a generic "Constraints not satisfied"; browsers give localized messages like "Please fill out this field.")

The `validity` flags

Flag	Set when	Caused by
`valueMissing`	Empty required field	`required`
`typeMismatch`	Not a valid email/URL	`type="email"`, `type="url"`
`patternMismatch`	Regex doesn't match the whole value	`pattern`
`tooShort` / `tooLong`	Length out of range (after user edits)	`minlength` / `maxlength`
`rangeUnderflow` / `rangeOverflow` / `stepMismatch`	Number/date out of range	`min` / `max` / `step`
`badInput`	Browser can't convert input (letters in a number field)	input type
`customError`	`setCustomValidity('msg')` with a non-empty message	your code
`valid`	None of the above	—

`code/form_validation.html` uses `novalidate` (turns off the browser's bubbles) and renders its own accessible messages from these flags — `aria-invalid` on the field, `aria-describedby` pointing to the message (ARIA attributes describe state and relationships to assistive technology such as screen readers). One delegated `input` listener gives live feedback; the `submit` handler checks everything. After a scripted submit with a personal email, age 9 and no password:

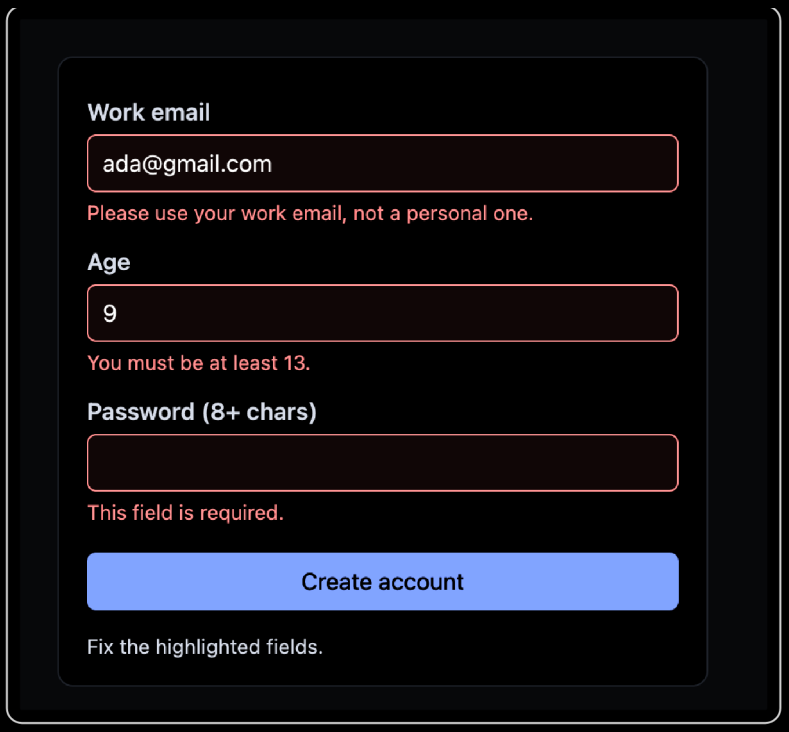


Figure — custom validation messages driven by the Constraint Validation API (headless Chrome).

Common mistakes

- Listening to the submit button's `click` → misses Enter-key submits. Listen to `submit`.
- `Object.fromEntries(new FormData(form))` keeps only the last value of multi-value fields (checkbox groups, `<select multiple>`); use `getAll`.
- Treating client-side validation as security — it's UX (user experience) only; **the server must validate everything again**.
- Missing `name` attributes — unnamed controls are left out of `FormData`.
- `form.submit()` skips validation *and* the `submit` event; `requestSubmit()` behaves like a real submit.

TL;DR

Listen to `submit`, `preventDefault`, read with `FormData`, validate with `validity` / `setCustomValidity` for UX, re-validate on the server. `input` = every keystroke, `change` = committed value.

13.10 Watching the DOM: `MutationObserver`

What it is

`MutationObserver` calls you back when the DOM changes — children added/removed, attributes changed, text edited. Records are **batched** and delivered as a **microtask** after the current script (Ch. 12 §12.2). It replaced the old, slow "mutation events".

`code/09_mutation_observer.mjs`:

```
const observer = new MutationObserver((records) => {
  console.log(`observer got ${records.length} records:`,
    records.map((r) => r.type === 'childList' ? `childList +${r.addedNodes.length} -${r.removedNodes.length}` :
`${r.type}:${r.attributeName}`));
});
observer.observe(feed, { childList: true, attributes: true, subtree: true });

for (let i = 1; i <= 3; i++) feed.insertAdjacentHTML('beforeend', `<li>post ${i}</li>`);
feed.firstElementChild.remove();
feed.setAttribute('data-count', '2');
console.log('sync code done - no callback yet (it is a microtask)');
await null;
observer.disconnect();
```

Output:

```
sync code done - no callback yet (it is a microtask)
observer got 5 records: [
  'childList +1 -0',
  'childList +1 -0',
  'childList +1 -0',
  'childList +0 -1',
  'attributes:data-count'
]
done
```

Uses: reacting to DOM changes by third-party code (ads, chat widgets), auto-initialising widgets when they appear, test utilities that wait for elements (Testing Library's `waitFor` / `findBy*`). Don't use it to watch your own changes. Size/visibility observers (`ResizeObserver`, `IntersectionObserver`): Chapter 14 §14.9.

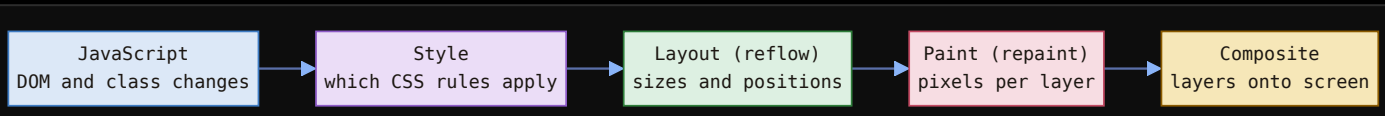
TL;DR

`new MutationObserver(cb).observe(node, { childList, attributes, subtree, characterData })`; batched microtask callbacks; `disconnect()` when done.

13.11 Rendering performance: reflow, repaint and layout thrashing

What it is

After DOM or style changes, the browser's **rendering pipeline** works out what to draw:



- **Reflow / layout** — computing geometry. Changing width, height, font size, adding elements dirties layout — often for the whole subtree or document.
- **Repaint** — redrawing pixels without geometry changes (color, background, shadow).
- **Composite** — assembling layers (separately drawn sheets the GPU stacks); changing only `transform` and `opacity` can often skip layout and paint — hence the go-to animation properties.

The browser normally **batches** your changes and runs the pipeline once before the next frame (§12.2 step 3). But **reading** geometry (`offsetWidth`, `offsetTop`, `clientHeight`, `scrollTop`, `getBoundingClientRect()`, `getComputedStyle(...)` values) after a write forces layout **synchronously** — a **forced synchronous layout**. Write → read → write → read in a loop forces one per iteration: **layout thrashing**.

Measured

`code/reflow.html` widens 2,000 boxes by 10 px two ways (median of 7 runs, headless Chrome):

```
// Interleaved (thrashing): each read forces a layout because of the previous write
for (const b of boxes) { const w = b.offsetWidth; b.style.width = (w + 10) + 'px'; }

// Batched: all reads (one layout), then all writes (one layout before the next frame)
const ws = boxes.map((b) => b.offsetWidth);
boxes.forEach((b, i) => { b.style.width = (ws[i] + 10) + 'px'; });
```

Resize 2,000 boxes to 10 px wider than they are (median of 7 runs)

Strategy	Pattern	Time
Interleaved (thrashing)	`for each box: read offsetWidth, then write style.width`	346.3 ms
Batched reads, then writes	`widths = boxes.map(read); boxes.forEach(write)`	1.5 ms

Interleaving forced 2000 synchronous layouts; batching needed one. Here that was about 231x slower (Chrome, this machine - your numbers will differ). First box: 110px (was 100px).

Figure — same result, two orders of magnitude apart (one run; exact numbers vary, the gap doesn't).

How to keep rendering fast

1. **Batch reads, then writes** — never interleave in loops (fastdom formalizes this).
2. **Change classes, not many individual styles.**
3. **Build off-DOM** (fragment/template), insert once (§13.3).
4. **Animate** `transform` **and** `opacity`, not `top` / `left` / `width`; `requestAnimationFrame` for JS animation (Ch. 14 §14.5).
5. **Debounce/throttle** high-frequency handlers (`scroll`, `resize`, `input`); passive listeners.
6. **Virtualize** long lists (render only visible rows); CSS `content-visibility: auto` for long off-screen sections.
7. **Measure** in the DevTools Performance panel — purple "Layout" blocks with a "Forced reflow" warning point at the culprit.

In the interview

30-second answer: "The browser recalculates style, layout, paint and composite. It batches DOM writes until the next frame, but reading geometry like `offsetWidth` right after a write forces a synchronous layout. Doing that in a loop is layout thrashing. The fix is to batch all reads, then all writes, animate with `transform` / `opacity`, and use `requestAnimationFrame`."

- *Q: Reflow vs repaint?* A: Reflow recomputes geometry (expensive, cascades); repaint redraws pixels without geometry changes.
- *Q: Which properties avoid layout?* A: `transform` and `opacity` typically only need compositing.

TL;DR

Writes are cheap until you read layout. Batch reads then writes; animate `transform`/`opacity`; measure in DevTools.

13.12 When scripts run: `DOMContentLoaded`, `load` and `defer`

A script can only find elements already parsed.

Moment	Event	Meaning
HTML parsed, DOM ready, deferred scripts run	`DOMContentLoaded` on `document`	Safe to query any element
Everything loaded (images, styles, iframes)	`load` on `window`	Needed only for image sizes etc.
User leaves / tab hidden	`pagehide`, `visibilitychange`	Save state; `beforeunload` only to warn about unsaved changes

Tag	Blocks parsing?	Runs when	Order
`<script src>` in `<head>`	Yes (download + run)	Immediately	Document order
`<script defer src>`	No	After parsing, before `DOMContentLoaded`	Document order
`<script async src>`	No (pauses only to run)	As soon as downloaded	Unpredictable
`<script type="module">`	No	Deferred by default	Document order

Recommended: scripts in `<head>` with `defer` (or `type="module"`, deferred automatically — Chapter 10 §10.7): the DOM is ready when your code runs, no `DOMContentLoaded` wrapper needed. `async` for independent scripts like analytics.

```
<!-- (HTML — not run) -->
<head>
  <script src="app.js" defer></script>
</head>
```

> **TL;DR**
>
> `defer` or modules so scripts run after parsing. `DOMContentLoaded` = DOM ready; `load` = all resources loaded.

Chapter 13 TL;DR

- **DOM** = live tree of nodes (element, text, comment) built by the parser; element-only navigation (`children`, `nextElementSibling`).
- **Select** with `querySelector(All)` / `getElementById`; relate with `closest`, `matches`, `contains`. `querySelectorAll` static; `getElementsBy*` live.
- **Modify** with `createElement`, `append`/`prepend`/`before`/`after`/`replaceWith`/`remove`, `replaceChildren`. `textContent` for data; `innerHTML` only for trusted markup. Fragments and `<template>` for bulk rendering.
- **Attributes vs properties**: initial HTML strings vs live typed state. `classList`, `dataset`, `style`, `getComputedStyle`.
- **Events**: `addEventListener(type, fn, { once, passive, signal, capture })`; `target` vs `currentTarget`; `CustomEvent` + `dispatchEvent` (synchronous).
- **Propagation**: capture ↓, target, bubble ↑; stop sparingly; `focus`/`mouseenter` don't bubble; `new Event` needs `{ bubbles: true }`.
- **Delegation**: one container listener + `event.target.closest()`.
- **`preventDefault`** cancels default actions; passive listeners for scroll/touch.
- **Forms**: `submit`, `FormData` (`getAll` for multi-values), `validity`, `setCustomValidity`; `requestSubmit()` not `submit()`; re-validate on the server.
- **MutationObserver**: batched microtask callbacks.
- **Performance**: batch reads then writes (thrashing measured ~230× slower), animate `transform`/`opacity`, `requestAnimationFrame`.
- **Loading**: `defer`/modules; `DOMContentLoaded` vs `load`.

Next: talking to servers, storing data, keeping pages safe — Chapter 14: Browser APIs and Security.

Chapter 14 — Browser APIs and Security

Beyond the DOM, browsers give JavaScript a large toolbox: talking to servers (`fetch`), remembering things (Web Storage, cookies, IndexedDB), scheduling visual updates (`requestAnimationFrame`), managing URLs and history, running code on other threads (Web Workers), and watching elements (observers). That power makes pages a target, so the second half covers the security model — the same-origin policy and CORS — and the attacks every web developer must prevent: XSS (cross-site scripting) and CSRF (cross-site request forgery), plus the defence-in-depth (an extra layer in case another fails) header CSP (Content Security Policy).

What you'll learn

1. `fetch`: requests, responses, JSON, headers, HTTP vs network errors, timeouts.
2. The same-origin policy and CORS, including preflights (demonstrated in Chrome).
3. `localStorage` and `sessionStorage`.
4. Cookies and IndexedDB; choosing a storage mechanism.
5. Timers vs `requestAnimationFrame`.
6. `URL` and `URLSearchParams`.
7. The History API and client-side routing.
8. Web Workers for CPU-heavy work.
9. `IntersectionObserver` and `ResizeObserver`.
10. XSS and sanitizing; CSRF; Content Security Policy.

Before you start. Promises and `async`/`await` (Chapter 12), DOM/events (Chapter 13), JSON (Chapter 5), error classes (Chapter 11).

How the code runs. Programs in `code/` run in Node 24. `fetch` demos start a real `node:http` **server inside the same script** on a free local port, so nothing touches the internet. Storage, URL and History demos use jsdom (`tools_dom.mjs`), which implements `localStorage`, `document.cookie` and `history`. Real-browser features — CORS enforcement, CSP, Web Workers, observers, IndexedDB — are pages served by `code/10_browser_demo_server.mjs` on `localhost:4101` (the "site") and `localhost:4102` (a second origin, the "API"), run in headless Chrome and screenshotted.

14.1 `fetch` and HTTP

What it is

`fetch(url, options?)` sends an HTTP request and returns a promise for a `Response`. Built into browsers and, since Node 18, Node (same API). It replaced the callback-based `XMLHttpRequest` (XHR). HTTP background: System Design Fundamentals.

HTTP in brief: a **request** has a **method** (`GET` read, `POST` create, `PUT`/`PATCH` update, `DELETE` remove), a URL, **headers** (metadata such as `Content-Type` — the body's format — and `Authorization` — credentials) and optionally a **body**. The **response** has a **status code** (2xx success, 3xx redirect, 4xx client error such as 404 Not Found or 401 Unauthorized, 5xx server error), headers and a body.

How it works

1. `fetch` resolves as soon as the **headers** arrive — with a `Response`, whatever the status.
2. Read the body with `res.json()`, `res.text()`, `res.blob()` (raw binary such as an image), `res.arrayBuffer()` or `res.formData()` — each a promise; a body can be read **once**.
3. `fetch` **rejects only on network failure** (DNS failure, refused connection, CORS block, abort). A 404 or 500 is a successful exchange: check `res.ok` (status 200–299).

`code/01_fetch.mjs` — the server part:

```
const server = http.createServer(async (req, res) => {
  const url = new URL(req.url, 'http://localhost');
  const send = (status, body, headers = {}) => {
    res.writeHead(status, { 'Content-Type': 'application/json', ...headers });
    res.end(JSON.stringify(body));
  };
  if (req.method === 'GET' && url.pathname === '/api/users') return send(200, users, { 'X-Total-Count':
String(users.length) });
  if (req.method === 'POST' && url.pathname === '/api/users') { /* read body, push, send(201, user, { Location })
*/ }
  if (url.pathname === '/api/slow') return setTimeout(() => send(200, { ok: true }), 2000);
  if (url.pathname === '/api/crash') return send(500, { error: 'database unavailable' });
  if (url.pathname === '/page') { res.writeHead(200, { 'Content-Type': 'text/html' }); return res.end('<!doctype
html><h1>Not JSON</h1>'); }
  send(404, { error: `no route ${req.method} ${url.pathname}` });
});
await new Promise((r) => server.listen(0, '127.0.0.1', r));   // port 0 = any free port
const BASE = `http://127.0.0.1:${server.address().port}`;
```

The client part:

```
// 1) GET + JSON
const res = await fetch(`${BASE}/api/users`);
console.log('1) status:', res.status, res.statusText, '| ok:', res.ok, '| type:', res.headers.get('content-type'),
'| total:', res.headers.get('x-total-count'));
console.log('   body  :', await res.json());

// 2) POST JSON
const created = await fetch(`${BASE}/api/users`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ name: 'Grace' }),
});
console.log('2) POST ->', created.status, created.headers.get('location'), await created.json());

// 3) HTTP errors do NOT reject: check res.ok yourself
const notFound = await fetch(`${BASE}/api/nope`);
console.log('3) 404 resolved, ok =', notFound.ok, '| body:', (await notFound.json()).error);

// 4) A reusable helper that turns HTTP errors into exceptions
class HttpError extends Error {
  constructor(res, body) { super(`HTTP ${res.status} for ${res.url}`); this.name = 'HttpError'; this.status =
res.status; this.body = body; }
}
async function getJSON(path, init) {
  const r = await fetch(`${BASE}${path}`, init);
  const isJson = r.headers.get('content-type')?.includes('application/json');
  const body = isJson ? await r.json() : await r.text();
  if (!r.ok) throw new HttpError(r, body);
  if (!isJson) throw new TypeError(`expected JSON, got ${r.headers.get('content-type')}`);
  return body;
}

// 5) Network failure DOES reject (TypeError) - a port where nothing listens
await fetch(`http://127.0.0.1:${deadPort}/`).catch((e) => console.log('5) network error:', e.name, '-', e.message,
'| cause:', e.cause?.code));

// 6) Timeout / cancel with AbortSignal
await fetch(`${BASE}/api/slow`, { signal: AbortSignal.timeout(100) })
  .catch((e) => console.log('6) aborted:', e.name, /* ... */));

// 7) A body can be read only once; clone() first if you need it twice
const r7 = await fetch(`${BASE}/api/users`);
const copy = r7.clone();
await r7.json();
await r7.text().catch((e) => console.log('7) second read:', e.name, '-', e.message));
// 8) (file) query string via URL.searchParams; a Request object with Headers
```

Output:

```
1) status: 200 OK | ok: true | type: application/json | total: 1
   body  : [ { id: 1, name: 'Ada' } ]
2) POST -> 201 /api/users/2 { id: 2, name: 'Grace' }
3) 404 resolved, ok = false | body: no route GET /api/nope
4) /api/users -> 2 users
4) /api/crash -> HttpError | 500 database unavailable
4) /page -> TypeError |  expected JSON, got text/html
5) network error: TypeError - fetch failed | cause: ECONNREFUSED
6) aborted: TimeoutError after ~100 ms
7) second read: TypeError - Body is unusable: Body has already been read
   clone still readable: 2 users
8) url: /api/nope?search=Ada+Lovelace+%26+co
   Request: DELETE Bearer demo-token -> 404
```

Network errors are worded `TypeError: Failed to fetch` (Chrome) or `NetworkError when attempting to fetch resource` (Firefox); Node says `fetch failed` with the system error in `cause`.

Useful `fetch` options

Option	Values	Purpose
`method`	`'GET'` (default), `'POST'`, …	HTTP method
`headers`	object or `Headers`	`Content-Type`, `Authorization`, `Accept`
`body`	string, `FormData`, `URLSearchParams`, `Blob`, `ArrayBuffer`, stream	Not with GET/HEAD. `FormData` sets a multipart (fields plus files) `Content-Type` for you — don't set it yourself
`signal`	`AbortSignal`	Cancel / timeout (§12.11)
`credentials`	`'same-origin'` (default), `'include'`, `'omit'`	Whether cookies are sent (§14.2)
`mode`	`'cors'` (default), `'same-origin'`, `'no-cors'`	CORS behaviour
`cache`	`'default'`, `'no-store'`, `'reload'`, …	HTTP cache use
`keepalive`	`true`	Let a small request outlive the page (analytics sent on close)

Common mistakes

- Not checking `res.ok` → parsing an error page or treating a 500 as data.
- `res.json()` on non-JSON → `SyntaxError: Unexpected token '<'` (§11.10).
- Forgetting `Content-Type: application/json` when sending JSON — many servers won't parse it.
- Concatenating query strings (`'?q=' + text`) — `&` or `#` breaks the URL. Use `URLSearchParams` (§14.6).
- No timeout: a hung server spins your UI forever. Always pass a `signal`.
- Secrets (API keys) in front-end code — anything shipped to the browser is public. Call your own backend, which holds the key.

In the interview

30-second answer: "`fetch` returns a promise for a `Response` that resolves when headers arrive, even for 404 or 500 — it only rejects on network errors or aborts, so you check `res.ok`. You read the body once with `json()` or `text()`. For sending JSON you set `Content-Type` and `JSON.stringify` the body. I always wrap it in a helper that throws on non-2xx and use `AbortSignal.timeout` so requests can't hang."

- *Q: Does `fetch` send cookies?* A: Same-origin yes (default `credentials: 'same-origin'`); cross-origin only with `credentials: 'include'` and the right CORS headers.

- *Q: `fetch` vs axios?* A: axios adds auto JSON, throw on non-2xx, interceptors (hooks on every request/response); `fetch` is built in and a small wrapper covers most needs.

TL;DR

`fetch` → `Response`; check `res.ok`; read the body once; HTTP errors resolve, network errors reject; JSON = header + `JSON.stringify`; always time out.

14.2 The same-origin policy and CORS

What it is

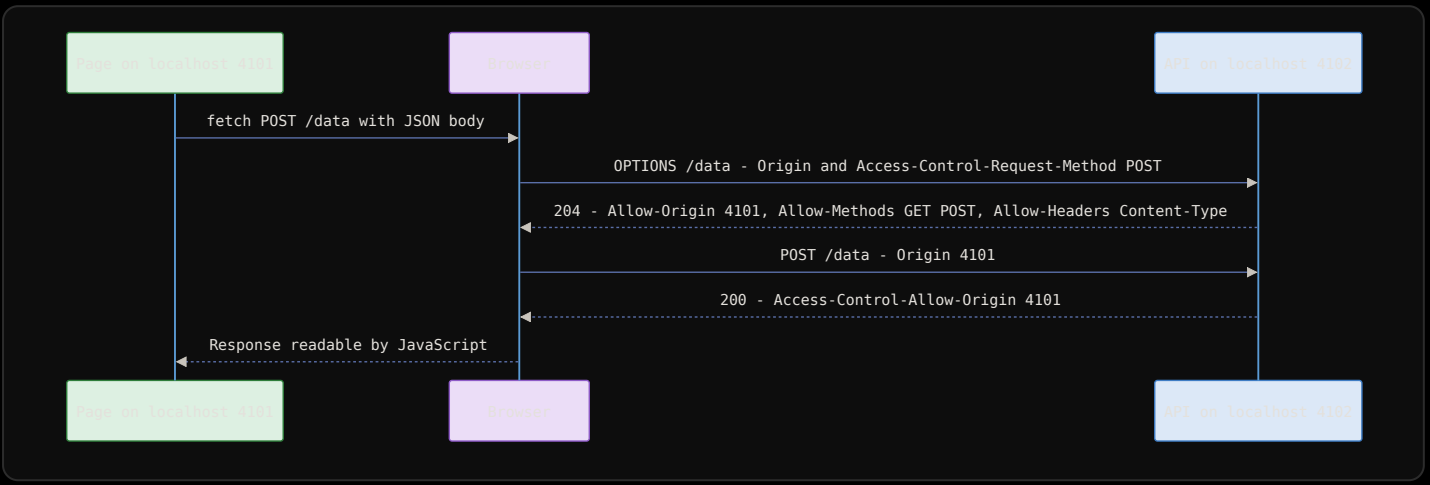
An **origin** is **scheme + host + port** (`https://shop.example.com:443`; 443 is HTTPS's default port). `http://localhost:4101` and `:4102` are different origins, as are `http://` vs `https://` and `a.example.com` vs `b.example.com`.

The **same-origin policy (SOP)**: a page may **send** requests anywhere, but its JavaScript may **read responses** only from its own origin. Without it, any site could `fetch('https://your-bank.example/account')` — with your bank cookies attached — and read your balance.

CORS (Cross-Origin Resource Sharing) is the controlled exception: a server declares, with response headers, which other origins may read its responses.

How it works

1. A page on `http://localhost:4101` calls `fetch('http://localhost:4102/public')`.
2. The browser adds `Origin: http://localhost:4101`.
3. **Simple requests** (GET/HEAD/POST with only "safelisted" headers and `Content-Type` limited to `text/plain`, `multipart/form-data` or `application/x-www-form-urlencoded`) are sent directly.
4. Anything else — `PUT`/`DELETE`, JSON `Content-Type`, an `Authorization` header — first triggers a **preflight**: an automatic `OPTIONS` request with `Access-Control-Request-Method`/`-Headers`. The real request is sent only if allowed.
5. On the response the browser checks `Access-Control-Allow-Origin`. Match (or `*`) → JavaScript gets it; otherwise `fetch` rejects with a `TypeError` and details go only to the DevTools console.



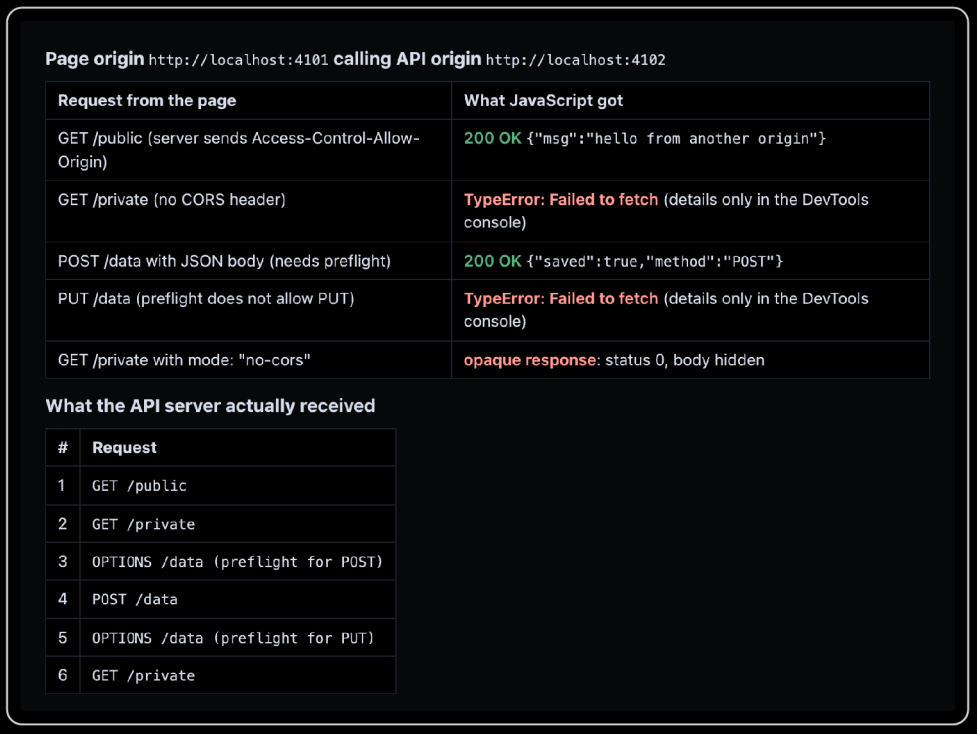
The CORS part of the API server in `code/10_browser_demo_server.mjs`:

```
const allow = { 'Access-Control-Allow-Origin': SITE, Vary: 'Origin' };   // SITE = 'http://localhost:4101'
if (req.method === 'OPTIONS') {                          // preflight: allow POST with a JSON body, but not PUT
  return res.writeHead(204, { ...allow, 'Access-Control-Allow-Methods': 'GET, POST',
    'Access-Control-Allow-Headers': 'Content-Type', 'Access-Control-Max-Age': '600' }).end();
}
if (path === '/public') return json(200, { msg: 'hello from another origin' }, allow);
if (path === '/private') return json(200, { secret: 'no CORS header on this one' });   // browser will hide it
if (path === '/data') return json(200, { saved: true, method: req.method }, allow);
```

Run without arguments, the script inspects the headers with Node's `fetch` — which does **not** enforce CORS (only browsers do):

```
preflight -> 204 {
  'access-control-allow-headers': 'Content-Type',
  'access-control-allow-methods': 'GET, POST',
  'access-control-allow-origin': 'http://localhost:4101',
  'access-control-max-age': '600'
}
/public  ACAO: http://localhost:4101
/private ACAO: null | Node still reads it: no CORS header on this one
csp.html CSP : default-src 'self'; script-src 'self'; img-src 'self'; style-src 'self' 'unsafe-inline'
```

In Chrome, `code/cors.html` (served from `:4101`) makes five requests to `:4102`, then asks the API what it received:



Page origin `http://localhost:4101` **calling API origin** `http://localhost:4102`

Request from the page	What JavaScript got
GET /public (server sends Access-Control-Allow-Origin)	200 OK `{"msg":"hello from another origin"}`
GET /private (no CORS header)	TypeError: Failed to fetch (details only in the DevTools console)
POST /data with JSON body (needs preflight)	200 OK `{"saved":true,"method":"POST"}`
PUT /data (preflight does not allow PUT)	TypeError: Failed to fetch (details only in the DevTools console)
GET /private with mode: "no-cors"	opaque response: status 0, body hidden

What the API server actually received

#	Request
1	GET /public
2	GET /private
3	OPTIONS /data (preflight for POST)
4	POST /data
5	OPTIONS /data (preflight for PUT)
6	GET /private

Figure — CORS as enforced by Chrome, and what actually reached the server.

What it proves:

- `/private` **reached the server** (log line 2) — CORS stops the page **reading** the response, not the request. Hence CORS is no CSRF defence (§14.11).
- The JSON `POST` triggered an `OPTIONS` preflight first (lines 3–4).
- For `PUT` the preflight said no, so the `PUT` was **never sent** (line 5).
- `mode: 'no-cors'` gives an **opaque** response: status 0, unreadable body — only for fire-and-forget.

CORS headers

Header (response)	Meaning
`Access-Control-Allow-Origin`	`*` or one exact origin (not a list — echo an allow-listed `Origin` and add `Vary: Origin` so caches key by origin)
`Access-Control-Allow-Methods` / `-Headers`	Preflight answer: allowed methods and request headers
`Access-Control-Allow-Credentials: true`	Allow cookies with `credentials: 'include'`; then `Allow-Origin` **can't** be `*`
`Access-Control-Max-Age`	Seconds the browser may cache the preflight answer
`Access-Control-Expose-Headers`	Non-safelisted response headers JS may read (e.g. `X-Total-Count`)

Common mistakes

- "Fixing" CORS in the front end — impossible; the **server** must send the headers (or proxy the API through your own origin, e.g. Vite's dev-server proxy).
- `Access-Control-Allow-Origin: *` with credentials — browsers reject it.
- Reflecting any `Origin` with credentials allowed — effectively disables the SOP. Use an allow-list.
- Reading a CORS error as "server down" — usually it answered without the right headers.

In the interview

> **30-second answer:** "The same-origin policy lets a page send requests anywhere but only read responses from its own scheme-host-port. CORS lets a server opt in with `Access-Control-Allow-Origin`. Non-simple requests — JSON bodies, custom headers, PUT/DELETE — first get an OPTIONS preflight. CORS is enforced by the browser, not the server, and it protects reading, not sending."

- *Q: Why does Postman/curl work but the browser fails?* A: Only browsers enforce CORS.
- *Q: Is CORS a security feature for the server?* A: No — it relaxes the browser's SOP; servers still need authentication and CSRF protection.

TL;DR

Origin = scheme + host + port. SOP blocks reading cross-origin responses; CORS headers relax it per origin. Non-simple requests get an OPTIONS preflight. Browsers enforce; servers configure.

14.3 Web Storage: `localStorage` and `sessionStorage`

What it is

Two **key–value stores of strings** per origin, with a synchronous API:

- `localStorage` — persists until cleared; shared by all tabs of the origin.
- `sessionStorage` — per **tab** (and origin); survives reloads, cleared when the tab closes.

API: `setItem`, `getItem` (→ `null` if missing), `removeItem`, `clear()`, `key(i)`, `length`. A change in one tab fires a `storage` **event** in the **other** tabs of the origin (not the writer) — handy for syncing logout or theme.

`code/03_storage.mjs` — key parts:

```
localStorage.setItem('theme', 'dark');
localStorage.count = 5;                              // property style works but is discouraged
console.log('   numbers become strings:', typeof localStorage.getItem('count'),
JSON.stringify(localStorage.getItem('count')));
localStorage.setItem('user', { name: 'Ada' });       // WRONG: stores "[object Object]"

const store = {
  get(key, fallback = null) {
    try { const raw = localStorage.getItem(key); return raw === null ? fallback : JSON.parse(raw); }
    catch { return fallback; }                          // corrupt value or storage disabled
  },
  set(key, value) {
    try { localStorage.setItem(key, JSON.stringify(value)); return true; }
    catch (e) { console.log('   set failed:', e.name); return false; }  // QuotaExceededError, private mode...
  },
};
store.set('cart', { items: [{ sku: 'A-1', qty: 2 }], updated: '2026-09-28' });
// ... enumerate, removeItem
store.set('huge', 'x'.repeat(6_000_000));            // over the ~5 MB quota
sessionStorage.setItem('wizardStep', '2');
```

Output (lines 1–6):

```
1) getItem      : dark | missing -> null
   numbers become strings: string "5"
   object stored raw     : [object Object]
2) JSON round trip: { sku: 'A-1', qty: 2 } | corrupt -> fallback
3) length: 5 | keys: [ 'theme', 'count', 'user', 'cart', 'bad' ]
   after removeItem: 4
   set failed: QuotaExceededError
5) sessionStorage: 2 | separate from localStorage: null
6) after clear(): 0
```

Trade-offs

✓ Good for	✗ Not for
UI preferences, drafts, small caches	**Secrets and auth tokens** — any script on the page (an XSS payload, a compromised npm package) can read them
Data that must survive reloads, a few KB–MB	Large data (quota ~5 MB per origin; use IndexedDB)
	Hot paths — it's **synchronous** and blocks the main thread on disk access
	Data for the server — never sent automatically (unlike cookies)

Common mistakes

- Storing objects without `JSON.stringify` → `"[object Object]"`.
- No `try`/`catch`: quota errors, and storage may be disabled (privacy modes, sandboxed iframes) so even *reading* can throw.
- Expecting `sessionStorage` to be shared across tabs (a duplicated tab gets a copy).

TL;DR

String key–value storage per origin; `localStorage` persists, `sessionStorage` is per tab. JSON + try/catch; never secrets.

14.4 Cookies and IndexedDB — choosing storage

Cookies

A **cookie** is a small (≈4 KB) name=value pair stored per site and **sent automatically with every matching HTTP request** in the `Cookie` header. Servers set them with `Set-Cookie`; JavaScript reads/writes non-`HttpOnly` cookies through the awkward `document.cookie` string.

```
document.cookie = 'lang=en; Path=/; Max-Age=86400; SameSite=Lax';
document.cookie = 'consent=yes; Path=/';
document.cookie = 'session=secret; HttpOnly';       // IGNORED: HttpOnly cookies can only be set by the server
console.log('7) document.cookie:', JSON.stringify(document.cookie));
const cookies = Object.fromEntries(document.cookie.split('; ').map((c) => c.split('=').map(decodeURIComponent)));
document.cookie = 'lang=; Path=/; Max-Age=0';        // delete = expire it
```

Output (end of `03_storage.mjs`):

```
7) document.cookie: "lang=en; consent=yes"
   parsed: { lang: 'en', consent: 'yes' }
   after delete: "consent=yes"
```

Assigning `document.cookie` adds/updates **one** cookie; reading returns all visible ones as one string, without attributes.

Attribute	Meaning
`Expires=<date>` / `Max-Age=<seconds>`	Lifetime; without either it's a **session cookie** (deleted when the browser session ends)
`Domain` / `Path`	Which hosts/paths receive it (omit `Domain` = this host only, safest)
`Secure`	Only sent over HTTPS (`localhost` counts as secure for development)
`HttpOnly`	Invisible to JavaScript — protects session cookies from XSS theft
`SameSite=Strict` / `Lax` / `None`	Whether it's sent on **cross-site** requests (§14.11). `None` requires `Secure`. Chrome and Edge treat a missing attribute as `Lax`; set it explicitly
`__Host-` name prefix	Browser enforces `Secure`, `Path=/`, no `Domain` — hardest to tamper with

Session-ID recipe: `Set-Cookie: __Host-sid=…; HttpOnly; Secure; SameSite=Lax; Path=/`. (An async `cookieStore` API exists; check support.)

IndexedDB

IndexedDB is the browser's real database: asynchronous, **transactional** (a group of changes all happen or none), storing **structured-cloneable** values (objects, arrays, `Date`, `Map`, `Blob`, typed arrays) in **object stores** (≈ tables) with **keys** and **indexes** (lookup structures for fast search by a field); quotas typically hundreds of MB to GB. The raw API is event-based and verbose, so apps use a thin promise wrapper (`req2p` below, the `idb` library) or a higher-level one (Dexie).

`code/idb.html` — key parts:

```
const req2p = (req) => new Promise((resolve, reject) => { req.onsuccess = () => resolve(req.result); req.onerror =
() => reject(req.error); });

const open = indexedDB.open('notes-demo', 1);
open.onupgradeneeded = () => {                              // runs when the version number increases
  const store = open.result.createObjectStore('notes', { keyPath: 'id', autoIncrement: true });
  store.createIndex('byTag', 'tag');
};
const db = await req2p(open);

const tx = db.transaction('notes', 'readwrite');           // all writes commit together, or none
for (const n of notesToAdd) tx.objectStore('notes').add(n);
await new Promise((r) => (tx.oncomplete = r));

const ro = db.transaction('notes').objectStore('notes');   // read-only transaction
await req2p(ro.get(2));
await req2p(ro.index('byTag').getAll('home'));             // query by index
await req2p(ro.getAll(IDBKeyRange.bound(2, 3)));           // key range
```

IndexedDB: a notes database in the browser

```
upgradeneeded: created store "notes" + index "byTag"
added 3 notes (Dates are stored as real Date objects — structured clone)
count      : 3
get(2)     : Ship chapter 14 | created is a Date: true
tag="home" : Buy milk, Fix bike
ids 2..3   : 2:Ship chapter 14, 3:Fix bike
storage.estimate(): quota is about 11 GB for this origin (varies by browser and disk)
```

Figure — IndexedDB in headless Chrome. The quota is from `navigator.storage.estimate()` and depends on browser and free disk.

Choosing

	Cookies	localStorage	sessionStorage	IndexedDB
Capacity	~4 KB each	~5 MB/origin	~5 MB/origin	Large (share of disk)
API	String; sync	Sync	Sync	Async, transactions
Data types	Strings	Strings	Strings	Structured clone (objects, Dates, Blobs…)
Sent to server	**Automatically**	No	No	No
Lifetime	Expiry / session	Until cleared	Tab	Until cleared (may be evicted under storage pressure unless `navigator.storage.persist()` is granted)
Readable by JS	Unless `HttpOnly`	Yes	Yes	Yes
Typical use	Session ID, CSRF, consent	Preferences	Wizard progress	Offline data, caches, large datasets

A fifth option, the **Cache API**, stores whole HTTP responses; service workers use it for offline apps (§14.8).

TL;DR

Cookies = tiny, auto-sent, the place for `HttpOnly` session IDs. Web Storage = small, sync, strings. IndexedDB = async database for big or structured data.

14.5 Timers and `requestAnimationFrame`

What it is

`requestAnimationFrame(callback)` (**rAF**) runs `callback(timestamp)` **just before the next repaint** — once per frame at the display's refresh rate (60 Hz → ~16.7 ms, 120 Hz → ~8.3 ms). It's the scheduler for anything visual; timers (§12.5) are for everything else.

	`setTimeout` / `setInterval`	`requestAnimationFrame`
When	After a delay, as a task	Right before paint, in the event loop's render step
Synced to display	No (can run twice per frame or miss frames → jank)	Yes
Background tabs	Throttled (≥1 s typically)	**Paused** — saves battery
Callback argument	Your args	High-resolution frame timestamp
Cancel	`clearTimeout`	`cancelAnimationFrame(id)`

`code/06_raf.mjs` (jsdom simulates 60 fps):

```
function animate(el, to, duration) {
  return new Promise((resolve) => {
    let start;
    const frames = [];
    function step(now) {                        // now = frame timestamp (ms), same clock as performance.now()
      start ??= now;
      const t = Math.min((now - start) / duration, 1);
      const eased = 1 - (1 - t) ** 3;           // ease-out cubic
      el.style.width = `${Math.round(eased * to)}px`;
      frames.push(el.style.width);
      if (t < 1) requestAnimationFrame(step);
      else resolve(frames);
    }
    requestAnimationFrame(step);
  });
}
const frames = await animate(bar, 300, 200);

// Coalesce many events into one update per frame (e.g. scroll or pointermove)
let scheduled = false;
function onPointerMove() {
  events++;
  if (scheduled) return;
  scheduled = true;
  requestAnimationFrame(() => { scheduled = false; updates++; /* read layout, then write styles */ });
}
```

Output (intermediate widths vary with real frame timing; 0px and 300px don't):

```
1) about 12 frames in 200 ms at 60 fps: true
   first widths: 0px, 69px, 130px, 178px ... last: 300px
2) cancelled a frame callback
3) 50 events -> 1 DOM update(s)
```

Rules

1. **Base progress on time, not frame count** — rates vary (60/120/144 Hz, dropped frames).
2. rAF runs **once**; re-request inside the callback to continue.
3. **Coalesce** bursts (`scroll`, `pointermove`, `resize`) into one DOM update per frame.
4. Prefer **CSS transitions/animations** or the Web Animations API (`el.animate()`) for simple effects — they can run on the compositor thread. Use rAF when JS must compute each frame (games, canvas, physics, drag).
5. Animate `transform`/`opacity` (§13.11).

TL;DR

rAF = "run before the next paint", once per frame, paused in background tabs. Time-based progress, re-request each frame, coalesce events through it.

14.6 `URL` and `URLSearchParams`

What it is

`new URL(input, base?)` parses a URL into parts; `url.searchParams` is a `URLSearchParams` for the query string. Both handle **percent-encoding** (unsafe characters as `%` + hex, e.g. space → `%20`), so never glue URLs with `+`.

`code/04_url.mjs` — key parts:

```
const u = new URL('https://shop.example.com:8443/search/results?q=red+shoes&size=9&size=10#reviews');
console.log('1) parts:', { protocol: u.protocol, host: u.host, hostname: u.hostname, port: u.port, pathname:
u.pathname, search: u.search, hash: u.hash, origin: u.origin });
const p = u.searchParams;
p.set('page', '2'); p.append('size', '11'); p.delete('q'); p.sort();
console.log('3)', new URL('../img/logo.png', 'https://example.com/docs/guide/').href);
const search = new URLSearchParams({ q: 'Tom & Jerry', tag: 'c++', emoji: '✓' });

function safeHttpUrl(input) {
  try {
    const url = new URL(input);
    return ['http:', 'https:'].includes(url.protocol) ? url.href : null;
  } catch { return null; }
}
```

Output:

```
1) parts: {
  protocol: 'https:',
  host: 'shop.example.com:8443',
  hostname: 'shop.example.com',
  port: '8443',
  pathname: '/search/results',
  search: '?q=red+shoes&size=9&size=10',
  hash: '#reviews',
  origin: 'https://shop.example.com:8443'
}
2) q = red shoes | size = 9 | getAll = [ '9', '10' ] | has page: false
   now: ?page=2&size=9&size=10&size=11 | entries: [
  [ 'page', '2' ],
  [ 'size', '9' ],
  [ 'size', '10' ],
  [ 'size', '11' ]
]
3) https://example.com/docs/img/logo.png
   https://example.com/api?x=1
4) q=Tom+%26+Jerry&tag=c%2B%2B&emoji=%E2%9C%93
   manual concat would break: q=Tom & Jerry <- the & starts a new parameter
   encodeURIComponent: Tom%20%26%20Jerry%2F%3F | encodeURI keeps / and ?: /a%20b/?x=1&y=2
5) https://ada.dev          -> https://ada.dev/
5) javascript:alert(1)     -> null
5) not a url               -> null
5) data:text/html,<b>x</b>  -> null
   URL.canParse: true false
6) { a: '1', b: 'two' } | k=v1&k=v2
```

Notes

- `new URL('relative')` **without a base throws** `TypeError: Invalid URL`; `URL.canParse()` checks without throwing.
- `URLSearchParams` encodes spaces as `+` (form encoding); `encodeURIComponent` uses `%20`. Servers decode both.
- `get` returns the first value, `getAll` all — repeated keys are normal (`?size=9&size=10`).
- In the browser `location` is URL-like; `new URL(location.href)` gives a mutable copy.
- **Validate user-supplied URLs** by parsing and allow-listing the protocol (item 5); blocking `javascript:` prevents a classic XSS (§14.10).

> **TL;DR**

Parse with `new URL`, build queries with `URLSearchParams`, resolve relative URLs with a base, allow-list protocols for user links.

14.7 The History API

What it is

The **History API** changes the address-bar URL and adds back/forward entries **without loading a new page** — the basis of **single-page application (SPA)** routing (one HTML page; JavaScript swaps views; a router maps each URL to a view, e.g. React Router, Vue Router).

API	Does
`history.pushState(state, '', url)`	Add an entry and change the URL (same origin only). No request, no `popstate`
`history.replaceState(state, '', url)`	Change the current entry — e.g. update filter query params
`history.state`	Current entry's state object (structured-cloned)
`history.back()` / `forward()` / `go(n)`	Navigate; async
`popstate` event on `window`	Fires on back/forward to an entry — **re-render from `location`**

`code/05_history.mjs` — a 30-line router:

```
const routes = [
  [/^\/$/, () => 'Home page'],
  [/^\/products$/, () => 'All products'],
  [/^\/products\/(\d+)$/, (id) => `Product #${id}`],
];
function render() { /* match location.pathname against routes, write the view with textContent */ }

// Intercept same-origin link clicks (delegation) and navigate without reloading
document.addEventListener('click', (e) => {
  const a = e.target.closest('a');
  if (!a || a.origin !== location.origin || e.ctrlKey || e.metaKey) return;   // let the browser handle it
  e.preventDefault();
  history.pushState({ from: location.pathname }, '', a.href);                  // new history entry, no request
  render();
});
// Back/forward buttons fire popstate: re-render from the URL
window.addEventListener('popstate', () => { render(); show('popstate (back/fwd)'); });

render(); show('initial');
document.querySelector('a[href="/products"]').click(); show('click Products');
document.querySelector('a[href="/products/42"]').click(); show('click Lamp');
history.replaceState({ tab: 'reviews' }, '', '/products/42?tab=reviews'); show('replaceState (no new entry)');
history.back();
```

Output:

```
initial                     /             | Home page | state: null | length: 1
click Products              /products     | All products | state: {"from":"/"} | length: 2
click Lamp                  /products/42  | Product #42 | state: {"from":"/products"} | length: 3
replaceState (no new entry) /products/42  | Product #42 | state: {"tab":"reviews"} | length: 3
popstate (back/fwd)         /products     | All products | state: {"from":"/"} | length: 3
external link: same origin? false -> not intercepted
```

Must-knows for SPA routing

1. **The server must serve the app for every route** (fallback to `index.html`) — else reloading `/products/42` is a 404.
2. Respect modifier keys (Ctrl/Cmd/Shift-click), `target="_blank"` and external links.
3. Manage **focus and scroll** after navigation (focus the new heading; restore scroll on back) — screen-reader users otherwise miss the change.
4. Put shareable UI state (filters, tabs, search) in the URL (`replaceState` + `URLSearchParams`).

5. Hash routing (`#/products` + `hashchange`) is the older approach needing no server config.

The newer **Navigation API** (`navigation.navigate()`, one `navigate` event for all navigations) simplifies routers; check support.

TL;DR

`pushState` / `replaceState` change the URL without loading; `popstate` on back/forward — re-render from `location`. Configure the server fallback; handle focus.

14.8 Web Workers

What it is

A **Web Worker** runs a script on a **separate background thread** with its own event loop and global (`self`). No DOM, `window` or `document`, but it has `fetch`, timers, IndexedDB, `crypto` and more. Page and worker communicate only by **messages**:

```
// main page
const w = new Worker('prime_worker.js');
w.onmessage = (e) => show(e.data.count);
w.postMessage(5_000_000);
```

```
// prime_worker.js
self.onmessage = (e) => {
  const count = countPrimes(e.data);
  self.postMessage({ upTo: e.data, count });
};
```

Messages are copied with the **structured clone algorithm** (plain objects, arrays, `Date`, `Map`, `Set`, `Blob`, typed arrays… **not** functions, DOM nodes or class prototypes). Big `ArrayBuffer`s can be **transferred** instead (`postMessage(buf, [buf])`): ownership moves, zero-copy, the sender loses access.

Why it matters — measured

`code/worker.html` counts primes up to 5 million on the main thread and in `code/prime_worker.js`; a 10 ms `setInterval` tick counter shows whether the page could do anything else meanwhile:

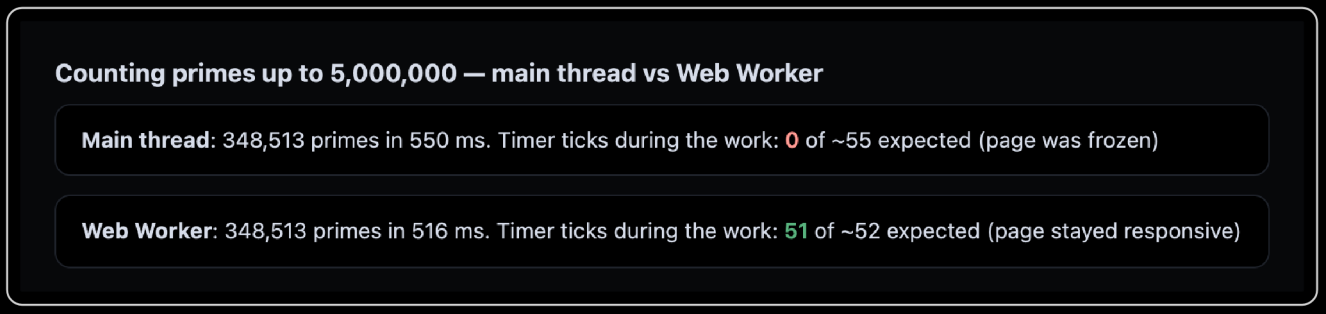


Figure — same work, same time; only the worker leaves the main thread free (headless Chrome).

A worker doesn't make computation faster — it moves it off the thread that handles clicks, scrolling and painting. Node has the same model in `worker_threads`; `code/07_workers_node.mjs` shows message passing and clone rules (Date and Map survive; posting `{ fn: () => 1 }` throws):

```
worker result : { upTo: 3000000, count: 216816, sampleIsCopy: true, mapSize: 1 }
main thread kept running while the worker computed: true
clone error   : DataCloneError
structuredClone keeps Date/Map: true
```

Kinds of workers

Kind	Purpose
Dedicated `Worker`	Background computation for one page. `new Worker(url, { type: 'module' })` allows `import`
`SharedWorker`	One worker shared by several tabs of the origin
Service worker	Network proxy between page and server — offline support, caching, push notifications (PWAs — installable, offline-capable websites). Event-driven, not for computation
Worklets	Tiny specialized scripts (audio, CSS paint)

When not to use it

For I/O (`fetch` is already async) or small tasks — message copying and start-up cost more than they save. Use it for parsing big files, image/audio processing, compression, cryptography, syntax highlighting, search indexes. Comlink makes messaging feel like function calls.

TL;DR

Workers = real parallel threads without DOM, talking via `postMessage` (structured clone or transfer). Keep CPU-heavy work off the main thread.

14.9 `IntersectionObserver` and `ResizeObserver`

What it is

Observers are browser callbacks **when something changes** — instead of polling in `scroll`/`resize` handlers that fire constantly and force layout reads.

Observer	Tells you when…	Typical uses
`IntersectionObserver`	An element enters/leaves the viewport (or a scroll container) by a given fraction	Lazy loading, infinite scroll, "seen" analytics, starting animations
`ResizeObserver`	An element's size changes (any reason, not just window resize)	Components that adapt to available space, charts that fit their container
`MutationObserver`	The DOM changes	Chapter 13 §13.10

`code/observers.html` — key parts:

```
const io = new IntersectionObserver((entries, obs) => {
  for (const e of entries) {
    if (!e.isIntersecting) continue;
    e.target.classList.add('loaded');
    e.target.textContent = 'item ' + e.target.dataset.i + ' loaded';
    obs.unobserve(e.target);                    // load once
  }
}, { root: scroller, rootMargin: '20px' });     // root: null would mean the viewport
scroller.querySelectorAll('.item').forEach((el) => io.observe(el));

const ro = new ResizeObserver((entries) => {
  for (const e of entries) {
    const { inlineSize, blockSize } = e.contentBoxSize[0];
    log('RO callback: content box ' + Math.round(inlineSize) + ' x ' + Math.round(blockSize) + ' px');
  }
});
ro.observe(box);
// scripted: scrollTop = 300, then 600; then box.style.width = '240px'
```

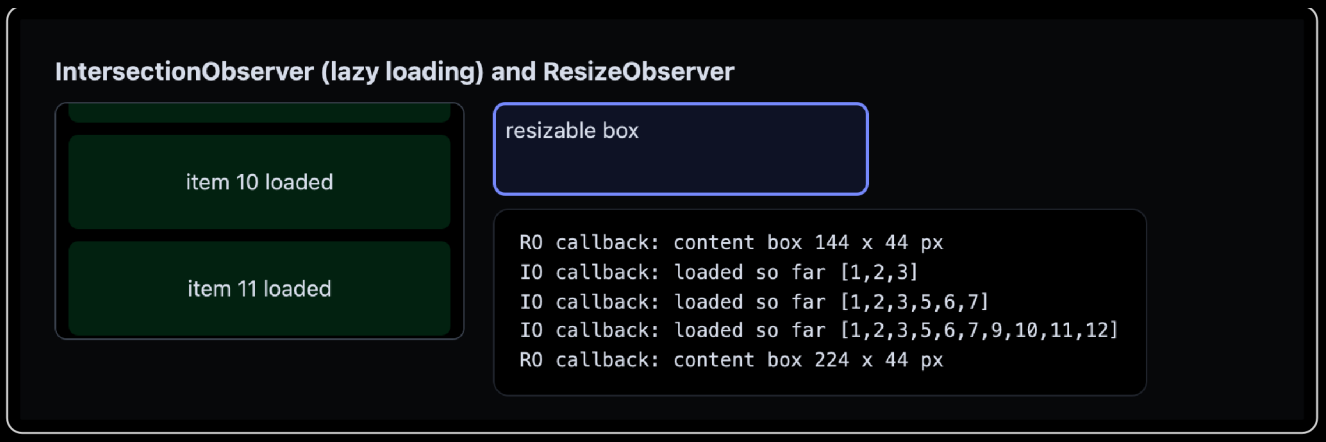


Figure — observers in headless Chrome. Items 4 and 8 never loaded: the scroll jumps skipped them, as a fast user scroll would.

- Both call back **once initially** per observed element, then on changes.
- Options: `root` (container or `null` = viewport), `rootMargin` (grow/shrink the root box, e.g. `'200px'` to load early), `threshold` (`0` = any pixel, `1` = fully visible, or an array).
- For images and iframes prefer native `loading="lazy"`; IO for custom cases.
- Always `unobserve` / `disconnect` when done.
- A `ResizeObserver` callback that resizes its target can loop; the browser stops after one pass and reports "ResizeObserver loop completed with undelivered notifications".

TL;DR

`IntersectionObserver` for visibility, `ResizeObserver` for element size — no polling, no forced layouts. Disconnect when done.

14.10 Cross-site scripting (XSS) and sanitizing

What it is

Cross-site scripting (XSS): an attacker gets **their JavaScript to run in your page** with your page's privileges — reading displayed data, calling your APIs as the user, reading non-HttpOnly cookies and `localStorage`, faking login forms. It has appeared in every edition of the OWASP Top 10 (the OWASP non-profit's ranking of critical web-app risks) — its own category until 2017, folded into "Injection" since 2021.

Type	How the payload arrives
Stored	Saved on the server (a comment, a profile name) and shown to others
Reflected	Echoed from the request (a search term in the URL) into the response
DOM-based	Client code moves untrusted data (URL, `postMessage`, API data) into a dangerous **sink** like `innerHTML`

How it works — seen in a real browser

`code/xss.html` renders the same comment twice. It contains `<img src="does-not-exist.png" onerror="…">`: the image fails, so the browser runs `onerror` as code.

```
document.getElementById('safe').textContent = comment;   // shown as text
document.getElementById('unsafe').innerHTML = comment;   // parsed: the onerror handler runs
```

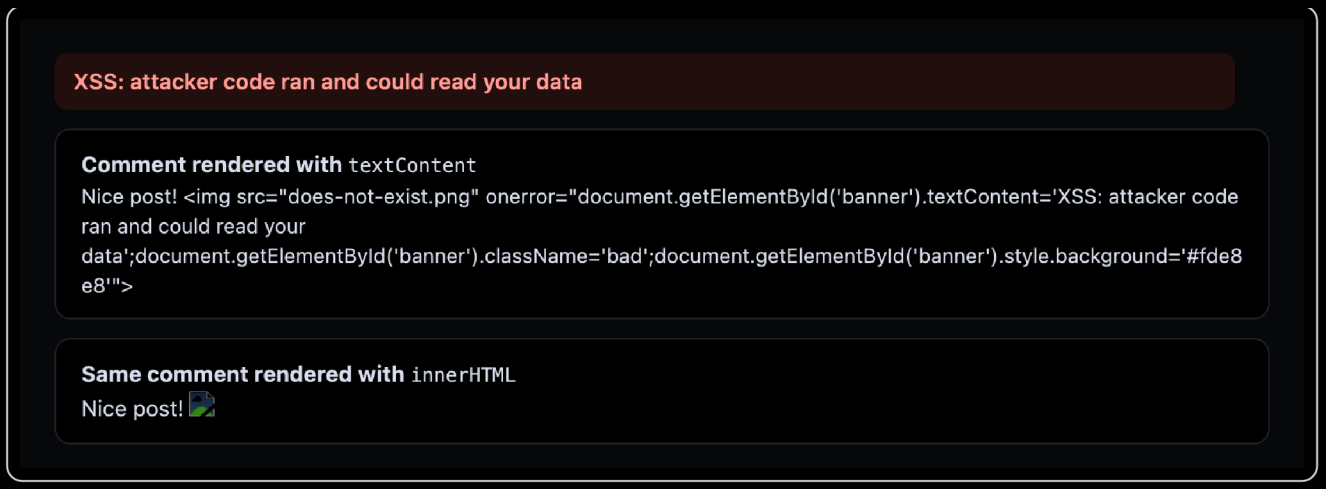


Figure — one line of `innerHTML` let the attacker's code take over the page (headless Chrome).

`code/08_xss.mjs` takes the payload apart in jsdom:

```
box.innerHTML = comment;
console.log('1) innerHTML created <img> with onerror handler:', img?.getAttribute('onerror'));
box.textContent = comment;
const escapeHtml = (s) => String(s).replace(/[&<>"']/g, (ch) => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;' })[ch]);
box.innerHTML = `<p class="comment">${escapeHtml(comment)}</p>`;
const website = 'javascript:alert(document.cookie)';
a.href = escapeHtml(website);                        // escaping does nothing useful here
```

Output:

```
1) innerHTML created <img> with onerror handler: stealCookies()
   <script> inserted via innerHTML exists but never runs: true (the <img onerror> is what attackers use instead)
2) textContent -> elements created: 0 | shown literally: Nice post! <img src="x...
3) escaped: &lt;img src=x onerror=&quot;go()&quot;&gt;
   elements inside <p>: 0
4) href after escaping : javascript:alert(document.cookie) <- still runs script when clicked
   href after allow-list: #
5) sinks to avoid with untrusted data:
   - el.innerHTML = s
   - el.outerHTML = s
   - el.insertAdjacentHTML(pos, s)
   - document.write(s)
   - eval(s) / new Function(s)
   - setTimeout('code string')
   - el.setAttribute("on...", s)
   - a.href = s (javascript:)
```

How to prevent it (in priority order)

1. **Treat external data as text**: `textContent`, `setAttribute` for non-event attributes, `createElement` (§13.3). Frameworks do this by default (React escapes `{value}` in JSX); the danger is escape hatches (`dangerouslySetInnerHTML`, Vue's `v-html`).
2. **Escape for the context** (`<` → `<` etc.) when building HTML strings on the server: HTML body, attribute, URL and JavaScript contexts each need different encoding (item 4: HTML-escaping doesn't make a `javascript:` URL safe). Use an auto-escaping template engine.
3. **Validate URLs** by parsing and allow-listing `http:` / `https:` (§14.6).
4. **Sanitize** (strip every dangerous part) **when you must render user HTML** (rich text, Markdown output) with a maintained sanitizer such as **DOMPurify** (`el.innerHTML = DOMPurify.sanitize(html)`), which removes scripts, event-handler attributes and dangerous URLs. The standard **HTML Sanitizer API** (`el.setHTML()`) still has limited browser availability (per MDN), so use it only with a fallback. Never write a regex sanitizer.
5. **Defence in depth**: a strict **CSP** (§14.12) so injected code doesn't run; `HttpOnly` cookies so sessions can't be stolen; **Trusted Types** (a CSP feature making dangerous sinks reject plain strings; check

support).

In the interview

> **30-second answer:** "XSS is attacker-controlled script running in my origin — stored, reflected or DOM-based. The root cause is untrusted data reaching a sink that parses HTML or code, like `innerHTML`, `eval` or a `javascript:` URL. I prevent it by rendering data as text (`textContent`, framework escaping), context-aware escaping on the server, URL protocol allow-lists, DOMPurify when HTML is truly needed, and a strict CSP plus HttpOnly cookies as backup."

- *Q: Does a `<script>` injected with `innerHTML` run?* A: No — but `<img onerror>`, `<svg onload>` and similar attributes do, so that's no protection.
- *Q: Is React immune?* A: Mostly for text; not for `dangerouslySetInnerHTML`, `href={userUrl}` with `javascript:` (React warns; newer versions block it), or injected server-rendered HTML.

TL;DR

XSS = untrusted data parsed as code. Render as text, escape by context, allow-list URL protocols, DOMPurify, plus CSP and HttpOnly.

14.11 Cross-site request forgery (CSRF)

What it is

CSRF tricks a logged-in user's browser into sending an unintended request — e.g. a hidden form on `evil.example` auto-submitting `POST https://bank.example/transfer`. Unless `SameSite` protects the cookie, the browser attaches the bank's cookies, so the bank sees a valid session. The attacker can't **read** the response (SOP), but a state-changing request doesn't need that.

How it's prevented

Defence	How it helps
`SameSite` cookies (`Lax` or `Strict`)	Cookie not sent on cross-site sub-requests and form POSTs. `Lax` still sends on top-level GET navigations (clicking a link), so **GET must never change state**
CSRF token (synchronizer token)	Server gives the page a random secret; state-changing requests must echo it in a header or field. `evil.example` can't read your page, so can't know it
Check `Origin` / `Sec-Fetch-Site` headers	Reject unexpected origins on state-changing requests
Require a custom header or JSON `Content-Type`	Forces a CORS preflight for cross-origin requests, which your server won't approve
Re-authentication	For critical actions (password change, large transfer)

Use at least two: `SameSite` + token (or origin check). `code/09_csrf_cookies_headers.mjs` implements session cookie + token on a local server; Node's `fetch` has no cookie jar, so the script sends the `Cookie` header by hand:

```
if (req.method === 'POST' && req.url === '/login') {
  const sid = randomBytes(16).toString('hex');
  const csrf = randomBytes(16).toString('hex');
  sessions.set(sid, { user: 'ada', csrf });
  return reply(200, { csrf }, {                          // token goes to the page, cookie to the browser
    'Set-Cookie': `sid=${sid}; HttpOnly; Secure; SameSite=Lax; Path=/; Max-Age=3600`,
  });
}
if (req.method === 'POST' && req.url === '/transfer') {
  if (!session) return reply(401, { error: 'not logged in' });
  if (!same(req.headers['x-csrf-token'], session.csrf)) return reply(403, { error: 'CSRF token missing or wrong'
});
  return reply(200, { ok: true, by: session.user });
}
// ...
const forged = await fetch(`${BASE}/transfer`, { method: 'POST', headers: { Cookie: cookie } });
const legit = await fetch(`${BASE}/transfer`, { method: 'POST', headers: { Cookie: cookie, 'X-CSRF-Token': csrf }
});
const noCookie = await fetch(`${BASE}/transfer`, { method: 'POST', headers: { 'X-CSRF-Token': csrf } });
```

Output:

```
1) Set-Cookie: sid=<random>; HttpOnly; Secure; SameSite=Lax; Path=/; Max-Age=3600
   CSP       : default-src 'self'; script-src 'self'; object-src 'none'; base-uri 'self'; frame-ancestors 'none'
2) forged request  -> 403 CSRF token missing or wrong
3) legit request   -> 200 { ok: true, by: 'ada' }
4) without cookie  -> 401 not logged in
```

Line 2: a forged request still carrying the cookie (as with `SameSite=None` or an old browser) is stopped by the missing token. Line 4 is what `SameSite=Lax` does to a cross-site POST in a modern browser: no cookie, no session. The token is compared with `crypto.timingSafeEqual` so timing doesn't leak how many characters matched.

Common mistakes

- State changes on `GET` (`/logout`, `/delete?id=5`) — an `<img src>` can trigger them.
- Believing CORS prevents CSRF — the forged request is still sent (§14.2's `/private`).
- Session token in `localStorage` + `Authorization` header avoids CSRF but exposes the token to any XSS — a trade-off. `HttpOnly` cookie + `SameSite` + CSRF token is the common recommendation (OWASP CSRF Prevention Cheat Sheet).

TL;DR

CSRF abuses auto-sent cookies. Defend with `SameSite` cookies, CSRF tokens (or origin checks), and never change state on GET.

14.12 Content Security Policy (CSP)

What it is

CSP is an HTTP response header (`Content-Security-Policy`) telling the browser **which sources of scripts, styles, images, frames etc. are allowed**; anything else is blocked and reported. It's **defence in depth**: if an XSS bug injects markup, CSP stops the code running.

Directive	Controls	Example
`default-src`	Fallback for all fetch directives	`'self'`
`script-src`	Scripts (`'unsafe-inline'` re-allows inline code — avoid; a *nonce* is a random one-time token)	`'self'` or `'nonce-r4nd0m'`
`script-src-attr`	Inline event-handler attributes (`onclick=…`)	falls back to `script-src`
`style-src` , `img-src` , `connect-src` (fetch/WebSocket — a persistent two-way connection), `font-src` , `frame-src`	Each resource type	`connect-src 'self' https://api.example.com`
`object-src 'none'`	Plugins (`<object>` , `<embed>`)	Always `'none'`
`base-uri 'self'`	Stops `<base>` hijacking relative URLs	
`frame-ancestors 'none'`	Who may embed you — **clickjacking** protection (hiding your page in an invisible frame to trick clicks); supersedes `X-Frame-Options`	
`report-to` / `report-uri`	Where to send violation reports	

Seen in a real browser

`code/csp.html` is served with `Content-Security-Policy: default-src 'self'; script-src 'self'; img-src 'self'; style-src 'self' 'unsafe-inline'`. It has an inline `<script>` that would replace a message, and loads `code/csp_app.js`, which injects an attacker comment via `innerHTML` and logs `securitypolicyviolation` events:

```
// csp_app.js (allowed: same origin)
document.addEventListener('securitypolicyviolation', (e) => {
  log.textContent += `blocked ${e.effectiveDirective.padEnd(15)} ${e.blockedURI || '(inline)'}\n`;
});
document.getElementById('comment').innerHTML =
  'Great article! <img src="https://evil.example/pixel.png" onerror="document.body.style.background=\'red\'">';
```



Figure — the injection that took over §14.10's page is neutralized: the external image and the inline `onerror` were both blocked. (The inline-script violation happened before the listener was registered; it's in the DevTools console.)

How to adopt CSP

1. Move inline scripts and `on…=` attributes into files and `addEventListener` (§13.5).

2. Start with `Content-Security-Policy-Report-Only` to collect violations without breaking the site.
3. For needed inline scripts, use a per-response random **nonce** (`script-src 'nonce-…'`); with `'strict-dynamic'`, scripts loaded by trusted scripts are allowed too. Google recommends this "strict CSP" over long domain allow-lists, which are often bypassable.
4. Enforce, keep reporting, review violations.

Other security headers worth sending

Header	Purpose
`Strict-Transport-Security: max-age=31536000; includeSubDomains`	HTTPS only (HSTS: HTTP Strict Transport Security)
`X-Content-Type-Options: nosniff`	Don't guess MIME types (format labels like `text/html`); stops some script injection via uploads
`Referrer-Policy: strict-origin-when-cross-origin`	Don't leak full URLs (the referrer) to other sites (also today's browser default)
`Permissions-Policy: camera=(), geolocation=()`	Turn off powerful features you don't use
`Cross-Origin-Opener-Policy: same-origin`	Isolate your window from cross-origin popups

Security checklist for front-end code

Risk	Rule
XSS	Data as text; escape by context; DOMPurify for HTML; allow-list URL protocols; strict CSP
Session theft	`HttpOnly; Secure; SameSite` cookies; never tokens in URLs
CSRF	SameSite + CSRF token or origin check; no state change on GET
Secrets	No API keys in front-end bundles
Dependencies	`npm audit` (known-vulnerability scan), lockfiles, few well-maintained packages (Chapter 10); Subresource Integrity (`integrity=` hash the browser checks) for third-party CDN scripts
Clickjacking	`frame-ancestors 'none'` or `'self'`
Validation	Client-side is UX; the server validates everything
Prototype pollution (attacker data adding properties to `Object.prototype`, affecting every object)	Don't deep-merge untrusted JSON into objects (Chapter 9)

TL;DR

CSP limits where scripts/styles/images may come from and blocks inline code, so injected markup can't run. Roll out report-only, prefer nonces + `'strict-dynamic'`, add HSTS, `nosniff`, `frame-ancestors`.

Chapter 14 TL;DR

- **fetch**: resolves on any HTTP status (check `res.ok`), rejects on network failure/abort; body read once; JSON needs `Content-Type` + `JSON.stringify`; time out with `AbortSignal.timeout`.
- **SOP & CORS**: origin = scheme + host + port; send anywhere, read only same-origin unless `Access-Control-Allow-Origin`; non-simple requests get an `OPTIONS` preflight; browsers enforce.
- **Storage**: cookies (tiny, auto-sent; `HttpOnly; Secure; SameSite` for sessions), `localStorage` (persistent strings, ~5 MB, sync, never secrets), `sessionStorage` (per tab), IndexedDB (async database, structured data, large quota).

- **rAF** for visual updates: once per frame, time-based, paused in background tabs.
- **URL/URLSearchParams** for parsing, building, encoding, validating URLs.
- **History API**: `pushState`/`replaceState` + `popstate` = SPA routing; server fallback to `index.html`.
- **Web Workers**: parallel threads without DOM; `postMessage` with structured clone.
- **Observers**: `IntersectionObserver` (visibility, lazy loading), `ResizeObserver` (element size).
- **XSS**: data as text, context-aware escaping, URL allow-lists, DOMPurify, CSP, HttpOnly.
- **CSRF**: SameSite cookies + tokens/origin checks; no state changes on GET; CORS isn't a CSRF defence.
- **CSP** and security headers are defence in depth.

Next: JavaScript on the server — Chapter 15: Node.js Basics.

Chapter 15 — Node.js Basics

Node.js takes JavaScript out of the browser and onto servers, laptops and build machines — every `npm install` (npm = Node's package manager), bundler run and `node --test` in this book already used it. This chapter covers what Node adds to the language and how to use its core modules (built into Node) for real work: files, paths, configuration, events, streams, HTTP and child processes.

What you'll learn

1. What Node.js is: V8 + libuv + core modules; how it differs from a browser.
2. The event loop, the libuv thread pool, blocking, and worker threads.
3. Modules in Node in one page (details in Chapter 10).
4. Files with `node:fs/promises`; paths and file URLs with `node:path` / `node:url`.
5. `process`: arguments, environment variables, `.env` files, exit codes, signals.
6. `Buffer` — raw bytes.
7. `EventEmitter` — the event pattern behind most Node APIs.
8. Streams: readable, writable, transform, `pipeline`, backpressure.
9. An HTTP server with `node:http`, called from the same script.
10. Child processes and npm scripts (`node --run`).
11. Node vs Deno vs Bun; everyday command-line flags.

Before you start. Promises and `async`/`await` (Chapter 12) — almost every API here is awaited. Event-loop phases, `process.nextTick` and `setImmediate`: §12.4; module systems, `package.json`, npm: Chapter 10; the Node inspector: §11.9.

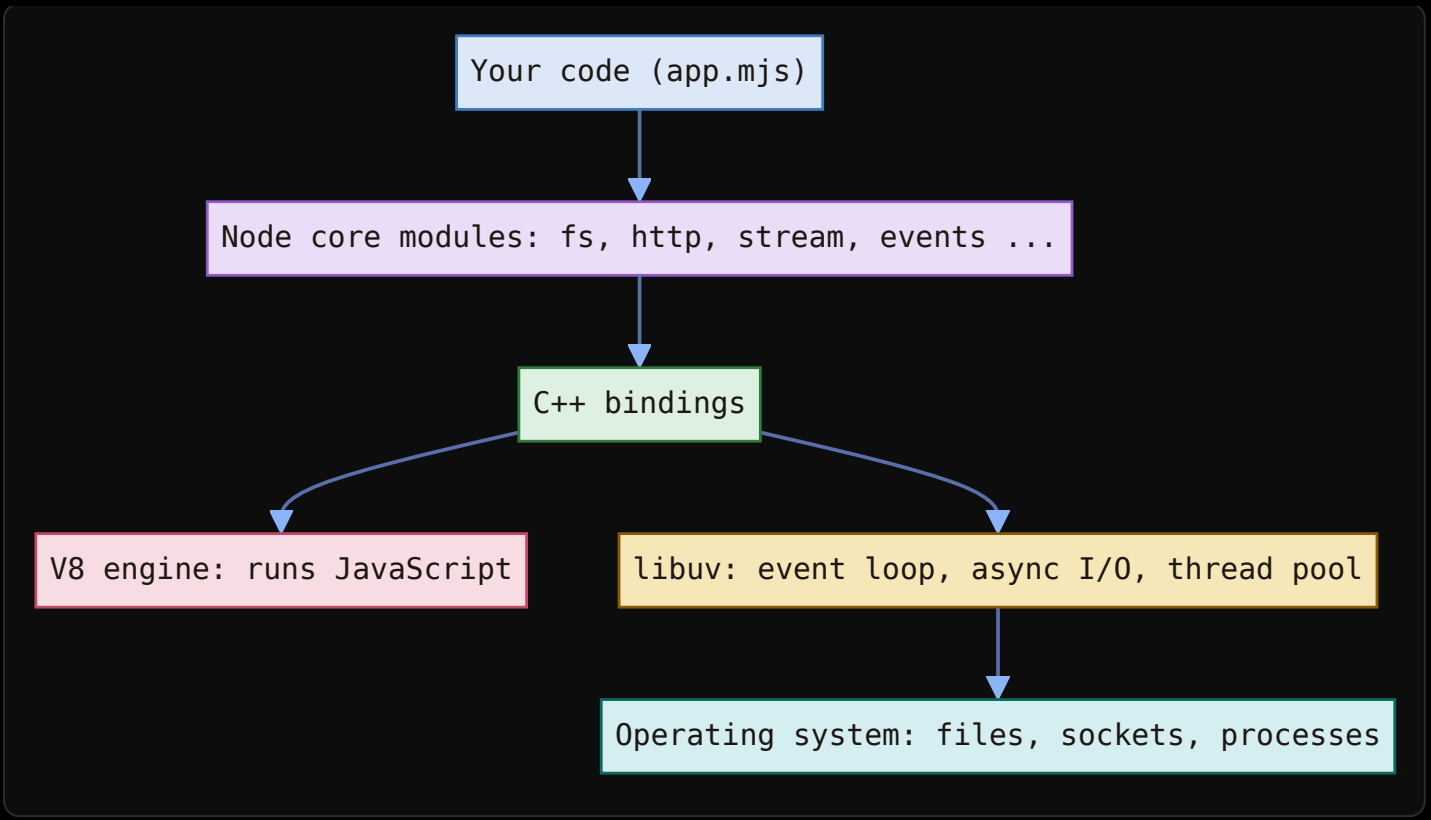
All runnable code is in `code/`, run with **Node 24.11.1**; outputs are real. Disk programs use a fresh `os.tmpdir()` folder and delete it; the HTTP example listens only on `127.0.0.1` on an OS-chosen free port.

15.1 What Node.js is

What it is

Node.js is a **JavaScript runtime** — a program that runs JavaScript outside a browser — created by Ryan Dahl in 2009. Three parts:

Part	What it does	Written in
V8	Google's JavaScript engine (same as Chrome) — parses, compiles, runs your JS (§1.3)	C++
libuv	Gives Node its **event loop**, async file and network I/O, timers, a **thread pool** (fixed helper threads for work that would otherwise block) and child processes — on Linux, macOS and Windows alike	C
Core modules + bindings	The APIs you import (`node:fs`, `node:http`, `node:stream` …) and the C++ glue ("bindings") connecting them to V8 and libuv	JS + C++



Why it matters

- **One language on both sides** — front-end and back-end share code, types and validation.
- **Great at I/O-heavy work** — one process juggles thousands of connections because it never waits idly on a disk or socket; it registers a callback and moves on (Chapter 12).
- **It runs the JS tool chain** — npm, Vite, ESLint, Prettier and the JavaScript-based TypeScript 5.x compiler all run on Node, even for front-end projects.

Node vs the browser

Same language (ECMAScript), different **host APIs**:

	Browser	Node.js
Global object	`window` (also `globalThis`)	`global` (also `globalThis`)
DOM (`document`, elements)	Yes	No (jsdom for tests, Chapter 13)
File system, processes, raw sockets	No (sandboxed)	Yes (`node:fs`, `node:child_process`, `node:net`)
Security model	Sandbox + same-origin policy	Full access of the user who started it (optional permission model, §15.12)
Modules	ES modules (`import`/`export`)	ES modules **and** CommonJS (Node's original `require` system)
Shared web APIs	`fetch`, `URL`, `AbortController`, `TextEncoder`, `structuredClone`, `crypto.subtle`, `Blob`, `WebSocket`, timers	The same — Node implements them too
You control the version?	No — the user's browser decides	Yes — so you can use every feature your version supports without transpiling

`code/01_node_runtime.mjs` checks these differences:

```js
import { builtinModules } from 'node:module';

console.log('typeof window   :', typeof window);        // no DOM in Node
console.log('typeof process  :', typeof process);       // Node-only global
console.log('globalThis === global:', globalThis === global);
console.log('has V8 & libuv  :', 'v8' in process.versions, 'uv' in process.versions);
const webGlobals = ['fetch', 'URL', 'AbortController', 'TextEncoder', 'structuredClone',
  'setTimeout', 'queueMicrotask', 'crypto', 'Blob', 'WebSocket'];
console.log('web APIs present:', webGlobals.filter((n) => n in globalThis).length, 'of', webGlobals.length);
console.log('this file       :', import.meta.filename.split('/').pop());
console.log('typeof __dirname:', typeof __dirname);
// ...
```

Output:

```
typeof window   : undefined
typeof document : undefined
typeof process  : object
globalThis === global: true
node major      : 24
has V8 & libuv  : true true
web APIs present: 10 of 10
this file       : 01_node_runtime.mjs
dirname ends    : 15-nodejs-basics/code
typeof __dirname: undefined
built-ins exist : true
built-in count >50: true
```

`__dirname` exists only in CommonJS; ES modules use `import.meta.dirname` / `import.meta.filename` (Node 20.11+; §10.6).

Versions and releases

- Up to Node 26, a new **major version** (first number of `24.11.1`; may break code) ships every six months (April and October). **Even-numbered** majors become **LTS** (Long-Term Support) that October with ~30 months of fixes; odd majors are short-lived "Current" releases.
- **From Node 27 (2027)**: **one major per year** (April), and **every** major becomes LTS each October.
- **Node 24**: released May 2025, LTS October 2025, supported until April 2028 (V8 13.6, npm 11). Production should run an **Active or Maintenance LTS**.

`node --version` (`node -v`) shows yours; **nvm**, **fnm** or **Volta** install several versions and switch per project (`.nvmrc` pins one).

Real-world examples

Netflix, PayPal, LinkedIn and Walmart have written about moving web servers or API layers to Node. Typical uses: REST/GraphQL APIs (REST = URLs + HTTP methods; GraphQL = one endpoint with a query language), backend-for-frontend servers (shaped around one UI), server-side rendering for React (Next.js, React Handbook ch13), real-time apps (chat, WebSockets), CLIs and build tooling.

When not to use it

Long CPU-bound work (video encoding, heavy numerics, ML training) is a poor fit for the main thread — Node shines when work *waits* on I/O. Use worker threads (§15.2), a native module (compiled C/C++ or Rust add-on), or a language built for it (Go, Rust, Java, Python with NumPy).

TL;DR

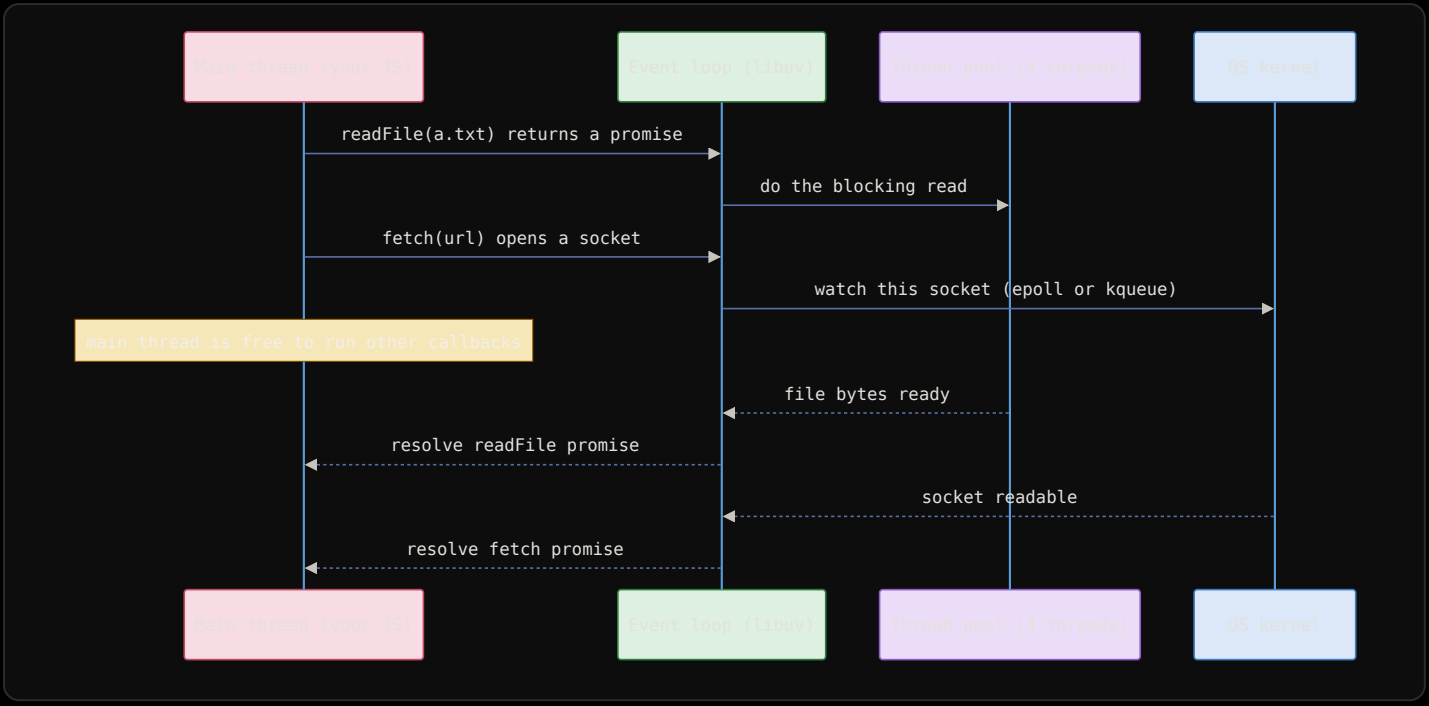
Node = V8 + libuv (event loop, async I/O, thread pool) + core modules. No DOM, but files, processes, sockets. Run LTS in production (Node 24 today; Node 26 becomes LTS in October 2026).

15.2 The event loop, the thread pool and blocking

What it is

Your JavaScript runs on **one main thread**; libuv's **event loop** repeatedly picks the next ready callback (expired timer, socket with data, finished file read) and runs it there. Phases and the `nextTick`/microtask queues: §12.4. This section adds **how the I/O gets done** while the thread is free:

1. **Network I/O** (TCP, HTTP, pipes) uses the OS's async notification mechanism — **epoll** (Linux), **kqueue** (macOS/BSD), **IOCP** (Windows). No extra threads: the OS says "socket has data" and libuv queues your callback.
2. **Work the OS can't do asynchronously** goes to the **libuv thread pool**: most `fs` operations, `dns.lookup()`, CPU-heavy `crypto` (`pbkdf2`, `scrypt` — deliberately slow password hashing — and `randomBytes`), and `zlib`. **4 threads by default**; set the environment variable `UV_THREADPOOL_SIZE` (up to 1024) *before* Node starts.
3. A finished pool job is queued on the loop; your callback/promise continuation runs on the **main thread**.



Why it matters — blocking the loop

Every callback shares one thread, so **one slow synchronous piece delays everything**: other requests, timers, health checks (a monitor's "are you alive?" requests). A 200 ms `JSON.parse` makes every other user wait 200 ms. Culprits:

- `*Sync` APIs (`readFileSync`, `execSync`) in request handlers;
- huge `JSON.parse` / `JSON.stringify`;
- regexes with catastrophic backtracking (exponential time on some inputs — "ReDoS", regular-expression denial of service);
- tight loops over large arrays; synchronous crypto (`pbkdf2Sync`).

Worker threads

`node:worker_threads` runs JavaScript on extra threads, each with its own V8 instance and event loop, communicating by **message passing** (`postMessage`, structured clone) or a shared `SharedArrayBuffer`. For **CPU-bound** work only; I/O is already async.

`code/02_blocking_and_workers.mjs`:

```
function busyFor(ms) {                  // simulate CPU-bound work
  const end = Date.now() + ms;
  while (Date.now() < end) {}
}

// 1) A 10 ms timer cannot fire while the main thread is busy for 200 ms.
const start = Date.now();
await new Promise((resolve) => {
  setTimeout(() => {
    console.log('timer asked for 10 ms, fired after >= 200 ms:', Date.now() - start >= 200);
    resolve();
  }, 10);
  busyFor(200);
});

// 2) Move the CPU work to a worker thread; the main thread stays responsive.
const workerCode = `
  const { parentPort, workerData } = require('node:worker_threads');
  const fib = (n) => (n < 2 ? n : fib(n - 1) + fib(n - 2));
  parentPort.postMessage(fib(workerData));
`;
let ticks = 0;
const ticker = setInterval(() => ticks++, 5);          // proves the loop is free
const result = await new Promise((resolve, reject) => {
  const w = new Worker(workerCode, { eval: true, workerData: 32 });
  w.once('message', resolve);
  w.once('error', reject);
});
clearInterval(ticker);
console.log('fib(32) from worker:', result);
console.log('main thread kept ticking meanwhile:', ticks > 0);
```

Output:

```
timer asked for 10 ms, fired after >= 200 ms: true
fib(32) from worker: 2178309
main thread kept ticking meanwhile: true
```

(`eval: true` keeps the demo's worker code in a string; real projects use `new Worker(new URL('./worker.mjs', import.meta.url))`. Workers cost milliseconds and memory to start, so apps keep a **pool** — e.g. Piscina.)

Scaling options compared

Tool	What it gives you	Shares memory?	Use for
Async I/O (default)	Thousands of concurrent I/O operations on one thread	—	Almost all server work
`worker_threads`	Extra JS threads in the same process	Only `SharedArrayBuffer`	CPU-heavy tasks (image resize, parsing, hashing)
`child_process` (§15.11)	Separate OS processes, any program	No	Running other programs, isolation
`cluster` / multiple processes	Several Node processes sharing one port	No	Using all CPU cores for an HTTP server (often via a process manager, Kubernetes, or a load balancer instead — System Design Fundamentals)

Common mistakes

- "Single-threaded" ≠ "one thing at a time": JS is one thread, but I/O proceeds in parallel in the OS and the pool.
- `*Sync` functions in request handlers (fine in CLIs and startup code).

- Expecting workers to speed up I/O (`fetch`, DB queries) — only overhead.
- Saturating the 4-thread pool with slow `fs` / `pbkdf2` calls, so unrelated `fs` calls queue behind them.

In the interview

Q: Is Node single-threaded? "Your JavaScript runs on a single main thread with an event loop. Network I/O is handled asynchronously by the OS (epoll/kqueue/IOCP), and file system, DNS lookup, some crypto and zlib work runs on libuv's thread pool (4 threads by default). CPU-heavy JS can be moved to worker threads."

- *Find what blocks the loop?* Measure event-loop delay (`perf_hooks.monitorEventLoopDelay()`), then profile with `node --cpu-prof` or the inspector (§11.9).
- *`readFile` slow under load with idle CPU?* Pool saturated; raise `UV_THREADPOOL_SIZE` or reduce concurrent blocking work.
- *Worker thread vs child process?* Worker: same process, cheaper, can share memory, JS only. Child: separate process, any executable, full isolation, more overhead.
- *Which use the pool: `fs.readFile`, an HTTP request, `crypto.pbkdf2`, `setTimeout`?* `fs.readFile` and `crypto.pbkdf2`; HTTP uses OS async I/O; timers are managed by the loop.
- *A handler runs a 300 ms sync loop — a second request meanwhile?* Waits; its latency grows by up to 300 ms.

TL;DR

One JS thread + event loop; OS does network I/O; libuv's pool (default 4) does files, DNS lookup, crypto, zlib. Never block the main thread in a server — move CPU work to worker threads (input via `workerData` or `postMessage`).

15.3 Modules in Node — the short version

Module syntax and loading: Chapter 10. What you need while writing Node code:

Question	Answer
Is my `.js` file ESM or CommonJS?	`.mjs` → ESM, `.cjs` → CommonJS; `.js` follows the nearest `package.json` `"type"` (`"module"` → ESM). Without `"type"`, Node detects ESM syntax automatically (Node 22.7+). §10.8
Importing a built-in?	Prefer the **`node:` prefix** (`import { readFile } from 'node:fs/promises'`): unambiguous (no npm package can shadow it) and required for newer modules like `node:test`, `node:sqlite`.
Where is `__dirname`?	CommonJS only. ESM: `import.meta.dirname` / `import.meta.filename`.
Can ESM use a CommonJS package?	Yes: `import pkg from 'cjs-pkg'` (default import = `module.exports`).
Can CommonJS `require()` an ES module?	Yes in Node 22.12+ / 20.19+, unless it uses top-level `await`.
Loading JSON?	`import data from './x.json' with { type: 'json' }`, or read + `JSON.parse`.

Most core modules offer several API styles; use the promise one in new code:

Module	Promise API	Callback API	Sync API
File system	`node:fs/promises`	`node:fs` (`fs.readFile(p, cb)`)	`node:fs` (`fs.readFileSync`)
Streams helpers	`node:stream/promises`	`node:stream`	—
Timers	`node:timers/promises` (`await setTimeout(100)`)	global `setTimeout`	—
DNS	`node:dns/promises`	`node:dns`	—
Child processes	`util.promisify(execFile)`	`node:child_process`	`execFileSync`

`util.promisify(fn)` turns any Node-style callback function (`(err, value) => …` last) into a promise-returning one.

15.4 Files with `node:fs/promises`

What it is

`node:fs` reads, writes, lists, copies, moves and deletes files/folders and inspects metadata; **`node:fs/promises`** is the same returning promises.

How it works — the core operations

Task	Function	Notes
Read a whole file	`readFile(path, 'utf8')`	Without an encoding you get a `Buffer` (§15.7)
Write (create/replace)	`writeFile(path, data)`	`{ flag: 'wx' }` = fail if it exists
Append	`appendFile(path, data)`	Creates the file if missing
Create folders	`mkdir(path, { recursive: true })`	Like `mkdir -p`; no error if it exists
List a folder	`readdir(path, { withFileTypes: true })`	`Dirent`s with `isFile()`/`isDirectory()`; `{ recursive: true }` walks subfolders (Node 20.1+)
Match patterns	`glob('**/*.txt', { cwd })` (wildcard pattern; `cwd` = folder to search from)	Async iterator (Node 22+)
Metadata	`stat(path)`	size, `mtime` (last modified), `isFile()`, `isDirectory()`
Copy / move	`copyFile(a, b)`, `rename(a, b)`, `cp(a, b, { recursive: true })`	`rename` across disks may fail — copy then delete
Delete	`rm(path, { recursive: true, force: true })`	Like `rm -rf`; `unlink` for one file
Temp folder	`mkdtemp(join(tmpdir(), 'prefix-'))`	Unique name each call
Low-level	`open(path, flags)` → `FileHandle`	`read`, `write`, `close`; close in `finally`

`code/03_fs_promises.mjs` (key parts):

```
import { mkdtemp, writeFile, appendFile, readFile, readdir, stat, rename,
  copyFile, rm, open, mkdir, access, constants, glob } from 'node:fs/promises';
import { tmpdir } from 'node:os';
import { join } from 'node:path';

const dir = await mkdtemp(join(tmpdir(), 'notes-'));   // unique temp folder
try {
  const file = join(dir, 'todo.txt');
  await writeFile(file, 'buy milk\n');
  await appendFile(file, 'write chapter 15\n');
  const text = await readFile(file, 'utf8');
  console.log('read:', JSON.stringify(text));

  const buf = await readFile(file);
  console.log('as Buffer:', buf.constructor.name, buf.length, 'bytes, first byte', buf[0]);
  // ... write config.json, JSON.parse it back

  await mkdir(join(dir, 'archive/2026'), { recursive: true });
  await copyFile(file, join(dir, 'archive/2026/todo-copy.txt'));
  await rename(join(dir, 'config.json'), join(dir, 'settings.json'));

  const entries = await readdir(dir, { withFileTypes: true });
  console.log('entries:', entries.map((e) => `${e.name}${e.isDirectory() ? '/' : ''}`).sort().join(', '));
  const all = await readdir(dir, { recursive: true });
  console.log('recursive:', all.sort().join(', '));
  const txt = [];
  for await (const p of glob('**/*.txt', { cwd: dir })) txt.push(p);
  console.log('glob *.txt:', txt.sort().join(', '));

  // ... stat(file); open(file, 'r') -> FileHandle, read 8 bytes, close() in finally

  try { await readFile(join(dir, 'nope.txt'), 'utf8'); }
  catch (err) { console.log('error code:', err.code, '| syscall:', err.syscall); }
  // ... access() and the 'wx' flag
} finally {
  await rm(dir, { recursive: true, force: true });
  console.log('cleaned up');
}
```

Output:

```
read: "buy milk\nwrite chapter 15\n"
as Buffer: Buffer 26 bytes, first byte 98
config.port: 8080
entries: archive/, settings.json, todo.txt
recursive: archive, archive/2026, archive/2026/todo-copy.txt, settings.json, todo.txt
glob *.txt: archive/2026/todo-copy.txt, todo.txt
stat: isFile true | size 26 | mtime is Date true
first 8 bytes: "buy milk"
error code: ENOENT | syscall: open
exists & readable: true
wx on existing file -> EEXIST
cleaned up
```

- Without `'utf8'`, `readFile` returns a `Buffer`; byte 98 is `b`.
- FS errors are `Error`s with a string `code`: `ENOENT` (no such file), `EEXIST` (exists), `EACCES` / `EPERM` (permission), `EISDIR`, `ENOTDIR`, `EMFILE` (too many open files). Branch on `err.code`, never the message.
- `try … finally` removes the temp folder even on errors.

Common mistakes

- **Check-then-act races** (TOCTOU — time of check vs time of use): `if (await exists(f)) await readFile(f)` can still fail. Just try and handle `ENOENT`; use `'wx'` for "create only if new".
- **Relative paths resolve against** `process.cwd()` (where Node was launched), *not* the source file's folder. Use `import.meta.dirname` for "next to this file".
- **Reading huge files whole** — a 5 GB log will crash or stall. Use streams (§15.9).

- **`*Sync` calls in servers** (§15.2).
- **Writes are not atomic** (all-or-nothing): a crash mid-`writeFile` leaves a half-written file. Write a temp file in the same folder, then `rename` over the original (atomic on the same file system).
- **Unclosed `FileHandle`s** leak file descriptors (OS handles for open files/sockets) until `EMFILE`.

Real-world examples

Static-site generators read Markdown and write HTML; bundlers write `dist/`; loggers append; scaffolding CLIs copy template folders with `cp(..., { recursive: true })`.

TL;DR

`node:fs/promises` with `await`; `'utf8'` for strings; handle errors by `err.code`; paths from `import.meta.dirname`; no check-then-act; no `readFile` on huge files; clean up in `finally`.

15.5 Paths and file URLs: `node:path`, `node:url`

What it is

`node:path` builds and splits paths correctly for the OS (`/` on Linux/macOS, `\` on Windows). Concatenation (`dir + '/' + name`) breaks on Windows, doubles slashes and ignores `..`.

How it works

Function	Does	Example result
`join(...parts)`	Glue segments, normalize `..` and `.`	`join('/srv','app','..','data')` → `/srv/data`
`resolve(...parts)`	**Absolute** path, processed right to left until absolute; falls back to `process.cwd()`	`resolve('/srv/app','logs')` → `/srv/app/logs`
`basename(p, ext?)`	Last segment	`price.test.mjs`
`extname(p)`	Last extension only	`.mjs`
`dirname(p)`	Everything before the last segment	`/home/ana/.../utils`
`parse(p)` / `format(obj)`	Split into / build from `{ root, dir, base, name, ext }`	
`relative(from, to)`	Path from one location to another	`../public/logo.png`
`normalize(p)`	Clean up `//`, `.`, `..`	`/a/c/d`
`isAbsolute(p)`	Starts from the root?	
`sep`, `delimiter`	`/` or `\`; `:` or `;` (for `PATH`)	
`path.posix`, `path.win32`	Force one platform's rules	

ES modules identify files by **URL** (`file:///home/ana/a.txt`); `node:url`'s `fileURLToPath(url)` and `pathToFileURL(path)` handle percent-encoding (space → `%20`) and Windows drive letters. `new URL('./data.json', import.meta.url)` points at a sibling file; most `fs` functions accept URL objects.

`code/04_path_and_urls.mjs` (excerpt):

```
import path from 'node:path';
import { fileURLToPath, pathToFileURL } from 'node:url';

const p = '/home/ana/projects/shop/src/utils/price.test.mjs';
console.log('basename :', path.basename(p));
console.log('  no ext :', path.basename(p, '.mjs'));
console.log('extname  :', path.extname(p));
console.log('parse    :', path.parse(p));
console.log('join     :', path.join('/srv', 'app', '..', 'data', './users.json'));
console.log('resolve abs wins:', path.resolve('/srv/app', '/etc', 'hosts'));
console.log('relative :', path.relative('/srv/app/src', '/srv/app/public/logo.png'));
console.log('win32 join:', path.win32.join('C:\\Users', 'ana', 'file.txt'));
console.log('fileURLToPath:', fileURLToPath(new URL('file:///home/ana/my%20notes/a.txt')));

// Security: stop "../" escaping a base folder (path traversal)
function safeJoin(base, userPath) {
  const full = path.resolve(base, userPath);
  if (full !== base && !full.startsWith(base + path.sep)) throw new Error(`blocked: ${userPath}`);
  return full;
}
console.log('safe  :', safeJoin('/srv/uploads', 'cat.png'));
try { safeJoin('/srv/uploads', '../../etc/passwd'); } catch (e) { console.log('unsafe:', e.message); }
```

Output:

```
basename : price.test.mjs
  no ext : price.test
extname  : .mjs
dirname  : /home/ana/projects/shop/src/utils
parse    : {
  root: '/',
  dir: '/home/ana/projects/shop/src/utils',
  base: 'price.test.mjs',
  ext: '.mjs',
  name: 'price.test'
}
join     : /srv/data/users.json
...
resolve abs wins: /etc/hosts
relative : ../public/logo.png
...
win32 join: C:\Users\ana\file.txt
...
fileURLToPath: /home/ana/my notes/a.txt
...
safe  : /srv/uploads/cat.png
unsafe: blocked: ../../etc/passwd
```

Common mistakes

- `join` vs `resolve`: `join` just glues (may be relative); `resolve` is always absolute and **restarts at any absolute segment** — user input passed to `resolve` can jump anywhere.
- **Path traversal**: serving `join(publicDir, req.url)` lets `GET /../../etc/passwd` read system files. Resolve, then check the result is inside the base (the `+ path.sep` stops `/srv/uploads-evil` passing `startsWith('/srv/uploads')`).
- `extname('archive.tar.gz')` is `.gz`.
- `new URL(import.meta.url).pathname` as a path stays percent-encoded (`my%20notes`) and is wrong on Windows — use `fileURLToPath` or `import.meta.dirname`.

TL;DR

Never concatenate paths: `join` to combine, `resolve` for absolute, `basename/extname/dirname/parse` to split; `fileURLToPath` for module URLs; guard user paths against traversal.

15.6 `process`: arguments, environment and exit codes

What it is

`process` is the global describing the running program: `argv`, `env`, `cwd()`, standard streams (`stdin`, `stdout`, `stderr`), process id, exit code and lifecycle events.

Command-line arguments

`process.argv` = `[pathToNode, pathToScript, ...yourArgs]` — user args start at index 2. Beyond one or two positional values, use the built-in `util.parseArgs` (stable since Node 20):

```
import { parseArgs } from 'node:util';
const { values, positionals } = parseArgs({
  args: ['--port', '8080', '-v', 'build', 'src/'],   // omit args to read process.argv
  options: {
    port: { type: 'string', default: '3000' },
    verbose: { type: 'boolean', short: 'v' },
    env: { type: 'string', default: 'development' },
  },
  allowPositionals: true,
});
```

Unknown flags throw by default (`strict: true`). For sub-commands and help text, Commander or yargs.

Environment variables

Environment variables are key–value strings the OS passes to every process (`PATH`, `HOME`, your own `DATABASE_URL`) — the standard way to configure one codebase per environment (development, test, production) and inject secrets without committing them to Git (the Twelve-Factor App's "config in the environment" rule).

1. `process.env.X` is **always a string or** `undefined`; assigned numbers become strings. `"false"` and `"0"` are *truthy* — compare explicitly.
2. `.env` **files**: Node 20.6+ loads them — `node --env-file=.env app.mjs` (stable in 24.10+; `--env-file-if-exists` for optional files) or `process.loadEnvFile(path)`. Real environment variables win over the file. The older `dotenv` package does the same.
3. **Never commit** `.env` **with secrets**; commit `.env.example` and list `.env` in `.gitignore`.
4. `NODE_ENV=production` is a library convention for disabling dev-only checks, not special to Node.
5. Read and **validate** config once at startup; fail fast with a clear message.

`code/05_process_and_env.mjs` (excerpt):

```
process.env.MAX_ITEMS = 25;                         // assigned a number...
console.log('typeof MAX_ITEMS:', typeof process.env.MAX_ITEMS);  // ...stored as a string
const maxItems = Number.parseInt(process.env.MAX_ITEMS ?? '10', 10);
const debug = process.env.DEBUG === 'true';         // "false" is a truthy string!

writeFileSync(envFile, 'DB_HOST=localhost\nDB_PORT=5432\n# comment\nGREETING="hello world"\n');
process.loadEnvFile(envFile);

function loadConfig(env) {
  const port = Number(env.PORT ?? 3000);
  if (!Number.isInteger(port) || port <= 0) throw new Error(`Invalid PORT: ${env.PORT}`);
  return Object.freeze({ port, dbHost: env.DB_HOST ?? 'localhost', isProd: env.NODE_ENV === 'production' });
}

process.on('exit', (code) => console.log(`exit event, code ${code}`));
process.exitCode = 0;
```

Output:

```
argv[0] is node: true
user args      : []
parsed values  : [Object: null prototype] {
  port: '8080',
  verbose: true,
  env: 'development'
}
positionals    : [ 'build', 'src/' ]
typeof MAX_ITEMS: string
parsed number  : 26
DEBUG flag     : false | truthy "false"? true
from .env      : localhost 5432 hello world
config         : { port: 8080, dbHost: 'localhost', isProd: true }
config error   : Invalid PORT: abc
...
exit event, code 0
```

(`[Object: null prototype]`: `parseArgs` returns an `Object.create(null)` object, so a `--constructor` flag can't collide with inherited properties.)

Exit codes, signals and graceful shutdown

- A process ends when the loop has nothing left. **Exit code 0** = success; anything else = failure — shells, CI systems (servers that build and test every change) and `npm test` rely on it.
- Prefer **`process.exitCode = 1`** and let Node finish; **`process.exit(1)`** stops immediately and can cut off pending async work (e.g. pipe writes).
- An uncaught exception or unhandled rejection exits with code 1 (Node 15+ for rejections).
- **Signals** (OS notifications to a process): **Ctrl+C** sends `SIGINT`; orchestrators and process managers send **`SIGTERM`** before killing. Handle them:

```
// Not run - pattern for a long-running server (see server in §15.10)
process.once('SIGTERM', async () => {
  console.log('SIGTERM: stop accepting requests, finish in-flight ones');
  server.close();                   // stop new connections
  await db.end();                   // close pools, flush logs
  process.exitCode = 0;             // let the loop drain
  setTimeout(() => process.exit(1), 10_000).unref();   // hard deadline
});
```

`.unref()` = "don't keep the process alive just for this timer".

Other `process` members worth knowing

Member	Use
`process.cwd()` / `process.chdir()`	Current working directory
`process.platform`, `process.arch`	`'linux'`, `'darwin'`, `'win32'`; `'x64'`, `'arm64'`
`process.versions`	Versions of Node, V8, libuv, OpenSSL …
`process.stdin/stdout/stderr`	Standard streams (§15.9); `console.log` → `stdout`, `console.error` → `stderr`
`process.hrtime.bigint()`	Nanosecond timer (`performance.now()` also works)
`process.memoryUsage()`	Heap (JS objects) and RSS (all process memory) for diagnosing leaks
`process.on('uncaughtException' / 'unhandledRejection')`	Last-resort logging — log and exit (Chapter 11)

Common mistakes

- `if (process.env.FEATURE)` with `"false"` — true!
- Using `process.env.PORT` as a number without converting.

- `process.exit()` right after `console.log` to a pipe or file — output may be lost.
- Logging all of `process.env` (secrets).

TL;DR

`argv` (args from index 2; `util.parseArgs`), `env` (strings only; `--env-file`; validate at startup), `process.exitCode` for failure, handle `SIGTERM` / `SIGINT` for graceful shutdown.

15.7 `Buffer`: raw bytes

What it is

A `Buffer` is a fixed-size chunk of raw bytes — Node's subclass of `Uint8Array` (a typed array of bytes 0–255) with encoding helpers. You meet it whenever data isn't (yet) text: files read without an encoding, network packets, stream chunks, images, hashes.

An **encoding** turns text into bytes and back: `utf8` (default; 1–4 bytes per character), `base64` (bytes as 64 safe text characters), `hex` (two characters per byte), `latin1`, `utf16le`.

`code/10_buffers.mjs`:

```
const text = 'héllo 👋';
const buf = Buffer.from(text, 'utf8');
console.log('string length (UTF-16 units):', text.length);
console.log('byte length (UTF-8)          :', buf.length, '=', Buffer.byteLength(text));
console.log('bytes (hex)                  :', buf.toString('hex'));
console.log('base64                       :', buf.toString('base64'));
// ... decode base64 back to text; buf instanceof Uint8Array

const whole = Buffer.from('abcdef');
const part = whole.subarray(0, 3);
part[0] = 0x7a;                                    // 'z'
console.log('subarray shares memory       :', whole.toString());
console.log('cut mid-character            :', JSON.stringify(buf.subarray(0, 2).toString('utf8')));
const joined = Buffer.concat([Buffer.from('Node '), Buffer.from('24')]);
console.log('concat                       :', joined.toString(), '| equals:', joined.equals(Buffer.from('Node
24')));
console.log('alloc(4) is zero-filled      :', Buffer.alloc(4));
```

Output:

```
string length (UTF-16 units): 8
byte length (UTF-8)          : 11 = 11
bytes (hex)                  : 68c3a96c6c6f20f09f918b
base64                       : aMOpbGxvIPCfkYs=
back to text                 : héllo 👋
is a Uint8Array              : true
subarray shares memory       : zbcdef
cut mid-character            : "h\ufffd"
concat                       : Node 24 | equals: true
alloc(4) is zero-filled      : <Buffer 00 00 00 00>
```

`é` is 2 bytes and the emoji 4, so 8 string units become 11 bytes (§6.3). Cutting mid-`é` gave `\ufffd` — why stream code must not decode chunks separately (use `setEncoding('utf8')`, `TextDecoder` with `{ stream: true }`, or `Buffer.concat` first).

Common mistakes

- `str.length` as the `Content-Length` (body size in bytes) — use `Buffer.byteLength(str)`.
- `Buffer.allocUnsafe(n)` may contain old data; use `Buffer.alloc` unless you overwrite every byte immediately.
- Forgetting `subarray` (and older `slice`) share memory.
- Treating base64 as encryption — it's only an encoding.

TL;DR

Buffer = bytes (`Uint8Array`). `Buffer.from(str, enc)` / `buf.toString(enc)`. Byte length ≠ string length. Don't decode UTF-8 chunk by chunk.

15.8 Events: `EventEmitter`

What it is

`EventEmitter` (`node:events`) is Node's **observer / publish-subscribe** pattern: a list of **listeners** per **event name**; `emit(name, ...args)` calls each. Servers, sockets, streams, child processes and `process` are all EventEmitters — learn it once, understand most of Node. The browser equivalent is `EventTarget` + `addEventListener` (Chapter 13); Node supports `EventTarget` too, but core modules use `EventEmitter`.

How it works

Method	Meaning
`on(name, fn)` / `addListener`	Add a listener (appended)
`once(name, fn)`	Removed after its first call
`prependListener(name, fn)`	Add to the **front**
`off(name, fn)` / `removeListener`	Remove — needs the **same function reference**
`emit(name, ...args)`	Call listeners **synchronously, in order**; returns `true` if any existed
`listenerCount(name)`, `removeAllListeners(name)`	Inspect / clear
`events.once(emitter, name)`	Promise of the event's arguments array (rejects on `'error'`)
`events.on(emitter, name, { signal })`	Async iterator over repeated events

`code/06_event_emitter.mjs` (excerpt):

```
import { EventEmitter, once, on } from 'node:events';

const bus = new EventEmitter();
function onOrder(order) { console.log('  handler A got order', order.id); }
bus.on('order', onOrder);
bus.once('order', (o) => console.log('  once-handler fires only the first time', o.id));
bus.prependListener('order', () => console.log('  prepended handler runs first'));
console.log('emit #1 ->', bus.emit('order', { id: 1 }));
console.log('emit #2');
bus.emit('order', { id: 2 });
bus.off('order', onOrder);
console.log('listeners left:', bus.listenerCount('order'));

const e3 = new EventEmitter();
try { e3.emit('error', new Error('disk full')); }         // no 'error' listener -> throws
catch (err) { console.log('unhandled error event threw:', err.message); }
// ... a Kettle class extends EventEmitter; once(e4, 'ready'); for await ... on(e4, 'tick', { signal })
```

Output (first lines and the error case):

```
  prepended handler runs first
  handler A got order 1
  once-handler fires only the first time 1
emit #1 -> true
emit #2
  prepended handler runs first
  handler A got order 2
listeners left: 1
...
unhandled error event threw: disk full
```

1. Listeners printed **before** `emit #1 -> true` — `emit` is synchronous; asynchrony only happens if a listener does something async.
2. **Emitting `'error'` with no listener throws** (and crashes the process if uncaught). Always attach `'error'` listeners to emitters you create or receive (streams, sockets, child processes).
3. More than 10 listeners on one event prints `MaxListenersExceededWarning` — usually a per-request listener never removed (a leak). `setMaxListeners(n)` only if intentional.

Trade-offs

Pattern	Good for	Weak at
EventEmitter	Many events over time, many listeners, decoupling	Manual error handling; harder-to-follow control flow
Promise	One result, once	Repeated events
Async iterator (`events.on`, streams)	A sequence processed in order with `for await`	Fan-out (same data to many consumers)
Callback	Legacy APIs	Composition

When not to use it

For one "done or failed" result, return a promise. An app-wide emitter is a hidden global dependency; across processes/machines use a message broker (Kafka, RabbitMQ, Redis streams).

Common mistakes

- `off('x', () => …)` with a new arrow — different reference, nothing removed.
- Throwing in a listener propagates out of `emit()` to its caller.
- `async` listener rejections aren't caught by the emitter (`new EventEmitter({ captureRejections: true })` routes them to `'error'`).
- Missing `'error'` listeners.

In the interview

Q: What is EventEmitter and is `emit` async? "It's Node's observer pattern: `on` registers listeners per event name, `emit` calls them synchronously in registration order and returns whether any existed. Streams, HTTP servers and sockets extend it. An `'error'` event without a listener throws."

- *Wait for one event?* `const [arg] = await events.once(emitter, 'name')`.
- *Avoid listener leaks?* Remove when done (`off`, `once`, or `{ signal }` in `events.on`/`once`); watch for `MaxListenersExceededWarning`.
- *`console.log('A'); em.emit('x'); console.log('C')` with a listener printing `B`?* A, B, C.

TL;DR

`on` / `once` / `off` / `emit`; emit is synchronous and ordered; always handle `'error'`; `events.once` / `events.on` bridge to promises and `for await`.

15.9 Streams

What it is

A **stream** is data arriving (or leaving) **piece by piece over time**. Each piece is a **chunk** (a `Buffer` or string, or any value in **object mode**). Four types, all EventEmitters:

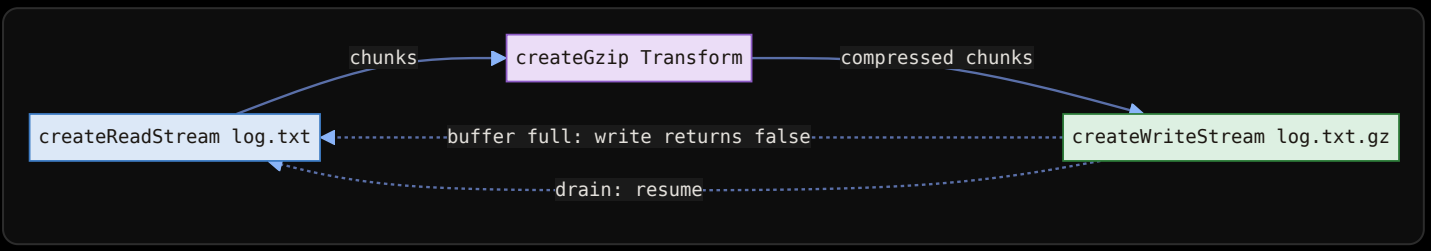
Type	Direction	Examples
Readable	Source	`fs.createReadStream`, HTTP request (server side), `process.stdin`, `Readable.from(iterable)`
Writable	Destination	`fs.createWriteStream`, HTTP response, `process.stdout`
Duplex	Both, independent sides	TCP socket
Transform	Duplex whose output is computed from input	`zlib.createGzip()`, `crypto` hashes/ciphers, your parsers

Why it matters

Memory (a 10 GB file needs only KBs of buffer), **time to first byte** (respond before all data exists), and **composition** of small stages (read → decompress → parse → write), like Unix pipes (`cmd1 | cmd2`).

How it works

1. A Readable fills an internal buffer; consumers take chunks (`for await`, `'data'` events, piping).
2. A Writable accepts `write(chunk)`. Buffers are limited by **`highWaterMark`**: 64 KiB (1 KiB = 1024 bytes) for byte streams since Node 22 (16 KiB before), 16 objects in object mode.
3. **Backpressure**: when a Writable's buffer passes `highWaterMark`, `write()` returns **`false`** — "slow down"; the producer should wait for **`'drain'`**. Ignoring it grows memory without limit when the source outpaces the destination (fast disk → slow client).
4. **`pipeline(src, ...transforms, dest)`** (`node:stream/promises`) connects stages, **handles backpressure**, **forwards errors from any stage**, **destroys all streams** on failure, and returns a promise. Prefer it over `src.pipe(dest)`, which neither propagates errors nor cleans up.
5. Readables are **async iterables**: `for await (const chunk of readable)` respects backpressure.



`code/07_streams.mjs` (key parts):

```
// imports: Readable, Writable, Transform (node:stream), pipeline (node:stream/promises),
// createReadStream/createWriteStream (node:fs), createGzip/createGunzip (node:zlib), createInterface
(node:readline)
// 2) A Transform (uppercase) and a Writable (collector)
const upper = () => new Transform({
  transform(chunk, _enc, done) { done(null, chunk.toString().toUpperCase()); },
});
function collector(out) {
  return new Writable({ write(chunk, _enc, done) { out.push(chunk.toString()); done(); } });
}
const out = [];
await pipeline(Readable.from(['hello ', 'streams']), upper(), collector(out));

// 3) Chunks do NOT align with lines: keep the incomplete tail between chunks
function splitLines() {
  let rest = '';
  return new Transform({
    readableObjectMode: true,
    transform(chunk, _enc, done) {
      const parts = (rest + chunk).split('\n');
      rest = parts.pop();
      for (const line of parts) this.push(line);
      done();
    },
    flush(done) { if (rest) this.push(rest); done(); },
  });
}

// 4) 50,000-line file -> gzip -> gunzip, constant memory
function* logLines() { for (let i = 1; i <= 50_000; i++) yield `line ${i} status=ok\n`; }
await pipeline(Readable.from(logLines()), createWriteStream(src));
await pipeline(createReadStream(src), createGzip(), createWriteStream(src + '.gz'));
await pipeline(createReadStream(src + '.gz'), createGunzip(), createWriteStream(src + '.copy'));

// 5) Line by line: for await (const line of createInterface({ input: createReadStream(src) })) ...
// 6) pipeline(createReadStream('missing.txt'), createGzip(), ...) -> the promise rejects

// 7) Backpressure by hand: write() -> false -> wait for 'drain'
const slow = new Writable({ highWaterMark: 16, write(_c, _e, done) { setTimeout(done, 1); } });
for (let i = 0; i < 5; i++) {
  const ok = slow.write('0123456789');
  if (!ok) { falses++; await new Promise((r) => slow.once('drain', r)); }
}
```

Output:

```
1) read chunks: [ 'a', 'b', 'c' ]
2) pipeline result: HELLO STREAMS
3) lines: [ 'id,name', '1,Ana', '2,Bob', '3,Cy' ]
4) original bytes: 1038894 | gzip smaller: true
   round trip identical: true
5) readline counted 50000 lines; last = "line 50000 status=ok"
6) pipeline rejected: ENOENT
7) write() returned false 2 times; waited for drain each time: true
```

- Part 3's chunks were `'…2,Bo'` and `'b\n3,Cy'` — `Bob` was split and stitched back. Chunk boundaries are arbitrary; never assume a chunk is a line or JSON object.
- The 1 MB file compressed to under a fifth, round-tripped byte-identically, never fully in memory.
- `ENOENT` surfaced as a rejected `pipeline` promise, catchable with `try/catch`.
- Part 7: 10 bytes fit under the 16-byte mark; the second write reached 20, so `write()` returned `false`.

Web Streams

Node implements the standard **Web Streams API** (`ReadableStream`, `WritableStream`, `TransformStream`) used by `fetch` (`response.body` is a web `ReadableStream`). Convert with `Readable.fromWeb()` / `Readable.toWeb()`. Node streams for Node APIs; web streams for `fetch` and cross-runtime code.

Trade-offs

Approach	Memory	Simplicity	Use when
`readFile` / `res.text()`	Whole payload	Simplest	Small data (config, JSON under a few MB)
`for await` over a stream	One chunk	Simple	Sequential processing
`pipeline` of stages	One chunk per stage	Moderate	Copy/transform/compress; server responses
Hand-written `write` / `drain`	One chunk	Error-prone	Rarely — let `pipeline` do it

Common mistakes

- `src.pipe(dest)` without error handlers — an error leaves `dest` open and leaks descriptors. Use `pipeline`.
- Ignoring `write()`'s return value in a loop (unbounded memory).
- Decoding each chunk as UTF-8 separately (§15.7).
- Mixing `'data'` events and `for await` on one stream.
- Forgetting `flush` in a stateful Transform (the last partial line is lost).

In the interview

Q: What is backpressure? "When a consumer is slower than the producer, data piles up in memory. Node streams signal it: `write()` returns false once the buffer passes `highWaterMark`, and the producer should pause until `'drain'`. `pipeline()` and `pipe()` do this automatically; `pipeline` also propagates errors and cleans up every stream."

- *Why stream a download instead of `readFile` + `res.end`?* Constant memory per request; first bytes arrive sooner.
- *Read a big file line by line?* `readline.createInterface({ input: createReadStream(f) })` + `for await`.
- *`zlib.createGzip()` is which type?* Transform.

TL;DR

Readable, Writable, Duplex, Transform. Backpressure = `write()` false → wait for `'drain'`. `stream/promises` `pipeline` to connect stages safely, `for await` to consume, `readline` for lines.

15.10 Building an HTTP server with `node:http`

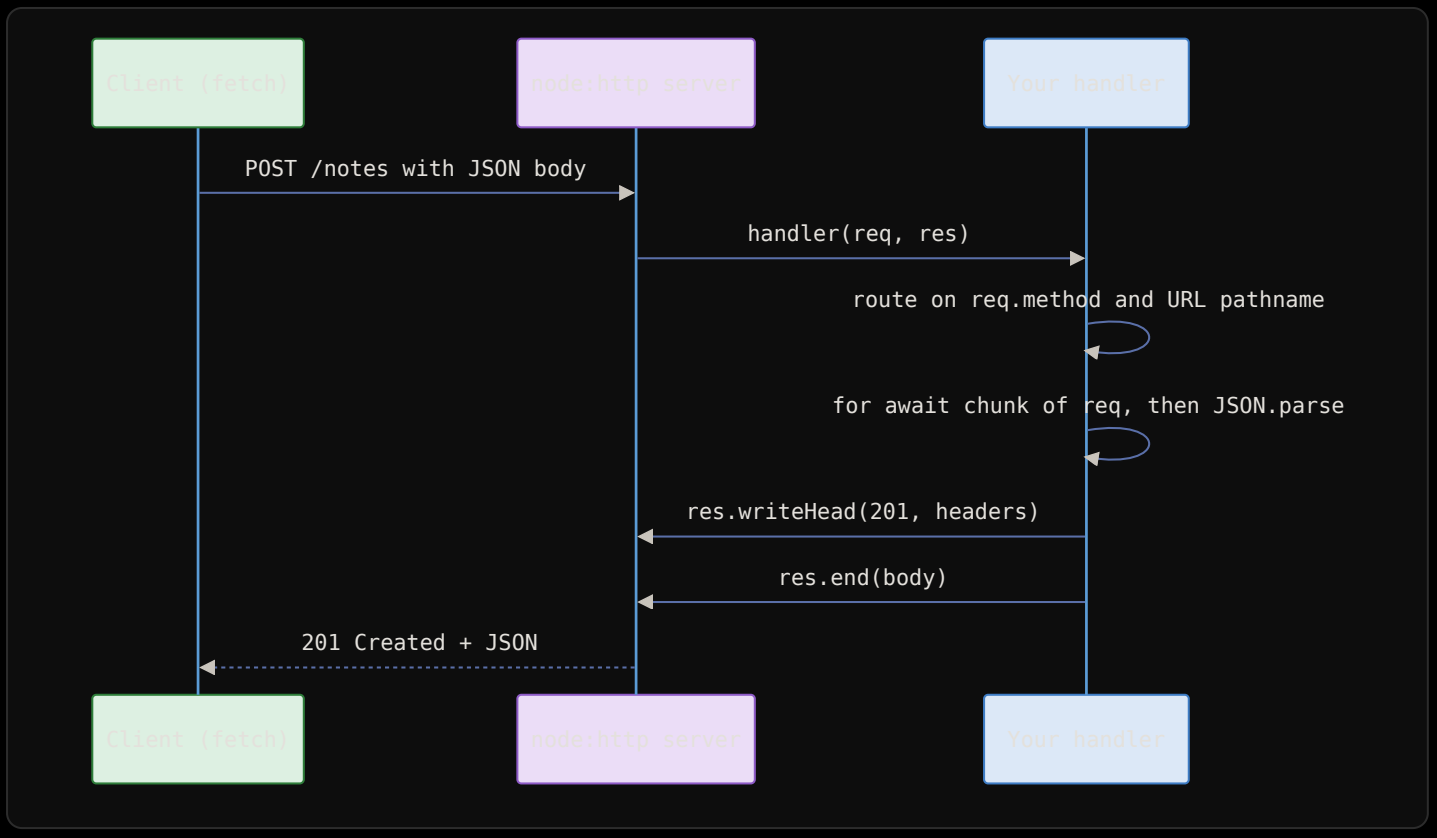
What it is

`node:http` implements HTTP/1.1 clients and servers. `http.createServer(handler)` calls `handler(req, res)` per request:

- `req` (`IncomingMessage`) — a **Readable stream** of the body, plus `req.method`, `req.url` (path + query), `req.headers` (lower-cased).
- `res` (`ServerResponse`) — a **Writable stream**: status and headers (`res.writeHead(status, headers)` or `res.statusCode` + `res.setHeader`), body (`res.write`), finish with **`res.end()`**.

Express, **Fastify**, **Koa** and **Hono** wrap this API, adding routing, middleware, body parsing and validation; knowing the raw layer explains their performance and error behaviour. HTTP from the browser side (methods, statuses, headers, CORS): Chapter 14.

How a request flows



Example — a JSON API and its client in one script

`code/08_http_server.mjs` starts a server on **port 0** (OS picks a free port), calls it with `fetch`, then shuts it down:

```
import { createServer } from 'node:http';
import { once } from 'node:events';
const notes = [{ id: 1, text: 'learn streams' }];

async function readJson(req, limit = 1_000_000) {
  let body = '';
  for await (const chunk of req) {                  // req is a Readable stream
    body += chunk;
    if (body.length > limit) throw Object.assign(new Error('Payload too large'), { status: 413 });
  }
  try { return JSON.parse(body); }
  catch { throw Object.assign(new Error('Invalid JSON'), { status: 400 }); }
}

function send(res, status, data) {
  const body = JSON.stringify(data);
  res.writeHead(status, { 'Content-Type': 'application/json', 'Content-Length': Buffer.byteLength(body) });
  res.end(body);
}

const server = createServer(async (req, res) => {
  const url = new URL(req.url, `http://${req.headers.host}`);
  try {
    if (req.method === 'GET' && url.pathname === '/health') return send(res, 200, { ok: true });
    if (req.method === 'GET' && url.pathname === '/notes') {
      const q = url.searchParams.get('q') ?? '';
      return send(res, 200, notes.filter((n) => n.text.includes(q)));
    }
    if (req.method === 'POST' && url.pathname === '/notes') {
      const { text } = await readJson(req);
      if (typeof text !== 'string' || !text.trim()) return send(res, 422, { error: 'text is required' });
      const note = { id: notes.length + 1, text: text.trim() };
      notes.push(note);
      res.setHeader('Location', `/notes/${note.id}`);
      return send(res, 201, note);
    }
    if (url.pathname === '/stream') {               // stream a response in chunks
      res.writeHead(200, { 'Content-Type': 'text/plain' });
      for (let i = 1; i <= 3; i++) res.write(`chunk ${i}\n`);
      return res.end();
    }
    send(res, 404, { error: `No route for ${req.method} ${url.pathname}` });
  } catch (err) {
    send(res, err.status ?? 500, { error: err.status ? err.message : 'Internal Server Error' });
  }
});

server.listen(0, '127.0.0.1');                      // port 0 = let the OS pick a free port
await once(server, 'listening');
const { port } = server.address();
// ... call() wraps fetch(`http://127.0.0.1:${port}${path}`) and logs status + body
await call('GET', '/health');
await call('GET', '/notes');
const created = await call('POST', '/notes', { text: '  write chapter 15 ' });
console.log('  Location header:', created.headers.get('location'));
await call('POST', '/notes', { text: '' });
await call('POST', '/notes', '{not json');
await call('GET', '/notes?q=chapter');
await call('GET', '/stream');
await call('DELETE', '/notes/1');

server.close();                                     // stop accepting new connections
server.closeAllConnections();                       // drop idle keep-alive sockets (Node 18.2+)
await once(server, 'close');
console.log('server closed');
```

Output:

```
listening on a free port: true
GET /health -> 200 { ok: true }
GET /notes -> 200 [ { id: 1, text: 'learn streams' } ]
POST /notes -> 201 { id: 2, text: 'write chapter 15' }
  Location header: /notes/2
POST /notes -> 422 { error: 'text is required' }
POST /notes -> 400 { error: 'Invalid JSON' }
GET /notes?q=chapter -> 200 [ { id: 2, text: 'write chapter 15' } ]
GET /stream -> 200 "chunk 1\nchunk 2\nchunk 3\n"
DELETE /notes/1 -> 404 { error: 'No route for DELETE /notes/1' }
server closed
```

1. **Routing** = `if`s on `req.method` and `url.pathname`; `new URL(req.url, base)` parses path and query safely.
2. **Body**: `for await` over `req`; the size limit protects memory (a real server also checks `Content-Type`).
3. **Statuses**: 200, 201 Created (+ `Location`), 400 bad JSON, 404 unknown route, 413 too large, 422 failed validation, 500 unexpected — internals hidden.
4. **`closeAllConnections()`**: `server.close()` stops *new* connections but waits for existing ones; `fetch` uses keep-alive (one connection reused for several requests), so without it `'close'` waits for idle sockets to time out.
5. **Port 0 + `127.0.0.1`**: no conflicts, unreachable from other machines — standard in test suites.

Production concerns (what frameworks and platforms add)

Concern	Raw `node:http`	Typical solution
Routing with params (`/notes/:id`)	Manual	Framework router (Express, Fastify, Hono)
Body parsing, size limits, validation	Manual	Framework + schema (Zod, JSON Schema)
Error handling	`try/catch` per handler	Central error middleware
HTTPS/TLS (the encryption under HTTPS)	`node:https` with certificates	Usually terminated at a reverse proxy or load balancer (nginx, cloud LB)
HTTP/2	`node:http2`	Proxy or `http2` module
Timeouts	`server.requestTimeout` (default 300 s), `headersTimeout`, `keepAliveTimeout`	Tune per workload
Graceful shutdown	`close()` + `SIGTERM` handler (§15.6)	Same, plus health checks
Using all cores	One process = one core for JS	Several processes behind a load balancer / container replicas

Common mistakes

- Forgetting `res.end()` — the client hangs until a timeout.
- Setting headers after the body started (`ERR_HTTP_HEADERS_SENT`).
- Unhandled rejections in async handlers — Node doesn't catch them; the request hangs and the process crashes. Wrap in `try/catch`.
- Unlimited body size (memory-exhaustion attack).
- `str.length` for `Content-Length` with non-ASCII text.
- Returning stack traces to clients.

In the interview

Q: How does a Node HTTP server handle many concurrent requests on one thread? "Each request is an event; the handler runs briefly, and whenever it awaits I/O (database, another API) the thread serves other requests. As long as handlers don't do long synchronous work, thousands of connections share one thread."

- *Graceful shutdown?* On `SIGTERM`: `server.close()`, finish in-flight requests, close idle keep-alive sockets, close DB pools, exit with a deadline.
- *Express vs Fastify?* Express: minimal, huge ecosystem, v5 (2024) handles rejected promises from async handlers. Fastify: schema-based validation/serialization, faster JSON, plugin system.
- *Query param `q` from `req.url`?* `new URL(req.url, 'http://x').searchParams.get('q')`.
- *Status for "JSON parsed but `text` missing"?* 422 (or 400).

TL;DR

`createServer((req, res) => …)`: `req` = readable stream + method/url/headers; `res` = writable; always `res.end()`. Parse with `new URL`, limit bodies, map errors to statuses, close gracefully.

15.11 Child processes and npm scripts

What it is

`node:child_process` starts other programs (git, ffmpeg, Python, another Node) as separate OS processes.

Function	Shell?	Output	Use for
`execFile(file, args)`	No	Buffered (collected, returned at the end)	Run a program with arguments — **safe default**
`spawn(cmd, args)`	No (unless `shell: true`)	Streams (`child.stdout`)	Long-running or large output
`exec(commandString)`	**Yes**	Buffered	Shell one-liners with **trusted** input only
`fork(modulePath)`	No	Streams + IPC (inter-process communication) channel (`send` / `'message'`)	Another Node script that talks to the parent
`*Sync` versions	—	Blocks	Build scripts and CLIs, never servers

Security: `exec(\`convert ${userFile}\`)` passes a string to a shell, so `x; rm -rf ~` becomes two commands — **command injection**. `execFile` / `spawn` pass an argument array with no shell interpretation.

`code/09_child_process_and_scripts.mjs` also builds a throw-away project and runs its scripts with **`node --run`** (Node 22+), a faster, simpler `npm run` (§10.9 covers npm, scripts, `npx` and pre/post hook differences):

```
const execFileP = promisify(execFile);
const { stdout } = await execFileP(process.execPath, ['-e', 'console.log(6 * 7)']);
console.log('child said:', stdout.trim());

const child = spawn(process.execPath, ['-e', 'console.log("working"); process.exit(3)']);
child.stdout.on('data', (d) => console.log('spawn stdout:', d.toString().trim()));
const [code] = await new Promise((r) => child.on('close', (...a) => r(a)));
console.log('spawn exit code:', code);

await writeFile(join(dir, 'package.json'), JSON.stringify({
  name: 'demo', private: true, type: 'module',
  scripts: { hello: 'node hello.js', greet: 'node hello.js' },
}, null, 2));
await writeFile(join(dir, 'hello.js'), 'console.log(`hello from ${process.argv[2] ?? "script"}`)');
const r1 = await execFileP(process.execPath, ['--run', 'hello'], { cwd: dir });
const r2 = await execFileP(process.execPath, ['--run', 'greet', '--', 'Ana'], { cwd: dir });
```

Output:

```
child said: 42
spawn stdout: working
spawn exit code: 3
node --run hello -> hello from script
node --run greet -- Ana -> hello from Ana
```

`process.execPath` is the absolute path of the running `node` binary — safer than assuming `node` is on the `PATH`. Arguments after `--` go to the script.

npm scripts in a Node project

```
{
  "scripts": {
    "start": "node src/server.mjs",
    "dev": "node --watch --env-file=.env src/server.mjs",
    "test": "node --test",
    "typecheck": "tsc --noEmit",
    "lint": "eslint ."
  }
}
```

(Not run.) `npm start`, `npm test`, `npm run dev` or `node --run dev`. Tests and type-checking: Chapter 16.

TL;DR

`execFile`/`spawn` with argument arrays (no shell); `exec` only with trusted strings. `node --run <script>` runs `package.json` scripts quickly; `npm run` adds pre/post hooks.

15.12 Everyday Node flags, and Node vs Deno vs Bun

Command-line flags worth knowing (Node 24)

Flag	What it does	Status
`node --watch app.mjs`	Restart on file changes	Stable
`node --env-file=.env app.mjs`	Load environment variables from a file	Stable (24.10+)
`node --run <script>`	Run a `package.json` script	Added 22
`node --test`	Run tests (Chapter 16)	Stable (20+)
`node app.ts`	Run TypeScript by **stripping types** (deleting annotations without checking) (§16.9)	On by default since 23.6; stable in 24.12
`node --inspect` / `--inspect-brk`	Attach Chrome DevTools / VS Code (§11.9)	Stable
`node --cpu-prof`, `--heap-prof`	Write CPU / heap profiles	Stable
`node --permission --allow-fs-read=./data app.mjs`	**Permission model**: deny file-system reads/writes, child processes, workers, native addons … unless allowed	Stable since 23.5 / 22.13
`node -e "code"` / `node -p "expr"`	Evaluate / evaluate-and-print	Stable

The permission model limits what a compromised dependency can do, but the docs state it is not a sandbox against deliberately malicious code.

Node vs Deno vs Bun

All three run JavaScript and TypeScript on servers, and most npm packages.

	Node.js	Deno	Bun
First release	2009	2020 (1.0); **Deno 2** (Oct 2024) added full npm / `package.json` compatibility	2023 (1.0, September)
Creator / owner	Ryan Dahl; now OpenJS Foundation	Ryan Dahl; Deno Land Inc.	Jarred Sumner / Oven; **acquired by Anthropic in December 2025**, still MIT-licensed
Engine	V8	V8 (runtime in Rust)	**JavaScriptCore** (Safari's engine; runtime in Zig)
TypeScript	Type stripping built in (erasable syntax only); check with `tsc`	Built in; `deno check` type-checks	Built in (transpiles, no type check)
Security default	Full access (opt-in `--permission`)	**Deny by default** — `--allow-net`, `--allow-read` …	Full access
Built-in tools	test runner, watch mode, `--run`	formatter, linter, test runner, `compile`, package manager	package manager (fast `bun install`), bundler, Jest-like test runner
Strength	Maturity, ecosystem, LTS policy, hosting everywhere	Secure defaults, web-standard APIs, all-in-one tooling	Install/startup speed, all-in-one tooling

Which? Node for production by default — largest ecosystem, predictable LTS, supported by every host and observability tool. Deno for secure-by-default permissions and integrated tooling. Bun when install/startup speed matters (many teams use `bun install` or its test runner with a Node production runtime). Check native add-ons and less common Node APIs before switching.

> **TL;DR**

Learn `--watch`, `--env-file`, `--run`, `--test`, `--inspect`, running `.ts` directly. Node is the default; Deno (secure-by-default, V8) and Bun (fast, JavaScriptCore) are credible alternatives.

Chapter 15 TL;DR

- **Node.js** = V8 + libuv + core modules. No DOM; files, processes, sockets. Run an **LTS** (Node 24 today).
- **One JS thread**: network I/O is OS-async; fs, DNS lookup, crypto, zlib use the **libuv thread pool** (4). Never block the loop; **worker threads** for CPU work.
- Built-ins via **`node:`**; ESM uses **`import.meta.dirname`**.
- **`node:fs/promises`**: `readFile(p,'utf8')`, `writeFile`, `mkdir({recursive})`, `readdir`, `stat`, `rm({recursive,force})`; errors by **`err.code`**; no check-then-act.
- **`node:path`**: `join`, `resolve`, `basename/extname/dirname/parse`; `fileURLToPath`; guard **path traversal**.
- **`process`**: `argv` (+ `util.parseArgs`), **`env` is strings only**, `--env-file`, validate config, `process.exitCode`, handle `SIGTERM`.
- **Buffer** = bytes; byte length ≠ string length.
- **EventEmitter**: `on/once/off/emit`; emit is synchronous; always handle `'error'`.
- **Streams**: **backpressure** (`write()` false → `'drain'`); use **`pipeline`** from `stream/promises`.
- **`node:http`**: `req` readable + `res` writable; always `res.end()`; limit bodies; map errors; port 0 in tests; graceful shutdown.
- **child_process**: `execFile`/`spawn` with arrays (no shell injection); `node --run` for scripts.
- **Deno** (secure by default) and **Bun** (fast, JavaScriptCore, owned by Anthropic) are alternatives; Node remains the default.

Next: prove your code works and catch type errors before running it — Chapter 16: Testing and TypeScript.

Chapter 16 — Testing and TypeScript

Two habits separate code that *seems* to work from code you can change without fear. **Automated tests** prove behaviour by running code and checking results. **Static types** (TypeScript) catch whole classes of mistakes — typos, wrong argument types, forgotten `null` checks — before the code runs. They overlap little: types say *the shapes fit*; tests say *the answers are right*. This chapter teaches both with tools that ship with Node 24 and the TypeScript compiler.

What you'll learn

1. Why we test, kinds of tests, the runner landscape (node:test, Vitest, Jest, Playwright).
2. Assertions with `node:assert/strict`.
3. `node:test`: `test` / `describe` / `it`, hooks, subtests, skip/todo/only, reporters, watch mode.
4. Async tests and the classic failures (wrong expectation, timeout, missing `await`).
5. Test doubles: fakes, stubs, spies, `mock.fn`, `mock.method`, fake timers, module mocks.
6. Snapshot tests and code coverage.
7. TypeScript: what it is, `tsc`, `tsconfig.json`, TypeScript 7.
8. Running `.ts` directly with Node (type stripping — deleting annotations so plain JavaScript remains) and its rules.
9. Types: primitives, arrays, tuples, object types, interfaces, unions, literals, `any` vs `unknown`.
10. Narrowing, discriminated unions, exhaustive checks.
11. Generics and utility types.
12. Classes, `as const`, `satisfies`, enums and alternatives.
13. Tests in TypeScript; JSDoc types (types in `/** … */` comments) for plain JavaScript.

Before you start. Functions (Chapter 4), classes (Chapter 9), modules and `package.json` scripts (Chapter 10), errors (Chapter 11), `async` / `await` (Chapter 12). React testing and TypeScript with React: React Handbook ch11 and ch12.

The code. `code/` is a tiny project with its own `package.json` (`"type": "module"`, scripts `test`, `coverage`, `typecheck`) and `tsconfig.json`; modules under test live in `code/src/`. Outputs are real: **Node 24.11.1**, **TypeScript 7.0.2** (`tsc`). Test files are ordinary scripts — `node cart.test.mjs` runs its tests and exits 1 on failure. Files that fail *on purpose* (`demo_failing.mjs`, `ts_errors.ts`, `jsdoc_errors.mjs`) or need a flag (`module_mock_demo.mjs`) are marked `// tools_run: skip`, with the exact command in their first line and in the text. Timings like `(0.42ms)` vary.

16.1 Why test, and what kinds of tests exist

What it is

An **automated test** runs code with known inputs and **asserts** (checks) the result. A **test runner** finds test files, runs them and reports passes/failures. A **test suite** is a group of related tests.

Why it matters

- **Regression safety** (a *regression* = something that used to work and broke): after a refactor or upgrade, the suite tells you in seconds.
- **Design feedback**: hard-to-test code (hidden global state, hard-wired network calls) is usually hard to reuse.
- **Executable documentation** and a **CI gate**: the test command's exit code (0 = success) decides whether a pull request can merge.

Kinds of tests

Kind	What it exercises	Speed	Typical tool (JS)
Unit	One function/class/module in isolation; dependencies replaced by doubles (§16.5)	Milliseconds	node:test, Vitest, Jest
Integration	Real pieces together (HTTP handler + a real database in a container, e.g. Docker)	Tens of ms to seconds	Same runners + Testcontainers (starts such containers for tests)
End-to-end (E2E)	The whole app through its real interface (a browser clicking through pages)	Seconds	Playwright, Cypress
Static checks	No execution: types, lint rules	Fast	TypeScript, ESLint

The **test pyramid**: many unit, fewer integration, a handful of E2E tests — higher layers are slower and **flakier** (pass/fail randomly with no code change). The **testing trophy** (popular in front-end) makes integration the fattest layer, for the most confidence per test. Both agree: static checks at the bottom, few E2E, test **behaviour**, not implementation details.

Test runners in 2026

Runner	Strengths	Notes
`node:test` (built in)	No dependencies, fast, runs TypeScript via type stripping; mocks, fake timers, snapshots, coverage built in	Stable since Node 20; used here
Vitest	Jest-compatible API, Vite's config and transforms, browser mode	Common for Vite / React projects
Jest	Very large ecosystem	Older projects; ESM historically needed extra config
Playwright Test	Real browsers (Chromium, Firefox, WebKit — the engines behind Chrome, Firefox, Safari), auto-waiting, tracing (recording each step)	E2E and component tests
Bun test / Deno test	Built into those runtimes (§15.12)	Jest-like APIs

Concepts (describe/it, hooks, assertions, mocks, snapshots) are nearly identical across runners; §16.7 has a translation table.

TL;DR

Tests run code and check results automatically. Unit = fast, isolated; integration = real pieces together; E2E = whole app. Test behaviour, keep most tests fast, let CI run them on every change.

16.2 Assertions with `node:assert/strict`

What it is

An **assertion** throws an `AssertionError` when it fails; a test *fails* exactly when something in it throws. Import the **strict** variant, `node:assert/strict`: `equal` compares with `Object.is` and `deepEqual` also compares types and prototypes. (Legacy non-strict mode uses `==` — avoid.)

Assertion	Passes when
`equal(actual, expected)`	`Object.is(actual, expected)` — same primitive or **same object reference**
`deepEqual(actual, expected)`	Same structure and values, recursively, **type-strict** (`1` ≠ `'1'`), same prototypes
`notEqual` / `notDeepEqual`	The opposites
`ok(value)`	`value` is truthy
`match(string, regex)` / `doesNotMatch`	Regex matches / doesn't
`throws(fn, expected?)`	`fn()` throws; `expected` = error class, message regex, `{ name, message }` object, or a function
`rejects(promiseOrFn, expected?)`	The promise **rejects** — must be `await`ed
`doesNotThrow` / `fail(msg)`	Rarely needed (just call the code) / always fails
`partialDeepStrictEqual(actual, expected)`	`actual` contains at least `expected`'s properties (newer; check your Node version)

`code/01_assert_basics.mjs` prints PASS or FAIL per assertion:

```
import assert from 'node:assert/strict';

function check(label, fn) {
  try { fn(); console.log('PASS', label); }
  catch (err) { console.log('FAIL', label, '->', err.code, '|', err.message.split('\n')[0]); }
}

check('equal uses Object.is', () => assert.equal(0.1 + 0.2, 0.3));
check('equal numbers', () => assert.equal(2 + 2, 4));
check('equal objects (reference!)', () => assert.equal({ a: 1 }, { a: 1 }));
check('deepEqual objects', () => assert.deepEqual({ a: 1, tags: ['x'] }, { a: 1, tags: ['x'] }));
check('deepEqual is type-strict', () => assert.deepEqual({ n: 1 }, { n: '1' }));
// ... ok([].length), match, throws with { name, message }, notDeepEqual
check('throws but nothing thrown', () => assert.throws(() => 42));
check('float with tolerance', () => assert.ok(Math.abs(0.1 + 0.2 - 0.3) < Number.EPSILON));

try {
  assert.deepEqual({ id: 7, items: ['pen', 'ink'] }, { id: 7, items: ['pen', 'paper'] });
} catch (err) {
  console.log('--- AssertionError message ---\n' + err.message);
}

await assert.rejects(Promise.reject(new Error('timeout')), /timeout/);
console.log('PASS rejects');
```

Output:

```
FAIL equal uses Object.is -> ERR_ASSERTION | Expected values to be strictly equal:
PASS equal numbers
FAIL equal objects (reference!) -> ERR_ASSERTION | Values have same structure but are not reference-equal:
PASS deepEqual objects
FAIL deepEqual is type-strict -> ERR_ASSERTION | Expected values to be strictly deep-equal:
FAIL ok (truthy) -> ERR_ASSERTION | The expression evaluated to a falsy value:
PASS match regex
PASS throws with class + message
FAIL throws but nothing thrown -> ERR_ASSERTION | Missing expected exception.
PASS notDeepEqual
PASS float with tolerance
--- AssertionError message ---
Expected values to be strictly deep-equal:
+ actual - expected

  {
    id: 7,
    items: [
      'pen',
+     'ink'
-     'paper'
    ]
  }

PASS rejects
```

- `0.1 + 0.2 !== 0.3` in floating point (§2 Number) — compare floats with a tolerance, or keep money in integer cents (as below).
- `equal` on two separate objects fails even with identical contents; use `deepEqual`.
- The `+ actual - expected` diff makes failures quick to diagnose — prefer one `deepEqual` on a whole object over many field `equal`s.

TL;DR

`node:assert/strict`: `equal` for primitives/same reference, `deepEqual` for structures, `throws`/`await rejects` for errors, `match` for strings. A test fails when anything in it throws.

16.3 Writing and running tests with `node:test`

What it is

`node:test` is Node's built-in runner (stable since Node 20); the `node:` prefix is mandatory.

Function	Purpose
`test(name, [options], fn)` / alias `it`	One test. `fn` may be `async`; it receives a **test context** `t`
`describe(name, fn)` / alias `suite`	Group tests (can nest)
`before`/`after`, `beforeEach`/`afterEach`	Once around the suite / around every test in it (and nested suites)
`t.test(name, fn)`	A **subtest** (`await` it)
`test.skip`, `test.todo`, `test.only` (+ `{ skip, todo, only, timeout }` options)	Skip; not yet written (runs, failures don't count); run only these (with `--test-only`)
`t.mock`, `t.assert`, `t.plan(n)`, `t.signal`, `t.diagnostic(msg)`	Auto-restoring mocks (§16.5), assertions incl. snapshots (§16.6), expected assertion count, an `AbortSignal`, extra report output

How it works

1. `node --test` (usually via `npm test`) with no arguments searches the cwd recursively (skipping `node_modules`) for `*.test.{js,mjs,cjs}`, `*-test.*`, `*_test.*`, `test-*.*`, `test.*`, or anything in a

`test/` folder — and the same with `.ts/.mts/.cts`.

2. Each file runs in **its own child process** by default (isolation: globals and mocks can't leak; `--test-isolation=none` uses one process for speed).
3. Within a file tests run **in order**; `describe` callbacks run immediately to collect tests; hooks wrap them.
4. The **reporter** prints results: `spec` (default), `tap`, `dot`, `junit` (CI dashboards), `lcov` (coverage); `--test-reporter-destination` writes to a file.
5. **Exit code** 1 if any test failed — what CI checks.

```
node --test                                # all matching files under the cwd
node --test test/cart.test.mjs             # specific files or globs
node --test --watch                        # re-run on file changes
node --test --test-name-pattern="Cart"     # only tests whose name matches a regex
node --test --test-reporter=dot            # compact output (--test-only: run only test.only)
node cart.test.mjs                         # a single file also runs as a plain script
```

Example — a unit test suite

`code/src/cart.mjs` is a cart storing prices in integer cents, plus `applyDiscount`, `formatPrice` and an async `checkout`:

```
export class Cart {
  #items = new Map();                                  // sku -> { sku, price, qty }
  add(sku, price, qty = 1) {
    if (typeof sku !== 'string' || sku === '') throw new TypeError('sku must be a non-empty string');
    if (!Number.isInteger(price) || price < 0) throw new RangeError('price must be integer cents >= 0');
    if (!Number.isInteger(qty) || qty < 1) throw new RangeError('qty must be an integer >= 1');
    // ... merge into the existing line for this sku; return this
  }
  // remove(sku), get count(), lines() (copies), subtotal()
}
export function applyDiscount(cents, percent) {
  if (percent < 0 || percent > 100) throw new RangeError(`bad percent: ${percent}`);
  return Math.round(cents * (100 - percent) / 100);
}
export function formatPrice(cents, currency = 'USD', locale = 'en-US') { /* Intl.NumberFormat */ }
export async function checkout(cart, { priceService, now = () => new Date() }) { /* §16.5 */ }
```

`code/cart.test.mjs`:

```
import { describe, it, test, before, beforeEach, afterEach, after } from 'node:test';
import assert from 'node:assert/strict';
import { Cart, applyDiscount, formatPrice } from './src/cart.mjs';

describe('Cart', () => {
  let cart;
  beforeEach(() => { cart = new Cart(); });          // fresh object for every test

  it('starts empty', () => {
    assert.equal(cart.count, 0);
    assert.equal(cart.subtotal(), 0);
  });

  it('adds items and merges the same sku', () => {
    // Arrange - Act - Assert
    cart.add('pen', 150, 2).add('ink', 900).add('pen', 150);
    assert.equal(cart.count, 4);
    assert.deepEqual(cart.lines(), [
      { sku: 'pen', price: 150, qty: 3 },
      { sku: 'ink', price: 900, qty: 1 },
    ]);
    assert.equal(cart.subtotal(), 1350);
  });
  // ... 'removes items'

  describe('validation', () => {
    const bad = [
      ['', 100, 1, TypeError],
      ['pen', -1, 1, RangeError],
      ['pen', 1.5, 1, RangeError],
      ['pen', 100, 0, RangeError],
    ];
    for (const [sku, price, qty, ErrorType] of bad) {     // table-driven tests
      it(`rejects add(${JSON.stringify(sku)}, ${price}, ${qty})`, () => {
        assert.throws(() => cart.add(sku, price, qty), ErrorType);
      });
    }
  });

  // ... 'lines() returns copies, not internal state'
});

describe('applyDiscount', () => {
  it('takes a percentage off and rounds to cents', () => {
    assert.equal(applyDiscount(1000, 10), 900);
    assert.equal(applyDiscount(999, 15), 849);         // 849.15 -> 849
    assert.equal(applyDiscount(1000, 0), 1000);
    assert.equal(applyDiscount(1000, 100), 0);
  });
  it('rejects percentages outside 0-100', () => {
    assert.throws(() => applyDiscount(1000, 101), { name: 'RangeError', message: 'bad percent: 101' });
  });
});

test('formatPrice', async (t) => {                     // subtests with t.test
  await t.test('US dollars', () => assert.equal(formatPrice(123456), '$1,234.56'));
  await t.test('euros in Germany', () => assert.equal(formatPrice(123456, 'EUR', 'de-DE'), '1.234,56 €'));
});

test.skip('multi-currency carts', () => {});           // reported as skipped
test.todo('coupon codes');                               // reported as todo

describe('hooks order', () => {       // before/beforeEach/afterEach push to a log; after() prints it
  // ... it('one'), it('two')
});
```

Output of `node --test cart.test.mjs` (abridged):

```
# hooks ran: before > beforeEach > test1 > afterEach > beforeEach > test2 > afterEach
▶ Cart
  ✔ starts empty (0.425292ms)
  ▶ validation
    ✔ rejects add("", 100, 1) (0.228958ms)
    ...
- multi-currency carts (0.020625ms) # SKIP
✔ coupon codes (0.016958ms) # TODO
ℹ tests 17
ℹ pass 15
ℹ fail 0
ℹ skipped 1
ℹ todo 1
```

1. **Arrange–Act–Assert (AAA)**: set up, one action, check. One behaviour per test, named as a sentence so the report reads like a spec.
2. `beforeEach` **builds a fresh** `Cart` — no dependence on other tests' leftovers.
3. **Table-driven tests**: one named test per bad-input row — cheap to extend, failures reported separately.
4. **Edge cases** explicit: zero, 100 %, rounding (`849.15 → 849`), mutation of returned data.
5. `'1.234,56 €'` contains a **non-breaking space** before `€` — locale-dependent output is a classic source of surprising failures.
6. The `after` hook's `console.log` printed at the top: each file's child-process stdout is printed before the formatted results. Use `t.diagnostic()` for messages inside the report.

Common mistakes

- Shared mutable state between tests (a module-level `cart`) — order-dependent, flaky.
- Testing implementation details (`#items`) — every refactor breaks tests.
- Committing `test.only` / `{ only: true }`. Node still runs *all* tests unless `--test-only` is passed (it prints "'only' and 'runOnly' require the --test-only command-line option"), but Jest and Vitest really skip the rest — a suite can silently stop running.
- Many unrelated assertions in one test — the first failure hides the rest.
- Tests with no assertion (always pass). `t.plan(n)` enforces a count when callbacks are involved.

TL;DR

`import { test, describe, it, beforeEach } from 'node:test'`; run `node --test`. One behaviour per test, AAA, fresh fixtures (prepared test data) in `beforeEach`, table-driven tests, `skip`/`todo` for gaps. Exit code 1 on failure.

16.4 Async tests and how tests fail

How async tests work

If a test returns a promise (any `async` function does), the runner **waits**: fulfilled = pass, rejected = fail. So `await` everything, especially `await assert.rejects(...)`. A test also fails past its `timeout` option (default: none — set one for code that might hang).

Example — three ways to fail

`code/demo_failing.mjs` (skip-marked; run `node --test demo_failing.mjs`):

```
test('a wrong expectation', () => {
  assert.equal(applyDiscount(999, 15), 850);          // real answer is 849
});

test('a test that hangs', { timeout: 50 }, async () => {
  await new Promise((r) => setTimeout(r, 1000));
});

test('forgot to await the rejection', async () => {
  assert.rejects(Promise.resolve('no error'));          // missing await -> failure escapes the test
});
```

Output (shortened with `…`):

```
✖ a wrong expectation (0.612875ms)
✖ a test that hangs (51.228334ms)
✔ forgot to await the rejection (0.396875ms)
ℹ Error: Test "forgot to await the rejection" at demo_failing.mjs:14:1 generated asynchronous activity after the
test ended. This activity created the error "AssertionError [ERR_ASSERTION]: Missing expected rejection." and
would have caused the test to fail, but instead triggered an unhandledRejection event.
ℹ tests 3
...
ℹ pass 1
ℹ fail 1
ℹ cancelled 1
...

✖ failing tests:

test at demo_failing.mjs:6:1
✖ a wrong expectation (0.612875ms)
  AssertionError [ERR_ASSERTION]: Expected values to be strictly equal:

  849 !== 850

      at TestContext.<anonymous> (file:///…/16-testing-and-typescript/code/demo_failing.mjs:7:10)
      …

test at demo_failing.mjs:10:1
✖ a test that hangs (51.228334ms)
  'test timed out after 50ms'
exit=1
```

1. **Wrong expectation** — actual vs expected plus file:line. Decide whether the code or the test is wrong (here the test: 999 × 0.85 = 849.15).
2. **Timeout** — reported as *cancelled*; usually a promise that never settles or a server never closed.
3. **Missing `await`** — the dangerous one: ✔ **passed**, because the test ended before the assertion rejected; only a warning a busy CI log hides. `await` *every* `assert.rejects` and async call (`@typescript-eslint/no-floating-promises` catches this).

TL;DR

`async` tests, `await` everything, `timeout` for code that could hang; a missing `await` can make a failing test green.

16.5 Test doubles and mocking

What it is

A **test double** stands in for a real dependency (network API, database, clock, randomness) so the unit runs fast, **deterministically** (same result every run) and offline.

Double	What it is	Example
Dummy	Passed but never used	`{}` for an unused options argument
Stub	Returns canned answers	`getDiscount: async () => ({ percent: 10 })`
Spy	Records how it was called (may call through)	`mock.fn()` → `.mock.calls`
Mock	A spy with call expectations verified in the test	"called once with 500"
Fake	A working lightweight implementation	In-memory repository instead of Postgres

Everyday speech says "mock" for all of these.

Why it matters — and dependency injection

The easiest code to test takes dependencies **as parameters** (**dependency injection**) instead of importing and calling them. `checkout` receives `priceService` and `now`:

```
export async function checkout(cart, { priceService, now = () => new Date() }) {
  if (cart.count === 0) throw new Error('cart is empty');
  const { percent } = await priceService.getDiscount(cart.subtotal());
  return { total: applyDiscount(cart.subtotal(), percent), at: now().toISOString() };
}
```

Production passes the real HTTP client and clock; tests pass a stub and a fixed clock — no mocking library needed.

The `node:test` mocking API

API	Does
`mock.fn(impl?)`	A spy; `fn.mock.calls` (each `{ arguments, result, error, this }`), `fn.mock.callCount()`
`fn.mock.mockImplementation(f)` / `mockImplementationOnce(f)`	Change behaviour (all / next call)
`mock.method(obj, 'name', impl?)`	Replace a method, keeping a spy on it
`mock.getter` / `setter` / `property`	Replace accessors / a property value
`t.mock.*`	Same, **auto-restored when the test ends** — prefer over global `mock` (needs `mock.restoreAll()`)
`t.mock.timers.enable({ apis: ['setTimeout', 'Date', …], now })` + `tick(ms)`	**Fake timers** — control `setTimeout` / `setInterval` / `setImmediate` / `Date` without waiting
`t.mock.module(specifier, { namedExports, defaultExport })`	Replace what `import` returns (needs `--experimental-test-module-mocks`)

`code/checkout.test.mjs`:

```
import { describe, it, mock } from 'node:test';
import assert from 'node:assert/strict';
import { Cart, checkout } from './src/cart.mjs';

const fixedNow = () => new Date('2026-01-15T10:00:00Z');   // an injected, fixed clock

describe('checkout (async + dependency injection)', () => {
  it('applies the discount returned by the price service (fake object)', async () => {
    const fakePriceService = { getDiscount: async () => ({ percent: 10 }) };
    const cart = new Cart().add('pen', 1000);
    const receipt = await checkout(cart, { priceService: fakePriceService, now: fixedNow });
    assert.deepEqual(receipt, { total: 900, at: '2026-01-15T10:00:00.000Z' });
  });
  // ... 'rejects an empty cart', 'propagates service failures' (await assert.rejects)
  it('calls the service once with the subtotal (mock.fn spy)', async () => {
    const getDiscount = mock.fn(async () => ({ percent: 0 }));
    await checkout(new Cart().add('ink', 250, 2), { priceService: { getDiscount }, now: fixedNow });
    assert.equal(getDiscount.mock.callCount(), 1);
    assert.deepEqual(getDiscount.mock.calls[0].arguments, [500]);
  });
});

describe('mock.method and t.mock', () => {
  const api = { async fetchUser(id) { throw new Error('real network call - must not run in tests'); } };
  it('replaces a method for one test; t.mock restores it automatically', async (t) => {
    const m = t.mock.method(api, 'fetchUser', async (id) => ({ id, name: 'Test User' }));
    assert.deepEqual(await api.fetchUser(7), { id: 7, name: 'Test User' });
    assert.equal(m.mock.callCount(), 1);
  });
  // ... 'after the previous test the real method is back' (assert.rejects on the real one)
  it('mockImplementationOnce: fail first, then succeed (retry logic)', async (t) => {
    const flaky = t.mock.fn(async () => 'ok');
    flaky.mock.mockImplementationOnce(async () => { throw new Error('ECONNRESET'); });
    assert.equal(await withRetry(flaky), 'ok');      // withRetry: try up to 3 times
    assert.equal(flaky.mock.callCount(), 2);
  });
});

describe('fake timers', () => {
  function debounce(fn, ms) {
    let id;
    return (...args) => { clearTimeout(id); id = setTimeout(() => fn(...args), ms); };
  }
  it('debounce calls once, after the quiet period - without really waiting', (t) => {
    t.mock.timers.enable({ apis: ['setTimeout'] });
    const save = t.mock.fn();
    const debouncedSave = debounce(save, 300);
    debouncedSave('a'); debouncedSave('ab'); debouncedSave('abc');
    t.mock.timers.tick(299);
    assert.equal(save.mock.callCount(), 0);
    t.mock.timers.tick(1);
    assert.equal(save.mock.callCount(), 1);
    assert.deepEqual(save.mock.calls[0].arguments, ['abc']);
  });
  // ... 'Date can be frozen too': enable({ apis: ['Date'], now: new Date('2030-05-01T00:00:00Z') })
});
```

Output of `node --test checkout.test.mjs` (abridged): all 9 tests in 3 suites pass; the debounce test runs in 0.29 ms.

```
▶ fake timers
  ✔ debounce calls once, after the quiet period - without really waiting (0.290834ms)
  ✔ Date can be frozen too (0.12175ms)
✔ fake timers (0.520417ms)
ℹ tests 9
ℹ suites 3
ℹ pass 9
ℹ fail 0
```

Fake timers move time only on `tick`, so a 300 ms delay is checked in under half a millisecond. (Debounce itself: Chapter 17.)

Module mocking

When code imports a dependency directly, replace the **module**. `code/module_mock_demo.mjs` mocks `node:os`, which `src/greeting.mjs` imports:

```
test('greeting uses the mocked os module', async (t) => {
  t.mock.module('node:os', { namedExports: { userInfo: () => ({ username: 'ana' }) } });
  const { greeting } = await import('./src/greeting.mjs');   // import AFTER mocking
  assert.equal(greeting(), 'Hello, ana!');
});
```

Output of `node --test --experimental-test-module-mocks module_mock_demo.mjs`:

```
(node:77997) ExperimentalWarning: Module mocking is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
✔ greeting uses the mocked os module (2.704792ms)
ℹ tests 1
...
ℹ pass 1
ℹ fail 0
...
```

Module mocking is **experimental** in Node 24 (flag + warning), and the module must be imported *after* the mock — hence dynamic `import()` (Vitest's `vi.mock` and Jest's `jest.mock` hoist mocks above imports for you). Prefer dependency injection; module mocks when you can't change the code.

When not to mock

- **Don't mock what you own and can run cheaply** (pure functions — no side effects, same input → same output — and small modules).
- **Don't mock the unit under test.**
- Too many mocks → tests pass while production fails (doubles drift). Balance with integration tests against real (containerized) databases and **contract tests** (both sides of an API still agree on its format).
- For HTTP prefer a real local server (§15.10's port-0 pattern) or an interception library (MSW, undici `MockAgent`) over hand-mocking `fetch`.

In the interview

Q: Stub vs mock vs fake? "A stub returns canned data; a mock (or spy) records calls so you can assert on them; a fake is a simple working implementation, like an in-memory DB. I prefer injecting dependencies so tests can pass fakes without a mocking framework."

- *Testing `setTimeout` or `Date.now()` code?* Fake timers (`t.mock.timers`), or inject a clock.
- *What makes tests flaky?* Real time, real network, shared state, test order, unawaited promises, locale/timezone differences.

TL;DR

Inject dependencies and pass stubs/fakes; `t.mock.fn` / `t.mock.method` spies (auto-restored); fake timers for time; module mocks last. Don't over-mock.

16.6 Snapshot tests and code coverage

Snapshot testing

A **snapshot test** saves output the first time and fails later if it changes — for large, stable outputs (a receipt, CLI help) tedious to write by hand. `t.assert.snapshot(value)` is **stable since Node 23.4**.

`code/snapshot.test.mjs`:

```
function receiptText(cart) {
  const rows = cart.lines().map((l) => `${l.sku.padEnd(6)} x${l.qty}  ${(l.price * l.qty /
100).toFixed(2).padStart(7)}`);
  return [...rows, '-'.repeat(20), `TOTAL ${(cart.subtotal() / 100).toFixed(2).padStart(13)}`].join('\n');
}
test('receipt layout', (t) => {
  const cart = new Cart().add('pen', 150, 3).add('ink', 900);
  t.assert.snapshot(receiptText(cart));
});
```

1. First run `node --test --test-update-snapshots snapshot.test.mjs` writes `snapshot.test.mjs.snapshot`:

   ```
   exports[`receipt layout 1`] = `
   "pen    x3     4.50\\nink    x1     9.00\\n--------------------\\nTOTAL         13.50"
   `;
   ```

2. Commit it. Later runs compare; without the file the test fails with "Missing snapshots can be generated by rerunning the command with the --test-update-snapshots flag".
3. When output changes **on purpose**, rerun with `--test-update-snapshots` and review the diff in code review.

Danger: updating snapshots without reading them. Keep them small; review like code.

Code coverage

Coverage measures which lines, **branches** (each path through `if`/`else`, `switch`, `?:`) and functions tests executed. `node --test --experimental-test-coverage` in `code/`:

```
ℹ start of coverage report
ℹ ---------------------------------------------------------
ℹ file      | line % | branch % | funcs % | uncovered lines
ℹ ---------------------------------------------------------
ℹ src       |        |          |         |
ℹ  cart.mjs | 100.00 |   100.00 |   90.91 |
ℹ  money.ts | 100.00 |    88.89 |  100.00 |
ℹ ---------------------------------------------------------
ℹ all files | 100.00 |    97.14 |   93.33 |
ℹ ---------------------------------------------------------
ℹ end of coverage report
```

(Test files excluded by default.) `cart.mjs` functions = 90.91 %: the default clock `now = () => new Date()` never ran because every test injected one. Flags: `--test-coverage-include`/`-exclude`, `--test-coverage-lines=80` (fail below a threshold), `--test-reporter=lcov` for CI services. Still "experimental" in Node 24.

Coverage is a **floor, not a goal**: 100 % means every line *ran*, not that results were *checked*.

> **TL;DR**
>
> Snapshots record output and flag changes — commit and review them. Coverage shows unexecuted code — a hint, not a target.

16.7 Good tests, and other runners

What makes a good test (F.I.R.S.T. and more)

Property	Meaning
Fast	Milliseconds; slow suites stop being run
Independent/Isolated	No shared state, any order
Repeatable	Same result every run: fixed clock, seeded randomness (fixed start value, so numbers repeat), no real network
Self-validating	Pass/fail by assertions, not by reading logs
Timely	Written with (or before — **TDD**, test-driven development: red → green → refactor) the code
Behaviour-focused	Tests public behaviour; survives refactoring
Readable	Sentence names; AAA; failure message says what broke

Same ideas in Vitest and Jest

Concept	node:test	Vitest	Jest
Import	`from 'node:test'` + `node:assert/strict`	`from 'vitest'` (or globals)	globals
Test / group	`test` / `it`, `describe`	same	same
Assert equal	`assert.equal(a, b)`	`expect(a).toBe(b)`	same
Deep equal	`assert.deepEqual`	`expect(a).toEqual(b)` / `toStrictEqual`	same
Throws	`assert.throws(fn, Err)`	`expect(fn).toThrow(Err)`	same
Rejects	`await assert.rejects(p)`	`await expect(p).rejects.toThrow()`	same
Spy	`t.mock.fn()`	`vi.fn()`	`jest.fn()`
Method mock	`t.mock.method(o, 'm')`	`vi.spyOn(o, 'm')`	`jest.spyOn`
Fake timers	`t.mock.timers.enable()` / `tick`	`vi.useFakeTimers()` / `vi.advanceTimersByTime`	`jest.useFakeTimers()`
Module mock	`t.mock.module` (flag)	`vi.mock()` (hoisted)	`jest.mock()` (hoisted)
Snapshot	`t.assert.snapshot`	`toMatchSnapshot()`	same
Run	`node --test`	`vitest`	`jest`

> **TL;DR**

Fast, isolated, repeatable, behaviour-focused, readable. API names change between runners; concepts don't.

16.8 TypeScript: what it is and how it's checked

What it is

TypeScript (TS) is JavaScript plus a **static type system**, developed by Microsoft since 2012. You add **type annotations** (`let pages: number`); the **type checker** (`tsc`, or your editor using it) verifies consistent use *before* running. Types are then **erased**: what executes is plain JavaScript — TS has **no runtime**.

- **Static typing**: checked by analyzing code, not running it (JS itself is dynamically typed — §2).
- **Type inference**: TS works out many types (`let pages = 30` is `number`); annotate mainly parameters and public APIs.
- **Structural typing**: compatibility by **shape**, not name — any `{ id: number; name: string }` is a valid `User` ("duck typing" at compile time).
- **Gradual typing**: adopt file by file; `any` switches checking off for a value.
- **Soundness**: TS is deliberately not fully sound — `any`, assertions and some index operations let wrong types through. Types are evidence, not proof.

Why it matters

Catches typos, wrong argument orders, missing `null` checks and forgotten cases at edit time; powers accurate autocomplete and safe renames; documents APIs. The industry default for serious front-end and Node projects (React, Angular, Vue, Next.js support it first-class).

Trade-offs

Benefit	Cost
Bugs caught before running; editor support	A check step and `tsconfig.json`; learning curve for advanced types
Self-documenting APIs	Types lie if you misuse `any` / `as`, and never validate **runtime** data (API JSON still needs a check or a schema library such as Zod)

`tsc` and TypeScript 7

`tsc` **type-checks** and **emits** JavaScript. Emitting is often left to faster tools (esbuild, Vite, SWC — Go/Rust compilers — or Node's type stripping, §16.9), so `tsc` runs only to check: `tsc --noEmit`.

TypeScript 7.0 (released 8 July 2026) is a **native port of the compiler written in Go**; Microsoft reports full builds roughly 8–12× faster than TypeScript 6, same `typescript` package and `tsc` command. It changes defaults and removes legacy options:

In TypeScript 7	Details
New defaults	`strict: true`, `module: esnext`, `target`: current stable ECMAScript, `types: []` (list `@types` packages — community type definitions for JS libraries — explicitly), `noUncheckedSideEffectImports: true`
Removed	`target: es5`, `downlevelIteration`, `moduleResolution: node` / `node10` / `classic`, `module: amd/umd/systemjs/none`, `baseUrl` (use `paths`)
Not yet	A stable programmatic (JavaScript-callable) API — expected in 7.1; tools that call the compiler API may lag

```
npm install -D typescript @types/node   # compiler + Node's type definitions
npx tsc --init                          # create a tsconfig.json
npx tsc --noEmit                        # type-check the project (what CI runs)
npx tsc --noEmit --watch                # re-check on every save
```

(Not run — the project already has TypeScript.)

tsconfig.json

code/tsconfig.json:

```
{
  "compilerOptions": {
    "target": "es2024",
    "lib": ["es2024", "dom"],
    "module": "nodenext",
    "strict": true,
    "noEmit": true,
    "noUncheckedIndexedAccess": true,
    "allowImportingTsExtensions": true,
    "erasableSyntaxOnly": true,
    "verbatimModuleSyntax": true,
    "allowJs": true,
    "checkJs": false
  },
  "include": ["*.ts", "src/*.ts", "15_jsdoc.mjs"],
  "exclude": ["ts_errors.ts", "no_typecheck_demo.ts", "*.test.ts"]
}
```

Option	Why
`target`, `lib`	Output syntax and available built-ins (`dom` provides `console`, as `@types/node` isn't installed)
`module: nodenext`	Follow Node's real module rules (reads `"type"` from `package.json`)
`strict`	`strictNullChecks`, `noImplicitAny`, … — always on (default in TS 7)
`noEmit`	Check only; something else runs the code
`noUncheckedIndexedAccess`	`arr[i]` and `record[key]` are `T | undefined` — catches out-of-range reads
`allowImportingTsExtensions`	Allow `import './shapes.ts'` (Node requires the real extension)
`erasableSyntaxOnly`	Reject TS features that generate code (`enum` — named constants; namespaces with code; parameter properties) — exactly what Node can't strip (TS 5.8+)
`verbatimModuleSyntax`	Force `import type` for type-only imports, so removing types never changes which modules load

```
$ tsc --noEmit
$ echo $?
0
```

No output + exit code 0 = no type errors.

TL;DR

TypeScript = JavaScript + static types checked by `tsc` and erased before running. `strict`; check with `tsc --noEmit`. TS 7 = Go-native, ~10× faster compiler with stricter defaults.

16.9 Running TypeScript: Node's type stripping and other tools

What it is

Since **Node 23.6** (backported to 22.18), `node file.ts` works with no flags: Node **strips the annotations** — replacing them with whitespace, so line/column numbers stay correct without source maps — and runs the JavaScript. No experimental warning since Node 24.3; **stable in Node 24.12**.

Stripping is not checking. code/no_typecheck_demo.ts:

```ts
const price: number = 5;
const qty: number = '2';                  // type error: string is not assignable to number
const total: number = price + qty;        // JS concatenates: 5 + '2' -> '52'
console.log('total =', JSON.stringify(total), '| typeof', typeof total);
```

Output of `node no_typecheck_demo.ts`, then `tsc` on the same file:

```
total = "52" | typeof string
no_typecheck_demo.ts(4,7): error TS2322: Type 'string' is not assignable to type 'number'.
```

Node ran code whose `number` holds `"52"`. **Always run** `tsc --noEmit` too (editor, pre-commit hook, CI).

The rules for Node's type stripping

1. **Only erasable syntax.** Code-generating features fail with `ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX`: `enum`, `namespace`/`module` blocks with code, **parameter properties** (`constructor(public x: number)`), `import x = require()`, legacy decorators (`@name` on classes and members). (`--experimental-transform-types` enables them, with a warning.) `erasableSyntaxOnly` makes the editor flag them.
2. **Real file extensions** in relative imports: `import { area } from './shapes.ts'`.
3. `import type` (or inline `type`) for type-only imports (`verbatimModuleSyntax` enforces it).
4. `tsconfig.json` **is ignored** — no `paths` aliases, no `target` downlevelling (rewriting new syntax for older engines).
5. **Not in** `node_modules`: packages must publish JavaScript (+ `.d.ts` declarations).
6. `.ts` follows the same ESM/CommonJS rules as `.js`; `.mts`/`.cts` force one.

A first draft of `11_narrowing.ts` hit rule 1:

```
class HttpError extends Error { constructor(public status: number) { super(`HTTP ${status}`); } }
                                                ^^^^^^^^^^^^^^
SyntaxError [ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX]: TypeScript parameter property is not supported in strip-only mode
```

`tsc` with `erasableSyntaxOnly` reports `TS1294`. Fix: declare the field and assign it in the constructor.

`code/14_node_runs_ts.ts` follows all the rules:

```ts
import type { Shape } from './shapes.ts';
import { area } from './shapes.ts';

const shapes: Shape[] = [
  { kind: 'rect', w: 2, h: 5 },
  { kind: 'circle', r: 1 },
];
const total: number = shapes.reduce((sum, s) => sum + area(s), 0);
console.log('total area:', total.toFixed(3));
console.log('shapes:', shapes.map((s): string => s.kind).join(', '));
```

Output of `node 14_node_runs_ts.ts`:

```
total area: 13.142
shapes: rect, circle
```

Other ways to run TypeScript

Tool	What it does	Type-checks?
`node file.ts`	Strips erasable types	No
`tsx` (npm)	esbuild-powered runner; all TS syntax and `tsconfig` paths	No
`esbuild` / `Vite` / `SWC`	Transpile or bundle TS for browsers and Node (this book's runner uses esbuild)	No
`tsc`	Checks; can emit `.js` + `.d.ts` (needed when publishing libraries)	**Yes**
`Deno` / `Bun`	Run TS natively (§15.12); `deno check` type-checks	Deno: on request

Typical setup: *run* with Node/tsx/Vite, *check* with `tsc --noEmit`, *publish libraries* with `tsc` emitting declarations.

TL;DR

`node app.ts` works in Node 24 (stable type stripping) but never type-checks. Erasable syntax only, `.ts` extensions, `import type`; keep `tsc --noEmit` in CI.

16.10 Everyday types

How it works

`code/10_ts_basics.ts` (key parts):

```
// 1) Primitive annotations - and inference
let title: string = 'Testing & TypeScript';
let pages = 30;                          // inferred: number
const lang = 'ts';                       // inferred: the LITERAL type "ts" (const)
// ... also bigint and null annotations

// 2) Arrays and tuples
const scores: number[] = [90, 72, 85];
const point: [x: number, y: number] = [3, 4];          // tuple: fixed length & types

// 3) Object types: type alias vs interface; optional and readonly properties
type User = {
  readonly id: number;
  name: string;
  email?: string;                        // optional: string | undefined
};
interface Product { sku: string; price: number }
interface Product { tags: string[] }     // interfaces MERGE (declaration merging)

// 4) Functions: parameter & return types, default params, void
function greet(user: User, greeting = 'Hello'): string { return `${greeting}, ${user.name}!`; }

// 5) Union and literal types: type Status = 'draft' | 'published' | 'archived'; let id: number | string

// 6) any vs unknown: unknown forces you to check before use
function parse(json: string): unknown { return JSON.parse(json); }
const data = parse('{"count": 3}');
if (typeof data === 'object' && data !== null && 'count' in data && typeof data.count === 'number') {
  console.log('6) count + 1 =', data.count + 1);
}

// 7) Type assertions and the non-null assertion - you override the checker (use sparingly)
const maybe = scores.find((n) => n > 80);           // number | undefined
console.log('7)', (maybe as number).toFixed(1), maybe!.toFixed(0));
// 8) ... types disappear at runtime
```

Output of `node 10_ts_basics.ts`:

```
1) string number ts 10000000000000000000n null
2) 3 Ana+Bo 5
3) (no email) office
4) Hello, Ana!
4) Hi, Bo!
5) [LIVE] [DRAFT] A-42
6) count + 1 = 4
7) 90.0 90
8) typeof User at runtime: undefined
```

The type vocabulary

Type	Meaning
`string`, `number`, `boolean`, `bigint`, `symbol`, `null`, `undefined`	JS primitives (lower-case; `String` is the wrapper object)
`'draft'`, `42`, `true`	**Literal types** — exactly that value
`A | B`	**Union** — either
`A & B`	**Intersection** — both at once (combine object types)
`T[]` / `Array<T>`	Array
`[string, number]`	**Tuple** — fixed length, typed positions
`{ a: string; b?: number }`	Object type; `?` optional, `readonly` prevents reassignment
`(x: number) => string`	Function type
`any`	Opt out of checking — errors spread silently
`unknown`	"Could be anything" — must narrow before use. **Use for untrusted input** (JSON, `catch (e)`)
`never`	No value possible (always-throwing function; impossible switch branch)
`void`	A function returns nothing useful
`object`	Any non-primitive

`type` vs `interface`: both describe object shapes. `interface` can be **reopened** (declaration merging, used to extend library types) and `extends` others; `type` can also name unions, tuples and computed types (§16.12). Many teams use `interface` for object shapes and `type` for the rest — consistency matters more than the choice.

Seeing the checker work

`code/ts_errors.ts` (skip-marked) has nine deliberate mistakes:

```ts
type User = { id: number; name: string; email?: string };

const u: User = { id: 1, name: 'Ana', age: 30 };              // 1) excess property
u.id = '2';                                                   // 2) wrong type
console.log(u.email.toLowerCase());                           // 3) possibly undefined

function total(prices: number[]): number {
  return prices.reduce((a, b) => a + b, '');                  // 4) string is not number
}
total(['1.50', '2.00']);                                      // 5) string[] is not number[]

type Status = 'draft' | 'published';
const s: Status = 'publised';                                 // 6) typo caught

function len(x: string | number) {
  return x.length;                                            // 7) number has no length
}

class Point { constructor(private x: number) {} }             // 8) not erasable (Node can't strip)
enum Color { Red, Green }                                     // 9) not erasable either
```

Output of `tsc --ignoreConfig --noEmit --strict --target es2024 --erasableSyntaxOnly ts_errors.ts`:

```
ts_errors.ts(4,39): error TS2353: Object literal may only specify known properties, and 'age' does not exist in
type 'User'.
ts_errors.ts(5,1): error TS2322: Type 'string' is not assignable to type 'number'.
ts_errors.ts(6,13): error TS18048: 'u.email' is possibly 'undefined'.
ts_errors.ts(9,3): error TS2322: Type 'string' is not assignable to type 'number'.
...
ts_errors.ts(14,7): error TS2820: Type '"publised"' is not assignable to type 'Status'. Did you mean
'"published"'?
ts_errors.ts(17,12): error TS2339: Property 'length' does not exist on type 'string | number'.
  Property 'length' does not exist on type 'number'.
ts_errors.ts(20,27): error TS1294: This syntax is not allowed when 'erasableSyntaxOnly' is enabled.
...
```

Format: `file(line,column): error TSxxxx: message` (`--ignoreConfig`: TS 7 won't combine a present `tsconfig.json` with command-line file names). Error 3 is the famous one: with `strict`, "possibly `undefined`" catches `TypeError: Cannot read properties of undefined` (§11.10) in advance. Error 1, the **excess property check**, applies only to object literals written where a type is expected; an object stored in a variable first may carry extras (structural typing).

Common mistakes

- `any` to silence an error — it spreads to everything derived from it. Use `unknown` and narrow.
- Overusing `as` and `!` — when wrong, the runtime error returns.
- Annotating every local instead of letting inference work.
- Trusting types for external data: `(await res.json()) as User` checks nothing. Validate at the boundary (schema library or hand-written guard).

TL;DR

Annotate parameters and public APIs; infer the rest. Unions and literals model real states; `unknown` for untrusted data, avoid `any`; `strict` catches `undefined` bugs.

16.11 Narrowing

What it is

Narrowing is how TS refines a broad type (a union, `unknown`) inside a branch, based on ordinary JavaScript checks it understands.

Check	Narrows by
`typeof x === 'string'`	Primitive type
`x === null`, `x !== undefined`, truthiness `if (x)`	Removing `null` / `undefined` (truthiness also removes `0` and `''` — prefer `x !== undefined` for `number | undefined`)
`x instanceof Date`	Class
`'code' in x`	Property existence
`s.kind === 'circle'`	**Discriminant** property of a union
`function isCat(p): p is Cat`	Your own **type predicate**
`function assertX(v): asserts v is X`	**Assertion function** — narrows everything after the call (must throw otherwise)

Discriminated unions and exhaustiveness

A **discriminated (tagged) union** is a union of object types sharing one literal property (the **discriminant**, here `kind`). Assigning the leftover to `never` in `default` **forces handling every case**: add `{ kind: 'hexagon' }` to `Shape` and `area` stops compiling until you add a branch.

`code/11_narrowing.ts` (key parts):

```ts
type Shape =
  | { kind: 'circle'; radius: number }
  | { kind: 'rect'; width: number; height: number }
  | { kind: 'triangle'; base: number; height: number };

function area(s: Shape): number {
  switch (s.kind) {
    case 'circle': return Math.PI * s.radius ** 2;
    case 'rect': return s.width * s.height;
    case 'triangle': return (s.base * s.height) / 2;
    default: {
      const unreachable: never = s;        // compile error here if a new kind is not handled
      throw new Error(`unknown shape ${JSON.stringify(unreachable)}`);
    }
  }
}

type Result<T> = { ok: true; value: T } | { ok: false; error: string };   // parsePort(text): Result<number>

function isCat(pet: Cat | Dog): pet is Cat { return 'meow' in pet; }   // type predicate

function assertIsString(v: unknown, name: string): asserts v is string {
  if (typeof v !== 'string') throw new TypeError(`${name} must be a string`);
}
```

Output of `node 11_narrowing.ts`:

```
1) text of length 3 | number 3.14 | nothing
2) http 404 | code ENOENT | error x
3) circle=3.14, rect=6.00, triangle=10.00
4) port 8080
4) bad port "http"
5) meow woof
5) after assert: HELLO
6) [ 'Pune', 'unknown' ]
```

Real-world examples

Redux actions (`{ type: 'added' } | { type: 'removed' }`, React Handbook ch6), API response states (`{ status: 'loading' } | { status: 'success'; data } | { status: 'error'; error }`), and `Result` types in exception-free libraries are all discriminated unions.

In the interview

Q: How do you make sure every case of a union is handled? "Model it as a discriminated union, switch on the discriminant, and assign the value to a `never`-typed variable in `default`. Adding a new member makes that assignment a compile error."

- *Type predicate?* A function returning `x is T`; when true, the argument is `T` in the caller.
- *After `if (typeof x === 'string') return;` with `string | number`?* `x` is `number`.

TL;DR

TS narrows through `typeof`, `instanceof`, `in`, equality and discriminants. Model states as discriminated unions; `never` for exhaustive switches; type predicates and assertion functions for custom checks.

16.12 Generics and utility types

What it is

A **generic** is a type parameter (`T`) filled in at use, letting one function work for many types **without losing** the specific type (as `any` would).

`code/12_generics.ts` (key parts):

```
function first<T>(items: readonly T[]): T | undefined { return items[0]; }
const n = first([10, 20]);          // number | undefined  (T inferred)

function longest<T extends { length: number }>(a: T, b: T): T { /* ... */ }   // constraint

function pluck<T, K extends keyof T>(rows: T[], key: K): T[K][] {   // keyof + indexed access
  return rows.map((r) => r[key]);
}
pluck(users, 'name');               // string[]  -  pluck(users, 'nme') would not compile

class TypedCache<K, V> { /* set(k: K, v: V): this; get(k: K): V | undefined - wraps a Map<K, V> */ }

interface Todo { id: number; title: string; done: boolean; tags: string[] }
type TodoPatch = Partial<Omit<Todo, 'id'>>;
type ById = Record<number, Pick<Todo, 'id' | 'title'>>;

type Flags<T> = { [K in keyof T]: boolean };              // mapped type
type ElementOf<T> = T extends readonly (infer E)[] ? E : never;   // conditional type
```

Output of `node 12_generics.ts`:

```
1) 10 a
2) apple [ 1, 2, 3 ]
3) [ 'Ana', 'Bo' ] [ true, false ]
4) 2 3
5) true write ch16
5) [ 1, 'x' ] true
6) { title: true, done: false } urgent
```

Built-in utility types

Utility	Result
`Partial<T>` / `Required<T>`	All properties optional / required
`Readonly<T>`	All properties `readonly` (shallow)
`Pick<T, K>` / `Omit<T, K>`	Keep / drop some properties
`Record<K, V>`	Object with keys `K`, values `V`
`Exclude<U, X>` / `Extract<U, X>`	Remove / keep union members
`NonNullable<T>`	Remove `null` and `undefined`
`ReturnType<F>` / `Parameters<F>`	Return type / parameter tuple
`Awaited<T>`	The value a promise resolves to

Mapped types (`{ [K in keyof T]: … }`) transform every property; **conditional types** (`T extends X ? A : B`, with `infer`) compute types from types. Utility types are built from them; application code rarely needs them directly.

Common mistakes

- A type parameter used only once (`function f<T>(x: T): void`) — `unknown` is simpler.
- `any[]` instead of `T[]` — loses the element type.
- Deep type gymnastics in application code — keep types boring.

TL;DR

Generics (`<T>`) keep precise types across reusable code; constrain with `extends`; `keyof` (union of key names) / `T[K]` (property `K`'s type) for property-safe helpers. Know `Partial`, `Pick`, `Omit`, `Record`, `ReturnType`, `Awaited`.

16.13 Classes, `as const`, `satisfies`, and enums

`code/13_classes_and_modern_ts.ts` (key parts):

```ts
interface Account { deposit(cents: number): void; readonly balance: number }

class BankAccount implements Account {
  #balance = 0;                             // JS private field (runtime)
  private readonly history: number[] = [];    // TS private (compile-time only)
  protected owner: string;
  // ... constructor(owner), deposit(cents), get balance()
}

const ROLES = ['viewer', 'editor', 'admin'] as const;
type Role = (typeof ROLES)[number];           // 'viewer' | 'editor' | 'admin'

const LogLevel = { Debug: 0, Info: 1, Warn: 2, Error: 3 } as const;   // enum alternative

type Route = { path: string; auth: boolean };
const routes = {
  home: { path: '/', auth: false },
  admin: { path: '/admin', auth: true },
} satisfies Record<string, Route>;
// stock['ink'] on a Record<string, number> is number | undefined (noUncheckedIndexedAccess)
```

Output of `node 13_classes_and_modern_ts.ts`:

```
1) Ana's savings: 5000 | accounts: 1
2) viewer:false editor:true admin:true
3) false true
4) home,admin /admin
5) 0
```

Feature	Meaning	Note
`implements Account`	The class must match the interface's shape	Compile time only
`private` / `protected` / `public`	Visibility **checked by TS only** — ordinary property at runtime	Use JS `#private` (§9) for runtime privacy
`readonly`	Can't be reassigned after construction	Compile time only
`abstract class`	Can't be instantiated; subclasses implement abstract members	Erasable
`as const`	Deeply `readonly` with literal types	Derive unions from data (`typeof ROLES[number]`) — one source of truth
`satisfies T` (TS 4.9+)	Check a value against `T` **without widening** it (widening = loosening `'admin'` to `string`, etc.)	`routes.admin` stays known; with `: Record<string, Route>` it would be "any string key"
`enum`	Named constants generating a runtime object	**Not erasable** — rejected by Node's type stripping. Prefer a literal union or an `as const` object

TL;DR

TS classes add `implements` and compile-time `private/protected/readonly`; use `#private` for runtime privacy. Prefer `as const` objects/unions to `enum`; `satisfies` validates config objects while keeping precise types.

16.14 Tests written in TypeScript

`node --test` also finds and type-strips `*.test.ts`, so TypeScript projects use the built-in runner with **no build step**. `code/src/money.ts` splits money without losing cents:

```ts
export type Currency = 'USD' | 'EUR' | 'INR';
export interface Money { readonly cents: number; readonly currency: Currency }
export function money(cents: number, currency: Currency = 'USD'): Money { /* validate, freeze */ }
export function add(a: Money, b: Money): Money { /* same currency only */ }
export function split(m: Money, n: number): Money[] {
  const base = Math.floor(m.cents / n);
  const extra = m.cents % n;
  return Array.from({ length: n }, (_, i) => money(base + (i < extra ? 1 : 0), m.currency));
}
```

`code/money.test.ts`:

```ts
import { describe, it } from 'node:test';
import assert from 'node:assert/strict';
import { money, add, split, type Money } from './src/money.ts';

describe('money', () => {
  it('adds amounts in the same currency', () => {
    assert.deepEqual(add(money(150), money(275)), { cents: 425, currency: 'USD' });
  });
  // ... 'refuses to mix currencies'
  it('splits without losing a cent', () => {
    const parts: Money[] = split(money(1000), 3);
    assert.deepEqual(parts.map((p) => p.cents), [334, 333, 333]);
    assert.equal(parts.reduce((t, p) => t + p.cents, 0), 1000);
  });
  it('values are immutable', () => {
    const m = money(5);
    assert.throws(() => { (m as { cents: number }).cents = 6; }, TypeError);
  });
});
```

Output of `node --test money.test.ts`: `ℹ tests 4`, `ℹ pass 4`, `ℹ fail 0`.

"values are immutable" relies on `Object.freeze` plus module strict mode (assigning a frozen property throws `TypeError`); the `as { cents: number }` cast steps around TS's (correct) `readonly` error so the test proves the *runtime* guard too.

Type-checking test files needs Node's type definitions. This project doesn't install `@types/node` (hence `tsconfig.json` excludes `*.test.ts`); checking the file alone shows what a fresh project gets:

```
money.test.ts(2,30): error TS2591: Cannot find name 'node:test'. Do you need to install type definitions for node?
Try `npm i --save-dev @types/node` and then add 'node' to the types field in your tsconfig.
...
```

Real projects: `npm i -D @types/node` and `"types": ["node"]` in `compilerOptions` (required in TS 7, where `types` defaults to `[]`).

The full suite in `code/` (`node --test` finds all four test files): `tests 31`, `suites 8`, `pass 29`, `fail 0`, `skipped 1`, `todo 1`.

16.15 JSDoc types: TypeScript checking for plain JavaScript

What it is

TypeScript can check **JavaScript** using types in **JSDoc comments** (`/** @param {number} x */`). `// @ts-check` at the top of a file (or `checkJs: true`) makes the editor and `tsc` check it — nothing to compile or strip. Svelte and webpack are well-known codebases typed this way.

JSDoc tag	Meaning
`@param {Type} name` / `@returns {Type}`	Parameter and return types
`@type {Type}`	A variable's type
`@typedef {object} Name` + `@property`	A named object type
`@template T`	A generic type parameter
`@import` / `import('./x.js').Type`	Types from other modules

`code/15_jsdoc.mjs`:

```
// @ts-check
/**
 * @typedef {object} LineItem
 * @property {string} sku
 * @property {number} price  unit price in cents
 * @property {number} [qty]  optional, defaults to 1
 */

/**
 * @param {LineItem[]} items
 * @returns {number} total in cents
 */
export function sum(items) {
  return items.reduce((t, i) => t + i.price * (i.qty ?? 1), 0);
}

/** @template T @param {T[]} xs @returns {T | undefined} */
export function last(xs) { return xs[xs.length - 1]; }

/** @type {Record<string, 'admin' | 'user'>} */
const roles = { ana: 'admin', bo: 'user' };
```

Output of `node 15_jsdoc.mjs`:

```
sum  : 1200
last : c
roles: { ana: 'admin', bo: 'user' }
```

The file passes the project's `tsc --noEmit`. With mistakes — `code/jsdoc_errors.mjs`:

```
/**
 * @param {number} cents
 * @returns {string}
 */
function formatCents(cents) {
  return (cents / 100).toFixed(2);
}

formatCents('150');                       // string passed where number expected
/** @type {number} */
const count = formatCents(99);            // string result assigned to a number
console.log(count.toUpperCase());         // number has no toUpperCase
```

Output of `tsc --ignoreConfig --noEmit --allowJs --checkJs --strict --target es2024 jsdoc_errors.mjs`:

```
jsdoc_errors.mjs(10,13): error TS2345: Argument of type 'string' is not assignable to parameter of type 'number'.
jsdoc_errors.mjs(12,7): error TS2322: Type 'string' is not assignable to type 'number'.
jsdoc_errors.mjs(13,19): error TS2339: Property 'toUpperCase' does not exist on type 'number'.
```

Trade-offs

	`.ts` files	JSDoc + `@ts-check`
Syntax	Concise	Verbose comments
Build/strip step	Needed (or Node type stripping)	None
Advanced types	Everything	Most things (complex types can live in a `.d.ts`)
Adoption	Rename files	One comment line per file — easiest first step for an existing JS codebase

TL;DR

`// @ts-check` + JSDoc give TypeScript checking to plain `.js` with zero build step — a gentle way to adopt types.

Chapter 16 TL;DR

- **Tests** run code and assert results; unit (fast, isolated) → integration → E2E (few). Test behaviour, not internals; exit code gates CI.
- **`node:assert/strict`**: `equal` (Object.is), `deepEqual` (structure, type-strict), `throws`, **`await rejects`**, `match`.
- **`node:test`**: `test`/`it`, `describe`, hooks, subtests, `skip`/`todo`/`only`, `timeout`; `node --test` (finds `*.test.*` and `test/`), `--watch`, reporters.
- **Async**: return/await promises; a missing `await` can make a failing test pass.
- **Doubles**: dependency injection + stubs/fakes first; `t.mock.fn`/`t.mock.method` spies (auto-restored); **fake timers**; module mocks need `--experimental-test-module-mocks`.
- **Snapshots** (`t.assert.snapshot`, `--test-update-snapshots`) — review them; **coverage** (`--experimental-test-coverage`) — a hint, not a goal.
- **TypeScript** = JS + static types, erased at runtime; structural typing; `strict`; **`tsc --noEmit`**. **TS 7** (July 2026): Go-native, ~10× faster, stricter defaults, `types: []`.
- **`node file.ts`** works (stable type stripping, Node 24) but **never type-checks**. Erasable syntax only, `.ts` extensions, `import type`.
- Types: literals, unions, tuples, `interface` vs `type`, **`unknown`** **over** **`any`**, `never`. **Narrowing** + **discriminated unions** with a `never` exhaustive check.
- **Generics** with constraints and `keyof`; `Partial`, `Pick`, `Omit`, `Record`, `ReturnType`, `Awaited`.
- Classes: `implements`, TS-only `private`/`readonly` vs runtime `#private`; **`as const`** over `enum`; **`satisfies`** checks without widening.
- **`*.test.ts`** run directly with `node --test`; type-checking them needs `@types/node`.
- **JSDoc +** **`// @ts-check`**: type-check plain JavaScript with no build step.

Next: put it all together for interviews — Chapter 17: JavaScript for Interviews.

Chapter 17 — JavaScript for Interviews

A JavaScript interview usually has three parts: a **coding round** (an algorithm problem, in JavaScript — so know which built-ins are fast and which are traps), a **"build it yourself" round** (a utility libraries normally give you — `debounce`, `Promise.all`, a deep clone), and a **language round** ("what does this print, and why?"). This chapter prepares all three without re-teaching the language: answers link to the sections that teach it, and to the Chapter 19 question bank for say-it-out-loud answers.

What you'll learn

1. The JavaScript **DSA toolkit** (data structures and algorithms, §17.1): arrays as stacks/queues, `Map` / `Set`, sort comparators and stability, number pitfalls, building strings, `BigInt` — with a complexity table.
2. **Twenty-one coding patterns** (§17.2), each with two or three classic problems in runnable, asserted code — including a hand-written `MinHeap`, since JavaScript has none.
3. **Implement-from-scratch classics** with tests (§17.3): `debounce`, `throttle`, `curry`, `compose` / `pipe`, `memoize`, `flatten`, a `bind` polyfill, deep clone, deep equal, an LRU cache, an `EventEmitter`, `Promise` combinator polyfills.
4. **Fifty-three tricky questions** with real outputs (§17.4): coercion (automatic type conversion), equality, hoisting and the TDZ, closures, `this`, prototypes, event-loop order, copies, garbage collection (automatic freeing of unreachable memory) and leaks.

Before you start. This chapter assumes the whole book up to Chapter 16. The tests use `node:assert/strict` and fake timers from `node:test`, explained in §16.2 and §16.5. The same patterns are solved in Python in the Python Handbook's Chapter 19: Python for Coding Interviews — handy for comparing; here we focus on what is specific to JavaScript.

All code is in `code/`. Every program checks itself with `assert` and exits with an error if any answer is wrong; outputs below are real, from Node 24.

File	Contents
`01_toolkit.mjs`	Arrays, queue, grids, Map/Set, sorting, numbers, strings, BigInt
`02_patterns_arrays.mjs`	Two pointers, sliding window, hashing, stack, prefix sums, intervals
`03_patterns_search_graphs_dp.mjs`	MinHeap, two heaps, k-way merge, binary search, BFS, DFS, DP, backtracking
`04_classics_functions.mjs`	debounce, throttle, curry, compose/pipe, memoize, flatten, bind/call
`05_classics_data.mjs`	structuredClone, deepClone, deepEqual, LRU cache, EventEmitter
`06_promise_polyfills.mjs`	all/allSettled/race/any, timeout, promisify, pool, retry
`07_qa_language.js`	Outputs for the language questions (CommonJS, sloppy — non-strict — mode on purpose)
`08_qa_async_and_memory.mjs`	Outputs for event-loop, strict-mode and memory questions
`09_patterns_lists_trees_tries.mjs`	Linked lists, fast & slow pointers, trees, tries (Patterns 13–16)
`10_patterns_graphs_greedy_bits_math.mjs`	Topological sort, union-find, greedy, quickselect/Dutch flag/merge sort, bits, math & geometry (Patterns 17–21)

17.1 The JavaScript DSA toolkit

What it is

You must turn an idea ("use a queue", "count with a hash map") into working code fast. JavaScript has fewer ready-made structures than Java or Python — no priority queue (always returns the smallest/most

urgent item), no deque (add/remove at both ends), no sorted map — so know what the built-ins cost and how to fill the gaps.

- **Hash map (hash table):** key → value storage where a *hash function* turns the key into a slot index, so lookups don't scan. `Map`, `Set` and objects are built on them; O(1) *on average* (colliding keys need a little extra work).
- **Big-O notation:** how work (or memory) grows with input size `n`, ignoring constants: O(1) constant, O(log n) halving each step, O(n) linear, O(n log n) good sorting, O(n²) quadratic (10× input ≈ 100× work). State both *time* and extra *space*.

Why it matters

Two JavaScript traps sink otherwise-correct solutions: **hidden O(n) operations** (`shift`, `unshift`, `includes` inside a loop) and **wrong defaults** (`sort()` comparing numbers as text, floating point). The table is the difference between O(n) and O(n²).

Complexity of the built-ins

Operation	Cost	Note
`arr[i]`, `arr[i] = x`, `arr.length`, `arr.at(-1)`	O(1)	
`push`, `pop`	O(1) amortized (averaged over calls)	Use the end of an array as a **stack**
`shift`, `unshift`, `splice(i, …)`	O(n)	Every later element moves — avoid in loops
`indexOf`, `includes`, `find`, `some`	O(n)	Put values in a `Set` if you look them up repeatedly
`slice`, spread `[...a]`, `concat`, `Array.from`	O(k)	k = number of elements copied
`map`, `filter`, `reduce`, `forEach`	O(n)	Plus the callback's cost
`sort`, `toSorted`	O(n log n)	Stable (guaranteed since ES2019); V8 uses TimSort (merge + insertion sort hybrid)
`Map` / `Set` `get`, `set`, `has`, `add`, `delete`	O(1) average	The spec only requires *sublinear* on average; engines use hash tables
Object property read/write, `in`, `Object.hasOwn`	O(1) average	
`Object.keys` / `entries`, `[...map]`	O(n)	
`str[i]`, `str.length`, `charCodeAt`	O(1)	Strings are immutable
`str.slice`, `split`, `join`, `[...str]`	O(n)	
`JSON.stringify`, `structuredClone`	O(size)	
Heap push/pop (you write it, §17.2)	O(log n)	

Arrays as stack, queue and grid

From `code/01_toolkit.mjs`:

```
const stack = [];
stack.push(1); stack.push(2);
console.log('stack pop:', stack.pop(), 'peek:', stack.at(-1));   // O(1) at the end

const q = [1, 2, 3];
console.log('queue shift:', q.shift(), 'left:', q);             // shift is O(n)!

// A real O(1) queue: keep a head index instead of shifting
class Queue {
  #items = []; #head = 0;
  push(x) { this.#items.push(x); }
  shift() {
    if (this.#head === this.#items.length) return undefined;
    const x = this.#items[this.#head++];
    if (this.#head > 1024 && this.#head * 2 > this.#items.length) {   // compact occasionally
      this.#items = this.#items.slice(this.#head); this.#head = 0;
    }
    return x;
  }
  get size() { return this.#items.length - this.#head; }
}

// 2D grid: Array.from, NOT fill with an array (shared rows!)
const bad = new Array(2).fill(new Array(2).fill(0));
bad[0][0] = 9;
const grid = Array.from({ length: 2 }, () => new Array(2).fill(0));
grid[0][0] = 9;
```

Output:

```
stack pop: 2 peek: 1
queue shift: 1 left: [ 2, 3 ]
Queue: a 1
fill(array) shares rows: [[9,0],[9,0]]
Array.from rows are separate: [[9,0],[0,0]]
```

1. **Stack** (LIFO) = `push` / `pop`; peek with `at(-1)`.
2. **Queue** (FIFO) with `shift()` is fine when small, but each `shift` moves every element. Measured in Node 24: emptying 10,000 elements took 0.3 ms, 100,000 took 0.6 s, 400,000 took 10 s — quadratic (V8 shortcuts small arrays only). For BFS on large graphs (nodes joined by edges) keep a **head index** — the `Queue` above, or `for (let head = 0; head < queue.length; head++)` (§17.2).
3. `fill(x)` puts the *same* value in every slot — with an array, every row is one object. `Array.from({ length }, () => …)` calls the function per row (Chapter 7).

`Map` and `Set`

```
const freq = new Map();
for (const ch of 'banana') freq.set(ch, (freq.get(ch) ?? 0) + 1);
const visited = new Set();
visited.add(`${1},${2}`);                          // compound key as a string
new Set([[1, 2]]).has([1, 2]);                     // false: arrays compare by reference
const a = new Set([1, 2, 3]), b = new Set([2, 3, 4]);
a.union(b); a.intersection(b); a.difference(b);    // ES2025
```

Output:

```
freq: [ [ 'b', 1 ], [ 'a', 3 ], [ 'n', 2 ] ]
set: [ 3, 1, 2 ] has 2? true
visited has "1,2"? true | Set of arrays: false
union [ 1, 2, 3, 4 ] intersection [ 2, 3 ] difference [ 1 ]
```

- Prefer `Map` over a plain object for counting and lookups: any key type, no inherited keys like `constructor`, a real `size`, and insertion-order iteration (§8.1).
- `Map` and `Set` compare keys with **SameValueZero** (like `===`, except `NaN` equals `NaN`). Two arrays with equal contents are *different* keys, so encode coordinates as a string (`` `${r},${c}` ``) or a number (`r *`

`cols + c`).

- The ES2025 set methods (`union`, `intersection`, `difference`, `symmetricDifference`, `isSubsetOf`, …) are in Node 22+ and all current browsers (§8.2).

Sorting: comparators and stability

```
[10, 9, 1, 100].sort();                              // compared as STRINGS
[10, 9, 1, 100].sort((x, y) => x - y);               // numeric
people.toSorted((p, r) => p.age - r.age);            // stable: ties keep input order
people.toSorted((p, r) => r.age - p.age || p.name.localeCompare(r.name));  // two keys
```

Output:

```
default sort: [ 1, 10, 100, 9 ]
numeric sort: [ 1, 9, 10, 100 ]
by age (stable): Bob Dee Ann Cid
age desc, name asc: Ann Cid Bob Dee
original untouched: Ann Bob Cid Dee
```

1. A **comparator** `(a, b) => number`: negative = `a` first, positive = `b` first, `0` = tie. `a - b` ascending, `b - a` descending.
2. **Multi-key**: chain with `||` — a `0` from the first key lets the second decide.
3. **Stability**: equal elements keep their input order (Bob before Dee); guaranteed since ES2019, so you can sort by the secondary key first, then the primary.
4. `sort` **mutates**; `toSorted` (ES2023) returns a new array — say which you use.
5. Mistake: a boolean comparator (`(a, b) => a > b`) never returns negative — engine-dependent, wrong (§7.6).

Number pitfalls

```
0.1 + 0.2;                          // 0.30000000000000004
Number.MAX_SAFE_INTEGER;            // 9007199254740991 = 2**53 - 1
2 ** 53 + 1;                        // 9007199254740992 (can't be represented)
Math.trunc(-7 / 2); Math.floor(-7 / 2);   // -3 and -4
-7 % 3;                             // -1 (sign follows the dividend)
(2 ** 31 + 2) >> 1;                 // -1073741823: bit operators use 32-bit ints
Math.max(...[]);                    // -Infinity
```

Output:

```
0.1 + 0.2 = 0.30000000000000004 | === 0.3? false
close(0.1+0.2, 0.3): true
MAX_SAFE_INTEGER: 9007199254740991 | 2**53 + 1 = 9007199254740992
isSafeInteger(2**53): false
7 / 2 = 3.5 | Math.trunc: 3 | -7/2 trunc: -3 floor: -4
-7 % 3 = -1 | true modulo: 2
mid without overflow worry: 5 | but >> breaks above 2**31: -1073741823
Infinity sentinels: Infinity -Infinity
Math.max(...[]) = -Infinity | parseInt("08") = 8 | Number("") = 0
NaN === NaN: false | Number.isNaN(NaN): true | isNaN("x"): true Number.isNaN("x"): false
```

Pitfall	What to do
All numbers are 64-bit floats (IEEE 754 doubles, which can't store most fractions exactly), so `0.1 + 0.2 !== 0.3`	Compare with a tolerance (`Math.abs(a - b) < Number.EPSILON * scale`), or work in integers (cents, not dollars)
No integer division	`Math.trunc(a / b)` (towards zero) or `Math.floor(a / b)` (towards −∞) — they differ for negatives
`%` is a *remainder*, sign follows the left side	For array wrap-around use `((i % n) + n) % n`
Integers are exact only up to `2**53 - 1`	Check with `Number.isSafeInteger`; use `BigInt` beyond (e.g. factorials, large Fibonacci, 64-bit IDs)
Bitwise operators (`|`, `^`, `&`, `<<`, `>>`, `>>>`, `~`) convert to 32-bit signed ints	
`Infinity` / `-Infinity`	Handy initial values for min/max; but `Math.max(...[])` is `-Infinity` and spreading a very large array (about 125,000+ elements in Node 24 on this machine — the limit depends on stack size) throws `RangeError` — use a loop or `reduce`
`isNaN('x')` coerces first	Use `Number.isNaN`

Background: §2.10 Numbers in depth.

Strings: immutable, so build with arrays

```
const parts = [];
for (let i = 0; i < 5; i++) parts.push(i);
parts.join('-');                          // '0-1-2-3-4'
'd'.charCodeAt(0) - 97;                   // 3: letter -> index 0..25
[...'hello'].reverse().join('');          // 'olleh'
```

- `s[0] = 'x'` silently does nothing (throws in strict mode). Edit via `[...s]`, change, `join('')`.
- `+=` in a loop is fine in modern engines (V8 uses *ropes* — trees of pieces), but `push` + `join` is the portable idiom and makes the O(n) cost obvious.
- A 26-slot counting array (`charCodeAt(0) - 97`) beats a `Map` for lowercase-letter problems.
- `[...s]` splits by **code point** (emoji stay whole); `s.split('')` by UTF-16 code unit (§6.5).

`BigInt`

```
const big = 2n ** 64n;                    // 18446744073709551616n, typeof 'bigint'
const factorial = (n) => (n <= 1n ? 1n : n * factorial(n - 1n));
factorial(25n);                           // 15511210043330985984000000n
7n / 2n;                                  // 3n (division truncates)
1n == 1; 1n === 1;                        // true, false
1n + 1;                                   // TypeError: can't mix BigInt and Number
JSON.stringify(1n);                       // TypeError
```

`BigInt` (ES2020): integers of any size (`n` suffix or `BigInt(x)`); no fractions, no `Math.*`, no mixing with numbers (convert explicitly), slower — use only when values may pass `2**53`.

In the interview

30-second answer — "What should I watch for when doing algorithms in JavaScript?" Arrays are stacks (`push` / `pop`); queues need a head index, not `shift`. `Map` / `Set` give O(1) lookups (string keys for coordinates). Always pass `sort` a comparator; it mutates. Numbers are doubles: `Math.trunc` for integer division, `%` can be negative, exact only to $2^{53} - 1$ (then `BigInt`). No heap — write one.

- *Is `Array.prototype.sort` stable?* Yes, required since ES2019.

- *What does `sort()` do with no comparator?* Converts to strings and compares UTF-16 code units, with `undefined` last — so `[10, 9]` stays `[10, 9]`.
- *Why a `Map` rather than `{}` for counting?* Any key type, no prototype keys (`'constructor' in {}` is `true`), `size`, insertion order, optimised for frequent adds/removes.

Self-check

1. Locale-aware alphabetical sort? → `arr.sort((a, b) => a.localeCompare(b))`.
2. `Number.isSafeInteger(2 ** 53)`? → `false` — the largest safe integer is `2 ** 53 - 1`.

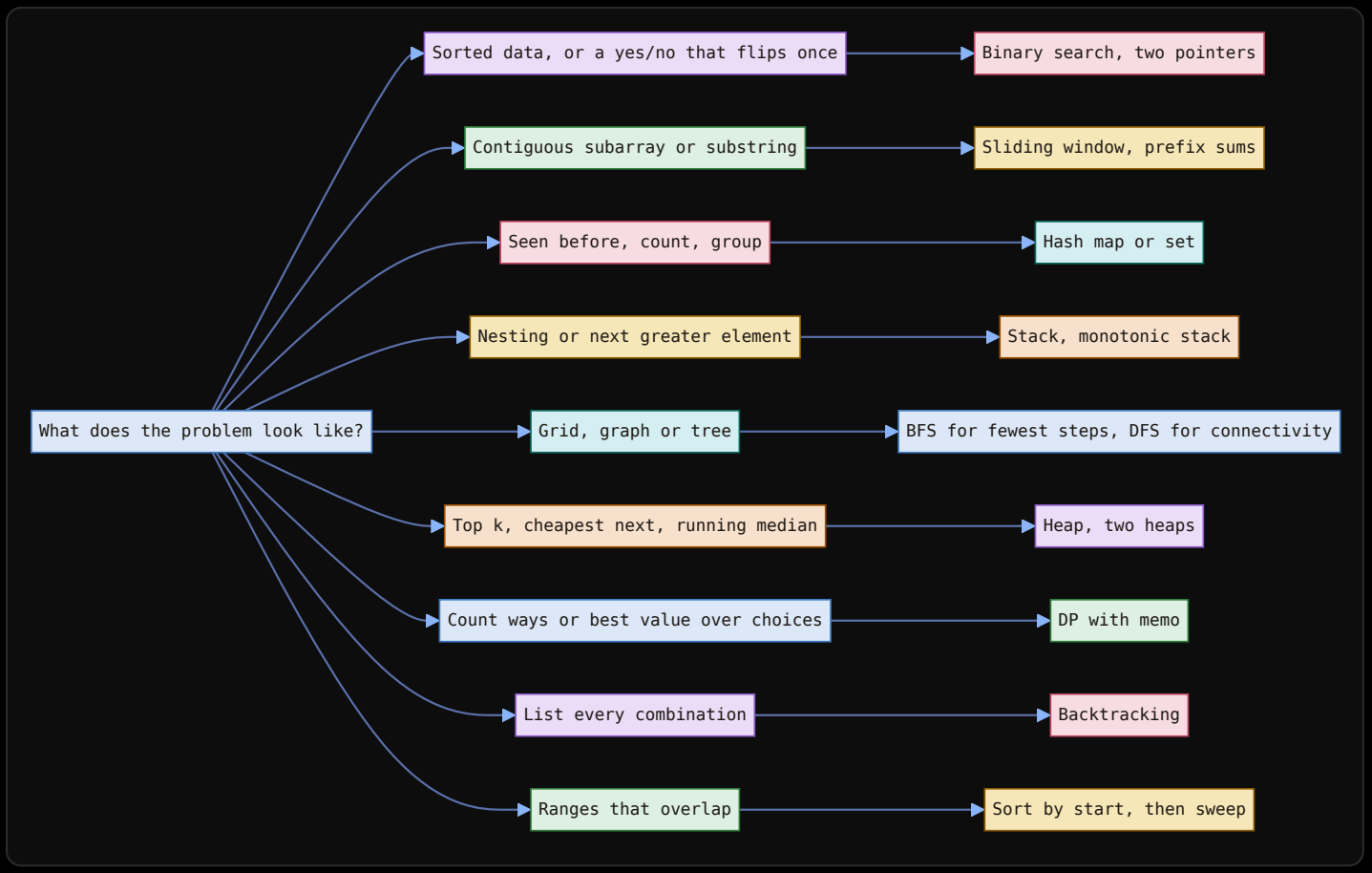
TL;DR

Stack = `push`/`pop`; queue = head index; grid = `Array.from`. `Map`/`Set` for O(1) lookups (string keys for pairs). Always give `sort` a comparator; it's stable; `toSorted` doesn't mutate. Floats: tolerance for equality, `Math.trunc` for division, safe up to $2^{53} - 1$, `BigInt` beyond.

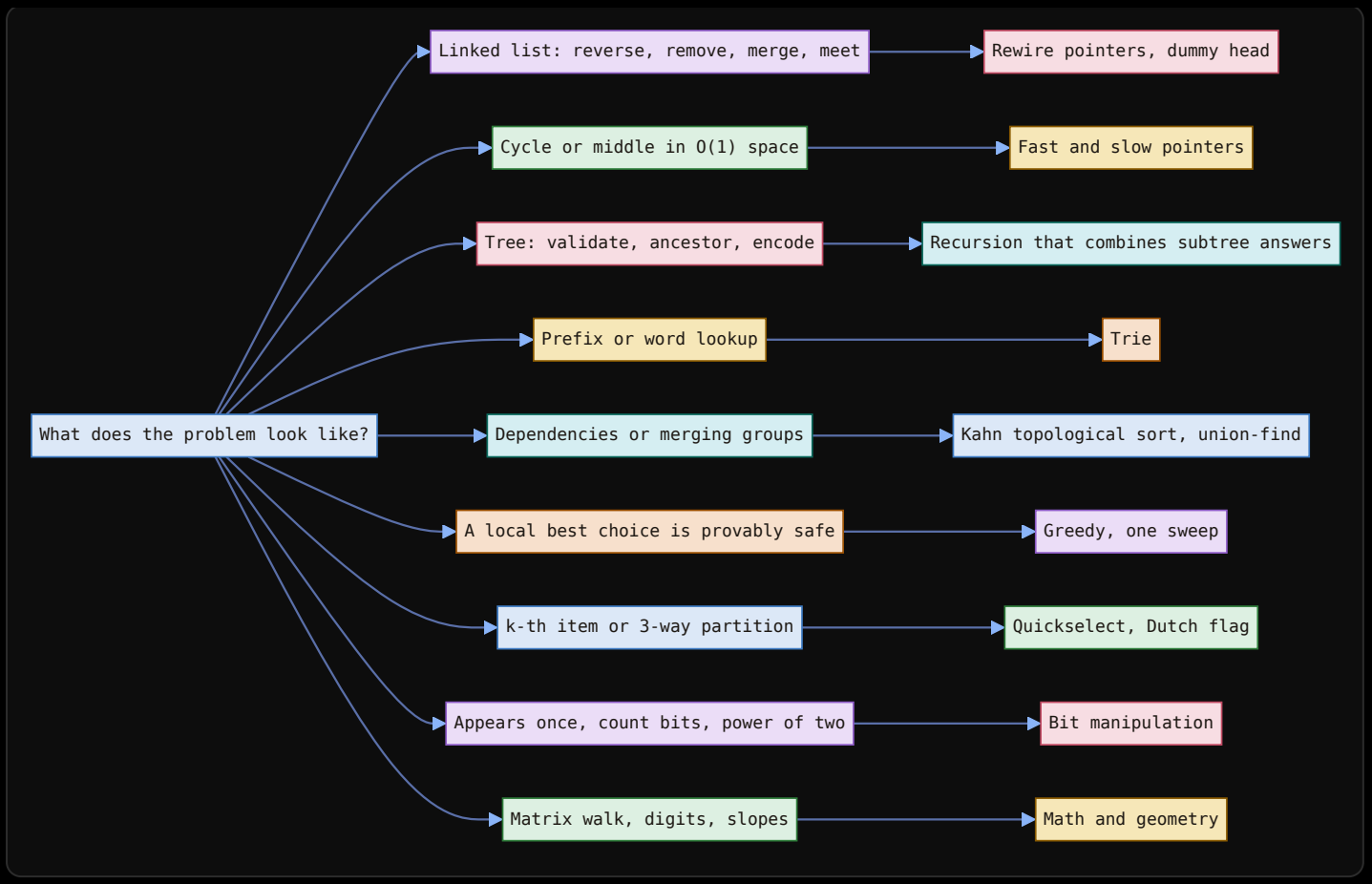
17.2 Coding patterns

What it is

A **pattern** is a reusable problem-solving shape; most interview problems are one of about twenty in disguise, and recognising it gives you the algorithm and complexity almost for free. The chart is a starting point, not a law.



Patterns 13–21 — specific data structures and algorithm families:



Each pattern: idea, signal, asserted code, cost. Full programs: `code/02_patterns_arrays.mjs`, `code/03_patterns_search_graphs_dp.mjs`, `code/09_patterns_lists_trees_tries.mjs` and `code/10_patterns_graphs_greedy_bits_math.mjs`. Each asserts every result shown below and ends by printing `all … asserts passed`.

Pattern 1 — Two pointers

Idea: two indexes move towards each other (or in the same direction) so each element is looked at once. **Signal:** sorted array, pairs, palindromes, in-place partitioning. **Cost:** O(n) time, O(1) space.

```
function pairSum(sorted, target) {
  let lo = 0, hi = sorted.length - 1;
  while (lo < hi) {
    const s = sorted[lo] + sorted[hi];
    if (s === target) return [lo, hi];
    if (s < target) lo++; else hi--;      // too small: need a bigger left value
  }
  return null;
}
assert.deepEqual(pairSum([1, 3, 4, 6, 9], 12), [1, 4]);

function isPalindrome(s) {
  const t = s.toLowerCase().replace(/[^a-z0-9]/g, '');
  for (let i = 0, j = t.length - 1; i < j; i++, j--) if (t[i] !== t[j]) return false;
  return true;
}
assert.equal(isPalindrome('A man, a plan, a canal: Panama'), true);
```

Why it's correct: if the sum is too small, the smallest element can't be part of any answer with a *smaller* partner, so discarding it (`lo++`) loses nothing.

Pattern 2 — Sliding window

Idea: keep a window `[start, end)` over a sequence; grow the end, shrink the start when a rule breaks. **Signal:** "longest/shortest/max-sum **contiguous** subarray or substring". **Cost:** O(n) — each index enters and

leaves the window once.

```
function longestUnique(s) {
  const last = new Map();                  // char -> last index seen
  let start = 0, best = 0;
  for (let end = 0; end < s.length; end++) {
    const ch = s[end];
    if (last.has(ch) && last.get(ch) >= start) start = last.get(ch) + 1;   // shrink
    last.set(ch, end);
    best = Math.max(best, end - start + 1);
  }
  return best;
}
assert.equal(longestUnique('abcabcbb'), 3);

function maxSumK(nums, k) {                // fixed-size window: add the new, drop the old
  let sum = 0;
  for (let i = 0; i < k; i++) sum += nums[i];
  let best = sum;
  for (let i = k; i < nums.length; i++) { sum += nums[i] - nums[i - k]; best = Math.max(best, sum); }
  return best;
}
assert.equal(maxSumK([2, 1, 5, 1, 3, 2], 3), 9);
```

Pattern 3 — Hashing

Idea: trade memory for time — remember what you've seen in a `Map` / `Set` so a later lookup is O(1). **Signal:** "find a pair/duplicate", "group by", "count". **Cost:** O(n) time, O(n) space.

```
function twoSum(nums, target) {
  const indexOf = new Map();
  for (let i = 0; i < nums.length; i++) {
    const need = target - nums[i];
    if (indexOf.has(need)) return [indexOf.get(need), i];   // check BEFORE inserting
    indexOf.set(nums[i], i);
  }
  return null;
}
assert.deepEqual(twoSum([2, 7, 11, 15], 9), [0, 1]);
assert.deepEqual(twoSum([3, 3], 6), [0, 1]);

function groupAnagrams(words) {
  const groups = new Map();
  for (const w of words) {
    const key = [...w].sort().join('');      // canonical form = the hash key
    if (!groups.has(key)) groups.set(key, []);
    groups.get(key).push(w);
  }
  return [...groups.values()];
}
```

Checking before inserting makes `[3, 3]` work (the first `3` isn't paired with itself). Sorting an anagram's letters gives every anagram the same *canonical* key; `Map.groupBy` (§8.7 does the grouping in one call.

Pattern 4 — Stack (and monotonic stack)

Idea: a stack remembers "things still open" — brackets, unfinished elements. A **monotonic stack** keeps its contents increasing or decreasing and answers "next greater/smaller element" questions. **Cost:** O(n) — every index is pushed and popped at most once.

```
function isBalanced(s) {
  const pairs = { ')': '(', ']': '[', '}': '{' };
  const st = [];
  for (const ch of s) {
    if ('([{'.includes(ch)) st.push(ch);
    else if (ch in pairs) { if (st.pop() !== pairs[ch]) return false; }
  }
  return st.length === 0;                  // leftovers = unclosed brackets
}

function dailyTemperatures(t) {            // days until a warmer day
  const ans = new Array(t.length).fill(0);
  const st = [];                           // indices whose temperatures are decreasing
  for (let i = 0; i < t.length; i++) {
    while (st.length && t[i] > t[st.at(-1)]) { const j = st.pop(); ans[j] = i - j; }
    st.push(i);
  }
  return ans;
}
assert.deepEqual(dailyTemperatures([73, 74, 75, 71, 69, 72, 76, 73]), [1, 1, 4, 2, 1, 1, 0, 0]);
```

Pattern 5 — Prefix sums

Idea: precompute running totals `p[i] = nums[0] + … + nums[i-1]`; then any range sum is one subtraction. Combined with a hash map it counts subarrays with a given sum, even with negative numbers (where a sliding window fails). **Cost:** O(n) setup, O(1) per query.

```
function prefix(nums) {
  const p = [0];
  for (const x of nums) p.push(p.at(-1) + x);
  return (i, j) => p[j + 1] - p[i];        // sum of nums[i..j] inclusive
}
assert.equal(prefix([3, 1, 4, 1, 5, 9])(2, 5), 19);

function subarraySum(nums, k) {            // how many subarrays sum to k?
  const count = new Map([[0, 1]]);         // the empty prefix
  let sum = 0, total = 0;
  for (const x of nums) {
    sum += x;
    total += count.get(sum - k) ?? 0;      // earlier prefixes that leave exactly k
    count.set(sum, (count.get(sum) ?? 0) + 1);
  }
  return total;
}
assert.equal(subarraySum([1, -1, 0], 0), 3);
```

Pattern 6 — Intervals

Idea: sort by start, then sweep once, merging or counting overlaps. **Signal:** meetings, bookings, ranges. **Cost:** O(n log n) for the sort.

```js
function mergeIntervals(intervals) {
  const sorted = intervals.toSorted((a, b) => a[0] - b[0]);   // don't mutate the input
  const out = [];
  for (const [s, e] of sorted) {
    const last = out.at(-1);
    if (last && s <= last[1]) last[1] = Math.max(last[1], e);   // overlap: extend
    else out.push([s, e]);
  }
  return out;
}
assert.deepEqual(mergeIntervals([[8, 10], [1, 3], [2, 6], [15, 18]]), [[1, 6], [8, 10], [15, 18]]);

function minRooms(meetings) {              // sweep line: sorted starts vs sorted ends
  const starts = meetings.map(m => m[0]).sort((a, b) => a - b);
  const ends = meetings.map(m => m[1]).sort((a, b) => a - b);
  let rooms = 0, best = 0, j = 0;
  for (const s of starts) {
    while (ends[j] <= s) { j++; rooms--; }  // a meeting ending at s frees its room
    rooms++; best = Math.max(best, rooms);
  }
  return best;
}
assert.equal(minRooms([[0, 30], [5, 10], [15, 20]]), 2);
```

Pattern 7 — Heap (write your own `MinHeap`)

What it is: a **binary heap** is a complete binary tree (every level full except the last, filled from the left) stored in an array, every parent ≤ its children (*min*-heap); the smallest is at index 0. Children of `i`: `2i+1`, `2i+2`; parent `(i-1)>>1` (so node 6's parent is 2). **Signal:** "k largest/smallest", "merge k sorted lists", "median of a stream", "always process the cheapest next" (Dijkstra, scheduling). **Cost:** `push`/`pop` O(log n), `peek` O(1).

JavaScript has **no built-in priority queue**, so write one (or describe it) — 25 lines:

```
export class MinHeap {
  #a = [];
  #less;
  constructor(less = (x, y) => x < y) { this.#less = less; }   // custom ordering
  get size() { return this.#a.length; }
  peek() { return this.#a[0]; }
  push(x) {                                    // add at the end, sift UP
    const a = this.#a; a.push(x);
    let i = a.length - 1;
    while (i > 0) {
      const p = (i - 1) >> 1;
      if (!this.#less(a[i], a[p])) break;
      [a[i], a[p]] = [a[p], a[i]]; i = p;
    }
  }
  pop() {                                      // move the last to the root, sift DOWN
    const a = this.#a;
    if (a.length === 0) return undefined;
    const top = a[0], last = a.pop();
    if (a.length) {
      a[0] = last;
      let i = 0;
      for (;;) {
        const l = 2 * i + 1, r = l + 1;
        let m = i;
        if (l < a.length && this.#less(a[l], a[m])) m = l;
        if (r < a.length && this.#less(a[r], a[m])) m = r;
        if (m === i) break;
        [a[i], a[m]] = [a[m], a[i]]; i = m;
      }
    }
    return top;
  }
}

function kLargest(nums, k) {                   // min-heap of size k: O(n log k)
  const heap = new MinHeap();
  for (const x of nums) { heap.push(x); if (heap.size > k) heap.pop(); }
  const out = []; while (heap.size) out.push(heap.pop());
  return out.reverse();
}
assert.deepEqual(kLargest([3, 2, 1, 5, 6, 4], 2), [6, 5]);
```

The `less` parameter makes it a general **priority queue**: pass `(x, y) => x > y` for a max-heap, or compare a field. Dijkstra's shortest paths uses `[distance, node]` pairs:

```
function dijkstra(graph, src) {
  const dist = new Map([[src, 0]]);
  const pq = new MinHeap((x, y) => x[0] < y[0]);   // [distance, node]
  pq.push([0, src]);
  while (pq.size) {
    const [d, u] = pq.pop();
    if (d > dist.get(u)) continue;                 // stale entry: a shorter path was found
    for (const [v, w] of graph[u] ?? []) {
      const nd = d + w;
      if (nd < (dist.get(v) ?? Infinity)) { dist.set(v, nd); pq.push([nd, v]); }
    }
  }
  return Object.fromEntries(dist);
}
const roads = { A: [['B', 4], ['C', 1]], C: [['B', 2], ['D', 7]], B: [['D', 1]] };
assert.deepEqual(dijkstra(roads, 'A'), { A: 0, C: 1, B: 3, D: 4 });
```

Two heaps: the running median. To report the median of a stream after every new number, keep the smaller half in a **max-heap** (`low`) and the larger half in a **min-heap** (`high`), with `low` holding the same number of items or one more. The median is then `low`'s top, or the average of both tops. Each `add` is O(log n); `median` is O(1).

```
class MedianFinder {
  #low = new MinHeap((x, y) => x > y);        // max-heap: the smaller half
  #high = new MinHeap();                      // min-heap: the larger half
  add(x) {
    if (this.#low.size === 0 || x <= this.#low.peek()) this.#low.push(x); else this.#high.push(x);
    if (this.#low.size > this.#high.size + 1) this.#high.push(this.#low.pop());   // rebalance
    else if (this.#high.size > this.#low.size) this.#low.push(this.#high.pop());
  }
  median() {
    return this.#low.size > this.#high.size ? this.#low.peek() : (this.#low.peek() + this.#high.peek()) / 2;
  }
}
const mf = new MedianFinder();
assert.deepEqual([5, 15, 1, 3].map(x => { mf.add(x); return mf.median(); }), [5, 10, 5, 4]);
```

k-way merge. To merge k sorted lists, keep one "front" item per list in a min-heap; pop the smallest, then push the next item from the same list. With N items in total the cost is O(N log k) — better than concatenating and sorting (O(N log N)) when k is small.

```
function mergeKSorted(lists) {
  const pq = new MinHeap((x, y) => x[0] < y[0]);    // [value, list index, position]
  lists.forEach((l, i) => { if (l.length) pq.push([l[0], i, 0]); });
  const out = [];
  while (pq.size) {
    const [v, i, j] = pq.pop();
    out.push(v);
    if (j + 1 < lists[i].length) pq.push([lists[i][j + 1], i, j + 1]);   // refill from the same list
  }
  return out;
}
assert.deepEqual(mergeKSorted([[1, 4, 7], [2, 5, 8], [0, 3, 6, 9], []]), [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]);
```

Shortcut worth knowing: if you only need to sort once, `toSorted` is simpler and also O(n log n). A heap wins when items arrive over time or you need only the top k.

Pattern 8 — Binary search

Idea: halve the search space each step. **Signal:** sorted data, or a yes/no condition that flips once as a number grows ("the smallest speed that works"). **Cost:** O(log n) steps.

```
function lowerBound(a, target) {            // first index with a[i] >= target
  let lo = 0, hi = a.length;                // half-open range [lo, hi)
  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (a[mid] < target) lo = mid + 1; else hi = mid;
  }
  return lo;                                 // a.length if every value is smaller
}
assert.equal(lowerBound([1, 3, 3, 5, 8], 3), 1);
assert.equal(lowerBound([1, 3, 3, 5, 8], 4), 3);

function minEatingSpeed(piles, hours) {     // binary search on the ANSWER
  let lo = 1, hi = Math.max(...piles);
  const ok = (k) => piles.reduce((t, p) => t + Math.ceil(p / k), 0) <= hours;
  while (lo < hi) { const mid = (lo + hi) >> 1; if (ok(mid)) hi = mid; else lo = mid + 1; }
  return lo;
}
assert.equal(minEatingSpeed([3, 6, 7, 11], 8), 4);
```

One template (half-open `[lo, hi)`, `lo < hi`, `hi = mid`) avoids binary search's famous off-by-one bugs. JavaScript has no `bisect` built-in; `indexOf` / `findIndex` are linear.

Pattern 9 — Breadth-first search (BFS)

Idea: explore in rings — all nodes 1 step away, then 2 steps, … using a queue. The first time you reach a node is by a **shortest path** (in steps). **Signal:** "minimum number of moves", unweighted shortest path, level-order traversal. **Cost:** O(V + E) (V vertices, E edges; a grid is O(rows × cols)).

```
function shortestPath(grid, [sr, sc], [tr, tc]) {
  const R = grid.length, C = grid[0].length;
  const dist = Array.from({ length: R }, () => new Array(C).fill(-1));   // -1 = unvisited
  const queue = [[sr, sc]]; dist[sr][sc] = 0;
  for (let head = 0; head < queue.length; head++) {       // index instead of shift()
    const [r, c] = queue[head];
    if (r === tr && c === tc) return dist[r][c];
    for (const [dr, dc] of [[1, 0], [-1, 0], [0, 1], [0, -1]]) {
      const nr = r + dr, nc = c + dc;
      if (nr >= 0 && nr < R && nc >= 0 && nc < C && grid[nr][nc] !== '#' && dist[nr][nc] === -1) {
        dist[nr][nc] = dist[r][c] + 1; queue.push([nr, nc]);   // mark when ENQUEUED
      }
    }
  }
  return -1;
}
assert.equal(shortestPath(['..#.', '..#.', '....'], [0, 0], [0, 3]), 7);
```

Mark a cell visited when you **enqueue** it, not when you dequeue it — otherwise duplicates enter the queue (still correct, but time and memory can blow up).

Pattern 10 — Depth-first search (DFS)

Idea: go as deep as possible, then back up. **Signal:** connectivity ("number of islands"), cycle detection, topological order (dependencies first — Pattern 17), exploring every path. **Cost:** O(V + E).

```
function numIslands(rows) {
  const g = rows.map(r => [...r]);          // copy: don't destroy the caller's grid
  let count = 0;
  for (let r = 0; r < g.length; r++) for (let c = 0; c < g[0].length; c++) {
    if (g[r][c] !== '1') continue;
    count++;
    const st = [[r, c]]; g[r][c] = '0';      // explicit stack = iterative DFS
    while (st.length) {
      const [x, y] = st.pop();
      for (const [nx, ny] of [[x + 1, y], [x - 1, y], [x, y + 1], [x, y - 1]]) {
        if (g[nx]?.[ny] === '1') { g[nx][ny] = '0'; st.push([nx, ny]); }   // ?. handles edges
      }
    }
  }
  return count;
}
assert.equal(numIslands(['11000', '11000', '00100', '00011']), 3);

function topoSort(n, edges) {               // recursive DFS with three colours
  const adj = Array.from({ length: n }, () => []);
  for (const [u, v] of edges) adj[u].push(v);
  const state = new Array(n).fill(0);       // 0 new, 1 on the current path, 2 done
  const order = [];
  const visit = (u) => {
    if (state[u] === 1) throw new Error('cycle');
    if (state[u] === 2) return;
    state[u] = 1;
    for (const v of adj[u]) visit(v);
    state[u] = 2; order.push(u);            // post-order
  };
  for (let u = 0; u < n; u++) visit(u);
  return order.reverse();
}
assert.deepEqual(topoSort(4, [[0, 1], [0, 2], [1, 3], [2, 3]]), [0, 2, 1, 3]);
```

JavaScript-specific: recursion depth is limited (roughly ten thousand frames in V8, depending on frame size) and V8 has no tail-call optimisation (reusing the frame when the last action is a call). For inputs that can be deep — a 1000×1000 grid of land — use the iterative version with an explicit stack.

Pattern 11 — Dynamic programming with memoization

What it is: **dynamic programming (DP)** solves a problem by combining answers to smaller versions of the same problem, storing each answer so it is computed once. **Top-down** = recursion + a cache (**memoization**); **bottom-up** = fill a table in order. **Signal:** "count the ways", "minimum cost", "longest …", when the plain recursion repeats the same calls. **Cost:** (number of states) × (work per state).

```
function climb(n, memo = new Map()) {       // ways to climb n stairs taking 1 or 2 steps
  if (n <= 1) return 1;
  if (memo.has(n)) return memo.get(n);
  const v = climb(n - 1, memo) + climb(n - 2, memo);
  memo.set(n, v);
  return v;
}
assert.equal(climb(70), 308061521170129);   // exact: below 2**53
assert.equal(Number.isSafeInteger(climb(80)), false);   // too big: use BigInt

function coinChange(coins, amount) {        // bottom-up: fewest coins for each amount
  const dp = new Array(amount + 1).fill(Infinity); dp[0] = 0;
  for (let a = 1; a <= amount; a++)
    for (const c of coins) if (c <= a) dp[a] = Math.min(dp[a], dp[a - c] + 1);
  return dp[amount] === Infinity ? -1 : dp[amount];
}
assert.equal(coinChange([1, 2, 5], 11), 3);

function lcs(a, b) {                        // longest common subsequence: 2D table
  const dp = Array.from({ length: a.length + 1 }, () => new Array(b.length + 1).fill(0));
  for (let i = 1; i <= a.length; i++) for (let j = 1; j <= b.length; j++)
    dp[i][j] = a[i - 1] === b[j - 1] ? dp[i - 1][j - 1] + 1 : Math.max(dp[i - 1][j], dp[i][j - 1]);
  return dp[a.length][b.length];
}
assert.equal(lcs('abcde', 'ace'), 3);
```

Without the memo, `climb(70)` would make about 6 × 10^{14} calls; with it, about 140. Note the JavaScript twist: the answer for 80 stairs is larger than `2**53`, so a real solution would switch to `BigInt` (§17.1). For multi-argument states, use a string key (`` `${i},${j}` ``) or a 2D array.

Pattern 12 — Backtracking

Idea: build a solution one choice at a time; after exploring a choice, **undo** it and try the next (choose → explore → un-choose). Prune branches that can't succeed. **Signal:** "list all subsets/permutations/combinations", puzzles (N-Queens, Sudoku). **Cost:** exponential — 2^n subsets, $n!$ permutations — so inputs are small.

```js
function subsets(nums) {
  const out = [], cur = [];
  const go = (i) => {
    if (i === nums.length) { out.push([...cur]); return; }   // COPY the current path
    cur.push(nums[i]); go(i + 1); cur.pop();                 // choose, explore, un-choose
    go(i + 1);                                               // skip nums[i]
  };
  go(0);
  return out;
}
assert.deepEqual(subsets([1, 2]), [[1, 2], [1], [2], []]);

function nQueens(n) {                          // pruning with Sets of attacked lines
  const cols = new Set(), d1 = new Set(), d2 = new Set();
  let count = 0;
  const place = (r) => {
    if (r === n) { count++; return; }
    for (let c = 0; c < n; c++) {
      if (cols.has(c) || d1.has(r - c) || d2.has(r + c)) continue;
      cols.add(c); d1.add(r - c); d2.add(r + c);
      place(r + 1);
      cols.delete(c); d1.delete(r - c); d2.delete(r + c);
    }
  };
  place(0);
  return count;
}
assert.equal(nQueens(8), 92);
```

The most common JavaScript bug here is `out.push(cur)` without copying: every entry would be the *same* array, which ends up empty. (`permutations` with a `used` array is in the code file.)

Pattern 13 — Linked lists

What it is: a **linked list** is a chain of nodes; each node holds a value and a `next` reference to the following node (`null` at the end). Unlike an array there is no index: reaching the k-th node means walking k steps, but inserting or removing a node you already hold is O(1) — you only rewire references. JavaScript has no built-in linked list, so problems hand you a `{ val, next }` shape. **Signal:**

"reverse", "remove the k-th from the end", "merge", "where do two lists meet". **Cost:** usually O(n) time, O(1) extra space.

```
class ListNode { constructor(val, next = null) { this.val = val; this.next = next; } }

function reverseList(head) {                // flip every arrow while walking
  let prev = null, cur = head;
  while (cur) {
    const next = cur.next;                  // save before overwriting
    cur.next = prev;
    prev = cur; cur = next;
  }
  return prev;                              // the old tail is the new head
}
assert.deepEqual(toArray(reverseList(fromArray([1, 2, 3, 4]))), [4, 3, 2, 1]);

function removeKthFromEnd(head, k) {        // one pass: `lead` runs k nodes ahead
  const dummy = new ListNode(0, head);      // dummy head: removing the head needs no special case
  let lead = dummy, trail = dummy;
  for (let i = 0; i < k; i++) lead = lead.next;
  while (lead.next) { lead = lead.next; trail = trail.next; }
  trail.next = trail.next.next;             // trail sits just before the node to remove
  return dummy.next;
}
assert.deepEqual(toArray(removeKthFromEnd(fromArray([1, 2, 3, 4, 5]), 2)), [1, 2, 3, 5]);

function intersection(a, b) {               // first node shared by two lists, or null
  let p = a, q = b;
  while (p !== q) { p = p ? p.next : b; q = q ? q.next : a; }
  return p;
}
```

`intersection`: `p` walks A then B, `q` walks B then A; both travel `lenA + lenB` steps, so they reach the shared node together (or `null` together). It compares **nodes**, not values. The **dummy head** (a fake node before the real head) gives every node a predecessor, so changing the head needs no special case — e.g. `removeKthFromEnd` can stop *before* the head when k equals the length. `fromArray`/`toArray` are in the code file.

Pattern 14 — Fast & slow pointers

Idea: two pointers move through a sequence at different speeds — slow one step, fast two steps. If the sequence loops, fast eventually laps slow and they meet; if it ends, fast reaches the end when slow is halfway. **Signal:** "is there a cycle / where does it start", "middle of a list", any "does repeating this step loop forever?" question. **Cost:** O(n) time, **O(1) space** — the point, since a `Set` of visited nodes would also work but costs O(n).

```
function cycleStart(head) {                  // Floyd's algorithm
  let slow = head, fast = head;
  while (fast && fast.next) {
    slow = slow.next; fast = fast.next.next;
    if (slow === fast) {                     // met inside the loop
      let p = head;
      while (p !== slow) { p = p.next; slow = slow.next; }
      return p;                              // they meet again at the loop's first node
    }
  }
  return null;                               // fast fell off the end: no cycle
}

function middle(head) {                      // second middle for even lengths
  let slow = head, fast = head;
  while (fast && fast.next) { slow = slow.next; fast = fast.next.next; }
  return slow;
}
assert.equal(middle(fromArray([1, 2, 3, 4, 5])).val, 3);

function isHappy(n) {                        // n -> sum of squared digits; reaches 1 or loops
  const next = (x) => { let s = 0; while (x > 0) { const d = x % 10; s += d * d; x = Math.floor(x / 10); } return
s; };
  let slow = n, fast = next(n);
  while (fast !== 1 && slow !== fast) { slow = next(slow); fast = next(next(fast)); }
  return fast === 1;
}
assert.equal(isHappy(19), true);             // 19 -> 82 -> 68 -> 100 -> 1
assert.equal(isHappy(2), false);
```

Why the second phase finds the loop's start: say the start is `a` steps from the head and the pointers meet `b` steps into a loop of length `c`. Fast walked twice as far, so `2(a + b) = a + b + k·c` for some whole number `k`, which gives `a = k·c − b`. Walking `a` more steps from the meeting point therefore lands exactly on the loop's start — the same place a pointer starting at the head reaches after `a` steps. `isHappy` shows the pattern without any list: the "next" function defines an implicit chain.

Pattern 15 — Trees

What it is: a **binary tree** is a set of nodes where each node has at most two children (`left`, `right`); the top node is the **root**, nodes without children are **leaves**. A **binary search tree (BST)** adds a rule: every value in the left subtree is smaller than the node, every value in the right subtree larger. Most tree answers are a short recursive function that asks each subtree a question and combines the two answers. **Signal:** "validate", "lowest common ancestor", "path sum", "rebuild / encode a tree".

Cost: O(n) time, O(h) stack for a tree of height `h` (h = log n when balanced — both sides of every node about equally deep — n when it degenerates into a line).

```
class TreeNode { constructor(val, left = null, right = null) { this.val = val; this.left = left; this.right =
right; } }

function isValidBST(node, lo = -Infinity, hi = Infinity) {   // pass the allowed range DOWN
  if (!node) return true;
  if (node.val <= lo || node.val >= hi) return false;
  return isValidBST(node.left, lo, node.val) && isValidBST(node.right, node.val, hi);
}
assert.equal(isValidBST(fromLevels([5, 1, 7, null, null, 4, 8])), false);   // 4 is right of 5

function lca(root, p, q) {                  // lowest common ancestor of values p and q
  if (!root || root.val === p || root.val === q) return root;
  const left = lca(root.left, p, q), right = lca(root.right, p, q);
  return left && right ? root : left ?? right;   // found on both sides: this node splits them
}
const family = fromLevels([3, 5, 1, 6, 2, 0, 8, null, null, 7, 4]);
assert.equal(lca(family, 7, 4).val, 2);

function serialize(root) {                  // preorder, '#' marks an empty child
  const out = [];
  const go = (n) => { if (!n) { out.push('#'); return; } out.push(n.val); go(n.left); go(n.right); };
  go(root);
  return out.join(',');
}
function deserialize(text) {
  const tokens = text.split(',');
  let i = 0;
  const go = () => {
    const t = tokens[i++];
    if (t === '#') return null;
    const node = new TreeNode(Number(t));
    node.left = go(); node.right = go();      // same order as serialize
    return node;
  };
  return go();
}
assert.equal(serialize(fromLevels([1, 2, 3, null, null, 4, 5])), '1,2,#,#,3,4,#,#,5,#,#');
```

Classic BST bug: comparing a node only with its parent — in `[5, 1, 7, null, null, 4, 8]`, 4 is a valid left child of 7 but sits in 5's *right* subtree. Passing `(lo, hi)` down catches it. Serialization works because preorder (node, left, right) plus `#` markers fixes the shape: the reader always knows what the next token fills. (`fromLevels`, which builds a tree from a level-order array with `null` gaps, is in the code file; deep, unbalanced trees hit the recursion-depth limit from Pattern 10 — switch to an explicit stack.)

Pattern 16 — Tries

What it is: a **trie** (prefix tree, pronounced "try") stores strings one character per level; words that share a prefix share the same path from the root, and a flag marks "a word ends here". Looking up a word or prefix of length `L` costs O(L), no matter how many words are stored. **Signal:** "autocomplete",

"starts with", "search with a wildcard", "find many words in a grid". **Cost:** O(L) per insert/search; memory O(total characters).

```
class Trie {
  #root = { next: new Map(), end: false };
  insert(word) {
    let n = this.#root;
    for (const ch of word) {
      if (!n.next.has(ch)) n.next.set(ch, { next: new Map(), end: false });
      n = n.next.get(ch);
    }
    n.end = true;                          // "a word finishes here"
  }
  #walk(prefix) {
    let n = this.#root;
    for (const ch of prefix) { n = n.next.get(ch); if (!n) return null; }
    return n;
  }
  search(word) { return this.#walk(word)?.end === true; }
  startsWith(prefix) { return this.#walk(prefix) !== null; }
  match(pattern) {                         // '.' matches any one letter
    const go = (n, i) => {
      if (i === pattern.length) return n.end;
      if (pattern[i] !== '.') { const nx = n.next.get(pattern[i]); return nx ? go(nx, i + 1) : false; }
      for (const nx of n.next.values()) if (go(nx, i + 1)) return true;   // try every branch
      return false;
    };
    return go(this.#root, 0);
  }
}
// words: car, card, care, cat, dog
assert.equal(trie.search('ca'), false);      // a prefix, not a stored word
assert.equal(trie.match('c.r.'), true);      // card, care
```

Word search on a board (find every listed word traceable through neighbouring cells, each cell once per word) combines a trie with backtracking: one DFS per cell walks board and trie together and stops as soon as the letters aren't a prefix of any word — the prune that beats searching word by word.

```
function findWords(board, words) {
  const root = {};
  for (const w of words) { let n = root; for (const ch of w) n = n[ch] ??= {}; n.$word = w; }
  const grid = board.map(row => [...row]);   // copy: cells are marked while exploring
  const R = grid.length, C = grid[0].length, found = [];
  const dfs = (r, c, node) => {
    const ch = grid[r][c], next = node[ch];
    if (!next) return;                       // no word continues this way: prune
    if (next.$word) { found.push(next.$word); delete next.$word; }   // report once
    grid[r][c] = '#';                        // choose
    for (const [dr, dc] of [[1, 0], [-1, 0], [0, 1], [0, -1]]) {
      const nr = r + dr, nc = c + dc;
      if (nr >= 0 && nr < R && nc >= 0 && nc < C && grid[nr][nc] !== '#') dfs(nr, nc, next);
    }
    grid[r][c] = ch;                         // un-choose
  };
  for (let r = 0; r < R; r++) for (let c = 0; c < C; c++) dfs(r, c, root);
  return found.sort();
}
assert.deepEqual(findWords(['cats', 'ored', 'waxy'], ['cat', 'cats', 'car', 'core', 'cow', 'dog', 'red', 'wax',
'tax']),
  ['car', 'cat', 'cats', 'core', 'cow', 'red', 'wax']);
```

JavaScript notes: a `Map` per node is the clean choice; a plain object (`n[ch] ??= {}`) is shorter, but then a stored key such as `$word` must never collide with a letter. For lowercase-only input, a 26-slot array per node is the fastest variant (§17.1).

Pattern 17 — Topological sort and union-find

What it is: two graph tools that plain BFS/DFS (Patterns 9–10) don't cover directly. A **topological order** lists the nodes of a directed graph (one whose edges point one way, like "A before B") so every edge goes from earlier to later ("take prerequisites first"); it exists only if the graph has no cycle. **Kahn's algorithm** builds it with BFS: repeatedly take a node whose **in-degree** (number of incoming edges still unprocessed) is 0. A **union-find** (disjoint-set union) keeps a collection of groups and answers "are x and y in the same group?" and "merge their groups" in almost constant time. **Signal:** "order of tasks with dependencies", "can all courses be finished" → topological sort; "number of connected groups", "does this edge create a cycle", merging accounts or friend circles → union-find. **Cost:** Kahn O(V + E); union-find O(α(n)) per operation, where α (the inverse Ackermann function) is below 5 for any real input.

```
function courseOrder(n, prereqs) {          // prereqs: [course, mustTakeFirst]
  const adj = Array.from({ length: n }, () => []);
  const indeg = new Array(n).fill(0);
  for (const [c, p] of prereqs) { adj[p].push(c); indeg[c]++; }
  const order = [];
  for (let i = 0; i < n; i++) if (indeg[i] === 0) order.push(i);
  for (let h = 0; h < order.length; h++)     // `order` doubles as the queue
    for (const v of adj[order[h]]) if (--indeg[v] === 0) order.push(v);
  return order.length === n ? order : null;  // null = cycle: impossible schedule
}
assert.deepEqual(courseOrder(4, [[1, 0], [2, 0], [3, 1], [3, 2]]), [0, 1, 2, 3]);
assert.equal(courseOrder(2, [[0, 1], [1, 0]]), null);

class UnionFind {
  constructor(n) {
    this.parent = Array.from({ length: n }, (_, i) => i);   // everyone starts alone
    this.size = new Array(n).fill(1);
    this.sets = n;
  }
  find(x) {                                 // follow parents to the group's root
    while (this.parent[x] !== x) {
      this.parent[x] = this.parent[this.parent[x]];        // path halving: flatten as we go
      x = this.parent[x];
    }
    return x;
  }
  union(a, b) {
    let ra = this.find(a), rb = this.find(b);
    if (ra === rb) return false;            // already in the same group
    if (this.size[ra] < this.size[rb]) [ra, rb] = [rb, ra];
    this.parent[rb] = ra;                   // hang the smaller tree under the bigger one
    this.size[ra] += this.size[rb];
    this.sets--;
    return true;
  }
}
const countComponents = (n, edges) => { const uf = new UnionFind(n); for (const [a, b] of edges) uf.union(a, b);
return uf.sets; };
assert.equal(countComponents(5, [[0, 1], [1, 2], [3, 4]]), 2);

function redundantEdge(edges) {             // first edge that closes a cycle (nodes 1..n)
  const uf = new UnionFind(edges.length + 1);
  for (const e of edges) if (!uf.union(e[0], e[1])) return e;
  return null;
}
assert.deepEqual(redundantEdge([[1, 2], [2, 3], [3, 4], [1, 4], [1, 5]]), [1, 4]);
```

Kahn vs Pattern 10's DFS sort: both O(V + E); Kahn detects cycles by counting ("fewer than n came out"), needs no recursion, and works in "waves" (minimum number of semesters). Union-find handles edges arriving one at a time (rerunning BFS each time would cost O(V + E) per edge); `union` returning `false` means the ends already shared a root — the edge closes a cycle. Path halving and union by size make it fast; without them `find` can degrade to O(n).

Pattern 18 — Greedy

What it is: a **greedy** algorithm makes the choice that looks best right now and never revisits it. It is fast (usually one pass, often after a sort) but only correct when you can argue that a locally best choice never blocks a globally best answer — so in the interview, always say *why* the greedy choice is safe, or give a counterexample (one input where it gives the wrong answer) and switch to DP (Pattern 11). **Signal:** "minimum number of …", "can you reach …", scheduling, and problems where a sort plus one sweep feels natural. **Cost:** O(n) or O(n log n) with a sort.

```
function canJump(nums) {                    // nums[i] = max jump length from i
  let reach = 0;                            // farthest index reachable so far
  for (let i = 0; i < nums.length; i++) {
    if (i > reach) return false;            // stuck before i
    reach = Math.max(reach, i + nums[i]);
  }
  return true;
}
assert.equal(canJump([3, 2, 1, 0, 4]), false);

function gasStation(gas, cost) {            // start index for a full circle, or -1
  let total = 0, tank = 0, start = 0;
  for (let i = 0; i < gas.length; i++) {
    const d = gas[i] - cost[i];
    total += d; tank += d;
    if (tank < 0) { start = i + 1; tank = 0; }   // nobody in start..i can be the answer
  }
  return total >= 0 ? start : -1;
}
assert.equal(gasStation([1, 2, 3, 4, 5], [3, 4, 5, 1, 2]), 3);

function candies(ratings) {                 // everyone >= 1; higher rating than a neighbour => more
  const n = ratings.length, c = new Array(n).fill(1);
  for (let i = 1; i < n; i++) if (ratings[i] > ratings[i - 1]) c[i] = c[i - 1] + 1;
  for (let i = n - 2; i >= 0; i--) if (ratings[i] > ratings[i + 1]) c[i] = Math.max(c[i], c[i + 1] + 1);
  return c.reduce((a, b) => a + b, 0);
}
assert.equal(candies([1, 3, 4, 5, 2]), 11);  // 1, 2, 3, 4, 1
```

Safety arguments: **Jump game:** every index up to `reach` is reachable, so extending `reach` from each of them loses nothing. **Gas station:** if the tank goes negative at `i` after starting at `s`, every station between `s` and `i` arrived with at least 0 fuel from `s`, so starting there is no better — skip to `i + 1`; and if total gas covers total cost, the last surviving start completes the loop. **Candies:** the left-to-right pass satisfies every "better than my left neighbour" rule with the fewest candies, the right-to-left pass does the same for right neighbours, and `Math.max` keeps both. When greedy fails — coin change with coins `[1, 3, 4]` and amount 6 (greedy picks 4+1+1, the best is 3+3) — use DP. (Pattern 6's meeting-rooms and merge-intervals are greedy too: sort, then sweep.)

Pattern 19 — Sort & search: quickselect, Dutch flag, merge sort

Idea: some questions are really about the sorting algorithms themselves: find the k-th item *without* fully sorting, partition in place, or sort stably. **Quickselect** partitions like quicksort (a **pivot** value; smaller items to its left, the rest to its right) but then continues into only the side that contains the answer. **Signal:** "k-th largest/smallest", "median", "sort 0/1/2 in one pass", "sort a linked list", "count inversions" (pairs that are in the wrong order). **Cost:** quickselect O(n) average (O(n²) worst, made unlikely by a random pivot); Dutch flag O(n) one pass, O(1) space; merge sort O(n log n) always, O(n) extra space, **stable**.

```
function kthLargest(nums, k) {                // quickselect
  const a = [...nums], target = a.length - k;   // k-th largest = index n-k ascending
  let lo = 0, hi = a.length - 1;
  for (;;) {
    const r = lo + Math.floor(Math.random() * (hi - lo + 1));
    [a[r], a[hi]] = [a[hi], a[r]];            // random pivot, moved to the end
    const pivot = a[hi];
    let store = lo;                           // Lomuto partition
    for (let i = lo; i < hi; i++) if (a[i] < pivot) { [a[i], a[store]] = [a[store], a[i]]; store++; }
    [a[store], a[hi]] = [a[hi], a[store]];    // the pivot is now in its final sorted place
    if (store === target) return a[store];
    if (store < target) lo = store + 1; else hi = store - 1;   // keep ONE side
  }
}
assert.equal(kthLargest([3, 2, 1, 5, 6, 4], 2), 5);

function sortColors(a) {                      // Dutch national flag: 0s, 1s, 2s in one pass
  let lo = 0, mid = 0, hi = a.length - 1;     // [0,lo)=0s  [lo,mid)=1s  [mid,hi]=unknown  (hi,end]=2s
  while (mid <= hi) {
    if (a[mid] === 0) { [a[lo], a[mid]] = [a[mid], a[lo]]; lo++; mid++; }
    else if (a[mid] === 1) mid++;
    else { [a[mid], a[hi]] = [a[hi], a[mid]]; hi--; }   // don't move mid: new a[mid] is unchecked
  }
  return a;
}
assert.deepEqual(sortColors([2, 0, 2, 1, 1, 0]), [0, 0, 1, 1, 2, 2]);

function mergeSort(a, cmp = (x, y) => x - y) {
  if (a.length <= 1) return a;
  const mid = a.length >> 1;
  const L = mergeSort(a.slice(0, mid), cmp), R = mergeSort(a.slice(mid), cmp);
  const out = [];
  let i = 0, j = 0;
  while (i < L.length && j < R.length) out.push(cmp(L[i], R[j]) <= 0 ? L[i++] : R[j++]);   // <= : stable
  return out.concat(L.slice(i), R.slice(j));
}
const people = [['ann', 30], ['bob', 25], ['cy', 30], ['di', 25]];
assert.deepEqual(mergeSort(people, (x, y) => x[1] - y[1]).map(p => p[0]), ['bob', 'di', 'ann', 'cy']);
```

Quickselect (O(n) average, whole array, rearranges a copy) vs a size-k heap (O(n log k), works on a stream, no bad worst case). The code file checks `kthLargest` against `toSorted` on 200 random arrays — a cheap test for randomised algorithms. On a linked list merge sort needs no extra array (middle via Pattern 14, sort halves, merge). Everyday code: `toSorted` is already stable O(n log n); write these only when the algorithm is the question.

Pattern 20 — Bit manipulation

What it is: treating a number as its binary digits (**bits**) and using the bitwise operators: `&` (and), `|` (or), `^` (xor: 1 when the bits differ), `~` (not), `<<` (shift left = ×2), `>>` (shift right keeping the sign), `>>>` (shift right filling with 0). **Signal:** "without extra space, find the one that appears once", "count 1-bits", "power of two", subsets as bitmasks (an integer whose bits mark which items are chosen). **Cost:** O(1) per operation. **The JavaScript twist:** numbers are 64-bit floats, but every bitwise operator first converts to a **32-bit signed integer**, so results can come out negative and values above

2^{32} lose their high bits; `x >>> 0` reinterprets the 32 bits as unsigned (no sign bit, so never negative).

```
const singleNumber = (nums) => nums.reduce((acc, x) => acc ^ x, 0);   // x ^ x = 0, x ^ 0 = x
assert.equal(singleNumber([4, 1, 2, 1, 2]), 4);

function countBits(n) {                     // number of 1-bits for every i in 0..n
  const bits = new Array(n + 1).fill(0);
  for (let i = 1; i <= n; i++) bits[i] = bits[i >> 1] + (i & 1);   // DP: drop the last bit
  return bits;
}
assert.deepEqual(countBits(5), [0, 1, 1, 2, 1, 2]);

// swap every pair of neighbouring bits: 0xAAAAAAAA = odd positions, 0x55555555 = even
const swapOddEvenBits = (x) => (((x & 0xAAAAAAAA) >>> 1) | ((x & 0x55555555) << 1)) >>> 0;
assert.equal(swapOddEvenBits(0b10110), 0b101001);
assert.equal(swapOddEvenBits(0x40000000), 0x80000000);   // 2147483648, thanks to >>> 0
assert.equal(0x40000000 << 1, -2147483648);              // the trap without it

const isPowerOfTwo = (n) => n > 0 && (n & (n - 1)) === 0;   // n & (n-1) clears the lowest 1-bit
```

`singleNumber`: xor is commutative and associative, so pairs cancel in any order — O(1) space instead of a `Set`. `countBits` is a tiny DP (`i >> 1` = `i` minus its last bit, `i & 1` = that bit). The swap's final `>>> 0` matters: a 1 in bit 31 makes the signed result negative (the file asserts against a slow bit-by-bit version on `0`, `0x80000000`, `0xFFFFFFFF`). Beyond 32 bits, `BigInt` supports the same operators (`5n & 3n`) except `>>>`.

Pattern 21 — Math & geometry

Idea: simulation and number facts: walk a matrix in order, handle integer limits, compare slopes exactly — spot the invariant (true at every step) and the edge cases. **Signal:** "spiral / rotate a matrix", "reverse digits … 0 on overflow", "points on a line", gcd (greatest common divisor), modular arithmetic. **Cost:** problem-specific — here O(rows × cols), O(digits) and $O(n^2)$.

```js
function spiral(m) {                        // peel layers with four shrinking walls
  const out = [];
  let top = 0, bottom = m.length - 1, left = 0, right = m[0].length - 1;
  while (top <= bottom && left <= right) {
    for (let c = left; c <= right; c++) out.push(m[top][c]);
    top++;
    for (let r = top; r <= bottom; r++) out.push(m[r][right]);
    right--;
    if (top <= bottom) { for (let c = right; c >= left; c--) out.push(m[bottom][c]); bottom--; }
    if (left <= right) { for (let r = bottom; r >= top; r--) out.push(m[r][left]); left++; }
  }
  return out;
}
assert.deepEqual(spiral([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]), [1, 2, 3, 4, 8, 12, 11, 10, 9, 5, 6, 7]);

function reverseInt(x) {                    // 0 if the result leaves the 32-bit signed range
  const sign = x < 0 ? -1 : 1;
  let n = Math.abs(x), r = 0;
  while (n > 0) { r = r * 10 + (n % 10); n = Math.floor(n / 10); }
  r *= sign;
  return r < -(2 ** 31) || r > 2 ** 31 - 1 ? 0 : r;
}
assert.equal(reverseInt(-120), -21);
assert.equal(reverseInt(1534236469), 0);     // 9646324351 > 2^31 - 1

function maxPointsOnLine(points) {          // most points on one straight line
  if (points.length <= 2) return points.length;
  const gcd = (a, b) => (b === 0 ? Math.abs(a) : gcd(b, a % b));
  let best = 0;
  for (let i = 0; i < points.length; i++) {
    const byDir = new Map();
    let same = 1, most = 0;                 // `same` = the anchor plus exact duplicates
    for (let j = i + 1; j < points.length; j++) {
      let dx = points[j][0] - points[i][0], dy = points[j][1] - points[i][1];
      if (dx === 0 && dy === 0) { same++; continue; }
      const g = gcd(dx, dy);
      dx /= g; dy /= g;
      if (dx < 0 || (dx === 0 && dy < 0)) { dx = -dx; dy = -dy; }   // one sign per direction
      const key = `${dx}/${dy}`;
      const c = (byDir.get(key) ?? 0) + 1;
      byDir.set(key, c);
      most = Math.max(most, c);
    }
    best = Math.max(best, most + same);
  }
  return best;
}
assert.equal(maxPointsOnLine([[1, 1], [3, 2], [5, 3], [4, 1], [2, 3], [1, 4]]), 4);
```

`spiral`'s two `if`s stop a lone remaining row/column being walked twice (try 3×1). `reverseInt` is *easier* than in Java/C++: exact to 2^{53}, so no intermediate overflow — one range check at the end. In `maxPointsOnLine` a float slope `dy / dx` is imprecise (`94911150 / 94911151 === 94911151 / 94911152` is `true` though the directions differ, so non-collinear points would count as one line) and infinite for verticals; the gcd-reduced integer pair as a `Map` key avoids both (Pattern 3's canonical-key idea).

Summary table

#	Pattern	Signal words	Time	Extra space
1	Two pointers	sorted, pair, palindrome, in place	O(n)	O(1)
2	Sliding window	longest/shortest contiguous	O(n)	O(k)
3	Hashing	seen before, count, group	O(n)	O(n)
4	Stack / monotonic stack	brackets, next greater	O(n)	O(n)
5	Prefix sums	range sum, subarray sum = k	O(n) + O(1)/query	O(n)
6	Intervals	overlap, merge, rooms	O(n log n)	O(n)
7	Heap (two heaps, k-way merge)	top k, cheapest next, running median	O(n log k)	O(k)
8	Binary search	sorted, minimum value that works	O(log n) × check	O(1)
9	BFS	fewest steps, levels	O(V + E)	O(V)
10	DFS	connected, cycle, order	O(V + E)	O(V)
11	DP + memo	count ways, min cost, overlapping calls	states × transitions	states
12	Backtracking	all combinations, puzzles	exponential	O(depth)
13	Linked lists	reverse, k-th from end, merge, intersection	O(n)	O(1)
14	Fast & slow pointers	cycle, cycle start, middle, "does it loop?"	O(n)	O(1)
15	Trees	validate BST, ancestor, path, serialize	O(n)	O(h)
16	Tries	prefix, autocomplete, wildcard, words in a grid	O(L) per word	O(total chars)
17	Topological sort / union-find	dependencies, order; groups, redundant edge	O(V + E); ≈ O(1) per op	O(V)
18	Greedy	reach, minimum count, schedule (prove the choice)	O(n) or O(n log n)	O(1)–O(n)
19	Sort & search	k-th largest, 3-way partition, stable sort	O(n) avg; O(n log n)	O(1); O(n)
20	Bit manipulation	appears once, count bits, power of two, masks	O(n) or O(1)	O(1)
21	Math & geometry	spiral, overflow, collinear points, gcd	problem-specific	O(1)–O(n)

In the interview

30-second answer — "How do you approach a coding problem?" Restate it; agree on input/output and edge cases (empty, duplicates, negatives). State the brute force and its cost. Find the pattern — sorted → binary search/two pointers; contiguous → window; lookups → hash map; fewest steps → BFS — and state the better complexity before coding. Then test by hand with a normal and an edge case.

- *Why BFS rather than DFS for shortest path?* BFS reaches nodes in order of distance, so the first visit is the shortest; DFS may find a long path first.
- *Top-down or bottom-up DP?* Top-down is easier to derive and computes only needed states; bottom-up avoids recursion-depth limits (important in JavaScript) and allows rolling arrays (keep only the last row or two).

- *How do you implement a priority queue in JavaScript?* A binary heap over an array: push = append + sift up, pop = move last to root + sift down, both O(log n).
- *Why a dummy head in linked-list problems?* It gives every real node a predecessor, so removing or inserting at the head needs no special case.
- *How do you prove a greedy solution?* Show that the greedy choice can be swapped into any optimal answer without making it worse (an **exchange argument**), or that it keeps an invariant (jump game: everything up to `reach` is reachable). If you can't, try a small counterexample and fall back to DP.
- *Quickselect or heap for the k-th largest?* Quickselect: O(n) average, needs the whole array. Heap of size k: O(n log k), works on a stream and has no bad worst case.
- *Kahn's algorithm or DFS for topological order?* Both O(V + E). Kahn counts nodes to detect a cycle and needs no recursion; DFS uses three colours and post-order.

Self-check

1. Longest substring with at most two distinct letters? → Sliding window with a `Map` of counts.
2. Why does `subarraySum` start the map with `[0, 1]`? → So subarrays starting at index 0 (prefix sum exactly `k`) count.

TL;DR

Recognise the pattern (summary table above), state the complexity, then code. JavaScript specifics: hand-write the heap; head-index queues for BFS; iterative DFS for deep inputs; copy backtracking results; bitwise ops are 32-bit signed (`>>> 0` for unsigned); exact integer keys, not float slopes; greedy only with a safety argument.

17.3 Implement-from-scratch classics

What it is

"Implement `X` without the built-in" tests whether you know *how* everyday tools work: closures and timers (`debounce`, `throttle`), `this` and `apply` (`bind`), recursion and object types (deep clone, deep equal), `Map` ordering (LRU cache), promises (`Promise.all`). The versions below are the fuller, **tested** implementations; the Chapter 19 bank (Q245–Q269) has compact whiteboard versions and more tasks (`once`, `sleep`, retry, `get`, `instanceof`, a `Promise` class…).

How the tests control time

`code/04_classics_functions.mjs` uses `node:test` **fake timers** (§16.5): `setTimeout` and `Date.now()` move only when the test calls `tick(ms)`.

```
import { mock } from 'node:test';
mock.timers.enable({ apis: ['setTimeout', 'Date'] });
const tick = (ms) => mock.timers.tick(ms);
```

Debounce

A **debounced** function waits until calls *stop* for `wait` ms, then runs once with the latest arguments. **Use for:** search-as-you-type, auto-save, resize handling. (B Q245)

```
function debounce(fn, wait) {
  let timer = null;
  function debounced(...args) {
    clearTimeout(timer);                              // every call restarts the countdown
    timer = setTimeout(() => { timer = null; fn.apply(this, args); }, wait);
  }
  debounced.cancel = () => { clearTimeout(timer); timer = null; };
  return debounced;
}

const calls = [];
const search = debounce((q) => calls.push(q), 300);
search('j'); tick(100); search('ja'); tick(100); search('jav'); // user still typing
tick(299); assert.deepEqual(calls, []);             // 299 ms after the last key: nothing yet
tick(1);   assert.deepEqual(calls, ['jav']);        // fires once, with the LAST args
search('x'); search.cancel(); tick(1000);
assert.deepEqual(calls, ['jav']);                   // cancelled
```

Output: `debounce calls: [ 'jav' ]`

Interviewers check: the timer lives in a **closure** shared by all calls; `debounced` is a regular `function` forwarding `this` and arguments with `apply` (works as a method); a `cancel` method (Lodash also offers `flush` and `leading`). Superseded calls' arguments are simply dropped.

Throttle

A **throttled** function runs **at most once per `wait` ms**. **Use for:** scroll/mousemove handlers, rate-limited API calls. This version runs on the **leading** edge and once more on the **trailing** edge with the latest arguments, so the final position is never lost. (B Q246)

```
function throttle(fn, wait) {
  let last = -Infinity, timer = null, pendingArgs = null, pendingThis = null;
  function throttled(...args) {
    const now = Date.now(), remaining = wait - (now - last);
    if (remaining <= 0) {                           // window open: run now (leading edge)
      clearTimeout(timer); timer = null;
      last = now; fn.apply(this, args);
    } else {                                        // inside window: remember the latest call
      pendingArgs = args; pendingThis = this;
      timer ??= setTimeout(() => {                  // trailing edge
        last = Date.now(); timer = null;
        fn.apply(pendingThis, pendingArgs);
      }, remaining);
    }
  }
  return throttled;
}

const calls = [], t0 = Date.now();
const onScroll = throttle((y) => calls.push(`${y}@${Date.now() - t0}`), 100);
for (let t = 0; t <= 250; t += 10) { onScroll(t); tick(10); }  // an event every 10 ms
for (let i = 0; i < 20; i++) tick(10);                          // then 200 quiet ms
assert.deepEqual(calls, ['0@0', '90@100', '190@200', '250@300']);
```

Output: `throttle calls: [ '0@0', '90@100', '190@200', '250@300' ]` — 26 events became 4 calls, 100 ms apart, the last carrying the final `250`.

	Debounce	Throttle
Runs	Once, after the calls stop	Regularly, at most once per interval
During a long burst	Never (until it ends)	Every `wait` ms
Typical use	Search box, auto-save, validation	Scroll, drag, resize-driven layout, analytics

For scroll-tied visual updates, `requestAnimationFrame` (once per frame) is often better — §14.5.

Curry

Currying turns `f(a, b, c)` into `f(a)(b)(c)`; the usual version also accepts several arguments at once. It relies on `fn.length` — declared parameters, counting stops at the first default or rest parameter (`(a, b = 2, ...rest) => {}` has length 1). (B Q255, Q256)

```
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) return fn.apply(this, args);     // enough: call it
    return (...more) => curried.apply(this, [...args, ...more]);   // else collect more
  };
}
const add3 = curry((a, b, c) => a + b + c);
assert.equal(add3(1)(2)(3), 6);
assert.equal(add3(1, 2)(3), 6);
const add1 = add3(1);                                  // partials are reusable
assert.equal(add1(10)(20), 31); assert.equal(add1(0)(0), 1);

// Variant: unknown number of arguments, ended by an empty call
const sum = (a) => (b) => (b === undefined ? a : sum(a + b));
assert.equal(sum(1)(2)(3)(), 6);
```

Partials are reusable because each call builds a **new** argument array instead of pushing into a shared one — a common bug.

Compose and pipe

`compose(f, g)(x)` = `f(g(x))` (right to left, as in maths); `pipe(f, g)(x)` = `g(f(x))` (left to right). (B Q42, Q278)

```
const compose = (...fns) => (x) => fns.reduceRight((acc, f) => f(acc), x);
const pipe = (...fns) => (x) => fns.reduce((acc, f) => f(acc), x);
const inc = (x) => x + 1, dbl = (x) => x * 2;
assert.equal(compose(inc, dbl)(5), 11);                // inc(dbl(5))
assert.equal(pipe(inc, dbl)(5), 12);                   // dbl(inc(5))
assert.equal(pipe()(7), 7);                            // no functions = identity
```

Memoize

Memoization caches a pure function's results by its arguments. Design questions: the key (here `JSON.stringify(args)`, overridable) and where the cache lives (a `Map` grows forever — production bounds it, e.g. with the LRU below). (B Q257)

```
function memoize(fn, keyOf = (...args) => JSON.stringify(args)) {
  const cache = new Map();
  return function (...args) {
    const key = keyOf(...args);
    if (cache.has(key)) return cache.get(key);         // has(): a cached undefined still counts
    const value = fn.apply(this, args);
    cache.set(key, value);
    return value;
  };
}
let runs = 0;
const slowSquare = memoize((n) => { runs++; return n * n; });
slowSquare(4); slowSquare(4); slowSquare(5);
assert.equal(runs, 2);
// Recursive functions must call the MEMOIZED version, or only the top call is cached
const fib = memoize((n) => (n < 2 ? n : fib(n - 1) + fib(n - 2)));
```

Output: `memoize runs: 2 | fib(50): 12586269025`. `JSON.stringify` keys can't tell some values apart (`undefined` and `null` in arrays both become `null`; objects with different key orders differ) — offer a custom `keyOf`, or a `WeakMap` for single-object arguments.

Flatten

Flatten to a given depth, like `arr.flat(depth)` (§7.5). (B Q254, Q296)

```
function flatten(arr, depth = Infinity) {
  const out = [];
  for (const x of arr) {
    if (Array.isArray(x) && depth > 0) out.push(...flatten(x, depth - 1));
    else out.push(x);
  }
  return out;
}
function flattenDeep(arr) {                    // iterative: no recursion limit
  const stack = [...arr], out = [];
  while (stack.length) {
    const x = stack.pop();
    if (Array.isArray(x)) stack.push(...x); else out.push(x);
  }
  return out.reverse();                        // we took items from the end
}
const nested = [1, [2, [3, [4]], 5]];
assert.deepEqual(flatten(nested, 1), nested.flat(1));   // [1, 2, [3, [4]], 5]
assert.deepEqual(flattenDeep(nested), nested.flat(Infinity));

function flattenObject(obj, prefix = '', out = {}) {     // frequent variant: dotted keys
  for (const [k, v] of Object.entries(obj)) {
    const key = prefix ? `${prefix}.${k}` : k;
    if (v && typeof v === 'object' && !Array.isArray(v)) flattenObject(v, key, out);
    else out[key] = v;
  }
  return out;
}
assert.deepEqual(flattenObject({ a: 1, b: { c: 2, d: { e: 3 } } }), { a: 1, 'b.c': 2, 'b.d.e': 3 });
```

A `bind` polyfill (and `call`)

A **polyfill** implements a standard feature for environments that lack it (§10.12). `bind` must (1) fix `this`, (2) pre-fill arguments (partial application), (3) under `new`, ignore the bound `this` and construct normally (§9.1–9.2). (B Q250, Q251)

```
Function.prototype.myBind = function (thisArg, ...preset) {
  const target = this;                                  // the function myBind was called on
  if (typeof target !== 'function') throw new TypeError('myBind must be called on a function');
  function bound(...later) {
    if (new.target) return new target(...preset, ...later);   // called with `new`
    return target.apply(thisArg, [...preset, ...later]);
  }
  return bound;
};
const user = { name: 'Ada' };
function greet(greeting, punct) { return `${greeting}, ${this.name}${punct}`; }
const hi = greet.myBind(user, 'Hi');
assert.equal(hi('!'), 'Hi, Ada!');
assert.equal(hi.call({ name: 'Bob' }, '?'), 'Hi, Ada?');   // bound this can't be overridden
function Point(x, y) { this.x = x; this.y = y; }
const p = new (Point.myBind(null, 1))(2);
assert.ok(p instanceof Point);                          // { x: 1, y: 2 }

Function.prototype.myCall = function (thisArg, ...args) {
  const ctx = thisArg == null ? globalThis : Object(thisArg);   // sloppy-mode rule
  const key = Symbol('fn');                                     // unique: can't clash
  ctx[key] = this;                                              // call it AS A METHOD of ctx
  try { return ctx[key](...args); } finally { delete ctx[key]; }
};
```

Output: `myBind: Hi, Ada! | new bound -> {"x":1,"y":2}`

`myCall`'s trick: calling a function as a method of `ctx` makes `this === ctx`. (The real `call` neither boxes primitives — `Object(5)` — nor defaults to `globalThis` for strict functions; that's sloppy-mode behaviour.) Extending built-in prototypes is fine in an interview, **not** in real code — it can clash with future standard methods.

Deep clone: `structuredClone` vs manual

A **shallow copy** (`{ ...obj }`, `[...arr]`, `Object.assign`) shares nested objects; a **deep copy** copies every level. The built-in is **`structuredClone`** (browsers since 2022, Node 17+), using the *structured clone algorithm* (as `postMessage` does). (B Q74, Q252)

From `code/05_classics_data.mjs`:

```
const original = {
  when: new Date(0), tags: new Set(['a']), meta: new Map([['k', { n: 1 }]]),
  nested: { list: [1, { deep: true }] },
};
original.self = original;                                  // a cycle
const copy = structuredClone(original);                    // Date, Set, Map, cycles: all fine
structuredClone({ f() {} });                               // throws DataCloneError
structuredClone(new Point(-3));                            // becomes a plain { x: -3 }
JSON.parse(JSON.stringify({ d: new Date(0), u: undefined, m: new Map([[1, 2]]), n: NaN }));
```

Output:

```
structuredClone keeps Date/Set/Map/cycles: 1970-01-01T00:00:00.000Z [ 'a' ] true
function -> DataCloneError
class instance -> plain object: false {"x":-3}
JSON clone: { d: '1970-01-01T00:00:00.000Z', m: {}, n: null }
```

	Spread / `Object.assign`	`JSON.parse(JSON.stringify(x))`	`structuredClone(x)`	Manual `deepClone`
Depth	Top level only	Deep	Deep	Deep
`Date`	Shared	Becomes a string	✓	✓
`Map` / `Set`	Shared	Become `{}`	✓	✓
`undefined`, `NaN`, `Infinity`	✓	Dropped / `null`	✓	✓
Cycles	✓ (shared)	Throws `TypeError`	✓	✓ (with a `WeakMap`)
Functions	Shared	Dropped	Throws `DataCloneError`	Shared (as written)
Class prototype	Lost (plain object)	Lost	Lost	Kept (as written)
Getters, property descriptors (writable/enumerable/accessor settings)	Values copied	Values copied	Values copied	Values copied

The manual version: a recursive walk handling primitives, `Date`, `RegExp`, `Map`, `Set`, arrays, objects with their prototype, and **cycles** via a `WeakMap` of already-copied objects:

```
function deepClone(value, seen = new WeakMap()) {
  if (value === null || typeof value !== 'object') return value;   // primitives & functions
  if (seen.has(value)) return seen.get(value);                     // cycles / shared refs
  if (value instanceof Date) return new Date(value);
  if (value instanceof RegExp) return new RegExp(value.source, value.flags);
  if (value instanceof Map) {
    const m = new Map(); seen.set(value, m);                       // register BEFORE recursing
    for (const [k, v] of value) m.set(deepClone(k, seen), deepClone(v, seen));
    return m;
  }
  if (value instanceof Set) {
    const s = new Set(); seen.set(value, s);
    for (const v of value) s.add(deepClone(v, seen));
    return s;
  }
  const out = Array.isArray(value) ? [] : Object.create(Object.getPrototypeOf(value));
  seen.set(value, out);
  for (const key of Reflect.ownKeys(value)) {                      // strings AND symbols
    const desc = Object.getOwnPropertyDescriptor(value, key);
    if (desc.enumerable) out[key] = deepClone(value[key], seen);
  }
  return out;
}
```

Output: `deepClone keeps class: true | shared ref kept: true` — the copy of `new Point(-3)` still has `norm()`, and an object referenced twice stays one object referenced twice. `seen.set` happens *before* copying children so a child pointing back finds the copy instead of recursing forever.

Which? `structuredClone` for data; manual only to keep class instances — where a `clone()` method is usually clearer.

Deep equal

Decisions to state out loud: `NaN` equals `NaN`? (yes, via `Object.is`); `0` vs `-0`? (not equal with `Object.is` — say which you want); prototypes must match? (yes: `[1, 2]` ≠ `{0: 1, 1: 2}`); `{a: 1}` vs `{a: 1, b: undefined}`? (no — key sets differ). (B Q253)

```
function deepEqual(a, b, seen = new WeakMap()) {
  if (Object.is(a, b)) return true;                          // handles NaN and -0 vs +0
  if (typeof a !== 'object' || typeof b !== 'object' || a === null || b === null) return false;
  if (Object.getPrototypeOf(a) !== Object.getPrototypeOf(b)) return false;
  if (seen.get(a) === b) return true;                        // already comparing this pair (cycles)
  seen.set(a, b);
  if (a instanceof Date) return a.getTime() === b.getTime();
  if (a instanceof RegExp) return String(a) === String(b);
  if (a instanceof Map) {
    if (a.size !== b.size) return false;
    for (const [k, v] of a) if (!b.has(k) || !deepEqual(v, b.get(k), seen)) return false;
    return true;
  }
  if (a instanceof Set) {
    if (a.size !== b.size) return false;
    for (const v of a) if (!b.has(v)) return false;          // primitive members only (simplification)
    return true;
  }
  const ka = Reflect.ownKeys(a), kb = Reflect.ownKeys(b);
  if (ka.length !== kb.length) return false;
  return ka.every((k) => Object.hasOwn(b, k) && deepEqual(a[k], b[k], seen));
}
assert.ok(deepEqual({ a: [1, { b: NaN }] }, { a: [1, { b: NaN }] }));
assert.ok(!deepEqual({ a: 1 }, { a: 1, b: undefined }));
assert.ok(!deepEqual([1, 2], { 0: 1, 1: 2 }));
{ const x = { v: 1 }; x.me = x; const y = { v: 1 }; y.me = y; assert.ok(deepEqual(x, y)); }
```

In tests, `assert.deepStrictEqual` does this (§16.2).

LRU cache

An **LRU (least recently used) cache** holds at most `capacity` entries and evicts the one used longest ago. Textbook design: hash map + doubly linked list (each node links to previous and next). In JavaScript one `Map` suffices: it iterates in insertion order, so delete-and-reinsert moves a key to the "newest" end and `map.keys().next().value` is the oldest — both O(1). Caching in general: System Design Fundamentals. (B Q259)

```
class LRUCache {
  #cap; #map = new Map();
  constructor(capacity) { this.#cap = capacity; }
  get(key) {
    if (!this.#map.has(key)) return -1;
    const v = this.#map.get(key);
    this.#map.delete(key); this.#map.set(key, v);          // move to "most recent" end
    return v;
  }
  put(key, value) {
    this.#map.delete(key);                                  // updating also refreshes recency
    this.#map.set(key, value);
    if (this.#map.size > this.#cap) {
      const oldest = this.#map.keys().next().value;         // first key = least recent
      this.#map.delete(oldest);
    }
  }
  keys() { return [...this.#map.keys()]; }
}
const lru = new LRUCache(2);
lru.put('a', 1); lru.put('b', 2);
assert.equal(lru.get('a'), 1);          // a is now most recent
lru.put('c', 3);                        // evicts b
assert.equal(lru.get('b'), -1);
lru.put('a', 10);                       // update moves a to the end
assert.deepEqual(lru.keys(), ['c', 'a']);
```

Without the `Map` trick: nodes with `prev`/`next`, a `Map` from key to node, and dummy head/tail nodes so ends need no special case. The LLD handbook builds that version (in Java) with eviction policies and thread safety: §20.3.

EventEmitter

Publish/subscribe: listeners register for named events; `emit` calls them in registration order. Node's own is in §15.8; this is the interview version. (B Q258, Q298)

```
class EventEmitter {
  #listeners = new Map();                                  // event -> array of fns
  on(event, fn) {
    if (!this.#listeners.has(event)) this.#listeners.set(event, []);
    this.#listeners.get(event).push(fn);
    return () => this.off(event, fn);                      // handy unsubscribe
  }
  off(event, fn) {
    const list = this.#listeners.get(event);
    if (!list) return this;
    const i = list.findIndex((l) => l === fn || l.original === fn);
    if (i !== -1) list.splice(i, 1);
    return this;
  }
  once(event, fn) {
    const wrapper = (...args) => { this.off(event, wrapper); fn.apply(this, args); };
    wrapper.original = fn;                                 // so off(event, fn) still works
    return this.on(event, wrapper);
  }
  emit(event, ...args) {
    const list = this.#listeners.get(event);
    if (!list || list.length === 0) {
      if (event === 'error') throw args[0];                // same rule as Node
      return false;
    }
    for (const fn of [...list]) fn.apply(this, args);      // copy: listeners may unsubscribe
    return true;
  }
  listenerCount(event) { return this.#listeners.get(event)?.length ?? 0; }
}
```

Output: `EventEmitter log: [ 'A:1', 'once:1', 'B:1', 'B:2' ]`

Iterate a **copy** (a `once` listener removing itself mid-`emit` would otherwise skip the next one), let `off` remove a `once` listener by the original function, throw on an unhandled `'error'` like Node, return an unsubscribe function.

`Promise.all`, `allSettled`, `race`, `any` polyfills

Real behaviour: §12.8. From `code/06_promise_polyfills.mjs` (B Q247–Q249):

```
function promiseAll(iterable) {
  return new Promise((resolve, reject) => {
    const items = [...iterable];                        // accept any iterable
    const results = new Array(items.length);
    let remaining = items.length;
    if (remaining === 0) return resolve(results);       // all([]) resolves immediately
    items.forEach((item, i) => {
      Promise.resolve(item).then((v) => {               // wrap non-promises too
        results[i] = v;                                 // by index, not push -> keeps order
        if (--remaining === 0) resolve(results);
      }, reject);                                       // first rejection wins; later ones ignored
    });
  });
}

function promiseAllSettled(iterable) {                  // never rejects
  return promiseAll([...iterable].map((item) =>
    Promise.resolve(item).then(
      (value) => ({ status: 'fulfilled', value }),
      (reason) => ({ status: 'rejected', reason }))));
}

function promiseRace(iterable) {                        // first to SETTLE, either way
  return new Promise((resolve, reject) => {
    for (const item of iterable) Promise.resolve(item).then(resolve, reject);
  });                                                   // race([]) stays pending forever
}

function promiseAny(iterable) {                         // first to FULFIL
  return new Promise((resolve, reject) => {
    const items = [...iterable], errors = new Array(items.length);
    let remaining = items.length;
    if (remaining === 0) return reject(new AggregateError([], 'All promises were rejected'));
    items.forEach((item, i) => Promise.resolve(item).then(resolve, (e) => {
      errors[i] = e;
      if (--remaining === 0) reject(new AggregateError(errors, 'All promises were rejected'));
    }));
  });
}
```

The tests run each polyfill next to the built-in:

```
const all = await promiseAll([sleep(30, 'slow'), sleep(25, 'fast'), 42]);
assert.deepEqual(all, ['slow', 'fast', 42]);            // input order, not finish order
assert.deepEqual(await promiseAll([]), []);
await assert.rejects(promiseAll([sleep(20, 'ok'), sleep(5, 'bad', true)]), /bad/);
assert.equal(await promiseRace([sleep(20, 'a'), sleep(5, 'b')]), 'b');
assert.equal(await promiseAny([sleep(5, 'e1', true), sleep(15, 'win')]), 'win');
```

Output:

```
all: [ 'slow', 'fast', 42 ] | parallel (under 55 ms)? true
allSettled: [ 1, 'rejected(x)', 'y' ]
race: b
any all-rejected: AggregateError [ 'p', 'q' ]
pool results: [ 't0', 't1', 't2', 't3', 't4' ] peak concurrency: 2
all promise asserts passed
```

Four things interviewers look for in `Promise.all`: results **by index** (not `push`, which orders by finish time); a **counter**, not `results.length` (a pre-sized array is already "full"); `Promise.resolve(item)` so plain values work; **empty input** resolves immediately. A settled promise ignores later calls, so `reject` can be passed straight through.

The file also builds a **timeout** (`race` against a rejecting timer), **`promisify`** (Node ships `util.promisify`), a **concurrency-limited pool** (at most *n* tasks at once, results in order — peak asserted to be 2; same worker-pool shape as `mapLimit` in §12.13: `limit` workers share a `next++` index, safe

because nothing runs between the read and the increment) and **retry** (B Q262–Q264). Production patterns (back-off, stale responses): §12.13.

Common mistakes

- Arrow functions for `debounced` / `throttled` / `memoized` wrappers silently drop the caller's `this`.
- `Promise.all` with `results.push(v)` (wrong order) or `if (results.length === n)` (fires too early).
- Deep clone without a `seen` map — infinite recursion on the first cycle.
- `curry` mutating a shared argument array, so partials interfere.
- `EventEmitter.emit` iterating the live array while a `once` listener removes itself.

In the interview

> **30-second answer — "Implement debounce."** Return a wrapper holding a timer in a closure; each call clears it and starts a new `wait`-ms timer; when it fires, call the original with the latest arguments and `this`. Add `cancel`; mention the leading-edge option.

- *Debounce vs throttle?* Debounce waits for quiet and runs once; throttle runs at a steady maximum rate during activity.
- *Why store `Promise.all` results by index?* Promises finish in any order; results must match input order.
- *Why is a `Map` enough for an LRU cache?* Insertion order + O(1) delete/insert: "move to newest" = delete + set, "oldest" = first key.
- *`Promise.race([])`?* Never settles.

TL;DR

Closures hold state (`debounce` timer, `memoize` cache, `curry` arguments); forward `this` with `apply`; `bind` must handle `new`. `structuredClone` for data, manual clone for class instances, `WeakMap` for cycles. LRU = one `Map`. EventEmitter: copy the list before emitting. `Promise.all`: index, counter, `Promise.resolve`, empty input.

17.4 Fifty-three tricky interview questions

How to use this section

Each question shows the code and its **real output** (from `code/07_qa_language.js` — CommonJS in sloppy mode on purpose, so legacy behaviour is visible — and `code/08_qa_async_and_memory.mjs`), the one-line reason, and links: → **B** points to the say-it-out-loud answer in the Chapter 19 question bank (not repeated here), **§** to the section that teaches the topic. Cover the output, predict, then check.

17.4.1 Coercion and equality

Q1. What do `1 + '2'`, `'5' - 2`, `'3' * '4'`, `true + 1`, `[] + []`, `[] + {}` and `[1,2] + [3]` give?

```
Q1  1 + '2'                    "12"
Q1  '5' - 2                    3
Q1  '3' * '4'                  12
Q1  true + 1                   2
Q1  [] + []                    ""
Q1  [] + {}                    "[object Object]"
Q1  [1,2] + [3]                "1,23"
```

`+` first turns objects into primitives (arrays → `"1,2"`-style strings, plain objects → `"[object Object]"`); if **either** side is then a string it concatenates, otherwise it adds. `-`, `*`, `/` always convert to numbers. (At the start of a console line, `{} + []` is `0`: `{}` parses as an empty block and `+[]` is unary plus — `+x` converts `x` to a number.) → B Q6, Q230 · §2.5, §2.12

Q2. What do `null == undefined`, `null == 0`, `null >= 0`, `'' == 0` and `'0' == false` give?

```
Q2  null == undefined              true
Q2  null == 0                      false
Q2  null >= 0                      true
Q2  '' == 0                        true
Q2  '0' == false                   true
```

`null` and `undefined` are `==` only to each other, so `null == 0` is false; relational operators (`<`, `>=` …) have no such exception — `null` becomes `0`. → B Q3 · §2.6

Q3. Why is `[] == ![]` true?

```
Q3  [] == ![]                      true
```

`![]` is `false` (objects are truthy); then `[] == false` → `'' == 0` → `0 == 0`. — the best argument for `===`. → §2.6, §2.11

Q4. What is `typeof null`, and why?

```
Q4  typeof null                    "object"
```

A 1995 type-tag bug kept for compatibility. Test with `x === null`, and `x !== null && typeof x === 'object'` for "a real object". → B Q5 · §2.3

Q5. How does `NaN` behave?

```
Q5  NaN === NaN                    false
Q5  [NaN].includes(NaN)            true
Q5  [NaN].indexOf(NaN)             -1
Q5  typeof NaN                     "number"
```

`indexOf` uses `===`; `includes`, `Map` and `Set` use SameValueZero (`NaN` equals `NaN`). Use `Number.isNaN`, not the coercing global `isNaN`. → B Q8, Q236 · §2.10

Q6. How does `Object.is` differ from `===`?

```
Q6  Object.is(-0, 0) / -0 === 0    "false / true"
```

Only for `NaN` (equal) and `±0` (different). → B Q9

Q7. What does `typeof` return for arrays, functions, classes and undeclared variables?

```
Q7  typeof [] / function / class   "object / function / function"
Q7  typeof undeclaredVar           "undefined"
```

Use `Array.isArray`; classes are functions; `typeof` on a never-declared name doesn't throw (feature detection) — except in the TDZ (Q16). → B Q5

Q8. Which values are falsy? Is `[]` truthy? And `new Boolean(false)`?

```
Q8  Boolean([]), Boolean('0')      "true, true"
Q8  !!new Boolean(false)           true
```

Falsy: `false`, `0`, `-0`, `0n`, `''`, `null`, `undefined`, `NaN` (plus legacy `document.all`); everything else, objects included, is truthy. Test an empty array with `arr.length === 0`. → B Q7

Q9. What does `'b' + 'a' + +'a' + 'a'` give?

```
Q9  'b' + 'a' + +'a' + 'a'         "baNaNa"
```

`+'a'` (unary plus) is `NaN`, concatenated as text. → §2.5

Q10. Why does `['1', '7', '11'].map(parseInt)` give `[1, NaN, 3]`?

```
Q10 ['1','7','11'].map(parseInt)   [ 1, NaN, 3 ]
Q10 ['1','7','11'].map(Number)     [ 1, 7, 11 ]
```

`map` passes `(value, index)`, and `parseInt`'s second parameter is the **radix** (number base): `parseInt('1', 0)` (0 = default 10), `parseInt('7', 1)` (invalid → `NaN`), `parseInt('11', 2)` (binary 3). Use `map(Number)` or `(s) => parseInt(s, 10)`. → §7.5, §2.10

Q11. Why does `[10, 1, 2].sort()` give `[1, 10, 2]`?

```
Q11 [10, 1, 2].sort()              [ 1, 10, 2 ]
```

No comparator → string comparison. Pass `(a, b) => a - b`. → B Q90, Q233 · §7.6

Q12. What's the difference between `||` and `??`?

```
Q12 0 || 'x' / 0 ?? 'x'            "x / 0"
```

`||` replaces any falsy value; `??` (ES2020) only `null`/`undefined`. Use `??` for defaults. → B Q12 · §2.7

Q13. Why is `{} === {}` false?

```
Q13 {} === {}                      false
```

Objects compare by **reference**; two literals are two objects. Compare content with a deep-equal (§17.3). → B Q2 · §2.4

Q14. Why is `0.1 + 0.2 !== 0.3`? And why is `Math.max() < Math.min()`?

```
Q14 0.1 + 0.2 === 0.3              false
Q14 Math.max() < Math.min()        true
```

No exact IEEE 754 binary form → `0.30000000000000004` (§17.1). With no arguments `Math.max()` is `-Infinity` (the identity for "max") and `Math.min()` is `Infinity`. → B Q10 · §2.10

17.4.2 Scope, hoisting and closures

Q15. What are the differences between `var`, `let` and `const`?

	`var`	`let`	`const`
Scope	Function	Block `{ }`	Block `{ }`
Before the declaration line	`undefined`	`ReferenceError` (TDZ)	`ReferenceError` (TDZ)
Redeclare in the same scope	Allowed	`SyntaxError`	`SyntaxError`
Reassign	Yes	Yes	No (`TypeError`)
At top level of a script	Property of `globalThis`	No	No

`const` stops reassignment, not mutation (Q44). → B Q21 · §2.1

Q16. What happens when you read a variable before its declaration?

```
Q16 var read before declaration    "undefined"
Q16 let read before declaration    "ReferenceError: Cannot access 'tdz' before initialization"
Q16 typeof in the TDZ              "ReferenceError: Cannot access 'inTdz' before initialization"
```

All declarations are hoisted; only `var` is initialised (`undefined`). `let`/`const`/`class` sit in the **temporal dead zone** — even `typeof` throws. → B Q22, Q23 · §2.2

Q17. Can you call a function before it's defined?

```
Q17 declaration hoisted            "works"
Q17 expression not hoisted         "TypeError: late is not a function"
```

Declarations hoist with their body; a `var` function expression hoists only `undefined` (→ `TypeError`; with `let`/`const` a TDZ `ReferenceError`). → B Q24 · §4.2

Q18. What do these loops print?

```
for (var i = 0; i < 3; i++) setTimeout(() => out.var.push(i), 0);
for (let j = 0; j < 3; j++) setTimeout(() => out.let.push(j), 0);
for (var k = 0; k < 3; k++) ((k2) => setTimeout(() => out.fixed.push(k2), 0))(k);
```

```
Q18 closures in loops              {"var":[3,3,3],"let":[0,1,2],"fixed":[0,1,2]}
```

One shared `var i`; a fresh `let` binding per iteration; the pre-ES2015 fix was an IIFE (a function defined and called on the spot) copying the value. → B Q29, Q243 · §4.7

Q19. What is a closure? Show one.

```
const makeCounter = () => { let n = 0; return () => ++n; };
const c1 = makeCounter(), c2 = makeCounter();
c1(); c1();
```

```
Q19 counter via closure            "c1: 3, c2: 1"
```

A **closure** = a function plus its creation scope's variables, kept alive after the outer function returns; each `makeCounter()` makes a new `n`. → B Q28 · §4.7

Q20. What's wrong with `let a = b = 5`?

```
(function () { let a = b = 5; })();
```

```
Q20 let a = b = 5                  "outside: typeof a=undefined, typeof b=number"
```

`b = 5` creates an **accidental global** in sloppy mode (`ReferenceError` in strict). → B Q36 · §4.5

Q21. Is a `var` inside an `if` block visible outside it?

```
Q21 var in block leaks             "v=1, typeof l=undefined"
```

Yes: `var` ignores blocks; `let`/`const` don't. → §4.5

Q22. What does strict mode change?

```
Q22/Q23 strict: lost this      TypeError: Cannot read properties of undefined (reading 'name')
Q22 strict: undeclared assign  ReferenceError: undeclaredThing is not defined
```

Silent mistakes become errors; `this` in a plain call is `undefined`; automatic in ES modules and class bodies. → B Q34 · §1.8

17.4.3 `this`, arrow functions, `call`/`apply`/`bind`, `new`

Q23. Why does a method lose `this` when you pass it around?

```
const f = user.regular; f();
```

```
Q23 method call                    "Ada"
Q23 extracted method (sloppy)      undefined
```

`this` comes from the call: a plain `f()` gets `undefined` (strict → `TypeError`, Q22) or `globalThis` (sloppy). Fix with `bind` or `() => user.regular()`. → B Q54 · §9.1

Q24. How do arrow functions differ from regular functions?

```
Q24 arrow in object literal        "object"
```

No own `this` (lexical; `call`/`bind` can't change it), no `arguments` (use rest), no `new`/`prototype`. As an object-literal *method* an arrow sees the outer `this` — here CommonJS's `module.exports`; in an ES module, `undefined`. → B Q37, Q55 · §4.3

Q25. Inside a method, what is `this` in a nested regular function vs a nested arrow?

```
delayed() { return [function () { return this === globalThis; }(), (() => this.name)()]; }
```

```
Q25 inner function vs arrow        [ true, 'Ada' ]
```

The inner function is a plain call (global object in sloppy mode); the arrow inherits the method's `this` — why arrows suit `setTimeout(() => this.save())` in methods. → B Q226 · §9.1

Q26. What's the difference between `call`, `apply` and `bind`?

```
Q26 call / apply / bind            "Hi Lin! | Yo Lin? | Hey Lin."
```

`call` (args listed) and `apply` (args array) call now; `bind` returns a new fixed-`this` function. Apply = Array. → B Q53 · §9.2

Q27. What happens if you `bind` twice?

```
Q27 bind twice                     1
```

The first `bind` wins (only `new` overrides it). → B Q56 · §9.2

Q28. What does `new` do? What if the constructor returns something?

```
Q28 new returns object?            "{\"b\":2}"
Q28 new returns primitive?         "{\"a\":1}"
```

Creates an object linked to `F.prototype`, runs `F` with that `this`, returns it — unless `F` returns an *object*; returned primitives are ignored. → B Q59, Q237 · §9.4

17.4.4 Prototypes and classes

Q29. What's the difference between `prototype` and `__proto__`?

```
Q29 proto links                    "true true true"
Q29 own vs inherited               "true false true"
```

Every object has a `[[Prototype]]` link (`Object.getPrototypeOf`; `__proto__` is the legacy accessor); `F.prototype` is the object `new F()` installs as that link. Methods live on `Animal.prototype` (shared): `Object.hasOwn(rex, 'speak')` is false, `'speak' in rex` is true. → B Q57, Q58 · §9.3

Q30. How does `instanceof` work? Can it be fooled?

```
Q30 instanceof after proto swap    true
```

It walks the prototype chain for `F.prototype`, so a plain object given `Animal.prototype` passes; it fails across realms (iframe arrays — use `Array.isArray`). → B Q65 · §9.7

Q31. How are classes different from constructor functions?

```
Q31 class before declaration       "ReferenceError: Cannot access 'Later' before initialization"
Q31 class without new              "TypeError: Class constructor K cannot be invoked without 'new'"
```

TDZ, `new` required, strict bodies, non-enumerable methods, plus `#private`, `static` blocks, `super`. → B Q61 · §9.5

Q32. What is `Object.create(null)` for?

```
Q32 Object.create(null)            "undefined true"
```

An object with **no prototype** — no inherited `toString`, `constructor` or `__proto__` — a safe dictionary for user-supplied keys. A `Map` is usually better today. → §9.3

Q33. How do `#private` fields compare with closure-based privacy?

```
class Wallet { #cents = 0; add(c) { this.#cents += c; return this.#cents; } }
```

```
Q33 #private vs closure          "10 undefined 0 {}"
```

`#cents` is invisible to `w.cents`, `Object.keys`, `JSON.stringify`, and a *syntax* error outside the class. Closure privacy needs per-instance method copies; `#` fields keep methods shared on the prototype. → B Q30, Q63 · §9.5

Q34. What happens if a subclass constructor uses `this` before `super()`?

```
Q34 this before super()          "ReferenceError: Must call super constructor in derived class before accessing
'this' or returning from derived constructor"
```

In a derived class the **parent** constructor creates the object, so `this` exists only after `super(...)` returns. → §9.6

17.4.5 The event loop and async code

Rules (§12.2–12.3): all sync code; drain **all** microtasks (promise callbacks, code after `await`, `queueMicrotask`); one task (timer, I/O, event); repeat. More puzzles: B Q218–Q223.

Q35. The classic.

```
log('1');
setTimeout(() => log('2'), 0);
Promise.resolve().then(() => log('3'));
log('4');
```

```
Q35 classic                  1 4 3 2
```

The `.then` microtask runs as soon as the script finishes; the 0 ms timer waits for the next loop turn. → B Q108, Q218 · §12.2

Q36. `async`/`await` ordering.

```
async function a() { log('a1'); await b(); log('a2'); }
async function b() { log('b'); }
log('s1');
setTimeout(() => log('t'), 0);
a();
new Promise((res) => { log('p'); res(); }).then(() => log('then'));
log('s2');
```

```
Q36 async/await              s1 a1 b p s2 a2 then t
```

An async function runs synchronously until its first `await` (`a1`, `b`); the executor is synchronous (`p`); then microtasks in queue order (`a2`, `then`); the timer last. → B Q220 · §12.9

Q37. `process.nextTick`, promises and `setImmediate` in Node.

```
setImmediate(() => q.push('immediate'));
Promise.resolve().then(() => q.push('then'));
process.nextTick(() => q.push('nextTick'));
q.push('sync');
```

```
Q37 CommonJS main                sync nextTick then immediate
Q37 ESM (after await)        sync then nextTick immediate
```

`nextTick` normally drains before promises (CommonJS), but ESM code already runs inside a promise job, so promises go first; `setImmediate` runs in the *check* phase. Prefer `queueMicrotask`. → B Q119 · §12.4

Q38. When does a promise executor run, and what if you resolve twice?

```
Q38 executor + settle once   executor after first
```

Synchronously; the first `resolve`/`reject` wins. → B Q121 · §12.7

Q39. Why doesn't this `try`/`catch` catch the error?

```
try { setTimeout(() => { throw new Error('boom'); }, 0); } catch { /* never runs */ }
try { await Promise.reject(new Error('rejected')); } catch (e) { /* runs */ }
```

```
Q39 errors in callbacks        uncaught: boom | await caught: rejected
```

The timer callback runs after `try` left the stack (shown via `process.on('uncaughtException')`; normally Node crashes); `await` rethrows the rejection inside the function. → B Q128 · §11.5, §12.10

Q40. Does `await` inside `forEach` wait?

```
[20, 10].forEach(async (ms) => { await wait(ms); log.push(ms); });
log.push('forEach returned');
for (const ms of [20, 10]) { await wait(ms); log.push(`for-of ${ms}`); }
```

```
Q40 await in forEach           forEach returned | 10 | 20 | for-of 20 | for-of 10
```

No — `forEach` ignores the returned promises. → B Q116 · §12.9

Q41. Two independent `await`s — how long do they take?

```
Q41 sequential vs parallel     sequential >= 80ms: true, parallel < 80ms: true
```

`await wait(40); await wait(40);` ≈ 80 ms; `Promise.all` ≈ 40 ms. → B Q115 · §12.9

Q42. `Promise.all` vs `allSettled` vs `race` vs `any`?

`all`: all fulfil / first rejection; `allSettled`: never rejects; `race`: first to settle; `any`: first to fulfil, `AggregateError` if all reject. Polyfills: §17.3. → B Q112 · §12.8

17.4.6 Objects, arrays and copies

Q43. Shallow vs deep copy — what does this print?

```
const a = { n: { v: 1 } }; const b = { ...a }; b.n.v = 2; a.n.v;
```

```
Q43 spread copy is shallow         2
```

Spread copies the top level only; use `structuredClone` (or §17.3's manual clone). → B Q74 · §5.4

Q44. Can you change a `const` object? Does `Object.freeze` make it immutable?

```
Q44 const + mutate                 2
Q44 freeze is shallow              "2 undefined"
```

`const` protects the binding; `freeze` blocks changes to *own* properties (silent in sloppy, `TypeError` in strict) but is **shallow** — deep-freeze by recursing. → B Q76 · §5.6

Q45. What happens when you use an object as an object key?

```
const o = {}; o[{}] = 'a'; o[{ id: 2 }] = 'b'; o[1] = 'c';
```

```
Q45 object keys become strings     "[\"1\",\"[object Object]\"] b"
```

Keys are strings (or symbols): both objects become `"[object Object]"`, so the second overwrites the first; `1` becomes `"1"`. Use a `Map` for object keys. → §5.2, §8.1

Q46. What do `delete arr[i]` and `arr.length = n` do?

```
Q46 array holes                    "3 [1,null,3] false"
Q46 length truncates               "[1]"
Q46 length of sparse               6
```

`delete` leaves a **hole** (length unchanged; JSON shows `null`) — use `splice`. A smaller `length` truncates (`= 0` empties in place); assigning `a[5]` on `[]` gives length 6 with five holes. → B Q93, Q235 · §7.3

Q47. Is JavaScript pass-by-value or pass-by-reference?

```
const o = { v: 1 };
(function (p) { p = { v: 99 }; })(o);   // reassign the parameter
(function (p) { p.v = 2; })(o);          // mutate through it
o.v;
```

```
Q47 reassign param                2
```

Always **by value** — but an object's value *is a reference* ("call by sharing"): mutating through the parameter changes the caller's object; reassigning only changes the local. → §2.4

Q48. When should you use a `Map` instead of an object?

Dynamic keys, non-string keys, frequent size checks and adds/deletes, guaranteed order; objects for fixed-shape records and JSON (and they have inherited keys: `'toString' in {}` is true). → B Q84 · §7.8, §8.1

Q49. What does `JSON.stringify` drop?

```
Q49 JSON drops                    "{\"n\":null,\"d\":\"1970-01-01T00:00:00.000Z\"}"
Q49 JSON array holes              "[null,null]"
```

`undefined`/functions/symbols omitted (→ `null` in arrays); `NaN`/`Infinity` → `null`; dates → ISO strings; `Map`/`Set` → `{}`; `BigInt` and cycles throw. → B Q82 · §5.12

Q50. `for…in` vs `for…of`?

```
Q50 for...in vs for...of          "in: 0,1,extra | of: a,b"
```

`for…in`: enumerable string keys, inherited ones and extras like `extra` included; `for…of`: values of an iterable. → B Q80 · §3.10

17.4.7 Garbage collection and memory leaks

Q51. How does JavaScript free memory?

A **garbage collector (GC)** frees objects no longer **reachable** from a root (globals, call stack, active closures, pending callbacks) — **mark-and-sweep**, so cycles are fine. V8 is **generational**: a small young generation collected often (most objects die young), survivors promoted to an old generation collected less often, much of it on helper threads. Code can't (and needn't) trigger GC. → B Q182 · §8.3

Q52. What causes memory leaks in JavaScript, and how do you fix them?

A **leak** is memory still reachable but no longer needed:

Cause	Fix
Forgotten listeners and timers (their closures keep data alive)	`clearInterval`, `removeEventListener`, `AbortController` signal, React effect cleanup
Unbounded caches/collections keyed by request or DOM objects	Size limit (LRU), or a `WeakMap`
Closures capturing big data	Capture only what's needed; null out big references
Detached DOM nodes still referenced from JS	Drop the reference on removal
Accidental globals	Strict mode / modules

The demo stores ~8 KB per short-lived "request" object in a `Map` and in a `WeakMap`, then forces a GC (V8 flag, for the demo only):

```
Q52 Map vs WeakMap cache          Map retains > 10 MB: true, WeakMap retains < 2 MB: true
```

The `Map` keeps every key and value alive; the `WeakMap` lets both go once the key is unreachable. Find leaks with DevTools **Memory** heap snapshots before/after an action. → B Q183 · §4.7, §8.3, §11.9

Q53. What are `WeakMap`, `WeakRef` and `FinalizationRegistry` for?

```
Q53 WeakRef before            true
Q53 WeakRef after gc          collected
```

Weak keys/references don't keep objects alive; `deref()` returns the object or `undefined`; finalizers run some time after collection. GC timing is **not guaranteed** — optimisations only, never program logic. → B Q85, Q184 · §8.3

In the interview

30-second answer — "`this`, closures and the event loop in one breath?" `this` comes from the call site (method, plain call, `new`, `call` / `apply` / `bind`) except in arrows, which take it lexically. A closure is a function plus its creation scope's variables, kept alive with it. The event loop runs sync code, then all microtasks, then one task, repeat — so promise callbacks beat `setTimeout(0)`.

Self-check

1. `console.log([] + null + 1)`? → `"null1"`: `[]` becomes `''`, then string concatenation.
2. `async function f() { console.log(1); await null; console.log(2); } f(); console.log(3);`? → `1 3 2`.
3. `typeof typeof 1`? → `'string'`.

TL;DR

`===`; falsy list; `typeof null`; `NaN` rules; `+` concatenates if either side is a string. `var` = function scope, hoisted `undefined`; `let`/`const` = block scope + TDZ, one loop binding per iteration. `this` from the call; arrows inherit; first `bind` wins; `new` returns the object unless you return one. Sync → microtasks → one task. Spread is shallow; `structuredClone` deep. Leaks = things left reachable; clean up and use `WeakMap`.

Chapter 17 TL;DR

- **Toolkit:** `push`/`pop` stack; head-index queue (no `shift` in hot loops); `Array.from` grids; `Map`/`Set` with string keys for pairs; `sort` needs a comparator, is stable, mutates (`toSorted` doesn't); floats — tolerance, `Math.trunc`, `%` sign, safe to $2^{53} - 1$, then `BigInt`; build strings with `join`.
- **Patterns (21):** two pointers, sliding window, hashing, stack/monotonic stack, prefix sums, intervals (sort + sweep), heap (hand-written `MinHeap`, O(log n) push/pop; two heaps for a running median; k-way merge), binary search (on data or the answer), BFS (fewest steps, mark on enqueue), DFS (iterative for deep inputs), DP with memo, backtracking (choose/explore/un-choose, copy results), linked lists (dummy head, reverse, k-th from end), fast & slow pointers (cycle start, middle), trees (validate with ranges, LCA, serialize), tries (prefix, wildcard, board words), Kahn's topological sort and union-find, greedy (with a proof), quickselect / Dutch flag / merge sort, bit manipulation (32-bit signed, `>>> 0`), math & geometry (spiral, overflow, gcd-reduced slopes).
- **Classics:** `debounce` vs `throttle` (leading + trailing); `curry` via `fn.length`; `compose` = `reduceRight`, `pipe` = `reduce`; `memoize` with a `Map`; `flatten` recursive or with a stack; `bind` must support `new`; `structuredClone` vs manual `deepClone` with a `WeakMap`; `deepEqual` with `Object.is`; LRU = one `Map`; EventEmitter copies listeners before emitting; `Promise.all` = index + counter + `Promise.resolve` + empty case.
- **Q&A:** coercion, `==`, `typeof null`, `NaN`, `??`; hoisting/TDZ, loop closures, accidental globals, strict mode; `this`, arrows, `bind`, `new`; prototypes, `instanceof`, classes, `#private`; event-loop order, `nextTick`, callback errors, `await` in `forEach`; copies, `freeze`, string keys, holes, call by sharing, JSON; GC, leaks, weak references — short answers in Chapter 19.

Next: the cheat sheets and glossary — Chapter 18.

Chapter 18 — Cheat Sheets and Glossary

Chapters 1–17 squeezed into pages you can scan in a minute: one **cheat sheet** per area, a **"which API for which job"** table, a verified **ECMAScript version table** (ES2015 → ES2026), a study plan, an **interview question index**, and an A–Z **glossary** of 700+ terms — each defined once, in one line, linked to the section that explains it. Nothing here is new; follow a line's link (written **§12.2** = Chapter 12, section 2) when it doesn't make sense, and look up unfamiliar words in the glossary.

How to revise: read a sheet with the right-hand column covered and say the answer first. Last check before an interview: if you can't define a glossary term in one sentence, follow its link.

Every concrete value quoted on the sheets (outputs, orderings, return values, errors thrown) is asserted by `code/cheatsheet_checks.mjs`, which runs on Node 24 with jsdom for the DOM sheet:

```
OK   18.1 syntax & types
OK   18.2 operators & coercion
OK   18.3 control flow
OK   18.4 functions/closures/this
OK   18.5 objects/Math/Date/JSON
OK   18.6 classes & prototypes
OK   18.7 strings & regex
OK   18.8 arrays
OK   18.9 Map/Set/iterators
OK   18.10 modules
OK   18.11 errors
OK   18.12 async & event loop
OK   18.13 DOM & events
OK   18.14 browser APIs
OK   18.15 Node core
OK   18.16 testing
      Node v24.11.1 - not yet available: Math.sumPrecise (ES2026), Uint8Array.fromBase64 (ES2026), Map#getOrInsert
(ES2026), Iterator.concat (ES2026), Temporal (ES2027)
OK   18.18 versions (Node 24 availability)
all 17 cheat-sheet checks passed
```

18.1 Syntax, variables and types

Running code (§1.5): the browser console, `<script defer src="app.js">` or `<script type="module">` in a page, `node file.js` / `node file.mjs`, or the Node REPL (`node` with no file).

Basic rules (§1.6): case-sensitive; identifiers use letters, digits, `_` and `$` and can't start with a digit; statements end with `;` (automatic semicolon insertion exists, but never break a line straight after `return`). Comments: `//`, `/* */`, `/** JSDoc */` (§1.7).

Strict mode (§1.8) is on automatically in modules and class bodies, and opt-in with `"use strict"` at the top of a script or function. It turns silent mistakes into errors:

Mistake	Sloppy mode	Strict mode
`x = 1` with no declaration	creates a global	`ReferenceError`
`this` in a plain call `f()`	`globalThis`	`undefined`
Writing a frozen / read-only property	ignored	`TypeError`
`function f(a, a)`, `010`, `with`	allowed	`SyntaxError`

Declaring variables (§2.1, §2.2):

	`const` (default choice)	`let` (when you reassign)	`var` (legacy)
Scope	block `{}`	block `{}`	function
Reassign	✗ `TypeError`	✓	✓
Redeclare in same scope	✗ `SyntaxError`	✗	✓ silently
Read before its line	`ReferenceError` (TDZ)	`ReferenceError` (TDZ)	`undefined`
Top level of a classic script	not on `globalThis`	not on `globalThis`	becomes `globalThis.x`

`const` stops *reassignment*, not *mutation*: `const o = { n: 1 }; o.n = 2` is fine.

The eight types (§2.3): seven primitives plus objects.

Value	`typeof`	Remember
`42`, `3.14`, `NaN`, `Infinity`	`"number"`	one 64-bit float type for all numbers
`10n`	`"bigint"`	whole numbers of any size; can't mix with `number`
`'hi'`	`"string"`	immutable UTF-16 text
`true`	`"boolean"`	
`undefined`	`"undefined"`	"not set yet" — what the language gives you
`null`	**`"object"`**	a historic bug; test with `x === null`
`Symbol('id')`	`"symbol"`	unique keys
`{}`, `[]`, `new Date()`	`"object"`	arrays too — use `Array.isArray`
`function () {}`, `class {}`	`"function"`	functions are callable objects
an undeclared name	`"undefined"`	`typeof` never throws, except in the TDZ

Primitives vs objects (§2.4): primitives are copied by value and compared by value; objects are copied as a **reference** and compared by identity, so `{} === {}` is `false` and two variables can point at one object.

18.2 Operators and the coercion table

Arithmetic (§2.5):

Expression	Result	Why
`7 / 2`	`3.5`	`/` is always floating point; integer division is `Math.trunc(a / b)`
`-7 % 3`	`-1`	`%` is a remainder that keeps the **left** operand's sign; true modulo is `((a % n) + n) % n` → `2`
`2 ** 10`	`1024`	exponent (ES2016)
`n++` / `++n` (n = 5)	`5` / `7`	postfix returns the old value, prefix the new one
`1 + '2'`	`"12"`	`+` concatenates if either side is a string
`'5' - 2`, `'3' * '4'`	`3`, `12`	other arithmetic converts to numbers
`true + 1`	`2`	`true` → `1`
`[] + []` / `[] + {}` / `[1,2] + [3]`	`""` / `"[object Object]"` / `"1,23"`	objects become strings first

Equality (§2.6). Use `===`. The only common `==` idiom is `x == null` (true for `null` **or** `undefined`).

Expression	==	Rule behind it
`0 == ''`	`true`	string → number (`''` → `0`)
`0 == '0'`	`true`	string → number
`'' == '0'`	`false`	both strings: compared as text — so `==` is **not transitive**
`false == '0'`	`true`	boolean → number first: `0 == '0'`
`'2' == true`	`false`	`true` → `1`, then `2 == 1`
`null == undefined`	`true`	they equal each other…
`null == 0`	`false`	…and nothing else
`null >= 0`	`true`	relational operators turn `null` into `0`
`NaN == NaN`	`false`	`NaN` equals nothing, not even itself
`[1] == 1` , `[] == ![]`	`true` , `true`	object → primitive (`[]` → `''` → `0`)
`1n == 1`	`true`	BigInt vs number compares the maths value
`'10' < '9'` / `'10' < 9`	`true` / `false`	two strings compare by character code; otherwise numerically
`Object.is(NaN, NaN)` / `Object.is(0, -0)`	`true` / `false`	`===` except for those two cases

Conversions (§2.12):

Value	Number(v)	String(v)	Boolean(v)
`''` / `' '`	`0` / `0`	`''` / `' '`	`false` / `true`
`'42'` / `'4px'` / `'0x1f'`	`42` / `NaN` / `31`	same text	`true`
`null`	`0`	`'null'`	`false`
`undefined`	`NaN`	`'undefined'`	`false`
`true` / `false`	`1` / `0`	`'true'` / `'false'`	—
`[]` / `[7]` / `[1,2]`	`0` / `7` / `NaN`	`''` / `'7'` / `'1,2'`	`true` (all objects)
`{}`	`NaN`	`'[object Object]'`	`true`

`parseInt('08.5kg', 10)` → `8` (reads a prefix, **always pass the radix**) but `Number('08.5kg')` → `NaN` (all or nothing).

Falsy (§2.11) — the complete list: `false` , `0` , `-0` , `0n` , `''` , `null` , `undefined` , `NaN` . Everything else is truthy, including `[]` , `{}` , `'0'` , `'false'` and `new Boolean(false)` .

Logical operators (§2.7):

Operator	Returns	Example
`a && b`	`a` if falsy, else `b`	`0 && 'b'` → `0` ; `'a' && 'b'` → `'b'`
`a || b`	`a` if truthy, else `b`	`0 || 'x'` → `'x'`
`a ?? b`	`a` unless `null` / `undefined`	`0 ?? 'x'` → `0` , `'' ?? 'x'` → `''`
`a?.b` , `a?.[k]` , `a?.()`	`undefined` if `a` is `null` / `undefined`	`null?.deep.prop` → `undefined` (short-circuits the whole chain)
`a ||= b` , `a &&= b` , `a ??= b`	assign only when falsy / truthy / nullish	`cfg.port ??= 8080`

Mixing `??` with `||` or `&&` without parentheses is a `SyntaxError`.

Numbers (§2.10): `0.1 + 0.2` is `0.30000000000000004`; integers are exact only up to `Number.MAX_SAFE_INTEGER` = $2^{53} - 1$ (so `2 ** 53 === 2 ** 53 + 1` is `true`); `1 / 0` is `Infinity`; `Math.max()` is `-Infinity`. Use `Number.isNaN(x)` (the global `isNaN('abc')` converts first and says `true`), `toFixed(2)` for display (`(123.456).toFixed(2)` → `'123.46'`), integer cents for money, `BigInt` beyond 2^{53}. Numeric separators: `1_000_000`. Bitwise operators (§2.8) work on 32-bit integers: `5 & 3` = 1, `5 | 3` = 7, `5 ^ 3` = 6, `~5` = −6.

Precedence in one breath (§2.9): member access/calls → postfix → unary (`!`, `typeof`, `await`) → `**` → `* / %` → `+ -` → shifts → `< > in instanceof` → equality → bitwise → `&&` → `|| ??` → `? :` → assignment and `=>` → comma. When unsure, add parentheses.

18.3 Control flow and loops

Need	Use	Watch out
Choose between code paths	`if / else if / else` (§3.2)	first truthy branch wins — order specific → general
Choose between two **values**	`cond ? a : b` (§3.3)	don't nest deeply
One value vs many fixed options	`switch` (§3.4)	compares with `===`; forgetting `break` **falls through**; `switch (1) { case 1: … case 2: … break }` runs both cases
Pure value → value mapping	lookup object or `Map`	object keys are strings
Count / index loop	`for (let i = 0; i < n; i++)` (§3.6)	`<` vs `<=` off-by-one
Unknown number of repeats	`while (cond)` (§3.7)	update the condition or it never ends
Run at least once	`do { } while (cond)` (§3.8)	`do {…} while (false)` runs once
Values of an array, string, Map, Set	`for (const v of it)` (§3.10)	plain objects aren't iterable — use `Object.entries(obj)`
Keys of a plain object	`for (const k in obj)` (§3.9)	keys are **strings** (`for…in` over `['a','b']` gives `'0'`, `'1'`), includes inherited ones — avoid on arrays
Index and value	`for (const [i, v] of arr.entries())`	
Leave a loop / skip one pass	`break` / `continue` (§3.11)	`forEach` can't be stopped — use `for…of`, `some`, `find`
Leave an **outer** loop	`outer: for (…) { for (…) { break outer; } }`	labels work with `continue outer` too

Closures in loops (§4.7): `for (var i…)` shares one `i`, so three saved callbacks all return `3`; `for (let i…)` makes a fresh binding per iteration, giving `0, 1, 2`.

18.4 Functions, scope, closures and `this`

Forms (§4.1–§4.3):

```
function add(a, b) { return a + b; }        // declaration: hoisted with its body
const sub = function (a, b) { return a - b; }; // expression: follows const (TDZ)
const sq = (x) => x * x;                    // arrow, implicit return
const mk = (x, y) => ({ x, y });            // returning an object literal needs ( )
(() => { /* IIFE: runs immediately */ })();
```

	Regular `function`	Arrow `=>`
`this`	set by the call site	lexical — taken from where it's written; `call`/`bind` can't change it
`arguments`	yes	no — use `...args`
`new`	allowed	`TypeError` (no `prototype`)
Best as	object methods, constructors	callbacks, especially inside methods

Parameters (§4.4): with `f = (a, b = 2, ...rest) => [a, b, rest]`, `f(1)` → `[1, 2, []]`; `f(1, undefined, 3, 4)` → `[1, 2, [3, 4]]`; `f(1, null)` → `[1, null, []]` — **defaults replace only** `undefined`. `fn.length` counts parameters before the first default/rest (`(a, b = 1, c) => 0` has length `1`). Use an options object for named parameters: `function f({ a = 1, b = 2 } = {})`.

Scope and hoisting (§4.5, §4.6):

Declaration	Before its line
`function f() {}`	callable
`var x`	`undefined`
`let` / `const` / `class`	`ReferenceError` (TDZ)
`var f = function…`	`undefined` → calling it is a `TypeError`

Lookups walk the **lexical scope chain** outward from where the code is *written*; an inner name shadows an outer one.

Closures (§4.7): a function keeps live access to the variables of the scope it was created in.

```
function counter() { let c = 0; return () => ++c; }
const c1 = counter(); c1(); c1();
const c2 = counter();
c1(); // 3   - c1 has its own c
c2(); // 1   - c2 has a separate c
```

Uses: private state, factories, `once`, `memoize`, event handlers, debounce/throttle (§17.3).

Higher-order functions (§4.8): take or return functions — `map/filter/reduce`, `pipe` (left → right, `reduce`), `compose` (right → left, `reduceRight`), currying `f(a)(b)(c)`, partial application. **Recursion** (§4.10): base case + smaller case; too deep → `RangeError: Maximum call stack size exceeded`. **Pure functions** (§4.11): same input → same output, no side effects.

`this` — decided at call time (§9.1), highest priority first:

Call site	`this`
`new Fn()`	the new object
`fn.call(obj, a, b)`, `fn.apply(obj, [a, b])`, `fn.bind(obj)()`	`obj`
`obj.fn()`	`obj` (the thing before the dot)
plain `fn()`	`undefined` in strict code (`globalThis` in sloppy scripts)
arrow function	whatever `this` was where the arrow was written

Checked facts: `user.hi()` → `'Ada'`, but `const lost = user.hi; lost()` → `this` is `undefined` (the method "loses" its object when passed around — fix with an arrow wrapper or `bind`). Binding twice keeps the **first** binding. `call` runs now with arguments listed, `apply` runs now with an array, `bind` returns a new function (§9.2).

`['1', '7', '11'].map(parseInt)` → `[1, NaN, 3]`, because `map` passes `(value, index)` and `parseInt` treats the index as the radix.

18.5 Objects, Math, Date and JSON

Objects (§5.1–§5.5):

```
const key = 'dyn';
const o = { b: 1, 2: 'two', a: 2, 1: 'one', [key]: 3 };   // computed key
Object.keys(o);            // ['1', '2', 'b', 'a', 'dyn'] - integer-like keys first, then insertion order
const { a, b: renamed, missing = 'def' } = o;            // 2, 1, 'def'
const merged = { ...{ x: 1, y: 1 }, ...{ y: 2 } };       // { x: 1, y: 2 } - later wins
```

Task	Code	Note
Read with a fixed / dynamic key	`o.name` / `o[expr]`	brackets for spaces, dashes, variables
Own key?	`Object.hasOwn(o, k)`	`'toString' in {}` is `true` (inherited); `Object.hasOwn({}, 'toString')` is `false`
Remove	`delete o.k`	
List	`Object.keys / values / entries`, rebuild with `Object.fromEntries`	own, enumerable, string keys
Shallow copy / merge	`{ ...a, ...b }`, `Object.assign({}, a, b)`	nested objects are **shared**
Deep copy	`structuredClone(o)`	handles Date, Map, Set, cycles; throws on functions
Lock	`Object.preventExtensions` < `Object.seal` < `Object.freeze`	all **shallow**: a frozen object's nested objects stay writable
Prototype	`Object.create(proto)`, `Object.create(null)` for a bare dictionary	§18.6
Getter / setter	`get total() {…}`, `set total(v) {…}` (§5.7)	keep them cheap

Descriptors (§5.6): `writable`, `enumerable`, `configurable` (+ `get` / `set`). Normal assignment makes them all `true`; `Object.defineProperty` defaults them to `false`.

Math (§5.10):

Call	Result
`Math.round(2.5)` / `Math.round(-2.5)`	`3` / `-2` (halves go toward +∞)
`Math.floor(-1.5)` / `Math.trunc(-1.5)` / `Math.ceil(1.1)`	`-2` / `-1` / `2`
`Math.max(...[3, 9, 1])`	`9` (spread — `max` takes separate arguments)
Random int in [min, max]	`Math.floor(Math.random() * (max - min + 1)) + min`
Secure random	`crypto.getRandomValues`, `crypto.randomUUID()` — never `Math.random`

Date (§5.11): a `Date` is milliseconds since 1970-01-01 UTC. Months are **0-based** (`new Date(2024, 0, 31)` is 31 January); `getDay()` is the weekday (0 = Sunday), `getDate()` the day of month; setters mutate and roll

over (setting month 1 on 31 January gives a date in March); a date-only ISO string like `'2024-03-10'` is parsed as **UTC midnight**; `new Date(0).toISOString()` → `'1970-01-01T00:00:00.000Z'`. Show dates with `Intl.DateTimeFormat` / `toLocaleDateString`. **Temporal** reached Stage 4 in March 2026 (it goes into ES2027); it is not in Node 24, so feature-detect it.

JSON (§5.12):

Input to `JSON.stringify`	Output
`{ u: undefined, f() {}, n: NaN, d: new Date(0), m: new Map([[1, 2]]) }`	`{"n":null,"d":"1970-01-01T00:00:00.000Z","m":{}}`
`[undefined, () => 1]`	`[null,null]`
`undefined` (top level)	`undefined` (not a string)
`{ big: 1n }` or an object with a cycle	throws `TypeError`
`JSON.stringify(v, null, 2)`	pretty-printed with 2-space indent

`JSON.parse` is strict: double-quoted keys and strings only, no trailing commas, no comments (`JSON.parse("{'a':1}")` throws `SyntaxError`). A **reviver** `(key, value) => …` can turn ISO strings back into `Date`s.

18.6 Classes and prototypes

```
class Animal {
  #secret = 's';                    // private field (ES2022)
  static count = 0;                 // static field
  static { Animal.ready = true; }   // static block, runs once
  constructor(name) { this.name = name; Animal.count++; }
  speak() { return `${this.name} makes a sound`; }   // on Animal.prototype
  get upper() { return this.name.toUpperCase(); }    // accessor
  static isAnimal(x) { return #secret in x; }        // brand check
}
class Dog extends Animal {
  speak() { return `${super.speak()} - woof`; }
}
new Dog('Rex').speak();  // 'Rex makes a sound - woof'
```

Fact	Checked result
`Object.keys(new Dog('Rex'))`	`['name']` — methods are non-enumerable, `#private` is invisible
`Object.getPrototypeOf(Dog.prototype) === Animal.prototype`	`true` — `extends` links the chains
`Animal('x')` without `new`	`TypeError`
Using `this` before `super()` in a subclass constructor	`ReferenceError`
`typeof class {}`	`'function'` — a class is a special function
`Object.getPrototypeOf(Object.prototype)`	`null` — the end of every chain
`'toString' in Object.create(null)`	`false` — no inherited keys

How lookup works (§9.3): reading a missing property walks the `[[Prototype]]` chain until it finds it or reaches `null`; writing always creates an **own** property (shadowing). `Fn.prototype` is the object that *instances* of `new Fn()` get as their prototype, not `Fn`'s own prototype. `__proto__` is a legacy accessor — use `Object.getPrototypeOf` / `Object.create`.

`new Fn(args)` (§9.4): 1. create an object linked to `Fn.prototype`; 2. run `Fn` with `this` = that object; 3. return it, unless `Fn` returns a different **object** (a returned primitive is ignored).

Type checks (§9.7): `typeof` for primitives, `Array.isArray` for arrays, `Error.isError` (ES2026) for errors, `x instanceof C` for "made by C" (walks the chain; breaks across realms), `#brand in x` inside a class, duck typing (`typeof x?.then === 'function'`) for "behaves like", `Object.prototype.toString.call(x)` → `'[object Date]'` for built-in tags.

Design (§9.8, §9.9): inheritance (is-a) only for stable taxonomies such as `HttpError extends Error`; composition (has-a) for most application code; mixins to share a capability between unrelated classes. TypeScript's `private` is compile-time only; JavaScript's `#private` is enforced at runtime.

18.7 Strings and regular expressions

Strings (§6.2–§6.6) are immutable UTF-16 primitives. Methods always return a **new** string.

Job	Method	Example → result
Interpolate / multi-line	template literal	`` `${1 + 1}` `` → `'2'`
Last char, out-of-range	`at(-1)`, `s[i]`, `charAt(i)`	`'Hello'.at(-1)` → `'o'`; `'Hello'[10]` → `undefined`; `charAt(10)` → `''`
Substring	`slice(start, end?)` (negatives OK)	`'Hello'.slice(-3)` → `'llo'`, `slice(1, 3)` → `'el'`
Search	`includes`, `startsWith`, `endsWith`, `indexOf` (−1 if absent)	
Replace first / all	`replace` / `replaceAll`	`'a-b-c'.replace('-', '+')` → `'a+b-c'`; `replaceAll` → `'a+b+c'`
Pad, repeat	`padStart`, `padEnd`, `repeat`	`'5'.padStart(3, '0')` → `'005'`
Trim	`trim`, `trimStart`, `trimEnd`	
Split / join	`split(sep)`, `arr.join(sep)`	`'a,b,,c'.split(',')` → `['a', 'b', '', 'c']`
Case	`toUpperCase`, `toLowerCase`, `toLocale…`	
Human ordering	`a.localeCompare(b)`, `Intl.Collator`	default `sort` gives `'Cab'` for `['b','a','C']`; with `localeCompare` → `'abC'`

Unicode (§6.5): `length` counts 16-bit code units — `'😀'.length` is `2`, `[...'😀'].length` is `1` (code points), and a family emoji `'👨‍👩‍👧'` is one grapheme to `Intl.Segmenter`. Normalize with `normalize('NFC')` before comparing accented text.

Regex syntax (§6.7):

Piece	Meaning
`.` `\d` `\w` `\s` (`\D` `\W` `\S`)	any char (not newline) / digit / word char / whitespace (and complements)
`[abc]` `[^abc]` `[a-z]`	class / negated class / range
`*` `+` `?` `{n,m}`	0+ / 1+ / 0–1 / n to m — **greedy**; add `?` for lazy
`^` `$` `\b`	start / end / word boundary
`( )` `(?<name> )` `(?: )`	capture / named capture / group only
`(?= )` `(?! )` `(?<= )` `(?<! )`	lookahead / negative lookahead / lookbehind / negative lookbehind
`\1`, `\k<name>`	backreference
`\p{L}` (with `u` or `v`)	any letter in any script
Flags	`g` all, `i` ignore case, `m` multiline anchors, `s` dot matches newline, `u` Unicode, `v` Unicode sets, `y` sticky, `d` indices

Regex methods (§6.8):

Call	Result
`/^\d{3}-\d{4}$/.test('555-1234')`	`true`
`'x'.match(/y/)`	`null` (not `[]`)
`'a1b22'.match(/\d+/g)`	`['1', '22']`
`'2024-03-10'.match(/(?<y>\d{4})-…/).groups.y`	`'2024'`
`'2024-03-10'.replace(/(\d+)-(\d+)-(\d+)/, '$3/$2/$1')`	`'10/03/2024'`
`[...'a1b2'.matchAll(/\d/g)].map(m => m.index)`	`[1, 3]`; `matchAll` without `g` throws `TypeError`
`'<b>x</b>'.replace(/<.+>/, '')` vs `/<.+?>/`	`''` (greedy) vs `'x</b>'` (lazy)
`/a/g.test('aa')` then `lastIndex`	`1` — a `g` regex remembers its position between calls
`new RegExp(RegExp.escape(userText))`	matches the text literally (ES2025)

Prefer plain string methods for fixed text and real parsers (`URL`, `JSON.parse`, `DOMParser`) for structured formats (§6.9).

18.8 Arrays: mutating vs non-mutating

✎ = changes the original array (§7.4).

Mutates ✎	Returns	Copying twin / alternative (leaves the original alone)
`push(x)` / `pop()`	new length / removed item	`[...arr, x]` / `arr.slice(0, -1)`
`unshift(x)` / `shift()`	new length / removed item	`[x, ...arr]` / `arr.slice(1)`
`splice(i, n, ...items)`	array of **removed** items	`toSpliced(i, n, ...items)` (ES2023)
`sort(cmp)`	the **same** array	`toSorted(cmp)` (ES2023)
`reverse()`	the same array	`toReversed()` (ES2023)
`arr[i] = x`	—	`with(i, x)` (ES2023; negative `i` OK, out of range → `RangeError`)
`fill(v, s, e)`, `copyWithin(t, s, e)`	the same array	`Array.from` / `map`
`arr.length = n`	truncates	`slice(0, n)`

Never mutated: `slice`, `concat`, `map`, `filter`, `reduce`, `flat`, `flatMap`, `find*`, `some`, `every`, `includes`, `indexOf`, `at`, `join`, `keys/values/entries`, spread.

Checked behaviours: `const s = arr.sort(); s === arr` → `true`; `[1].push(2)` → `2` (not the array); `[10, 1, 2].sort()` → `[1, 10, 2]` (default sort compares **strings**) but `.sort((a, b) => a - b)` → `[1, 2, 10]`; `[3, 1, 2].toSorted()` → `[1, 2, 3]` with the original unchanged.

Iteration methods (§7.5):

Want	Method	Checked example
Transform each	`map`	
Keep some	`filter`	
Fold to one value	`reduce(fn, init)`	`[1,2,3].reduce((s, x) => s + x, 0)` → `6`; `[].reduce(fn)` with no init → `TypeError`
First match / its index	`find` / `findIndex`	`find(x => x > 1)` → `2`; `findIndex(x => x > 5)` → `-1`
Last match	`findLast` / `findLastIndex` (ES2023)	`findLast(x => x > 1)` → `3`
Any / all	`some` / `every`	`[].every(…)` → `true` (vacuous truth)
Contains	`includes`	`[NaN].includes(NaN)` → `true`, `[NaN].indexOf(NaN)` → `-1`
Flatten	`flat(depth = 1)`, `flatMap`	`[1,[2,[3,[4]]]].flat()` → `[1,2,[3,[4]]]`; `flat(Infinity)` → `[1,2,3,4]`
Side effects only	`forEach` / `for…of`	`forEach` can't `break` or `await`

Sorting (§7.6): numbers `(a, b) => a - b`; text `a.localeCompare(b)`; several keys chained with `||`: `(p, q) => p.age - q.age || p.name.localeCompare(q.name)`. `sort` is stable (guaranteed since ES2019).

Creating and destructuring (§7.2, §7.7): `Array.from({ length: 3 }, (_, i) => i * 2)` → `[0, 2, 4]`; `Array.of(3)` → `[3]` but `new Array(3)` has length 3 and holes; `Array(2).fill([])` puts the **same** inner array in both slots. `[x, y] = [y, x]` swaps; `const [first, , third = 'd', ...others] = [1, 2, undefined, 4, 5]` → `1`, `'d'`, `[4, 5]`. `join` turns `null`/`undefined` into empty strings.

Immutable updates (React/Redux style): add `[...arr, x]`; insert `arr.toSpliced(i, 0, x)`; remove by id `arr.filter(t => t.id !== id)`; replace `arr.with(i, x)`; update one `arr.map(t => t.id === id ? { ...t, done: true } : t)`.

Which collection (§7.8): array for ordered lists, object for fixed-shape records, `Map` for dynamic keyed lookup, `Set` for uniqueness. **Typed arrays** (§7.9) hold fixed-size binary numbers over an

`ArrayBuffer`; values wrap (`new Uint8Array([256, 257])` → `[0, 1]`).

18.9 Map, Set, iterators and generators

`Map` (§8.1) — keys of any type, insertion order, `size`:

```
const m = new Map([[1, 'num'], ['1', 'str']]);   // 1 and '1' are different keys
m.set({}, 'obj').set(NaN, 'nan');                // set() returns the map, so it chains
m.get(NaN);           // 'nan' - keys compare with SameValueZero
JSON.stringify(m);    // '{}'  - convert first: Object.fromEntries(m) or [...m]
for (const [k, v] of m) { /* insertion order */ }
```

`Set` (§8.2) — unique values, O(1) `has`: `[...new Set([3, 1, 3, 2, 1])]` → `[3, 1, 2]` (dedupe, first-seen order). ES2025 set algebra, with A = {1,2,3} and B = {2,3,4}:

Method	Result
`A.union(B)`	{1, 2, 3, 4}
`A.intersection(B)`	{2, 3}
`A.difference(B)`	{1}
`A.symmetricDifference(B)`	{1, 4}
`A.isSubsetOf(B)`, `isSupersetOf`, `isDisjointFrom`	booleans

Use	When
`Map`	dynamic dictionaries, counting, caches (an LRU cache is one `Map`), non-string keys
plain object	fixed-shape records, config, JSON
`Set`	membership tests, dedupe, visited sets in BFS/DFS
`WeakMap` / `WeakSet` (§8.3)	data attached to objects you don't own; entries vanish with the key object; no `size`, no iteration
`WeakRef` / `FinalizationRegistry`	advanced caches; GC timing is never guaranteed

Iteration protocol (§8.4): an **iterable** has a `[Symbol.iterator]()` method returning an **iterator**, an object whose `next()` returns `{ value, done }`. Arrays, strings, Maps, Sets, typed arrays, NodeLists and generators are iterable; plain objects are not (`[...{ a: 1 }]` → `TypeError`). Consumers: `for…of`, spread, array destructuring, `Array.from`, `new Map/Set`, `Promise.all`, `yield*`.

Generators (§8.5):

```
function* nat() { let n = 1; while (true) yield n++; }   // infinite, lazy
nat().next();                          // { value: 1, done: false }
const iterable = { *[Symbol.iterator]() { yield 1; yield 2; } };
[...iterable];                         // [1, 2]
function* two() { yield 'a'; return 'r'; }
[...two()];                            // ['a'] - the return value is not iterated
```

Iterator helpers (ES2025, §8.6) are lazy: `nat().filter(n => n % 2).map(n => n * n).take(3).toArray()` → `[1, 9, 25]`. Arrays need `.values()` first; `Iterator.from(x)` wraps custom iterators.

Grouping (ES2024, §8.7): `Object.groupBy([1,2,3,4], n => n % 2 ? 'odd' : 'even')` → `{ odd: [1, 3], even: [2, 4] }` (a null-prototype object); `Map.groupBy` when keys aren't strings.

Async iteration (§12.12): `async function*` + `for await (const page of pages())`; `await Array.fromAsync(pages())` (ES2026) collects them.

18.10 Modules and tooling

```
// math.mjs
export const PI = 3.14;                 // named export
export function area(r) { return PI * r * r; }
export default class Circle {}          // one default per module
// app.mjs
import Circle, { area, PI as pi } from './math.mjs';
import * as math from './math.mjs';     // namespace object: math.area, math.default
export { area } from './math.mjs';      // re-export (barrel file)
const { heavy } = await import('./heavy.mjs');  // dynamic import -> promise, code splitting
import data from './data.json' with { type: 'json' };  // JSON module (ES2025)
```

ES module fact (§10.3)	Consequence
Loads in 3 phases: parse → link → evaluate	imports are hoisted; a missing export fails before any code runs
Imports are **live, read-only bindings**	you see the exporter's updates; you can't assign to them
Each module is evaluated **once** (singleton)	module-level state is shared by every importer
Always strict; top-level `this` is `undefined`	checked by the sheet script
Top-level `await` allowed (§10.5)	importers wait for it
`import.meta.url`, `import.meta.dirname` / `filename` (Node) (§10.6)	replace CommonJS `__dirname` / `__filename`

CommonJS vs ESM in Node (§10.8): `.mjs` = ESM, `.cjs` = CommonJS, `.js` follows `"type"` in the nearest `package.json`. CommonJS uses `require()` / `module.exports`, loads synchronously, has `__dirname`. ESM can import CommonJS (default import = `module.exports`); since Node 22.12 / 20.19, CommonJS can `require()` an ES module unless it uses top-level `await` (`ERR_REQUIRE_ASYNC_MODULE`). **Browsers** (§10.7): `<script type="module">` is deferred, strict, needs CORS and a server (not `file://`); import maps resolve bare names.

npm (§10.9, §10.10):

Item	Meaning
`dependencies` / `devDependencies` / `peerDependencies`	needed at runtime / only to build and test / "use the host's copy"
`^1.2.3`	≥1.2.3 <2.0.0 (npm's default)
`~1.2.3`	≥1.2.3 <1.3.0
`^0.2.3`	≥0.2.3 <0.3.0 (caret is stricter below 1.0)
`package-lock.json`	commit it; CI runs `npm ci` (exact, clean install)
`npx tool`, `npm run x`, `node --run x`	run a package binary / a script

Tools (§10.11–§10.13): bundlers (Vite, esbuild, webpack, Rollup/Rolldown) do resolution, tree shaking, code splitting, minification, source maps and HMR. **Transpile syntax, polyfill APIs**: a transpiler rewrites new syntax for old engines; a polyfill adds missing built-ins. ESLint finds bugs (flat config `eslint.config.js`); Prettier formats.

18.11 Errors and debugging

Error type (§11.1)	Checked trigger
`TypeError`	`null.length`, calling a non-function, `1n + 1`, assigning to a `const`
`ReferenceError`	undeclared name, TDZ access
`SyntaxError`	`JSON.parse('{bad}')`, bad code in `eval` / `new Function`
`RangeError`	`new Array(-1)`, `(1).toFixed(101)`, stack overflow
`URIError`	`decodeURIComponent('%')`
`AggregateError`	`Promise.any` when all reject — `.errors` holds them

```
class ValidationError extends Error {                // §11.3
  constructor(msg, opts) { super(msg, opts); this.name = 'ValidationError'; this.code = 'E_VALID'; }
}
try {
  save();
} catch (err) {
  if (!(err instanceof ValidationError)) throw err;   // catch narrowly, rethrow the rest
  throw new Error('save failed', { cause: err });     // §11.4 error chaining
} finally {
  release();                                          // always runs; never return from it
}
```

Rules: always throw `Error` objects (they carry a `stack`); `String(err)` → `'ValidationError: save failed'`; a `return` in `try` still runs `finally` first; `catch {}` without a binding is legal (ES2019) but never swallow errors silently. **`try/catch` is synchronous**: it catches a rejected `await` inside an `async` function, but not an error thrown later in a timer callback (§11.5). Global last-resort handlers: browser `error` / `unhandledrejection` events; Node `uncaughtException` / `unhandledRejection` — log, then exit. `using` + `Symbol.dispose` gives automatic cleanup (§11.6; finished proposal, edition ES2027).

Debugging (§11.7–§11.9): `console.table`, `dir`, `group`, `count`, `time` / `timeEnd`, `trace`, `assert` (logs, doesn't throw). Stack traces list the newest frame first; source maps map bundled code back to your files. Breakpoints (conditional, logpoints, pause on exceptions), step over / into / out, Scope and Watch panes; `debugger;` statement; Node: `node --inspect` then `chrome://inspect`. Method: reproduce → read the message → hypothesize → inspect → fix the root cause → add a test. Common messages decoded: §11.10.

18.12 Async and the event loop

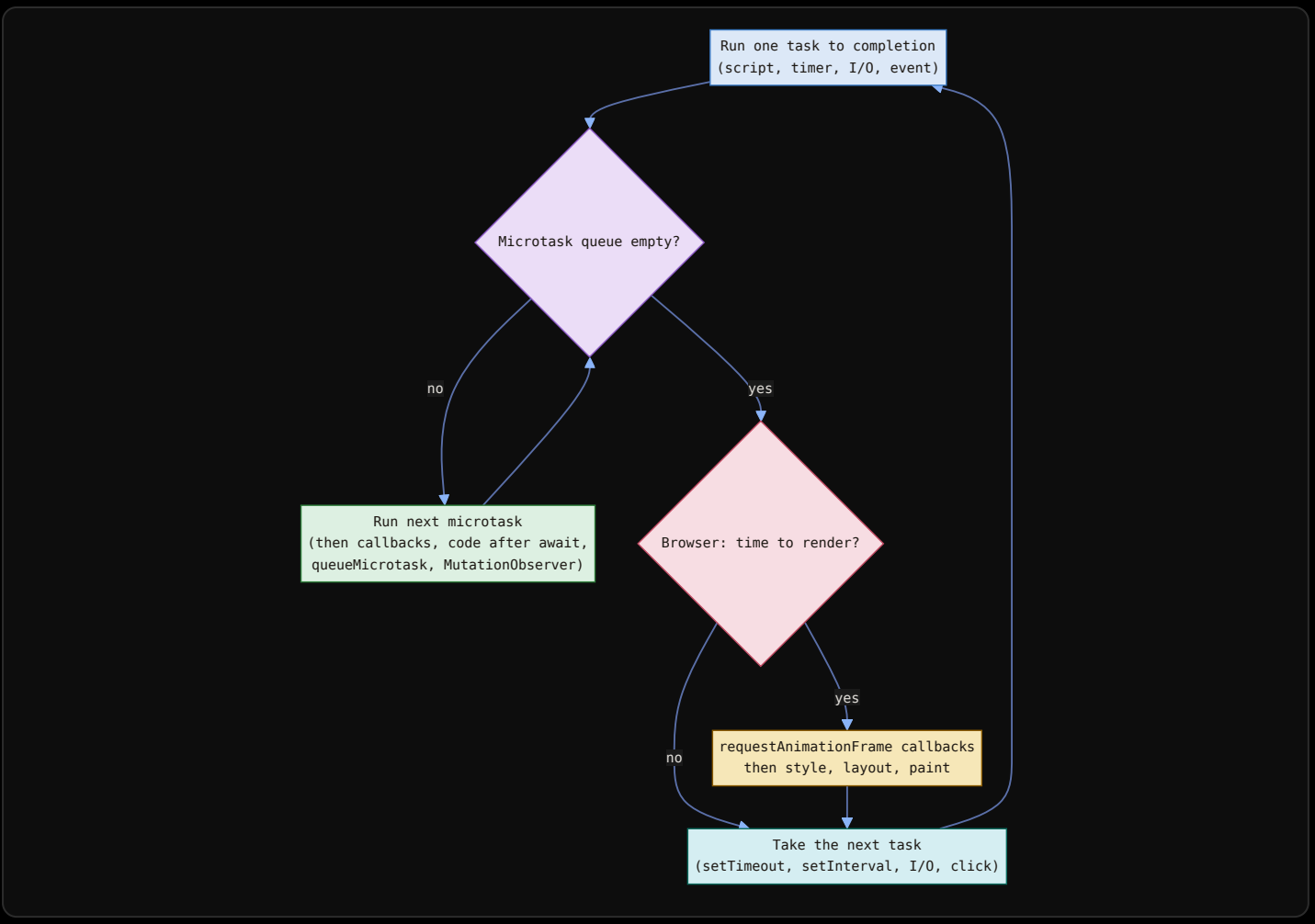


*Figure — one turn of the event loop: one task, then **all** microtasks, then maybe a render (§12.2).*

The ordering rule (§12.3): synchronous code first (including promise executors and an async function's body up to its first `await`), then microtasks in FIFO order, then the next task.

```
log('A');
setTimeout(() => log('B timeout'), 0);
Promise.resolve().then(() => log('C then')).then(() => log('D then2'));
queueMicrotask(() => log('M micro'));
new Promise((res) => { log('P executor'); res(); });
(async () => { log('F before await'); await null; log('F after await'); })();
log('E');
// A, P executor, F before await, E, C then, M micro, F after await, D then2, B timeout
```

Node extras (§12.4): phases timers → pending → poll (I/O) → check (`setImmediate`) → close. Inside a callback, `process.nextTick` runs before promise microtasks, and both run before `setImmediate` — checked order: `nextTick, promise, immediate`.

Promises (§12.7, §12.8): pending → fulfilled or rejected, settle **once**; `then` returns a new promise, so always `return` inside it and end the chain with `catch`.

Combinator	Fulfils when	Rejects when	Checked
`Promise.all`	all fulfil — values in **input order**	first rejection	`[ok(1, 5ms), ok(2, 1ms)]` → `[1, 2]`
`Promise.allSettled`	all settle	never	statuses `['fulfilled', 'rejected']`
`Promise.race`	first to **settle** fulfils	first to settle rejects	fastest wins
`Promise.any`	first to **fulfil**	all reject → `AggregateError`	skips a rejection
`Promise.withResolvers()` (ES2024)	`{ promise, resolve, reject }`		
`Promise.try(fn)` (ES2025)	a sync throw becomes a rejection		

None of the combinators cancel the losing operations.

async/await (§12.9): an `async` function always returns a promise; `await` unwraps it and **throws** on rejection (so `try/catch` works). Two independent awaits one after another take the sum of their times (checked: ~80 ms); `await Promise.all([...])` takes the longest (~40 ms). `arr.forEach(async …)` doesn't wait — use `for…of` (one at a time) or `Promise.all(arr.map(async …))` (all at once). Use `return await` inside `try` so the `catch` sees the rejection.

Timers (§12.5): delays are minimums; nested timers clamp to 4 ms in browsers; delays above $2^{31} - 1$ ms overflow; prefer recursive `setTimeout` to `setInterval` for polling. **Cancellation** (§12.11): `const ac = new AbortController(); fetch(url, { signal: ac.signal }); ac.abort()`; `AbortSignal.timeout(ms)`; `AbortSignal.any([...])`. **Patterns** (§12.13): retry with exponential backoff + jitter, concurrency limits, last-request-wins, debounce/throttle.

18.13 DOM and events

Task	API (§13.1–§13.4)
Find one / many	`document.getElementById('id')`, `el.querySelector(css)` / `querySelectorAll(css)` (static `NodeList`)
Live collections	`getElementsByTagName` / `ByClassName` — update automatically (checked: after appending an `<li>`, the live list has 2 items, the static one still 1)
Relatives	`el.closest(css)` (self or ancestor), `el.matches(css)`, `parent.contains(node)`, `children`, `nextElementSibling`
Create and insert	`document.createElement('li')`, `append`, `prepend`, `before`, `after`, `replaceWith`, `remove`, `replaceChildren`; `DocumentFragment` / `<template>` for bulk
Text vs HTML	`el.textContent = userText` (never parsed — `'<b>safe</b>'` shows as text); `innerHTML` only for trusted markup (XSS risk)
Classes, data, style	`classList.add/remove/toggle/contains`, `el.dataset.id` (from `data-id`, always a string), `el.style.x` (inline), `getComputedStyle(el)` (final)
Attribute vs property	`input.getAttribute('value')` = initial HTML value; `input.value` = current value

Events (§13.5–§13.8):

```
list.addEventListener('click', (e) => {          // one listener for all items (delegation)
  const item = e.target.closest('li');           // target = where it started
  if (!item || !list.contains(item)) return;
  console.log(item.dataset.id, e.currentTarget); // currentTarget = list (the listener's element)
}, { signal: controller.signal });                // options: once, passive, capture, signal
```

Fact	Checked
Propagation: capture (window → target), target, bubble (target → window)	order: `body capture`, `delegated 7`, `body bubble target=BUTTON current=BODY`
`new Event('ping')` doesn't bubble	parent heard 0; with `{ bubbles: true }` → 1
`{ once: true }`	handler ran once for two clicks
`e.preventDefault()`	cancels the default action (link navigation, form submit); `e.defaultPrevented` → `true`
`stopPropagation()`	stops the journey — use rarely
`dispatchEvent`	runs listeners **synchronously**

Forms (§13.9): listen for `submit`, call `preventDefault()`, read `new FormData(form)` (`getAll('t')` → `['a', 'b']` for repeated names), check `input.validity` / `setCustomValidity`, use `form.requestSubmit()`; always validate again on the server. **Watch changes**: `MutationObserver` (§13.10). **Performance** (§13.11): batch all layout reads, then all writes (interleaving them causes **layout thrashing**); animate `transform`/`opacity`; use `requestAnimationFrame`. **Script timing** (§13.12): `defer` and modules run after parsing, in order, before `DOMContentLoaded`; `async` runs whenever downloaded; `load` waits for images and styles.

18.14 Browser APIs and security

fetch (§14.1):

```
async function getJSON(url, { timeout = 5000 } = {}) {
  const res = await fetch(url, { signal: AbortSignal.timeout(timeout),
                                 headers: { Accept: 'application/json' } });
  if (!res.ok) throw new Error(`HTTP ${res.status}`);   // fetch resolves on 404/500!
  return res.json();                                    // the body can be read only once
}
await fetch('/api/items', { method: 'POST',
  headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(item) });
```

`fetch` rejects only on network failure or abort; a 404 `Response` has `ok === false`. Reading a body twice throws `TypeError`.

URL (§14.6): `new URL('https://shop.example.com:8443/search?q=red shoes')` → `.origin` = `'https://shop.example.com:8443'`, `.searchParams.get('q')` = `'red shoes'`; after `set('page','3')` and `append('tag','a&b')` the search string is `'?q=red+shoes&page=3&tag=a%26b'` (encoding done for you). `new URL('../img/a.png', 'https://x.dev/app/page/')` resolves to `https://x.dev/app/img/a.png`; `new URL('not a url')` throws `TypeError` (a cheap validator). `encodeURIComponent('a b&c')` → `'a%20b%26c'`.

Storage (§14.3, §14.4):

	Cookie	localStorage	sessionStorage	IndexedDB
Size	~4 KB	~5 MB	~5 MB	large
API	strings, sync	strings, sync (`setItem('n', 42)` stores `'42'` ; missing → `null`)	same, per tab	async, structured data
Sent to server	automatically	no	no	no
Use for	session ID (`HttpOnly; Secure; SameSite`)	preferences	wizard state	offline data, big caches

Never put tokens or secrets in `localStorage` — any script on the page can read it.

Other APIs: `requestAnimationFrame` for visual updates, once per frame, paused in background tabs (§14.5); History API `pushState` / `replaceState` + `popstate` for SPA routing (§14.7); Web Workers for CPU work off the main thread, talking via `postMessage` + structured clone (§14.8); `IntersectionObserver` (visibility, lazy loading) and `ResizeObserver` (element size) (§14.9). `structuredClone` copies Dates and Maps but throws `DataCloneError` on functions.

Security (§14.2, §14.10–§14.12):

Threat / rule	Remember
Origin	scheme + host + port; the browser lets a page *send* anywhere but *read* only same-origin responses
CORS	the **server** opts in with `Access-Control-Allow-Origin` ; non-simple requests get an `OPTIONS` preflight; `*` can't be combined with credentials
XSS	treat data as text (`textContent`), escape by context (`<` → `<`), sanitize HTML with DOMPurify, allow-list URL schemes, CSP, `HttpOnly` cookies
CSRF	`SameSite` cookies + CSRF token or `Origin` check; GET must never change state; CORS is not a CSRF defence
CSP	`Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none'` — blocks inline scripts and clickjacking
Other headers	HSTS, `X-Content-Type-Options: nosniff` , `Referrer-Policy` , `Permissions-Policy`
Supply chain	lockfiles, `npm audit` , few dependencies, Subresource Integrity for CDN scripts

---

18.15 Node.js core modules

Node = V8 + **libuv** (event loop, async I/O, a 4-thread pool for fs, DNS lookup, crypto, zlib) + core modules (§15.1, §15.2). Import built-ins with the `node:` prefix. Never block the loop; move CPU-heavy work to `worker_threads` .

Module	Everyday calls	Checked / remember
`node:fs/promises` (§15.4)	`readFile(p, 'utf8')`, `writeFile`, `appendFile`, `mkdir(p, { recursive: true })`, `readdir`, `stat`, `rm(p, { recursive: true, force: true })`	branch on `err.code` (`'ENOENT'`); don't check-then-act
`node:path` (§15.5)	`join`, `resolve`, `basename`, `extname`, `dirname`, `parse`, `relative`	`join('/srv','app','..','data')` → `/srv/data`; `extname('price.test.mjs')` → `.mjs`
`node:url`	`fileURLToPath(import.meta.url)`, `pathToFileURL`	or `import.meta.dirname`
`process` (§15.6)	`argv`, `env`, `exitCode`, `cwd()`, `on('SIGTERM')`	`process.env.X = 5` reads back `'5'` — env values are strings; `--env-file=.env`
`node:util`	`parseArgs`, `promisify`, `inspect`	`parseArgs` on `['--port','8080','file.txt']` → `port: '8080'`, positionals `['file.txt']`
`Buffer` (§15.7)	`Buffer.from(str)`, `.toString('hex' / 'base64')`	`Buffer.from('é').length` → `2` bytes, `'é'.length` → `1`
`node:events` (§15.8)	`on`, `once`, `off`, `emit`; `events.once(em, name)` → promise	`emit` is **synchronous**; an unhandled `'error'` event throws
`node:stream` (§15.9)	Readable, Writable, Duplex, Transform; `pipeline` from `node:stream/promises`	`pipeline` handles **backpressure** and errors
`node:http` (§15.10)	`createServer((req, res) => { … res.end() })`, `listen(0)` in tests	always end the response; limit body size
`node:child_process` (§15.11)	`execFile(cmd, args)`, `spawn`	pass argument arrays — no shell injection
`node:test`, `node:assert/strict`	§18.16	

Flags (§15.12): `--watch`, `--env-file`, `--run`, `--test`, `--inspect`, `--permission`; `node app.ts` strips types (no type check). **Alternatives**: Deno (secure by default, V8) and Bun (JavaScriptCore, fast tooling); Node stays the default choice.

18.16 Testing and TypeScript

```
// price.test.mjs - run with: node --test
import { test, describe, beforeEach } from 'node:test';
import assert from 'node:assert/strict';
import { applyDiscount } from './price.mjs';

describe('applyDiscount', () => {
  test('takes 10 percent off', () => assert.equal(applyDiscount(100, 10), 90));
  test('rejects negative percentages', () => assert.throws(() => applyDiscount(100, -1), RangeError));
  test('async loader rejects', async () => { await assert.rejects(load('bad'), /not found/); });
  test('calls the logger once', (t) => {
    const log = t.mock.fn();                    // spy, auto-restored
    applyDiscount(100, 10, { log });
    assert.equal(log.mock.callCount(), 1);
  });
});
```

Assertion (§16.2)	Passes when	Checked
`equal(a, b)`	`Object.is(a, b)` — same primitive or same object	`equal({a:1}, {a:1})` fails
`deepEqual(a, b)`	same structure, type-strict	`{a:1}` vs `{a:'1'}` fails
`throws(fn, Class | /regex/)`	`fn()` throws a match	
`await rejects(promise, …)`	the promise rejects — **must be awaited**	
`match(str, /re/)`	regex matches	

`node:test` (§16.3–§16.6): `test` / `it`, `describe`, hooks `before` / `beforeEach` / `afterEach` / `after`, `skip` / `todo` / `only`; `node --test` (finds `*.test.*`), `--watch`, `--test-name-pattern`; `t.mock.fn` / `t.mock.method` spies; fake timers `t.mock.timers.enable()` + `tick(ms)`; snapshots; `--experimental-test-coverage`. Test pyramid: many unit, some integration, few end-to-end (Playwright). Vitest/Jest use `expect(x).toBe(y)` (§16.7).

TypeScript (§16.8–§16.13):

```ts
type Shape = { kind: 'circle'; r: number } | { kind: 'rect'; w: number; h: number }; // discriminated union
function area(s: Shape): number {
  switch (s.kind) {                                  // narrowing by discriminant
    case 'circle': return Math.PI * s.r ** 2;
    case 'rect': return s.w * s.h;
    default: { const never: never = s; return never; }   // exhaustive check
  }
}
function first<T>(xs: readonly T[]): T | undefined { return xs[0]; }   // generic
const ROLES = ['admin', 'user'] as const;            // readonly literal tuple
type Role = (typeof ROLES)[number];                  // 'admin' | 'user'
```

Concept	One line
Types are erased	nothing is checked at runtime — validate JSON with a schema (e.g. Zod)
Structural typing	shapes match, names don't matter
`unknown` vs `any`	`unknown` must be narrowed before use; `any` switches checking off
Narrowing	`typeof`, `instanceof`, `in`, discriminant property, type predicates `x is T`, `asserts` functions
Utility types	`Partial`, `Required`, `Readonly`, `Pick`, `Omit`, `Record`, `ReturnType`, `Parameters`, `Awaited`, `NonNullable`
`interface` vs `type`	both describe objects; `type` also does unions/tuples; interfaces merge
`satisfies`	checks a value against a type without widening it
Running	`tsc --noEmit` type-checks; `node file.ts` only strips erasable syntax (no `enum`, no parameter properties)
JSDoc	`// @ts-check` + `/** @param {number} x */` type-checks plain `.js` files (§16.15)

18.17 Which API for which job

Job	Reach for	Not	See
Default variable	`const` (`let` if reassigned)	`var`	§2.1
Compare values	`===`, `Object.is` for `NaN` / `-0`	`==` (except `x == null`)	§2.6
Default for a missing value	`??` / `??=`	`||` when `0` or `''` are valid	§2.7
Read possibly-missing nested data	`a?.b?.c`	long `&&` chains	§5.8
Money	integer cents, `Intl.NumberFormat` for display	floats	§2.10
Integers beyond 2^{53}	`BigInt`	`number`	§2.10
Parse user input to a number	`Number(s)` + `Number.isNaN`	`parseInt` without a radix	§2.12
Walk an array / string / Map	`for…of`	`for…in`	§3.12
Walk a plain object	`Object.entries(obj)` + `for…of`	`for…in` without `hasOwn`	§5.6
Transform / select / fold	`map` / `filter` / `reduce`	`forEach` + push	§7.5
Sort without touching the original	`toSorted(cmp)`	`sort()` on shared data	§7.4
Last element	`arr.at(-1)`	`arr[arr.length - 1]`	§7.3
Copy an object / array (top level)	`{ ...o }`, `[...a]`	`Object.assign` into the original	§5.4
Deep copy	`structuredClone`	`JSON.parse(JSON.stringify(x))`	§5.4
Dictionary with dynamic or non-string keys	`Map`	plain object	§8.1
Unique values, fast membership	`Set`	`array.includes` in a loop	§8.2
Metadata on objects you don't own	`WeakMap`	`Map` (leaks)	§8.3
Group items by a key	`Object.groupBy` / `Map.groupBy`	hand-written `reduce`	§8.7
Lazy or infinite sequence	generator + iterator helpers	building a huge array	§8.5
Text search / replace of fixed text	`includes`, `replaceAll`	regex	§6.6
Pattern matching	regex (`test`, `matchAll`, named groups)	manual char loops	§6.8
Count visible characters	`Intl.Segmenter`	`length`	§6.5
Format dates / numbers for people	`Intl.DateTimeFormat`, `toLocaleString`	string concatenation	§5.11
Callback that keeps `this`	arrow function	`const self = this`	§9.1
Truly private state	`#private` fields or closures	`_underscore` names	§9.5

Job	Reach for	Not	See
Share code between modules	ES modules (`import` / `export`)	globals, IIFEs	§10.2
Load code only when needed	`import()`	one giant bundle	§10.4
Reproducible installs	lockfile + `npm ci`	`npm install` in CI	§10.10
Report a failure	`throw new Error(msg, { cause })` / custom subclass	throwing strings	§11.3
Several independent async jobs	`await Promise.all([...])`	sequential awaits	§12.9
Report every outcome	`Promise.allSettled`	`all`	§12.8
Timeout or cancel	`AbortSignal.timeout(ms)` , `AbortController`	`Promise.race` alone (doesn't cancel)	§12.11
Wrap an old callback API	`util.promisify` or `new Promise` once	nesting callbacks	§12.7
Paged / streamed data	`async function*` + `for await…of`	loading everything first	§12.12
Insert user text into the page	`textContent`	`innerHTML`	§13.3
Handle events on many children	delegation with `closest`	a listener per item	§13.7
Remove many listeners at once	`{ signal }` + `controller.abort()`	tracking every function	§13.5
Animate	CSS transitions, or `requestAnimationFrame` changing `transform`	`setInterval` changing `top` / `left`	§14.5
React to scroll visibility / element size	`IntersectionObserver` / `ResizeObserver`	`scroll` / `resize` handlers with measurements	§14.9
HTTP request	`fetch` + check `res.ok`	`XMLHttpRequest`	§14.1
Build or parse URLs	`URL` , `URLSearchParams`	string concatenation	§14.6
Store a session	`HttpOnly; Secure; SameSite` cookie	`localStorage` token	§14.4
Large offline data	IndexedDB	`localStorage`	§14.4
Heavy computation in a page	Web Worker	a long loop on the main thread	§14.8
Heavy computation in Node	`worker_threads`	blocking the event loop	§15.2
Big files / HTTP bodies	streams + `pipeline`	`readFile` of gigabytes	§15.9
Run another program	`execFile` / `spawn` with an argument array	`exec` with user input	§15.11
Tests without dependencies	`node:test` + `node:assert/strict`	ad-hoc `console.log` checks	§16.3
Time-dependent tests	fake timers (`t.mock.timers`)	real `setTimeout` waits	§16.5
Types for untrusted data	`unknown` + narrowing / a schema	`any`	§16.10

Job	Reach for	Not	See
A fixed set of names in TS	union of literals or `as const` object	`enum` (not erasable)	§16.13

18.18 ECMAScript versions ES2015 → ES2026

Each edition is published in June and contains every proposal that reached **Stage 4** in the year before (§1.2). The years below were checked against TC39's list of finished proposals (github.com/tc39/proposals, "finished-proposals"), and the Node 24 column was checked by running the features in Node v24.11 (the `18.18` block of `cheatsheet_checks.mjs`).

Edition	Main features	Node 24
ES2015 (ES6)	`let` / `const`, arrow functions, classes, ES modules, promises, template literals, destructuring, default / rest / spread parameters, `Map` / `Set` / `WeakMap` / `WeakSet`, `Symbol`, iterators and generators, `for…of`, `Proxy` / `Reflect`, typed arrays, `Array.from` / `of`, `find` / `findIndex`, `Object.assign`, `String` `includes` / `startsWith` / `repeat`, `Number.isNaN` / `isInteger`, regex `u` and `y` flags	✓
ES2016	`Array.prototype.includes`, exponent operator `**`	✓
ES2017	`async` / `await`, `Object.values` / `entries`, `padStart` / `padEnd`, `Object.getOwnPropertyDescriptors`, trailing commas in parameter lists, `SharedArrayBuffer` and `Atomics`	✓
ES2018	object rest/spread `{ ...o }`, async iteration (`for await…of`, async generators), `Promise.prototype.finally`, regex `s` flag, named groups, lookbehind, `\p{…}` escapes	✓
ES2019	`flat` / `flatMap`, `Object.fromEntries`, `trimStart` / `trimEnd`, optional `catch` binding, `Symbol.prototype.description`, well-formed `JSON.stringify`; `sort` required to be stable	✓
ES2020	`BigInt`, `??`, `?.`, dynamic `import()`, `import.meta`, `globalThis`, `Promise.allSettled`, `String.prototype.matchAll`, defined `for…in` order	✓
ES2021	`replaceAll`, `Promise.any` + `AggregateError`, logical assignment `||=` `&&=` `??=`, numeric separators `1_000`, `WeakRef` / `FinalizationRegistry`	✓
ES2022	class fields, `#private` methods and fields, `static` blocks, `#x in obj`, top-level `await`, `.at()`, `Object.hasOwn`, error `cause`, regex `d` flag (match indices)	✓
ES2023	`toSorted` / `toReversed` / `toSpliced` / `with`, `findLast` / `findLastIndex`, hashbang `#!` lines, symbols as `WeakMap` keys	✓
ES2024	`Object.groupBy` / `Map.groupBy`, `Promise.withResolvers`, regex `v` flag, `isWellFormed` / `toWellFormed`, resizable `ArrayBuffer`, `ArrayBuffer.prototype.transfer`, `Atomics.waitAsync`	✓
ES2025	iterator helpers + `Iterator.from`, `Set` methods (`union`, `intersection`, …), `Promise.try`, `RegExp.escape`, regex pattern modifiers `(?i:…)`, duplicate named groups, import attributes + JSON modules, `Float16Array` / `Math.f16round`	✓
ES2026	`Array.fromAsync`, `Error.isError`, `JSON.parse` source text access (`JSON.rawJSON`), `Math.sumPrecise`, `Uint8Array` to/from base64 and hex, `Iterator.concat`, upsert (`Map.prototype.getOrInsert` / `getOrInsertComputed`)	partial: first three ✓; the rest ✗ (feature-detect)
Next: ES2027	finished but not yet published: **Temporal**, explicit resource management (`using`, `Symbol.dispose`), `Atomics.pause`, joint iteration (`Iterator.zip`)	`using` and `Atomics.pause` already work; Temporal and `Iterator.zip` don't

"Node 24 ✓" means Node's V8 ships the feature. For browsers, check MDN's compatibility tables or the Baseline badge before relying on anything from the last two editions. A transpiler can rewrite new *syntax*, but only a polyfill can add missing *APIs* (§10.12).

18.19 Where to go next

A four-step study path built on this book.

Step 1 — Lock in the core (1–2 weeks)

1. Re-read the TL;DR at the end of each chapter, then do each sheet in this chapter with the right-hand column covered.
2. Work through the ordering puzzles in §12.3 and the 53 questions in §17.4 until you can explain each answer out loud.
3. Rebuild from memory: `debounce`, `throttle`, `curry`, `memoize`, a `Promise.all` polyfill and an LRU cache (§17.3).

Step 2 — Build small, real things (2–4 weeks)

Pick projects that force you to combine chapters:

Project	Exercises
To-do list with filters, saved in `localStorage`	DOM, events, delegation, immutable array updates, storage (Ch 13, Ch 14)
Search-as-you-type against a public API	`fetch`, `AbortController`, debounce, last-request-wins, error states (Ch 12)
A command-line tool that summarizes a CSV or log file	`node:fs/promises`, streams, `parseArgs`, exit codes (Ch 15)
A tiny JSON REST API with tests	`node:http`, validation, `node:test`, TypeScript via type stripping (Ch 15, Ch 16)

Write tests for each one, run ESLint and Prettier, and put it in a git repository.

Step 3 — Choose a direction

Direction	Next topics	In this library
Front end	semantic HTML, CSS layout (flexbox, grid), accessibility, then React (components, state, effects, routing, data fetching)	HTML & CSS Handbook, React Handbook
Back end with Node	a framework (Express, Fastify or Hono), SQL and schema design, authentication, caching, queues	SQL Handbook, Database modeling
Types at scale	strict TypeScript, generics, declaration files, typing React components	§16.8, TypeScript with React
Interviews	DSA patterns in JS, machine-coding rounds, object-oriented and system design	Ch 17, LLD Handbook, DDIA notes

Step 4 — Keep up to date

- **MDN Web Docs** is the reference for every API in this book, with browser-compatibility tables.
- **The ECMAScript spec** (tc39.es/ecma262) and the TC39 proposals repository show what's coming; features at Stage 3 are usually safe to learn.
- **Node.js release notes and docs** (nodejs.org): use an even-numbered LTS line.
- **Books for depth**: *You Don't Know JS Yet* (scope, closures, `this`, types), *Eloquent JavaScript* (fundamentals with exercises), and javascript.info as a free, well-structured tutorial.
- Read other people's code: small, well-maintained npm packages are a good place to start.

18.20 Interview question index

- **53 tricky questions with real output** — §17.4: coercion and equality Q1–14 (§17.4.1) · scope, hoisting, closures Q15–22 (§17.4.2) · `this`, arrows, `call` / `apply` / `bind`, `new` Q23–28 (§17.4.3) · prototypes and classes Q29–34 (§17.4.4) · event loop and async Q35–42 (§17.4.5) · objects, arrays, copies Q43–50 (§17.4.6) · memory Q51–53 (§17.4.7).

- **Built-in complexity table** — §17.1; **21 coding patterns** with a summary table — §17.2; **implement-from-scratch classics** (debounce, throttle, curry, compose/pipe, memoize, flatten, `bind`, deep clone, deep equal, LRU, EventEmitter, promise polyfills) — §17.3.
- **300+ short say-it-out-loud answers**, output puzzles and coding tasks by topic — Chapter 19.

18.21 Glossary (A–Z)

Each term has a one-line definition and a link to the section that explains it. Symbols and backticks are ignored when sorting.

A

- **AbortController** — Object whose `signal` is passed to `fetch`, listeners or timers; calling `abort()` cancels them all. §12.11
- `AbortSignal.any` — Combines several signals into one that aborts as soon as any of them does, e.g. a user's cancel button plus a timeout. §12.11
- **AbortSignal.timeout** — Ready-made signal that aborts itself after a number of milliseconds with a `TimeoutError`. §12.11
- **Abstract class** — Class meant only to be extended, never created directly; JavaScript has no `abstract` keyword (a constructor can throw if `new.target` is the base class), while TypeScript adds one. §9.6
- **Abstract syntax tree (AST)** — Tree-shaped description of your code's structure that a parser builds before the engine compiles or runs it; linters and transpilers work on it too. §1.3
- **Accessor property** — Property backed by a getter and/or setter function instead of a stored value. §5.7
- **Accidental global** — Variable that becomes a property of the global object because it was assigned without `let` / `const` / `var` in sloppy mode; strict mode turns it into a `ReferenceError`. §17.4
- `addEventListener` — Registers a function to run when an event of a given type reaches an element; accepts `once`, `passive`, `capture` and `signal` options. §13.5
- **AggregateError** — Error that wraps several errors in its `errors` array; `Promise.any` rejects with one when every input rejects. §11.1
- **AMD / UMD** — Older module formats: AMD loads modules asynchronously with `define()`; UMD wraps a file so it works as AMD, CommonJS or a global. §10.1
- **Amortized cost** — Average cost per operation over a long run: `push` is O(1) amortized even though the array sometimes has to grow and copy. §17.1
- **Anchor (regex)** — Position rather than a character: `^` start, `$` end, `\b` word boundary. §6.7
- `any` **(TS)** — The TypeScript type that switches checking off for a value; anything goes in and out, so prefer `unknown`. §16.10
- **API** — Application Programming Interface: the set of functions, objects and rules one piece of software offers for others to use, such as the DOM API or a web server's HTTP endpoints. §1.4
- **Argument** — Actual value you pass when calling a function; the parameter is the name that receives it. §4.1
- `arguments` — Array-like object holding every argument passed to a non-arrow function; replaced in modern code by rest parameters. §4.4
- **Arrange–Act–Assert (AAA)** — Layout for a test: set up the data, perform the one action under test, then check the result. §16.3
- **Array destructuring** — Unpacking values by position: `const [a, , c = 0, ...rest] = arr`. §7.7
- **Array-like** — Object with `length` and numeric indexes but no array methods (e.g. `arguments`, a `NodeList`); convert with `Array.from`. §4.4
- `Array.from` — Builds a real array from an iterable or array-like, with an optional mapping function. §7.2
- `Array.fromAsync` — Collects an async iterable (or iterable of promises) into an array, returning a promise (ES2026). §12.12
- `Array.isArray` — Reliable array test; works across realms, unlike `instanceof Array`. §7.1

- **ArrayBuffer** — Fixed block of raw bytes that typed arrays and `DataView` read and write. §7.9
- **Arrow function** — Short function syntax `(x) => x * 2` with lexical `this`, no `arguments` and no `new`. §4.3
- **`as const` (TS)** — Tells TypeScript to treat a literal as deeply read-only with its exact values (`'admin'` rather than `string`). §16.13
- **ASCII** — Old 7-bit character set covering 128 characters (English letters, digits, punctuation); its codes are the first 128 Unicode code points. §6.9
- **Assertion** — Check in a test that fails loudly when a value isn't what you expected (`assert.equal`, `assert.throws`). §16.2
- **Assertion function (TS)** — A TypeScript function declared `asserts v is T` that narrows its argument for the code after the call. §16.11
- **Assignment operators** — `=` plus compound forms such as `+=`, `**=`, `||=`, `&&=` and `??=`. §2.8
- **Associativity** — Which side groups first when operators of equal precedence meet: `-` is left-to-right (`8 - 3 - 1` is 4), while `**` and `=` are right-to-left. §2.9
- **`async` function** — Function that always returns a promise and may use `await` inside. §12.9
- **Async generator** — `async function*` that yields values over time; consumed with `for await…of`. §12.12
- **Async iteration** — Walking a sequence whose items arrive asynchronously, using `Symbol.asyncIterator` and `for await…of`. §12.12
- **`async` script** — `<script async>` downloads in parallel and runs as soon as it arrives, in no guaranteed order; fine for independent scripts such as analytics. §13.12
- **Attribute (HTML)** — Text written in markup (`value="init"`); read with `getAttribute`. Often the initial value only. §13.4
- **Auto-boxing** — Engine briefly wraps a primitive in its object wrapper so that `'abc'.toUpperCase()` works. §6.4
- **Automatic Semicolon Insertion (ASI)** — Parser's rule for adding missing semicolons; it is why a line break straight after `return` returns `undefined`. §1.6
- **`await`** — Pauses an async function until a promise settles; gives the value or throws the rejection reason. §12.9

B

- **Babel** — Transpiler that rewrites modern JavaScript (and JSX/TypeScript) into syntax older engines understand. §10.12
- **Backpressure** — Stream's signal that the consumer is slower than the producer; `write()` returns `false` until `'drain'`. §15.9
- **Backreference** — `\1` or `\k<name>` in a regex: match the same text a group matched earlier. §6.7
- **Backtracking** — Building a solution step by step and undoing the last choice when it leads to a dead end; used to list subsets, permutations and N-Queens boards. §17.2
- **Bare specifier** — Import name without a path, such as `'lodash'`; Node finds it in `node_modules`, a browser needs an import map. §10.7
- **Barrel file** — Module that only re-exports other modules (`export * from './a.mjs'`) to offer one import path. §10.2
- **Base case** — Condition that stops a recursive function from calling itself again. §4.10
- **Base64** — Way to write arbitrary bytes using only 64 safe text characters (A–Z, a–z, 0–9, `+`, `/`), so binary data can travel inside text such as JSON or URLs. §15.7
- **Baseline** — Label on MDN and web.dev saying a web feature works in all major current browsers ("newly" or "widely" available). §18.18
- **Basic Multilingual Plane (BMP)** — First 65,536 Unicode code points; characters outside it, like most emoji, need two UTF-16 code units (a surrogate pair). §6.5
- **BigInt** — Primitive for integers of any size, written `10n`; can't be mixed with `number` in arithmetic. §2.10
- **Big-O notation** — Way to describe how running time or memory grows with input size `n`, ignoring constants: O(1) constant, O(log n), O(n) linear, O(n log n), O(n²). §17.1
- **Binary search** — Finding a target in sorted data by repeatedly checking the middle and discarding half, taking O(log n) steps. §17.2

- **Binary search tree (BST)** — Binary tree where every left descendant is smaller and every right descendant larger than the node, so an in-order walk is sorted. §17.2
- **Binary tree** — Tree in which each node has at most two children, `left` and `right`. §17.2
- **`bind`** — Returns a new function with `this` (and optionally leading arguments) fixed permanently. §9.2
- **Binding** — Link between a name and the storage it refers to; `let` in a `for` loop creates a fresh binding each iteration. §4.7
- **Bitwise operators** — `& | ^ ~ << >> >>>`: operate on the 32-bit integer form of numbers. §2.8
- **Block** — Statements grouped in `{ }`, as in an `if` body or loop body; `let` and `const` are scoped to it. §1.6
- **Block scope** — Visibility limited to the nearest `{ }`; applies to `let`, `const` and `class`. §4.5
- **Blocking** — Code that keeps the single JavaScript thread busy (a long loop, synchronous file read), so nothing else, not even clicks or other requests, can run meanwhile. §12.1
- **Bound function** — New function returned by `bind`, with `this` (and optionally some arguments) fixed permanently. §9.2
- **Brand check** — Testing whether an object was really made by a class by checking for one of its private fields: `#secret in obj`. §9.7
- **Breadth-first search (BFS)** — Exploring a graph or grid level by level using a queue; finds the fewest-steps path when all steps cost the same. §17.2
- **Breakpoint** — Marker in DevTools or an editor that pauses execution at a line so you can inspect variables; can be conditional. §11.9
- **browserslist** — Shared config listing which browsers a project supports; Babel, autoprefixer and other tools read it to decide what to transform. §10.12
- **Bubbling** — Phase in which an event travels from the target back up through its ancestors. §13.6
- **Buffer** — Node's type for raw bytes; its length is in bytes, not characters. §15.7
- **Built-in objects** — Objects the language provides everywhere, such as `Object`, `Array`, `Math`, `JSON`, `Date`, `Map` and `Promise`. §5.9
- **Bun** — A JavaScript runtime built on JavaScriptCore with its own package manager, bundler and test runner. §15.12
- **Bundler** — Tool that follows imports and produces optimized files for the browser (Vite, esbuild, webpack, Rollup). §10.11
- **Busy-waiting** — Burning CPU in a loop until a condition changes, instead of waiting for an event or timer; it freezes the page in JavaScript. §3.7
- **Bytecode** — Compact, low-level instruction format an engine's interpreter runs; V8's Ignition turns your parsed code into it. §1.3

C

- **Cache API** — Browser store of request/response pairs, used mostly by service workers for offline support. §14.4
- **`call` / `apply`** — Invoke a function right now with a chosen `this`; `call` takes listed arguments, `apply` an array. §9.2
- **Call site** — Place in the code where a function is called; for regular functions it decides what `this` is. §9.1
- **Call stack** — Stack of function calls currently running; the newest frame is on top. §12.2
- **Callback** — Function passed to another function to be called later or for each item. §4.8
- **Callback hell** — Deeply nested callbacks where each step depends on the previous one; hard to read and to handle errors. §12.6
- **camelCase** — Naming style where the first word is lower-case and later words start with a capital (`totalPrice`); the JavaScript convention for variables and functions. §2.1
- **Cancelable event** — Event whose default action `preventDefault()` can stop; `event.cancelable` tells you. §13.8
- **Capture phase** — First event phase, travelling from `window` down to the target; listen with `{ capture: true }`. §13.6

- **Capturing group** — `( … )` in a regex: groups a sub-pattern and remembers its match as `$1`, `m[1]`. §6.7
- **Caret range (`^`)** — Npm version range allowing newer minor and patch releases of the same major (`^1.2.3` means at least 1.2.3, below 2.0.0). §10.10
- **`cause` (error)** — Option `new Error(msg, { cause })` that links an error to the one that caused it (ES2022). §11.4
- **Character class** — `[abc]`, `[a-z]` or shorthands like `\d`: matches one character from a set. §6.7
- **Child process** — Another OS program started from Node with `node:child_process` (`execFile`, `spawn`, `fork`). §15.11
- **Chunk** — One piece of data delivered by a stream (often a `Buffer` of up to 64 KB); also a separate file a bundler emits for code splitting. §15.9
- **CI (continuous integration)** — Server that automatically installs, lints, tests and builds every change pushed to a repository. §10.10
- **Circular dependency** — Two or more modules that import each other; ES modules allow it, but one side may see the other's bindings before they are initialised. §10.3
- **Class** — Syntax for a constructor plus prototype methods, with fields, `#private` members, statics and `extends`. §9.5
- **Class field** — Property declared in the class body (`count = 0`) and created on each instance (ES2022). §9.5
- **Classic script** — Normal `<script>` without `type="module"`: sloppy by default, top-level `var` becomes a global, and it blocks HTML parsing unless `defer` or `async` is set. §1.8
- **`classList`** — Element's list of CSS classes with `add`, `remove`, `toggle` and `contains`. §13.4
- **CLI** — Command-line interface: a program you run by typing commands in a terminal, such as `node`, `npm` or `tsc`. §10.9
- **Clickjacking** — Tricking a user into clicking something on your site hidden inside an invisible frame on another site; blocked with CSP `frame-ancestors`. §14.12
- **`closest`** — Returns the element itself or its nearest ancestor that matches a CSS selector, or `null`; the workhorse of event delegation. §13.2
- **Closure** — Function together with the variables of the scope where it was created, which it can still read and change. §4.7
- **Code point** — A Unicode character number (U+0000 to U+10FFFF); some need two UTF-16 code units. §6.5
- **Code splitting** — Breaking a bundle into chunks that load on demand, usually at each dynamic `import()`. §10.11
- **Code unit** — One 16-bit value of UTF-16; `length` and indexes count these. §6.5
- **Coercion** — Automatic type conversion done by operators such as `+`, `==` and `if`. §2.12
- **Combinator (promise)** — Function that turns several promises into one: `Promise.all`, `allSettled`, `race` and `any`. §12.8
- **Command injection** — Attack where user input passed to a shell is run as extra commands; avoided by `execFile`/`spawn` with an argument array. §15.11
- **CommonJS** — Node's original module system: `require()` and `module.exports`, loaded synchronously. §10.8
- **Comparator** — Function passed to `sort`: negative keeps `a` first, positive puts `b` first, 0 keeps their order. §7.6
- **Compile** — Translating source code into another form, such as machine code or bytecode, before it runs; JavaScript engines do this on the fly. §1.3
- **Composition** — Building behaviour by combining objects or functions (has-a) instead of inheriting (is-a). §9.9
- **Compound assignment** — Operator that combines an operation with assignment: `x += 1` means `x = x + 1`. §2.8
- **Computed property name** — Object key written as an expression in brackets: `{ [key]: value }`. §5.2
- **Concatenation** — Joining strings end to end, with `+`, `concat` or a template literal. §6.3
- **Concurrency limit** — Running many async jobs but only N at a time, so you don't flood a server or run out of file handles. §12.13

- **Conditional breakpoint** — Breakpoint that pauses only when an expression you give is truthy (e.g. `i === 500`). §11.9
- **Conditional statement** — Code that chooses what to run based on a condition: `if`, `else if`, `else` and `switch`. §3.1
- **Conditional types (TS)** — Types that pick one of two results based on a test, written `T extends U ? X : Y`. §16.12
- **`const`** — Declares a block-scoped binding that can't be reassigned (the value may still be mutated). §2.1
- **Constraint Validation API** — Built-in form validation from HTML attributes (`required`, `min`, `pattern`), read via `input.validity` and customised with `setCustomValidity`. §13.9
- **Constructor** — Function that sets up a new object: the `constructor` method of a class, or a function called with `new`. §9.5
- **Constructor function** — Regular function called with `new` to create objects; the pre-class way to make "types". §9.4
- **Content Security Policy (CSP)** — Response header that tells the browser which sources of scripts, styles and frames are allowed. §14.12
- **Content-Type / MIME type** — Header naming the format of a body, such as `application/json` or `text/html`; MIME types are those standard format names. §14.1
- **Cookie** — Small string the server sets and the browser sends back automatically with matching requests. §14.4
- **core-js** — Widely used library of polyfills for modern JavaScript built-ins. §10.12
- **CORS** — Cross-Origin Resource Sharing: headers by which a server allows pages from other origins to read its responses. §14.2
- **Coverage** — Share of code lines or branches your tests executed; a hint, not a goal. §16.6
- **CPU-bound / I/O-bound** — CPU-bound work is limited by computation (hashing, image resizing); I/O-bound work mostly waits for disks or networks. Node is great at the second, needs workers for the first. §15.2
- **CSRF** — Cross-site request forgery: another site makes the victim's browser send an authenticated request. §14.11
- **CSRF token** — Secret, unpredictable value the server puts in a form and checks on submit, proving the request came from its own page. §14.11
- **CSS** — Cascading Style Sheets: the language that controls how HTML looks (colours, layout, fonts). §1.1
- **`currentTarget`** — Element whose listener is running now, as opposed to `target`, where the event started. §13.5
- **Currying** — Turning `f(a, b, c)` into `f(a)(b)(c)` so arguments can be supplied one at a time. §4.8
- **CustomEvent** — Event you create with your own `detail` data and fire with `dispatchEvent`. §13.5

D

- **`dataset`** — Access to `data-*` attributes as camelCase string properties (`data-user-id` → `dataset.userId`). §13.4
- **`Date`** — Built-in object holding milliseconds since 1970-01-01 UTC, with local and UTC getters. §5.11
- **Daylight-saving time (DST)** — Seasonal clock changes that make some local days 23 or 25 hours long; a reason to store and compute dates in UTC. §5.11
- **Debounce** — Run a function only after calls have stopped for a given time (e.g. search-as-you-type). §17.3
- **`debugger`** — Statement that pauses execution when developer tools are attached. §11.9
- **Declaration** — Code that introduces a name: `let x`, `const y = 1`, `function f() {}` or `class C {}`. §2.1
- **Declarative code** — Code that says what result you want (`items.filter(isActive)`) rather than spelling out each step in a loop (imperative code). §7.5
- **Deep copy** — Copy in which nested objects are copied too, e.g. `structuredClone(obj)`. §5.4
- **Default action** — What the browser does on its own after an event, such as following a link or submitting a form; stopped with `preventDefault()`. §13.8

- **Default binding** — `this` rule for a plain call `f()`: `undefined` in strict code, the global object in sloppy scripts. §9.1
- **Default export** — Single unnamed export of a module, imported under any name: `import x from './m.mjs'`. §10.2
- **Default parameter** — Value used when an argument is missing or `undefined`: `function f(n = 1)`. §4.4
- **Defence in depth** — Using several independent protections (escaping, CSP, `HttpOnly` cookies) so one failure doesn't expose you. §14.10
- `defer` — Script attribute: download in parallel, run after the HTML is parsed, in document order. §13.12
- `delete` — Operator that removes an own property from an object. §5.2
- **Deno** — A V8-based runtime with deny-by-default permissions, built-in TypeScript and tooling. §15.12
- **Deoptimization** — When the optimizing compiler's assumptions about your code (e.g. "this is always a number") turn out wrong, so the engine throws away the fast machine code and goes back to slower code. §1.3
- **Dependency** — Package your project uses, listed in `package.json`; transitive dependencies are the packages your dependencies use in turn. §10.9
- **Dependency injection** — Passing collaborators (clock, database, HTTP client) in as arguments so tests can swap them for fakes. §16.5
- **Depth-first search (DFS)** — Exploring a graph or tree by going as deep as possible down one path before backing up, using recursion or a stack. §17.2
- **Destructuring** — Syntax that unpacks properties or elements into variables, with renames and defaults. §5.5
- `devDependencies` — Packages needed only to build and test (linters, bundlers), not at runtime. §10.9
- **DevTools** — Developer tools built into browsers (F12): console, element inspector, debugger, network and performance panels. §1.5
- **Dijkstra's algorithm** — Finds the cheapest path from one node to all others in a graph with non-negative edge weights, using a priority queue. §17.2
- **Directive** — String statement that changes how code is treated, such as `"use strict"` at the top of a script or function. §1.8
- **Discriminated union** — A TypeScript union whose members share a literal "kind" property used to narrow between them. §16.11
- `dispatchEvent` — Fires an event object at a node; its listeners run synchronously. §13.5
- **Document** — Root object of the DOM for one page (`document`); everything you select or create starts from it. §13.1
- **DocumentFragment** — Lightweight container for building many nodes off-page before inserting them at once. §13.3
- **DOM** — Document Object Model: the live tree of objects the browser builds from HTML and exposes to scripts. §13.1
- **DOM-based XSS** — XSS that happens entirely in the browser when script copies attacker-controlled data (often from the URL) into a dangerous sink like `innerHTML`. §14.10
- `DOMContentLoaded` — Event fired when the HTML is parsed and deferred scripts have run; images may still be loading. §13.12
- **DOMPurify** — Widely used library that sanitizes untrusted HTML before it goes into `innerHTML`. §14.10
- **dotAll flag** — Regex `s` flag: lets `.` match newline characters too. §6.7
- **Dual package hazard** — When both the CommonJS and ESM builds of one package get loaded, so the app ends up with two separate copies of its state. §10.8
- **Duck typing** — Checking whether a value has the methods you need rather than which class made it. §9.7
- **Dummy (test double)** — Placeholder argument a test passes only to fill a parameter; it is never actually used. §16.5
- **Duplex stream** — Stream that is both readable and writable, like a network socket. §15.9
- **Dutch national flag partition** — Sorting an array of three kinds of value (e.g. 0, 1, 2) in one pass with three pointers. §17.2

- **Dynamic import** — `import(specifier)`: loads a module at runtime and returns a promise for its namespace. §10.4
- **Dynamic programming (DP)** — Solving a problem by combining answers to overlapping smaller subproblems, storing each so it is computed once (memo top-down or table bottom-up). §17.2
- **Dynamic typing** — Types belong to values, not variables; a variable may hold a number and later a string. §1.1

E

- **Ecma International** — Standards organisation that publishes ECMAScript (as ECMA-262); its TC39 committee designs the language. §1.2
- **ECMAScript** — Official language standard (ECMA-262) that JavaScript engines implement; a new edition every June. §1.2
- **Electron** — Framework for desktop apps (VS Code, Slack) built from Chromium plus Node.js, written in HTML, CSS and JavaScript. §1.4
- **Element** — A DOM node that represents an HTML tag, such as `<li>` or `<button>`; it has attributes, children and methods like `querySelector`. §13.1
- **Encapsulation** — Hiding internal state so outside code can only use a public interface; achieved with closures, `#private` fields or module scope. §4.7
- **Encoding** — Rule that turns text into bytes and back; UTF-8 is the usual one (`'é'` is 2 bytes in UTF-8). §15.7
- **Endianness** — Byte order of multi-byte numbers in memory: little-endian stores the least significant byte first; `DataView` lets you choose. §7.9
- **End-to-end (E2E) test** — Test that drives the whole application through its real interface, often a browser. §16.1
- **Engine** — Program that parses, compiles and runs JavaScript: V8, SpiderMonkey, JavaScriptCore. §1.3
- `ENOENT` — Node's error code for "no such file or directory"; check `err.code` rather than the message. §15.4
- **Entry point** — First module or file that starts an app (or that a bundler starts from) and from which all imports are followed. §10.3
- **Enum (TS)** — A TypeScript construct that creates a named set of constants as a real runtime object; not erasable, so plain unions or `as const` objects are often preferred. §16.13
- **Enumerable** — Property attribute; enumerable properties appear in `for…in`, `Object.keys` and spread. §5.6
- **Environment variable** — Named string setting given to a process by the OS or shell (e.g. `PORT=3000`), read in Node with `process.env`. §15.6
- **Erasable syntax (TS)** — TypeScript syntax that disappears when types are deleted (annotations, interfaces); `enum` and parameter properties are not erasable because they generate code. §16.9
- **Error-first callback** — Node's callback convention `(err, result) => …`, where `err` is checked first. §12.6
- `Error.isError` — Reliable check that a value is a genuine error object, even across realms (ES2026). §9.7
- **ES module (ESM)** — Standard module system: static `import` / `export`, strict mode, live bindings. §10.2
- **esbuild** — Very fast bundler and transpiler written in Go. §10.11
- **Escape sequence** — Backslash code inside a string literal for a character that is hard to type: `\n` newline, `\t` tab, `\'`, `\u{1F600}`. §6.2
- **ESLint** — Configurable linter that reports likely bugs and style problems in JavaScript and TypeScript. §10.13
- **Event** — Object describing something that happened (click, key press, load) delivered to listeners. §13.5
- **Event delegation** — One listener on a container handles events from many children using `event.target.closest()`. §13.7
- **Event listener** — Function registered for an event type on a target; the browser calls it with an event object when that event happens. §13.5

- **Event loop** — Host scheduler that runs one task, then all microtasks, then possibly a render, repeatedly. §12.2
- **Event object** — Object passed to a listener, describing what happened: `type`, `target`, `currentTarget`, key or mouse details, and methods like `preventDefault`. §13.5
- **EventEmitter** — Node's publish/subscribe class: `on`, `once`, `off` and synchronous `emit`. §15.8
- **Exception** — Error that has been thrown and interrupts normal flow until a `catch` handles it; uncaught, it ends the script or process. §11.1
- **Excess property check (TS)** — TypeScript's error when an object literal has a property its target type doesn't declare, usually a typo. §16.10
- **Execution context** — Engine's record for a running piece of code (its variables, scope chain and `this`); each call pushes one onto the call stack. §4.1
- **Executor** — Function passed to `new Promise((resolve, reject) => …)`; it runs immediately and synchronously. §12.7
- **Exit code** — Number a process returns to the OS when it ends: 0 for success, anything else for failure; set with `process.exitCode`. §15.6
- **Explicit binding** — Setting `this` yourself with `call`, `apply` or `bind`. §9.1
- **Explicit resource management** — `using` / `await using` declarations plus `Symbol.dispose`, which clean up a resource automatically when its block ends (ES2027). §11.6
- **Exponential backoff** — Retrying a failed request with waits that double each time (100 ms, 200 ms, 400 ms …), usually plus random jitter. §12.13
- **Exponentiation operator** — `**`: raises to a power (`2 ** 10` is 1024). §2.5
- **`export`** — Makes a binding available to other modules, by name or as the default. §10.2
- **Express** — Most widely used Node.js web framework, adding routing and middleware on top of `node:http`. §15.10
- **Expression** — Code that produces a value (`2 + 3`, `f(x)`, `a ? b : c`). §1.6
- **`extends`** — Makes one class inherit from another (or from a built-in such as `Error`), linking their prototype chains. §9.6

F

- **Factory function** — Plain function that builds and returns a new object, an alternative to classes and `new`. §9.9
- **Fake (test double)** — Working but simplified stand-in, such as an in-memory database used in place of a real one. §16.5
- **Fake timers** — Test tool that replaces `setTimeout`/`Date` so you can move time forward instantly. §16.5
- **Fall-through** — In a `switch`, running on into the next `case` because a `break` (or `return`) is missing. §3.4
- **Falsy** — Value that becomes `false` in a boolean context: `false`, `0`, `-0`, `0n`, `''`, `null`, `undefined`, `NaN`. §2.11
- **Fast and slow pointers** — Two pointers moving through a list at different speeds; they meet if there is a cycle, and the slow one finds the middle. §17.2
- **Fastify / Hono / Koa** — Alternative Node.js web frameworks to Express: Fastify focuses on speed, Hono is tiny and runs on many runtimes, Koa is a minimal async-first design. §15.10
- **Feature detection** — Checking whether an API exists before using it (`if ('Temporal' in globalThis)`), instead of guessing from the browser name. §1.4
- **`fetch`** — Promise-based HTTP API in browsers and Node; resolves even for 4xx/5xx responses. §14.1
- **FinalizationRegistry** — Lets you ask for a callback some time after an object has been garbage-collected; timing is never guaranteed. §8.3
- **`finally`** — Block that runs after `try`/`catch` whatever happened, for cleanup. §11.2
- **First-class function** — Functions are values: stored in variables, passed as arguments and returned from other functions. §4.1
- **Fisher–Yates shuffle** — Correct way to shuffle an array fairly: walk from the end, swapping each item with a random earlier one (or itself). §7.6
- **`flat` / `flatMap`** — Flatten nested arrays by a depth / map then flatten one level. §7.5

- **Floating point** — Way computers store fractional numbers in binary with limited precision, which is why `0.1 + 0.2` isn't exactly `0.3`. §2.10
- **`for await…of`** — Loops over an async iterable, awaiting each value. §12.12
- **Forced synchronous layout** — When reading a layout value (like `offsetHeight`) after a style change makes the browser recalculate layout immediately; repeated, it causes layout thrashing. §13.11
- **`for…in`** — Loops over enumerable property names (strings), including inherited ones. §3.9
- **Formatter** — Tool that rewrites code to a consistent style (spacing, quotes, line breaks) without changing behaviour; Prettier is the common one. §10.13
- **`for…of`** — Loops over the values of any iterable (arrays, strings, Maps, Sets, generators). §3.10
- **FormData** — Object holding a form's field names and values; can be sent directly with `fetch`. §13.9
- **Framework** — Larger toolkit that shapes how a whole app is structured and calls your code (React, Angular, Express); a library is something your code calls. Ch 1
- **Fulfilled** — State of a promise that finished successfully with a value. §12.7
- **Function** — Reusable, named (or anonymous) block of code that takes inputs (parameters), runs, and may return a value; in JavaScript it is also an object. §4.1
- **Function composition** — Combining functions so the output of one feeds the next: `compose(f, g)(x)` is `f(g(x))`; `pipe` runs left to right. §4.8
- **Function declaration** — `function name() {}` as a statement; hoisted with its body. §4.2
- **Function expression** — Function used as a value (`const f = function () {}`); follows its variable's hoisting. §4.2
- **Function scope** — Scope created by each function call; `var` declarations and parameters live here. §4.5

G

- **Garbage collection** — Automatic freeing of memory for objects no longer reachable from roots such as globals and the stack. §17.4.7
- **Generational garbage collection** — Collecting young objects often and old ones rarely, because most objects die young; V8 works this way. §17.4
- **Generator** — `function*` that can pause at `yield`; calling it returns an iterator. §8.5
- **Generics (TS)** — Type parameters such as `<T>` that let one function or type work for many types safely. §16.12
- **`getComputedStyle`** — Returns the final CSS values the browser applied to an element after all stylesheets and inline styles. §13.4
- **Getter** — `get` method that runs when a property is read. §5.7
- **Global object** — Object that holds the host's global variables (`window` in browsers, `global` in Node), always reachable as `globalThis`. §1.4
- **Global scope** — Outermost scope, visible to all code; names declared there can clash across scripts. §4.5
- **`globalThis`** — Standard name for the global object in every environment (ES2020). §1.4
- **Graceful shutdown** — Stopping a server cleanly on a signal like `SIGTERM`: stop accepting new work, finish what's in flight, close connections, then exit. §15.10
- **Gradual typing** — Adding static types to a codebase a little at a time; typed and untyped code coexist (TypeScript, JSDoc checking). §16.8
- **Graph** — Set of nodes connected by edges, possibly directed or weighted; maps, dependency lists and social networks are graphs. §17.2
- **Grapheme cluster** — What a reader sees as one character, possibly several code points (flags, family emoji). §6.5
- **Greedy algorithm** — Making the best-looking local choice at each step, which is correct only when you can prove those choices never hurt the final answer. §17.2
- **Greedy quantifier** — Regex repetition that matches as much as possible; add `?` to make it lazy. §6.7
- **`groupBy`** — `Object.groupBy` / `Map.groupBy` put items into buckets by a key function (ES2024). §8.7
- **Guard clause** — Early `return` (or `throw`) at the top of a function for invalid or special cases, so the main logic needn't be nested. §3.2

H

- **Hash map** — Structure that maps keys to values with average O(1) lookup by hashing the key; in JavaScript use `Map` or a plain object. §17.1
- **Hashbang** — `#!/usr/bin/env node` first line that tells Unix shells which program runs the file; allowed by JavaScript since ES2023. §18.18
- `hasOwn` — `Object.hasOwn(obj, key)`: is this an own (not inherited) property? (ES2022) §5.2
- **Heap (data structure)** — Tree kept in an array so the smallest (min-heap) or largest (max-heap) item is always on top; push and pop are O(log n). §17.2
- **Heap (memory)** — Area of memory where objects live until the garbage collector frees them. §17.4
- **Higher-order function** — Function that takes or returns another function. §4.8
- **History API** — `pushState`, `replaceState` and `popstate` for changing the URL without reloading the page. §14.7
- **HMR** — Hot Module Replacement: a dev server swaps changed modules into the running page without a full reload. §10.11
- **Hoisting** — Declarations are registered before code runs: functions fully, `var` as `undefined`, `let` / `const` / `class` in the TDZ. §4.6
- **Hooks (testing)** — `before`, `beforeEach`, `afterEach` and `after`: functions a test runner calls around tests for setup and cleanup. §16.3
- **Host** — Environment around the engine that supplies extra APIs: a browser, Node.js, Deno, Bun. §1.4
- **HSTS** — HTTP Strict Transport Security: a response header telling the browser to use only HTTPS for this site from now on. §14.12
- **HTML** — HyperText Markup Language: the tag-based language that describes a web page's structure and content. §1.1
- **HTML Sanitizer API** — Newer browser API that cleans untrusted HTML of scripts and dangerous attributes, the built-in counterpart to DOMPurify. §14.10
- **HTTP** — HyperText Transfer Protocol: the request/response rules browsers and servers use to exchange data; HTTPS is HTTP encrypted with TLS. §14.1
- **HTTP header** — `Name: value` line of metadata on a request or response, such as `Content-Type`, `Authorization` or `Set-Cookie`. §14.1
- **HTTP method** — Verb of a request: `GET` reads, `POST` creates, `PUT` / `PATCH` update, `DELETE` removes, `OPTIONS` asks what is allowed. §14.1
- **HTTP status code** — Three-digit number in a response: 2xx success, 3xx redirect, 4xx client error (404 not found), 5xx server error. §14.1
- `HttpOnly` — Cookie attribute that hides the cookie from JavaScript, protecting sessions from XSS theft. §14.4
- **HTTPS** — HTTP over an encrypted TLS connection, so nobody in between can read or change the traffic; required for `Secure` cookies. §14.12

I

- **Idempotent** — Operation that has the same effect whether it runs once or several times (e.g. "set status to paid"); only these are safe to retry blindly. §12.13
- **Identifier** — Name you give to a variable, function, class or parameter; letters, digits, `_` and `$`, not starting with a digit. §1.6
- **IEEE 754** — The 64-bit floating-point format behind every JavaScript `number`. §2.10
- **Ignition / Sparkplug / Maglev / TurboFan** — V8's tiers: Ignition interprets bytecode, Sparkplug and Maglev compile it quickly to machine code, TurboFan produces the most optimized code for hot functions. §1.3
- **IIFE** — Immediately Invoked Function Expression `(() => { … })()`: an old way to get a private scope. §4.9
- **Immutability** — Not changing data in place; producing new values instead (important in React and Redux). §7.4
- **Imperative code** — Code that spells out each step (counters, loops, pushes) to reach a result; contrast with declarative code. §7.5
- **Implicit binding** — `this` rule for `obj.method()`: `this` is the object before the dot. §9.1

- **Implicit return** — Arrow function with an expression body returns that expression without `return`. §4.3
- `import` — Brings exported bindings from another module into this one. §10.2
- **Import attributes** — `with { type: 'json' }` on an import, telling the host how to load the module (ES2025). §10.6
- **Import map** — Browser `<script type="importmap">` that maps bare specifiers like `'lodash'` to URLs. §10.7
- `import.meta` — Per-module metadata such as `url`, `dirname`, `filename` and `resolve()`. §10.6
- **Impure function** — Function that has side effects or depends on outside state, so the same input may give a different result. §4.11
- **In-degree** — Number of edges pointing into a node of a directed graph; Kahn's topological sort starts from nodes with in-degree 0. §17.2
- **IndexedDB** — Browser's asynchronous, transactional database for large structured data. §14.4
- **Infinite loop** — Loop whose condition never becomes false, so it runs forever and freezes the page or process. §3.7
- `Infinity` — Special number bigger than any other, produced by `1 / 0` or overflow; `-Infinity` is its negative. §2.10
- **Inheritance** — One class extending another (`extends`) and reusing its behaviour through the prototype chain. §9.6
- **Inline style** — CSS set directly on one element via its `style` attribute or `el.style.x`; overrides most stylesheet rules. §13.4
- `innerHTML` — Gets or sets an element's children as an HTML string; setting it parses markup (XSS risk). §13.3
- **Inspector** — Debugging interface of an engine; `node --inspect` opens it so Chrome DevTools can attach to Node. §11.9
- `instanceof` — Tests whether `C.prototype` appears in an object's prototype chain. §9.7
- **Integer** — Whole number with no fractional part; JavaScript numbers hold integers exactly only up to $2^{53} - 1$. §2.10
- **Integration test** — Test that runs several real components together, such as a handler and a database. §16.1
- **Interface (TS)** — Named object type in TypeScript; can be extended and merged. §16.10
- **Interpolation** — Inserting values into a string, as with `${expr}` in a template literal. §6.3
- **Interpreter** — Program that runs code instruction by instruction without first compiling all of it to machine code; engines start this way and compile hot code later. §1.3
- **Intersection type (TS)** — `A & B`: a value that has all the properties of both types. §16.10
- **IntersectionObserver** — Reports when an element enters or leaves the viewport; used for lazy loading and infinite scroll. §14.9
- `Intl` — Internationalization namespace: `DateTimeFormat`, `NumberFormat`, `Collator`, `Segmenter` and more. §5.11
- `Intl.NumberFormat` — Formats numbers for people in a given locale, including currency and percentages (`€1,234.50`). §2.10
- **Invalid Date** — `Date` whose time value is `NaN` because parsing failed; `isNaN(d)` or `Number.isNaN(d.getTime())` detects it. §5.11
- **Inversion of control** — Handing your callback to someone else's code and trusting it to call it once, at the right time; promises reduce that trust problem. §12.6
- **ISO 8601** — International date-time text format `YYYY-MM-DDTHH:mm:ss.sssZ`; the safe format to send and parse. §5.11
- **Iterable** — Object with a `[Symbol.iterator]()` method, usable in `for…of` and spread. §8.4
- **Iterator** — Object with a `next()` method returning `{ value, done }`. §8.4
- **Iterator helpers** — Lazy `map`, `filter`, `take`, `drop`, `flatMap` and consuming `toArray`, `reduce` on iterators (ES2025). §8.6

J

- **JavaScriptCore** — Apple's JavaScript engine, used by Safari and Bun. §1.3

- **Jest** — Popular all-in-one JavaScript test runner with `expect` assertions, mocking and snapshots. §16.7
- **JIT compilation** — Just-in-time compilation: the engine compiles frequently run code to machine code while the program runs. §1.3
- **Jitter** — Random extra delay added to retries so many clients don't all retry at the same instant. §12.13
- **JSDoc** — Structured comments (`/** @param {number} x */`) that document code and can drive TypeScript checking. §16.15
- **jsdom** — A JavaScript implementation of the DOM that runs in Node, used to test browser code without a browser. §13.3
- **JSON** — Text data format with double-quoted keys and strings, numbers, booleans, `null`, arrays and objects. §5.12
- **JSON module** — `.json` file imported directly with `with { type: 'json' }` (ES2025). §10.6
- `JSON.parse` — Turns JSON text into values; strict syntax; an optional reviver transforms values. §5.12
- `JSON.stringify` — Turns values into JSON text; drops functions and `undefined` in objects, throws on BigInt and cycles. §5.12
- **JSX** — HTML-like syntax inside JavaScript used by React (`<Button />`); a build tool compiles it into function calls. §10.11
- **JWT** — JSON Web Token: a signed, Base64-encoded JSON token often used to prove who a user is; its timestamps are in seconds, not milliseconds. §5.11

K

- **Kahn's algorithm** — Topological sort by repeatedly taking nodes with no remaining incoming edges (in-degree 0) from a queue. §17.2

L

- **Label** — Name before a loop (`outer:`) that `break outer` / `continue outer` can target. §3.11
- `lastIndex` — Position where a `g` or `y` regex will start its next search; it makes repeated `test` calls stateful. §6.8
- **Last-request-wins** — Ignoring or cancelling responses from older requests so only the latest one updates the UI (e.g. in search-as-you-type). §12.13
- **Layout thrashing** — Alternating DOM writes and layout reads so the browser recalculates layout again and again. §13.11
- **Lazy evaluation** — Computing values only when they're requested, as generators and iterator helpers do. §8.5
- **Lazy quantifier** — Regex repetition with a trailing `?` (`.+?`) that matches as little as possible. §6.7
- `let` — Declares a block-scoped variable that can be reassigned. §2.1
- **Lexical scope** — Scope decided by where code is written, not where it's called from. §4.5
- **Lexical `this`** — Arrow function's `this` is the `this` of the surrounding code. §4.3
- **Library** — Reusable code you call from your own program, such as Lodash or date-fns; contrast with a framework, which calls your code. §1.1
- **libuv** — The C library that gives Node its event loop, async I/O and thread pool. §15.1
- **Linked list** — Chain of nodes where each holds a value and a reference (`next`) to the following node; O(1) insert at a known spot, O(n) lookup. §17.2
- **Linter** — Tool that analyses source code for bugs and style problems without running it (ESLint). §10.13
- **Literal** — Value written directly in code: `42`, `'hi'`, `true`, `[1, 2]`, `{ a: 1 }`, `/re/`. §1.6
- **Literal type (TS)** — Type that allows only one exact value, such as `'admin'` or `42`; unions of them make safe string options. §16.10
- **Live binding** — An ES module import stays connected to the exporter's variable and sees updates. §10.3

- **Live collection** — A DOM collection (`getElementsByTagName`) that updates automatically when the document changes. §13.2
- **`load` event** — Fires when the page and all its images, stylesheets and frames have finished loading, after `DOMContentLoaded`. §13.12
- **Local time** — Date and time in the user's own time zone, as `getHours()` and `toString()` show it; contrast with UTC. §5.11
- **`localeCompare`** — Compares two strings by a language's alphabet rules, returning a negative, zero or positive number; use it to sort text. §6.6
- **Localhost (`127.0.0.1`)** — Address that always means "this same computer"; servers bound to it can't be reached from other machines. §15.10
- **`localStorage`** — Per-origin string key-value storage that survives reloads; synchronous, about 5 MB. §14.3
- **Lockfile** — `package-lock.json`: records the exact installed version of every dependency. §10.10
- **Logical assignment** — `a ||= b`, `a &&= b`, `a ??= b`: assign only when the condition holds (ES2021). §2.7
- **Logpoint** — Breakpoint that logs a message instead of pausing, so you can add logging without editing code. §11.9
- **Lone surrogate** — Half of a surrogate pair with no partner, which makes a string not well-formed; `isWellFormed` detects it. §6.5
- **Lookahead / lookbehind** — Regex assertions `(?=…)`, `(?!…)`, `(?<=…)`, `(?<!…)` that check surrounding text without consuming it. §6.7
- **LRU cache** — Fixed-size cache that throws out the Least Recently Used entry when full; a `Map` (which keeps insertion order) can implement it. §17.3
- **LTS** — Long-Term Support: a Node.js release line (the even-numbered ones) that gets fixes for about 30 months; use one in production. §15.1

M

- **Macrotask** — See **Task**. §12.2
- **Main thread** — Single thread that runs your JavaScript, and in a browser also handles rendering and input; keep it free of long work. §12.1
- **Major / minor / patch** — Three parts of a semantic version `MAJOR.MINOR.PATCH`: breaking changes, new features that don't break anything, bug fixes. §10.10
- **`Map`** — Key-value collection with keys of any type, insertion order and a `size`. §8.1
- **Mapped types (TS)** — Types built by transforming every property of another type, e.g. `{ [K in keyof T]?: T[K] }` (how `Partial` works). §16.12
- **Mark-and-sweep** — Garbage collection by marking everything reachable from the roots, then freeing everything unmarked. §17.4
- **`Math`** — Namespace object with maths constants and functions (`round`, `max`, `random`). §5.10
- **MDN Web Docs** — Mozilla's free reference for JavaScript and web APIs, with browser-compatibility tables. §18.19
- **Memoization** — Caching a function's results by its arguments so repeated calls are instant. §4.7
- **Memory leak** — Memory that stays reachable by mistake (forgotten listeners, timers, growing caches, detached DOM nodes). §17.4.7
- **Merge sort** — Sorting by splitting the array in halves, sorting each recursively, and merging them; O(n log n) and stable. §17.2
- **Message passing** — Communicating between threads by sending copies of data (`postMessage`) instead of sharing variables. §14.8
- **Method** — Function stored as an object property and called through the object (`user.greet()`), where `this` is that object. §5.3
- **Method borrowing** — Calling one object's method on another object with `call`/`apply`, e.g. `Array.prototype.slice.call(arguments)`. §9.2
- **Method shorthand** — Writing `greet() { … }` inside an object literal instead of `greet: function () { … }`. §5.3

- **Microtask** — Job that runs right after the current task finishes: promise reactions, code after `await`, `queueMicrotask`. §12.2
- **Microtask queue** — Queue for promise callbacks, code after `await` and `queueMicrotask`; it is emptied completely after every task. §12.2
- **Microtask starvation** — When microtasks keep scheduling more microtasks, so the event loop never gets to timers, input or rendering. §12.2
- **Middleware** — Function in a web framework's request pipeline that can inspect or change the request and response, or pass control to the next one (logging, auth, parsing). §15.10
- **Minification** — Removing whitespace and shortening names so bundles download faster. §10.11
- **Mixin** — Reusable set of methods added to unrelated classes, by copying or by a subclass factory. §9.8
- **Mock** — Test double that records calls so the test can verify how it was used. §16.5
- **Module** — File with its own scope that exports and imports bindings. §10.1
- **Module graph** — All the modules an app loads, linked by their imports, starting from the entry point. §10.3
- **Module pattern** — An IIFE that returns an object of public functions while keeping private state in a closure. §4.9
- **Module scope** — Private top-level scope of an ES module; its names are invisible to other files unless exported. §4.5
- **Module specifier** — String after `from` in an import: relative (`'./a.js'`), absolute URL, or bare (`'lodash'`). §10.2
- **Monotonic stack** — Stack kept in increasing or decreasing order, used for "next greater element" style problems in O(n). §17.2
- **Multi-paradigm** — Language that supports several styles of programming; JavaScript allows procedural, object-oriented and functional code. §1.1
- **Mutating method** — Array method that changes the original array (`push`, `splice`, `sort`, `reverse`). §7.4
- **Mutation** — Changing an existing object or array in place (`o.n = 2`, `arr.push(x)`); everyone holding a reference sees the change. §2.4
- **MutationObserver** — Calls back (as a microtask) with a batch of changes when the observed DOM changes. §13.10

N

- **Named capture group** — `(?<name>…)` in a regex; its match appears as `match.groups.name`. §6.7
- **Named export** — Export referenced by its name: `export const x`, imported with `import { x }`. §10.2
- **Named function expression** — Function expression with its own name (`const f = function fact(n) {…}`); the name is visible inside for recursion and in stack traces. §4.2
- **Namespace import** — `import * as ns from './m.mjs'`: one object holding all of a module's exports. §10.2
- `NaN` — "Not a Number": a `number` value from failed maths; not equal to anything, including itself. §2.10
- **Narrowing (TS)** — Refining a broad type to a specific one via checks like `typeof`, `in` or a discriminant. §16.11
- **Navigation API** — Newer browser API (`navigation`) for single-page-app routing that intercepts all navigations, a cleaner successor to the History API. §14.7
- **NDJSON** — Newline-delimited JSON: one JSON value per line, handy for logs and streaming because each line parses on its own. §5.12
- `never` **(TS)** — Type with no values, used for code that can't be reached; assigning to it makes `switch` checks exhaustive. §16.11
- `new` — Creates an object linked to `Fn.prototype`, runs the constructor with it as `this` and returns it. §9.4
- `new` **binding** — `this` rule for `new Fn()`: `this` is the brand-new object; it has the highest priority. §9.1
- `new.target` — Inside a function, tells whether it was called with `new` (and which constructor). §9.4

- **Node (DOM)** — Any item in the DOM tree: elements, text nodes, comments and the document itself. §13.1
- **`node:` prefix** — Writing built-in Node modules as `node:fs` or `node:path`, which makes it clear they aren't npm packages. §15.3
- **`node_modules`** — Folder where npm installs a project's packages; never edit or commit it. §10.9
- **`node:test`** — Node's built-in test runner with suites, hooks, mocks, snapshots and coverage. §16.3
- **Node.js** — A JavaScript runtime built from V8 and libuv, for servers, CLIs and tools. §15.1
- **NodeList** — List of DOM nodes; `querySelectorAll` returns a static one that has `forEach`. §13.2
- **Non-blocking I/O** — Starting a disk or network operation and continuing with other work, getting a callback or promise when it finishes. §12.1
- **Non-capturing group** — `(?:…)`: groups part of a regex without remembering the match. §6.7
- **Nonce (CSP)** — Random value generated for each response and placed in both the CSP header and allowed `<script>` tags, so only those inline scripts run. §14.12
- **Normalization (Unicode)** — Converting text to a standard form (NFC) so visually identical strings, like a precomposed é and e plus an accent, compare equal. §6.5
- **npm** — Node's package manager and public registry. §10.9
- **`npm ci`** — Clean, exact install from the lockfile; the right command for CI and deploys. §10.10
- **npm scripts** — Commands named under `"scripts"` in `package.json` and run with `npm run name` or `node --run name`. §15.11
- **`npx`** — Runs a command from an npm package, using the local install or downloading it temporarily. §10.9
- **`null`** — Primitive meaning "deliberately empty"; `typeof null` is `'object'`. §2.3
- **Nullish** — Two "no value" values: `null` and `undefined`. §2.7
- **Nullish coalescing** — `a ?? b`: `b` only when `a` is `null` or `undefined` (ES2020). §2.7
- **Null-prototype object** — Object made with `Object.create(null)` that inherits nothing, so keys like `toString` or `__proto__` are safe to use as data. §8.7
- **`Number.EPSILON`** — Gap between 1 and the next double, about 2.2e-16; a tolerance for float comparisons. §2.10
- **`Number.MAX_SAFE_INTEGER`** — $2^{53} - 1$, the largest integer that numbers represent exactly. §2.10
- **Numeric separator** — Underscores inside number literals for readability: `1_000_000` (ES2021). §2.10

O

- **O(1), O(n), O(log n)** — Big-O labels for constant time, time growing in proportion to input size, and time growing with the logarithm of input size. §17.1
- **Object** — Collection of key–value properties; arrays, functions, dates and maps are all objects, and they are passed around by reference. §2.3
- **Object literal** — Creating an object directly with braces: `{ name: 'Ada', age: 36 }`. §5.1
- **`Object.create`** — Makes a new object with a chosen prototype, or none: `Object.create(null)`. §9.3
- **`Object.entries`** — Returns an object's own enumerable `[key, value]` pairs. §5.6
- **`Object.freeze`** — Makes an object's own properties read-only and blocks new ones; shallow. §5.6
- **`Object.is`** — Like `===` except `Object.is(NaN, NaN)` is true and `Object.is(0, -0)` is false. §2.6
- **Observer pattern** — Design where an object keeps a list of listeners and notifies them when something happens; `EventEmitter` and DOM events follow it (also called publish/subscribe). §15.8
- **Off-by-one error** — Loop or index that runs one step too many or too few, often `<=` used where `<` was meant. §3.6
- **OLOO** — "Objects linked to other objects": a style that skips classes and links plain objects with `Object.create`. §9.9
- **Operand** — Value an operator works on; in `a + b`, `a` and `b` are the operands. §2.5
- **Operator** — Symbol or keyword that computes a value from operands, such as `+`, `===`, `&&`, `typeof` or `?:`. §2.5
- **Optional catch binding** — `try { } catch { }` without naming the error (ES2019). §11.2
- **Optional chaining** — `a?.b`, `a?.[k]`, `a?.()`: stop and return `undefined` if the left side is `null` / `undefined`. §5.8

- **Options object** — Passing one object of named settings instead of many positional arguments: `connect({ host, port })`. §4.4
- **Origin** — Scheme, host and port of a URL; the unit of the same-origin policy. §14.2
- **Override** — Defining a method in a subclass with the same name as one in its parent so it replaces it; `super.method()` still reaches the parent's. §9.6
- **Own property** — Property stored directly on an object rather than inherited. §9.3

P

- **Package** — Folder of code with a `package.json`, published to a registry such as npm so others can install it. §10.9
- `package.json` — Project's manifest: name, version, scripts, dependencies, `type` and entry points. §10.9
- `padStart` / `padEnd` — Pad a string to a length with a fill string (`'5'.padStart(3, '0')` → `'005'`). §6.6
- **Parameter** — Name in a function's definition that receives a value (the argument) when the function is called. §4.1
- **Parameter properties (TS)** — TypeScript shorthand `constructor(private name: string)` that declares and assigns a field; not erasable. §16.9
- `parseInt` — Reads an integer from the start of a string, stopping at the first invalid character; always pass the radix (`parseInt(s, 10)`). §2.10
- **Partial application** — Fixing some of a function's arguments now and supplying the rest later. §4.8
- **PascalCase** — Naming style where every word starts with a capital (`ShoppingCart`); the convention for classes and constructors. §2.1
- **Passive listener** — Listener registered with `{ passive: true }`, promising not to call `preventDefault()`, so scrolling stays smooth. §13.8
- `path` **module** — Node's utilities for joining, resolving and splitting file paths. §15.5
- **Path traversal** — Attack using `../` in a user-supplied path to reach files outside an allowed folder; resolve the path and check its prefix. §15.5
- **Peer dependency** — Package a library expects the host app to install itself (e.g. a React plugin needs `react`), so only one copy exists. §10.9
- **Pending** — Starting state of a promise, before it is fulfilled or rejected. §12.7
- **Percent-encoding** — Writing unsafe URL characters as `%` plus hex bytes (`' '` → `%20`, `&` → `%26`); `URLSearchParams` and `encodeURIComponent` do it. §14.6
- **Permission model** — Node's `--permission` flag that denies file-system reads and writes, child processes, worker threads and more unless each is allowed (`--allow-fs-read=…`). §15.12
- **Phantom dependency** — Package your code imports that isn't in your own `package.json` and only works because some other package pulled it in. §10.9
- `pipeline` — Connects streams, handles backpressure and errors, and returns a promise (from `node:stream/promises`). §15.9
- **Playwright** — Tool from Microsoft for end-to-end testing that drives real Chromium, Firefox and WebKit browsers. §16.1
- **Poll phase** — The Node event-loop phase that waits for and runs I/O callbacks. §12.4
- **Polyfill** — Code that implements a missing built-in API for older environments. §10.12
- **Polymorphism** — Different classes offering the same method name so calling code can treat them alike (`shape.area()`), each doing its own thing. §9.6
- `popstate` — Event fired when the user moves back or forward through history entries. §14.7
- **Precedence** — Order in which operators are evaluated when parentheses don't decide it. §2.9
- **Pre-commit hook** — Script git runs before each commit (often lint and format) that can block a bad commit. §10.13
- **Prefix / postfix increment** — `++n` adds one and returns the new value; `n++` adds one but returns the old value. §2.5
- **Prefix sum** — Array where each entry is the total of all earlier values, letting you answer any range-sum question in O(1). §17.2

- **Preflight** — `OPTIONS` request a browser sends before a non-simple cross-origin request to ask permission. §14.2
- **Prettier** — Opinionated code formatter. §10.13
- `preventDefault` — Cancels the browser's default action for an event, such as following a link or submitting a form. §13.8
- **Primitive** — Immutable value that isn't an object: number, bigint, string, boolean, undefined, null, symbol. §2.4
- **Priority queue** — Queue that always hands out the highest-priority (smallest or largest) item next; usually built on a heap. §17.1
- **Private field** — `#name` class member only reachable inside the class body, enforced at runtime (ES2022). §9.5
- `process` — Node's global object for arguments, environment variables, exit codes and signals. §15.6
- `process.nextTick` — Node queue that runs before promise microtasks after the current operation. §12.4
- **Promise** — Object representing a value that will be available later; pending, then fulfilled or rejected. §12.7
- `Promise.all` — Waits for all promises; values in input order; rejects on the first rejection. §12.8
- `Promise.allSettled` — Waits for every promise and reports each outcome; never rejects. §12.8
- `Promise.any` — Fulfils with the first fulfilment; rejects with `AggregateError` if all reject. §12.8
- `Promise.race` — Settles like whichever input settles first. §12.8
- **Promisify** — Converting a callback-style function into one that returns a promise (`util.promisify`). §12.7
- **Property (DOM)** — Live JavaScript value on an element object (`input.value`), as opposed to the HTML attribute. §13.4
- **Property descriptor** — Settings of a property: `value`, `writable`, `enumerable`, `configurable`, or `get`/`set`. §5.6
- **Proposal (TC39)** — Suggested new language feature that moves through Stages 0–4, guided by a "champion" on the committee. §1.2
- **Prototype** — Object another object delegates missing property reads to, via its `[[Prototype]]` link. §9.3
- **Prototype chain** — Linked series of prototypes searched when a property isn't found on the object itself. §9.3
- **Prototype pollution** — Attack that writes to `Object.prototype` (e.g. via a `__proto__` key in merged JSON), affecting every object. §9.3
- **Proxy** — Object that wraps another and intercepts operations like get and set. §5.9
- **Pure function** — Function whose output depends only on its inputs and which has no side effects. §4.11

Q

- `querySelector` — Returns the first element matching a CSS selector, or `null`. §13.2
- **Queue** — First-in, first-out (FIFO) list: add at the back, remove from the front. §7.3
- `queueMicrotask` — Schedules a function as a microtask. §12.5
- **Quickselect** — Finding the k-th smallest item in average O(n) by partitioning around a pivot and recursing into one side only. §17.2

R

- **Race condition** — Bug where the result depends on which of several async operations finishes first. §12.13
- **Radians** — Unit `Math.sin`, `cos` and friends use for angles: 180° is π radians. §5.10
- **Radix** — Base of a number system: 10 for decimal, 16 for hex, 2 for binary; the second argument of `parseInt` and `toString`. §2.10
- `RangeError` — Thrown when a number is outside an allowed range: `new Array(-1)`, `toFixed(101)`, or too-deep recursion. §11.1
- **Reachability** — Object is reachable if some chain of references leads to it from a root (globals, the stack); unreachable objects are garbage-collected. §17.4

- **React** — Popular JavaScript library for building user interfaces from components; it relies heavily on immutable updates and closures. §2.4
- **Realm** — Separate global environment (an iframe, a `vm` context) with its own copies of built-ins. §9.7
- **Recursion** — Function solving a problem by calling itself on a smaller version of it. §4.10
- **ReDoS** — Regular-expression denial of service: a badly written pattern that takes exponential time on crafted input and freezes the thread. §6.9
- **`reduce`** — Folds an array into one value by running an accumulator function; pass an initial value. §7.5
- **Redux** — State-management library whose reducers must be pure functions that return new state instead of mutating the old one. §4.11
- **Reference** — Address an object variable holds; copying the variable copies the address, so both names point at the same object. §2.4
- **`ReferenceError`** — Thrown when you use a name that doesn't exist, or a `let`/`const` before its declaration (TDZ). §11.1
- **`Referrer-Policy`** / **`Permissions-Policy`** — Security headers: the first limits how much of the current URL is sent to other sites, the second turns off browser features (camera, geolocation) a page doesn't need. §14.12
- **Reflect** — Built-in object of methods mirroring basic object operations (`Reflect.get`, `Reflect.ownKeys`); handy inside `Proxy` handlers. §5.9
- **Reflected XSS** — XSS where the attack script comes from the request (e.g. a URL parameter) and the server echoes it straight back into the page. §14.10
- **Reflow** — Browser recalculating element positions and sizes (layout) after DOM or style changes. §13.11
- **`RegExp.escape`** — Escapes special regex characters so a string can be matched literally (ES2025). §6.9
- **Registry (npm)** — Online database (registry.npmjs.org) that npm downloads packages from and publishes them to. §10.9
- **Regression** — Bug in something that used to work; tests guard against regressions when code changes. §16.1
- **Regular expression** — Pattern for matching text, written `/pattern/flags` or with `new RegExp`. §6.7
- **Rejection** — Promise settling with an error (reason) instead of a value. §12.7
- **Remainder operator** — `%`: the remainder of division, with the sign of the left operand. §2.5
- **Rendering (event loop)** — Step where the browser runs `requestAnimationFrame` callbacks, then style, layout and paint. §12.2
- **Repaint** — Redrawing pixels after a visual change that doesn't affect layout. §13.11
- **REPL** — Read–Eval–Print Loop: an interactive prompt that runs each line you type and prints the result (`node` with no file, the browser console). §1.5
- **`replaceAll`** — Replaces every occurrence of a string or global regex (ES2021). §6.6
- **Replacer** — Optional second argument of `JSON.stringify`: a function or key list that changes or filters what is written. §5.12
- **Request / response** — Two halves of an HTTP exchange: the client sends a request (method, URL, headers, body); the server answers with a response (status, headers, body). §14.1
- **`requestAnimationFrame`** — Runs a callback right before the next paint; the right tool for animations. §14.5
- **ResizeObserver** — Reports when an element's size changes. §14.9
- **Rest parameter** — `...args` as the last parameter: collects remaining arguments into a real array. §4.4
- **Return value** — Value a function hands back with `return`; without one, a call gives `undefined`. §4.1
- **Reviver** — Function passed to `JSON.parse` to transform each parsed value. §5.12
- **Runtime** — Environment that runs your code (Node, a browser, Deno, Bun); also "at runtime" means while the program is running, as opposed to while compiling. §1.3

- **Run-to-completion** — Once a task starts, it runs until the call stack is empty; nothing interrupts it. §12.1

S

- **Same-origin policy (SOP)** — Browser rule that a page may read responses only from its own origin unless the other server allows it with CORS. §14.2
- `SameSite` — Cookie attribute (`Strict`, `Lax`, `None`) controlling whether it's sent on cross-site requests. §14.11
- **SameValueZero** — Equality `Map`, `Set` and `includes` use: like `===`, except `NaN` equals `NaN`. §8.1
- `satisfies` — TypeScript operator that checks a value against a type without widening the value's own type. §16.13
- **Scope** — Region of code where a name is visible: global, module, function or block. §4.5
- **Scope chain** — Nested scopes searched outward when a variable is looked up. §4.5
- **Seal** — `Object.seal`: no properties can be added or removed, but values can change. §5.6
- **Semantic versioning** — `MAJOR.MINOR.PATCH`: breaking changes, new features, bug fixes. §10.10
- **Serialization** — Turning an in-memory value into a format that can be stored or sent (such as a JSON string); deserialization turns it back. §5.2
- **Service worker** — Background script that acts as a network proxy for offline support, caching and push. §14.8
- **Session cookie** — Cookie with no expiry date that disappears when the browser session ends; also loosely, the cookie holding a login session ID. §14.4
- `sessionStorage` — Like `localStorage` but scoped to one tab and cleared when it closes. §14.3
- `Set` — Collection of unique values in insertion order with O(1) average `has`. §8.2
- **Set methods** — `union`, `intersection`, `difference`, `symmetricDifference` and subset tests on Sets (ES2025). §8.2
- `setImmediate` — Node timer that runs in the check phase, after I/O callbacks. §12.4
- `setInterval` — Runs a function repeatedly roughly every N milliseconds until cleared. §12.5
- **Setter** — `set` method that runs when a property is assigned. §5.7
- `setTimeout` — Runs a function once after at least N milliseconds, as a new task. §12.5
- **Settled** — Promise that is no longer pending: it has been either fulfilled or rejected, and never changes again. §12.7
- **Shadowing** — Inner variable or own property hiding one with the same name further out. §4.5
- **Shallow copy** — Copy of the top level only; nested objects are still shared. §5.4
- **SharedArrayBuffer / Atomics** — Memory that several workers can read and write at once, plus `Atomics` operations to update it safely without races. §15.2
- **Short-circuit evaluation** — `&&`, `||`, `??` and `?.` stop evaluating as soon as the result is known. §2.7
- **Side effect** — Anything a function does beyond returning a value: mutating inputs, I/O, logging, changing globals. §4.11
- `SIGTERM` / `SIGINT` — OS signals asking a process to stop: `SIGTERM` comes from tools such as Docker or `kill`, `SIGINT` from pressing Ctrl+C. §15.6
- **Simple request (CORS)** — Cross-origin `GET`, `HEAD` or `POST` with only basic headers and content types, sent without a preflight. §14.2
- **Single-page application (SPA)** — Web app that loads one page and updates the view with JavaScript, changing the URL with the History API instead of full page loads. §14.7
- **Single-threaded** — JavaScript runs one piece of your code at a time on one thread; waiting is handed to the host. §12.1
- **Singleton** — Something that exists only once; every importer of an ES module shares the same single instance of it. §10.3
- **Sink (XSS)** — An API that turns a string into HTML or code (`innerHTML`, `eval`, `document.write`); untrusted data must never reach it unescaped. §14.10
- **Sliding window** — Keeping a moving range `[left, right]` over an array or string and adjusting its ends, to solve contiguous-range problems in O(n). §17.2

- **Sloppy mode** — Default, non-strict mode of classic scripts, which silently allows mistakes strict mode reports. §1.8
- **Snapshot test** — Test that saves a serialized output and fails when it changes. §16.6
- **Soundness (TS)** — Whether a type system guarantees values really match their types; TypeScript is deliberately not fully sound (`any`, casts). §16.8
- **Source map** — File mapping minified or transpiled code back to the original source for debugging. §11.8
- **Sparse array** — Array with holes, indexes that were never set, as made by `new Array(3)` or `delete arr[i]`. §7.3
- **SpiderMonkey** — Firefox's JavaScript engine, the first one ever written. §1.3
- `splice` — Mutating array method that removes and/or inserts items at an index and returns the removed items. §7.4
- **Spread syntax** — `...` expanding an iterable into elements or arguments, or an object's properties into another object. §5.4
- **Spy** — Test double that records its calls (`t.mock.fn()`). §16.5
- **Stable sort** — Sort that keeps items with equal keys in their original order; `Array.prototype.sort` has been stable since ES2019. §7.6
- **Stack** — Last-in, first-out (LIFO) list: `push` and `pop` at the same end. §7.3
- **Stack overflow** — Error when too many nested calls fill the call stack, usually recursion without a reachable base case: `RangeError: Maximum call stack size exceeded`. §4.10
- **Stack trace** — List of function calls active when an error was created, newest first. §11.8
- **Stage (TC39)** — Proposal's maturity from 0 (idea) to 4 (finished, goes into the next edition). §1.2
- **Stale closure** — Callback still seeing old values because it closed over variables from an earlier render or call. §4.7
- **Statement** — Complete instruction such as a declaration, an `if` or a loop. §1.6
- **Static initialization block** — `static { … }` inside a class: code that runs once when the class is defined (ES2022). §9.5
- **Static member** — Method or field on the class itself, not on instances: `static create()`. §9.5
- **Static typing** — Checking types before the program runs, from annotations and inference; TypeScript adds it to JavaScript. §16.8
- `stdout` / `stderr` — Process's two output streams: normal output and error/diagnostic output; `console.log` writes to one, `console.error` to the other. §15.6
- **Step over / into / out** — Debugger controls: run the next line, enter the function being called, or run until the current function returns. §11.9
- **Sticky flag (`y`)** — Regex flag that only matches starting exactly at `lastIndex`, useful for writing tokenizers. §6.7
- `stopPropagation` — Stops an event from travelling to further elements. §13.6
- **Stored XSS** — XSS where the attacker's script is saved on the server (a comment, a profile name) and served to every visitor. §14.10
- **Stream** — Node's abstraction for data processed in chunks: Readable, Writable, Duplex, Transform. §15.9
- **Strict equality** — `===`: same type and value, no conversion. §2.6
- **Strict mode** — Restricted mode that turns silent mistakes into errors; automatic in modules and classes. §1.8
- **String** — Immutable sequence of UTF-16 code units. §6.1
- `String.raw` — Tag for template literals that keeps backslashes as written, handy for regex sources and Windows paths. §6.3
- **Structural typing** — TypeScript's rule that types are compatible if their shapes match, whatever their names. §16.8
- **Structured clone / `structuredClone`** — Deep-copy algorithm used by `structuredClone()`, `postMessage` and IndexedDB; handles Dates, Maps, Sets and cycles, throws `DataCloneError` on functions. §5.4, §14.8
- **Stub** — Test double that returns canned answers. §16.5

- **`submit` event** — Fires on a `<form>` when it is submitted; listen for it (not button clicks) and call `preventDefault()` to handle it in JavaScript. §13.9
- **Subresource Integrity** — `integrity` hash on a `<script>` so the browser refuses a tampered CDN file. §14.12
- **Subtest** — Test nested inside another test with `t.test()`, reported as part of its parent. §16.3
- **`super`** — Calls the parent class's constructor (`super()`) or methods (`super.m()`). §9.6
- **Supply-chain attack** — Malicious code slipped into a package you depend on (or its dependencies) so it runs in your app. §10.10
- **SuppressedError** — Error thrown when cleanup with `using` fails while another error is already in flight; it holds both. §11.6
- **Surrogate pair** — Two UTF-16 code units that together encode one character outside the Basic Multilingual Plane, like most emoji. §6.5
- **SWC** — Very fast transpiler written in Rust, used by Next.js and others to strip types and transform syntax. §10.12
- **`switch`** — Compares one value against `case` values with `===`; falls through without `break`. §3.4
- **Symbol** — Primitive creating unique identifiers, often used as property keys. §2.3
- **`Symbol.dispose`** — Well-known symbol naming the cleanup method that a `using` declaration calls automatically. §11.6
- **`Symbol.iterator`** — Well-known symbol naming the method that makes an object iterable. §8.4
- **Synchronous** — Code that runs step by step and blocks until each step finishes. §12.1
- **Syntax** — Grammar rules for how code must be written; breaking them gives a `SyntaxError` before anything runs. §1.6
- **`SyntaxError`** — Thrown when code or text can't be parsed: malformed source, bad `JSON.parse` input, an invalid regex. §11.1

T

- **Table-driven test** — One test body run over a table of inputs and expected outputs, so new cases are a single line. §16.3
- **Tagged template** — Function called with a template literal's pieces: `` tag`Hi ${name}` ``. §6.3
- **Task** (macrotask) — A unit of work in the event loop: a script, timer callback, I/O callback or UI event; one runs per loop turn. §12.2
- **TC39** — The Ecma committee that designs and maintains ECMAScript. §1.2
- **TCP** — Transmission Control Protocol: the reliable, ordered byte-stream connection that HTTP runs on top of. §15.2
- **Template literal** — Backtick string with `${}` interpolation and real line breaks. §6.3
- **Temporal** — Modern date-time API (Stage 4 in March 2026, planned for ES2027); not in Node 24. §5.11
- **Temporal Dead Zone (TDZ)** — Time between entering a scope and a `let`/`const`/`class` declaration running; access throws. §2.2
- **Terminal** — Text window where you type commands (the command line), such as `node app.js` or `npm install`. §1.5
- **Ternary operator** — `cond ? a : b`: chooses between two values. §3.3
- **Test double** — Any stand-in for a real dependency in a test: dummy, stub, spy, mock or fake. §16.5
- **Test pyramid** — Guideline to have many fast unit tests, fewer integration tests and only a few slow end-to-end tests. §16.1
- **Test runner** — Tool that finds tests, runs them and reports passes and failures: `node --test`, Vitest, Jest. §16.1
- **Test suite** — Group of related tests, or all of a project's tests together. §16.1
- **Test-driven development (TDD)** — Writing a failing test first, then the code that makes it pass, then tidying up (red, green, refactor). §16.7
- **Testing trophy** — Alternative to the test pyramid that puts most effort into integration tests, on top of static checks. §16.1
- **Text node** — A DOM node holding plain text inside an element, including the whitespace between tags. §13.1

- **`textContent`** — All text inside a node; setting it inserts plain text, never HTML. §13.3
- **`then`** — Registers callbacks on a promise and returns a new promise. §12.7
- **Thenable** — Any object with a `then` method; `await` and promise resolution treat it like a promise. §12.7
- **`this`** — Context object of a function call, set by how the function is called. §9.1
- **Thread** — Independent line of execution the CPU can run; JavaScript code runs on one, while the host may use others behind the scenes. §12.1
- **Thread pool** — libuv's worker threads (4 by default) that run file-system, DNS lookup, crypto and zlib jobs for Node. §15.2
- **Throttle** — Run a function at most once per interval during a burst of calls (e.g. scroll handlers). §17.3
- **`throw`** — Raises an exception, stopping normal flow until a `catch` handles it. §11.2
- **Tilde range (`~`)** — Npm version range that allows only patch updates (`~1.2.3` means at least 1.2.3, below 1.3.0). §10.10
- **Timers phase** — The Node event-loop phase that runs expired `setTimeout` and `setInterval` callbacks. §12.4
- **TOCTOU (check-then-act)** — "Time of check to time of use": a race where something changes between checking (does the file exist?) and acting on it; just try the action and handle the error. §15.4
- **`toJSON`** — Method `JSON.stringify` calls if present, letting an object choose its own JSON form (`Date` uses it to give an ISO string). §5.12
- **Top-level `await`** — Using `await` at the top level of an ES module; importers wait for it (ES2022). §10.5
- **Topological sort** — Ordering the nodes of a directed graph so every node comes after everything it depends on; impossible if there is a cycle. §17.2
- **`toSorted` / `toReversed` / `toSpliced` / `with`** — Non-mutating copies of `sort`, `reverse`, `splice` and index assignment (ES2023). §7.4
- **Transform stream** — Duplex stream that changes data as it passes through, like gzip compression. §15.9
- **Transitive dependency** — Package you don't install directly but get because one of your dependencies (or theirs) needs it. §10.9
- **Transpiler** — Tool that rewrites source code into another syntax (TypeScript or new JavaScript → older JavaScript). §10.12
- **Tree shaking** — Removing unused exports from a bundle, made possible by static ES module syntax. §10.11
- **Trie** — Tree of characters where each path from the root spells a prefix; fast for "starts with" lookups. §17.2
- **Trusted Types** — Browser feature that makes dangerous sinks like `innerHTML` accept only values produced by approved sanitizing policies. §14.10
- **Truthy** — Any value that is not falsy, including `[]`, `{}` and `'0'`. §2.11
- **`try…catch`** — Runs code and handles exceptions thrown synchronously inside it. §11.2
- **`tsc`** — The TypeScript compiler: it type-checks code and, unless given `--noEmit`, writes JavaScript output. §16.8
- **`tsconfig.json`** — Configuration file for a TypeScript project: which files to check and with which compiler options (e.g. `strict`). §16.8
- **`tsx`** — Popular command-line tool that runs TypeScript files directly in Node using esbuild. §16.9
- **Tuple (TS)** — Fixed-length array type with a type per position, e.g. `[string, number]`. §16.10
- **Two pointers** — Using two indexes that move through an array (often from both ends of sorted data) to solve pair problems in O(n). §17.2
- **Type annotation** — A TypeScript declaration of a value's type, such as `x: number`. §16.10
- **Type assertion (TS)** — `value as Type`: telling TypeScript to trust you about a type; it checks nothing at runtime. §16.8
- **Type checker** — Part of TypeScript that verifies values are used consistently with their types and reports errors before running. §16.8

- **Type erasure** — TypeScript types are deleted when code is compiled or stripped, so none of them exist at runtime. §16.8
- **Type inference** — TypeScript working out a type from how a value is created (`let n = 1` is `number`) so you needn't write it. §16.10
- **Type predicate** — A TypeScript return type `x is T` that narrows the argument when the function returns true. §16.11
- **Type stripping** — Node running `.ts` files by deleting type syntax without checking it. §16.9
- **Typed array** — Array-like view of fixed-size numbers over an `ArrayBuffer`, such as `Uint8Array`. §7.9
- `TypeError` — Thrown when a value is the wrong type for an operation: reading a property of `null`, calling a non-function, assigning to a `const`. §11.1
- `typeof` — Operator returning a string naming a value's type; `typeof null` is `'object'`. §2.3
- **TypeScript** — JavaScript plus static types that are checked by `tsc` and erased before running. §16.8

U

- **Unary operator** — Operator with one operand: `-x`, `+x` (converts to number), `!x`, `typeof x`, `++x`. §2.5
- `uncaughtException` — Node's `process` event for an error nobody caught; log it and exit, because the app may be in a broken state. §11.5
- `undefined` — Primitive meaning "no value assigned yet"; what missing properties and arguments give. §2.3
- **Unhandled rejection** — Rejected promise with no handler; reported by the browser and fatal by default in Node. §11.5
- **Unicode** — Standard that gives every character in every writing system a number (code point), plus encodings such as UTF-8 and UTF-16 to store them. §6.5
- **Unicode property escape** — `\p{…}` in a `u`/`v` regex, matching characters by Unicode property such as `\p{L}`. §6.9
- **Union type (TS)** — `A | B`: a value that may be any one of several types; narrow it before using type-specific features. §16.10
- **Union-find** — Structure (disjoint-set union) that tracks which items are in the same group and merges groups quickly. §17.2
- **Unit test** — Fast test of one function or module in isolation. §16.1
- **Unix epoch** — Moment 1970-01-01T00:00:00Z; JavaScript dates count milliseconds since it. §5.11
- `unknown` **(TS)** — Type-safe "anything" type: you must narrow it before using it. §16.10
- **Upsert (`getOrInsert`)** — Getting a key's value or inserting a default if it is missing, in one call: `map.getOrInsert(k, [])` (ES2026). §8.1
- `URIError` — Thrown when `encodeURIComponent` or `decodeURIComponent` gets malformed input, like `decodeURIComponent('%')`. §11.1
- **URL / URLSearchParams** — Built-ins for parsing, building and encoding URLs and query strings. §14.6
- `using` — Declaration that automatically calls `[Symbol.dispose]()` when the block ends (finished proposal, ES2027; works in Node 24). §11.6
- **UTC** — Coordinated Universal Time: the time-zone-free world clock; store and transmit dates in UTC and convert to local time only for display. §5.11
- **UTF-16** — Encoding JavaScript strings use: characters as one or two 16-bit code units. §6.5
- **UTF-8** — Most common text encoding on the web and in files: 1 to 4 bytes per code point, ASCII stays 1 byte. §6.5

V

- **V8** — Google's JavaScript engine, used in Chrome, Edge, Node.js and Deno. §1.3
- `var` — Original function-scoped, hoisted variable declaration; avoid in new code. §2.1
- **Variable** — Named box for a value, declared with `let`, `const` or `var`. §2.1
- **Virtualization (lists)** — Rendering only the rows currently visible in a long list and reusing them as you scroll, to keep the DOM small. §13.11
- **Visual Studio Code (VS Code)** — Free code editor from Microsoft with built-in JavaScript/TypeScript support, a terminal and a debugger. §1.5

- **Vite** — Dev server plus production bundler; the default choice for new front-end apps. §10.11
- **Vitest** — Fast test runner built on Vite with a Jest-compatible `expect` API. §16.7
- **Void operator** — `void expr` evaluates the expression and returns `undefined`. §2.9

W

- **Watch expression** — Expression the debugger re-evaluates and shows every time execution pauses. §11.9
- **WeakMap** — Map with object keys held weakly: entries disappear when the key object is garbage-collected. §8.3
- **WeakRef** — Weak reference to an object; `deref()` returns it or `undefined` once collected. §8.3
- **WeakSet** — Set of objects held weakly; used to tag objects without keeping them alive. §8.3
- **Web Streams API** — Standard `ReadableStream` / `WritableStream` / `TransformStream` classes, shared by browsers and Node (`fetch` bodies use them). §15.9
- **Web Worker** — Background thread with no DOM access that talks to the page via `postMessage`. §14.8
- **webpack** — Long-established, highly configurable bundler. §10.11
- **Well-known symbol** — Built-in `Symbol` the language uses as a hook, such as `Symbol.iterator`, `Symbol.asyncIterator` or `Symbol.dispose`. §8.4
- **Worker threads** — Node's `worker_threads` module for running CPU-heavy JavaScript in parallel. §15.2

X

- `X-Content-Type-Options: nosniff` — Header telling the browser to trust the declared `Content-Type` instead of guessing, so an uploaded text file can't be run as a script. §14.12
- **XOR** — "Exclusive or": a bit is 1 when exactly one input bit is 1 (`^` in JavaScript); `x ^ x` is 0, which helps find a single unpaired number. §2.8
- **XSS** — Cross-site scripting: attacker-controlled script runs in your page, usually through unescaped HTML. §14.10

Y

- `yield` — Pauses a generator and hands a value to the consumer; `yield*` delegates to another iterable. §8.5

Z

- **Zalgo** — Callback API that is sometimes synchronous and sometimes asynchronous, making ordering unpredictable. §12.6
- **Zero-width joiner (ZWJ)** — Invisible character (U+200D) that glues emoji into one picture, such as a family emoji made of several people. §6.5
- **Zod** — A TypeScript-first library for describing data shapes and validating untrusted input at runtime. §16.8

Chapter 18 TL;DR

- Use this chapter for **revision**: cover the answer column of a sheet, say the answer, then follow the § link for anything you missed.
- **Language core**: `const` by default, `===` always, `??` for defaults, eight types with `typeof null === 'object'`, eight falsy values, floats that aren't exact (`0.1 + 0.2`), closures that remember their scope, and `this` decided at the call site (arrows excepted).
- **Data**: shallow spread vs `structuredClone`; `Object.freeze` is shallow; `toSorted` / `with` instead of mutating shared arrays; `Map` / `Set` for keyed lookup and uniqueness; generators and iterator helpers for lazy sequences.
- **Async**: one task, then **all** microtasks, then render; sync code → microtasks → tasks; `Promise.all` for parallel work, `allSettled` for every outcome, `AbortController` to cancel.
- **Platform**: `textContent` over `innerHTML`, delegation with `closest`, check `res.ok` after `fetch`, `HttpOnly; Secure; SameSite` cookies for sessions, CSP against XSS; in Node, `node:` modules, streams with `pipeline`, never block the loop.

- **Quality**: `node:test` + `assert/strict`, `tsc --noEmit` for types, ESLint + Prettier, lockfile + `npm ci`.
- **Versions**: ES2015 was the big one; since then one edition every June. ES2026 is only partly in Node 24; Temporal and `using` belong to ES2027.

This is the end of the JavaScript Handbook. Next: the React Handbook, or back to Chapter 1 for a second pass.

Chapter 19 — Interview Question Bank

How to use this bank

This chapter collects the JavaScript questions interviewers ask most, worded the way they usually appear in LeetCode study lists (above all **30 Days of JavaScript**) and the big public question collections listed at the end. It covers the language (ES2025), the browser, Node 24 and basic TypeScript.

- ★ marks a question that shows up in many sources: expect it.
- Each question has a **short answer you can say out loud** in about 30 seconds, or a short code snippet plus a sentence or two. Answers are written in this book's own words.
- → **More** links to the section of this book that explains the topic in full. Chapter 17 already answers 53 tricky questions with real output, and §17.3 holds tested implementations of the coding classics; this bank points there instead of repeating them, and adds every common question that chapter lacks.
- Code answers are correct for Node 24 and current browsers. Outputs in the puzzle group were checked by running them.

How to drill. Cover the answer, say yours out loud (or type the code), then uncover and compare. Mark the ones you missed and come back to them the next day. Do the ★ questions first, then the coding tasks (groups 14 and 15) from memory, with a timer.

Terms used everywhere below. A **primitive** is a simple immutable value (a string, a number…). An **object** is a collection of properties held by **reference** (variables store a pointer to it, not a copy). A **callback** is a function you pass to another function so it can call it later. A **polyfill** is your own code that adds a built-in feature an engine lacks. The **event loop** is the mechanism that runs queued work once the current code finishes.

---

1. Types, values and coercion

Q1. ★ What are the data types in JavaScript? There are eight. Seven are **primitives**: `string`, `number` (64-bit floating point), `bigint` (whole numbers of any size), `boolean`, `undefined`, `null` and `symbol` (a unique identifier). The eighth is `object`, which includes arrays, functions, dates, maps and everything else built from objects. Primitives are immutable and compared by value; objects are mutable and compared by reference. → *More:* §2.3 Data types and `typeof`

Q2. ★ What is the difference between primitive and reference (non-primitive) values? A primitive is stored directly in the variable; copying the variable copies the value, and nothing can change it in place. An object lives on the heap (the memory area for dynamically created data) and the variable holds a reference to it; copying the variable copies the reference, so both names see the same object. That is why `{} === {}` is `false` (two different objects) but `a === a` is `true`. → *More:* §2.4 Primitives vs objects

Q3. ★ What is the difference between `==` and `===`? `===` (strict equality) is `true` only if both values have the same type and value, with no conversion. `==` (loose equality) first **coerces** (automatically converts) the operands to a common type using a set of rules: `null == undefined` is `true`, strings and booleans become numbers, objects become primitives. Use `===` always; the one idiom some teams allow is `x == null` to test for "null or undefined". → *More:* §2.6 Comparison operators and equality · §17.4.1 Q2–Q3

Q4. ★ What is the difference between `null`, `undefined` and an undeclared variable? How do you check each? `undefined` means "no value was ever assigned" (a declared variable, a missing property, a missing argument). `null` is a value a programmer assigns on purpose to mean "empty". An **undeclared** variable was never declared at all: reading it throws `ReferenceError`. Check with `x === undefined`, `x === null`, `x == null` (either), and `typeof x === 'undefined'`, which is the only test that does not throw for an undeclared name. → *More:* §2.3 Data types and `typeof`

Q5. ⋆ What does `typeof` return, and why is `typeof null` `"object"`? `typeof` returns one of `"string"`, `"number"`, `"bigint"`, `"boolean"`, `"undefined"`, `"symbol"`, `"object"` or `"function"`. Arrays give `"object"` (use `Array.isArray`), classes give `"function"`. `typeof null === "object"` is a mistake in the original 1995 implementation: values carried a type tag, the tag for objects was 0, and `null` (a zero pointer) was read as an object. It stays because changing it now would break existing websites. → *More:* §17.4.1 Q4 and Q7

Q6. ⋆ What is type coercion? What are its common pitfalls? Coercion is JavaScript converting a value to another type automatically. `+` with any string concatenates (`1 + '2'` is `'12'`), while `-`, `*`, `/` convert to numbers (`'5' - 2` is `3`). `if`, `!`, `&&`, `||` convert to boolean. Pitfalls: `==` surprises, `'' == 0` is `true`, `[] + {}` is `'[object Object]'`, form inputs are strings so `input.value + 1` concatenates. Convert explicitly with `Number()`, `String()`, `Boolean()`. → *More:* §2.12 Type conversion summary · §17.4.1 Q1

Q7. ⋆ Which values are falsy? Exactly eight: `false`, `0`, `-0`, `0n`, `''` (empty string), `null`, `undefined` and `NaN`. Everything else is truthy, including `'0'`, `'false'`, `[]`, `{}` and `new Boolean(false)` (an object). → *More:* §2.11 Booleans, truthy and falsy

Q8. ⋆ What is `NaN`? What is the difference between `isNaN` and `Number.isNaN`? `NaN` ("not a number") is the number value produced by a failed numeric operation, such as `Number('abc')` or `0 / 0`. It is the only value not equal to itself (`NaN === NaN` is `false`). The global `isNaN(x)` converts `x` to a number first, so `isNaN('abc')` is `true`; `Number.isNaN(x)` is `true` only for the actual `NaN` value. Prefer `Number.isNaN` or `Object.is(x, NaN)`. → *More:* §2.10 Numbers in depth · §17.4.1 Q5

Q9. What is the difference between `Object.is` and `===`? They agree except in two cases: `Object.is(NaN, NaN)` is `true` (but `NaN === NaN` is `false`), and `Object.is(0, -0)` is `false` (but `0 === -0` is `true`). React uses `Object.is` to decide whether state changed. → *More:* §17.4.1 Q6

Q10. ⋆ Why is `0.1 + 0.2 !== 0.3`? How do you compare decimals? Numbers are IEEE 754 binary floating point, and 0.1 and 0.2 have no exact binary form, so the sum is `0.30000000000000004`. Compare with a tolerance: `Math.abs(a - b) < Number.EPSILON` (or a tolerance that fits the scale), round for display with `toFixed`, and store money as integer cents (or `BigInt`). → *More:* §2.10 Numbers in depth · §17.4.1 Q14

Q11. What is `Number.MAX_SAFE_INTEGER`, and what is `BigInt`? `Number.MAX_SAFE_INTEGER` is $2^{53} - 1$ (9,007,199,254,740,991): the largest integer a `number` can hold exactly, after which `n + 1` may equal `n + 2`. `BigInt` (`123n`, `BigInt('123')`) stores integers of any size. You cannot mix `BigInt` and `number` in arithmetic (`1n + 1` throws `TypeError`), `BigInt` has no decimals, and `JSON.stringify` cannot serialize it without a custom replacer. → *More:* §2.10 Numbers in depth · §17.1

Q12. ⋆ What is the difference between `||` and `??`? What does `?.` do? `a || b` returns `b` when `a` is any falsy value, so `0 || 10` is `10`. `a ?? b` (nullish coalescing) returns `b` only when `a` is `null` or `undefined`, so `0 ?? 10` is `0` — right for defaults where `0` or `''` are valid. `a?.b` (optional chaining) returns `undefined` instead of throwing when `a` is `null`/`undefined`; it also works as `a?.[key]` and `fn?.()`. → *More:* §2.7 Logical operators, `??` and `?.` · §17.4.1 Q12

Q13. What do `"hello" || "world"` and `"foo" && "bar"` return? `"hello"` and `"bar"`. `||` and `&&` return one of their **operands**, not a boolean: `||` returns the first truthy operand (or the last one), `&&` returns the first falsy operand (or the last one). This is called short-circuit evaluation: the right side is not evaluated if the left side decides the result. → *More:* §2.7

Q14. What does `!!` do? It converts any value to its boolean: the first `!` converts and negates, the second negates back. `!!'text'` is `true`, `!!0` is `false`. `Boolean(x)` does the same and reads more clearly.

Q15. What are logical assignment operators? ES2021 added `a ||= b` (assign if `a` is falsy), `a &&= b` (assign if `a` is truthy) and `a ??= b` (assign if `a` is `null`/`undefined`). They short-circuit: if no assignment is needed, `b` is not evaluated and no write happens. Common use: `options.timeout ??= 5000`. → *More:* §2.8 Bitwise and assignment operators

Q16. How do you convert a string to a number? `Number` vs `parseInt` vs unary `+`? `Number(s)` and `+s` convert the whole string and give `NaN` if any part is not numeric (`Number('12px')` is `NaN`); empty or whitespace-only strings give `0`. `parseInt(s, 10)` reads digits from the start and stops at the first non-digit (`parseInt('12px', 10)` is `12`); always pass the radix (the base, 10). `parseFloat` does the same for decimals. → *More:* §2.12 Type conversion summary

Q17. How is an object converted to a primitive? When an object is used where a primitive is needed (`+`, `==`, template literals), the engine calls `obj[Symbol.toPrimitive](hint)` if defined; otherwise, for the `"number"` or `"default"` hint, it tries `valueOf()` then `toString()`, and for the `"string"` hint `toString()` first. Plain objects end up as `'[object Object]'`, arrays as their elements joined by commas (`[1, 2] + ''` is `'1,2'`), dates as a readable string with `+` but a timestamp with `-`.

```
const price = { [Symbol.toPrimitive]: (hint) => (hint === 'number' ? 42 : '$42') };
console.log(+price, `${price}`); // 42 '$42'
```

Q18. What are `Symbol`s used for? A symbol (`Symbol('id')`) is a unique, immutable primitive used as a property key that can never clash with another key, even one with the same description. Uses: hidden-ish metadata on objects you don't own, and **well-known symbols** that hook into language behaviour — `Symbol.iterator` (make an object iterable), `Symbol.toPrimitive`, `Symbol.asyncIterator`, `Symbol.dispose`. Symbol keys are skipped by `for…in`, `Object.keys` and `JSON.stringify`. `Symbol.for('k')` gives a shared, registry-wide symbol. → *More:* §2.3 · §8.4 Iterators

Q19. How can `'abc'.toUpperCase()` work if strings are primitives? When you access a property on a primitive, the engine temporarily wraps it in its **wrapper object** (`String`, `Number`, `Boolean`, `Symbol`, `BigInt`), calls the method, and discards the wrapper. That's also why `str.foo = 1` silently does nothing (in sloppy mode) or throws in strict mode. Never create wrappers yourself with `new String('x')`: they are objects, so they are truthy and `===` fails. → *More:* §6.4

Q20. How do you check whether a value is an array, a plain object, a number? `Array.isArray(v)` (works across iframes, unlike `instanceof Array`). Number: `typeof v === 'number' && Number.isFinite(v)`. Plain object: `v !== null && typeof v === 'object' && Object.getPrototypeOf(v) === Object.prototype` (or `=== null`). For a general type name, `Object.prototype.toString.call(v)` gives `'[object Date]'`, `'[object Map]'`, and so on. → *More:* §9.7 `instanceof` and type checks

---

2. Variables, hoisting, scope and closures

Q21. ⋆ What are the differences between `var`, `let` and `const`? `var` is **function-scoped** (visible in the whole function), can be re-declared, is hoisted and initialised to `undefined`, and at the top level creates a property on `window`. `let` and `const` are **block-scoped** (visible only inside the `{}` where they are declared), cannot be re-declared in the same scope, and sit in the temporal dead zone until their line runs. `const` must be initialised and cannot be reassigned — but the object it points to can still be changed. Default to `const`, use `let` when you reassign, avoid `var`. → *More:* §2.1 · §17.4.2 Q15

Q22. ⋆ What is hoisting? Before code in a scope runs, the engine registers all its declarations. That makes it look as if declarations were "hoisted" (moved) to the top. Function declarations are hoisted with their body, so you can call them before their line. `var` is hoisted and set to `undefined`. `let`, `const` and `class` are hoisted but left uninitialised (the TDZ), so reading them early throws. Function expressions and arrows follow the rule of the variable that holds them. → *More:* §2.2 · §4.6 Hoisting — the complete picture

Q23. ⋆ What is the temporal dead zone (TDZ)? The TDZ is the time between entering a scope and reaching a `let`/`const`/`class` declaration. During it the variable exists (it shadows any outer variable of the same name) but any access throws `ReferenceError: Cannot access 'x' before initialization`. Even `typeof x` throws inside the TDZ. It exists to catch use-before-define bugs.

```
let x = 'outer';
{ console.log(x); let x = 'inner'; } // ReferenceError, not 'outer'
```

→ *More:* §2.2 · §17.4.2 Q16

Q24. ⋆ What is the difference between `function foo() {}` and `const foo = function () {}`? The first is a **function declaration**: hoisted with its body, so it can be called anywhere in its scope. The second is a **function expression** assigned to a variable: it exists only after that line runs (before it, `foo` is in the TDZ for `const`/`let`, or `undefined` for `var`, giving `TypeError: foo is not a function`). Expressions can be anonymous or named; a name on an expression is visible only inside it (handy for recursion and stack traces). → *More:* §4.2 Declarations vs expressions

Q25. What is scope? What kinds are there? Scope is the region of code where a name is visible. JavaScript has **global** scope, **module** scope (top level of an ES module — not global), **function** scope, and **block** scope (any `{}` for `let`/`const`/`class`). When a name is used, the engine looks in the current scope, then the enclosing one, and so on up to global: this chain is the **scope chain**. If it is not found, reading throws `ReferenceError`. → *More:* §4.5 Scope

Q26. ⋆ What is lexical scope? Lexical (static) scope means a function's accessible variables are decided by **where it is written** in the source, not by where it is called. A function defined inside another can see the outer function's variables, wherever it is later called from. Closures are a consequence of lexical scope. (`this` is the exception: for regular functions it depends on the call.) → *More:* §4.5

Q27. What are shadowing and illegal shadowing? Shadowing is declaring a variable in an inner scope with the same name as an outer one; inside, the inner one hides the outer. **Illegal shadowing** is shadowing a `let`/`const` with a `var` in an inner block of the same function: because `var` belongs to the whole function, it would clash with the `let`, so it is a `SyntaxError`. Shadowing a `var` with a `let` in a block is fine.

```
function f() { let a = 1; { var a = 2; } } // SyntaxError: 'a' has already been declared
```

Q28. ⋆ What is a closure? Give a practical use. A closure is a function together with the variables of the scope it was created in. The function keeps access to those variables even after the outer function has returned, and each call of the outer function creates a fresh set. Uses: private state (counters, caches), function factories, `debounce`/`memoize`, event handlers that remember data, the module pattern.

```
function makeCounter() { let count = 0; return () => ++count; }
const next = makeCounter(); next(); next(); // 2 — `count` lives on in the closure
```

→ *More:* §4.7 Closures · §17.4.2 Q19

Q29. ⋆ What does this print, and how do you fix it? `for (var i = 0; i < 3; i++) setTimeout(() => console.log(i));` It prints `3 3 3`. `var` creates a single `i` for the whole function; the callbacks run after the loop, when `i` is already 3. Fix with `let i` (a `for` loop creates a fresh binding per iteration, so it prints `0 1 2`), or capture the value with an IIFE or `setTimeout(console.log, 0, i)`. → *More:* §17.4.2 Q18

Q30. How do you create private variables in JavaScript? Three ways: a **closure** (variables of a factory function, reachable only through the functions it returns); `#private` **class fields** (`#balance`, enforced by the language, ES2022); or a **module** (anything not exported is private to the file). A `WeakMap` keyed by instance is an older pattern. A leading underscore (`_x`) is only a naming convention. → *More:* §4.7 · §17.4.4 Q33

Q31. ⋆ What is an IIFE and why was it used? An IIFE (immediately invoked function expression) is a function that runs as soon as it is defined: `(function () { … })();` or `(() => { … })();`. Before `let`/`const` and modules, it was the way to create a private scope so variables didn't leak into the global scope, and to capture loop values. Today modules and block scope cover most uses; it's still handy for an `async` IIFE in old CommonJS code. → *More:* §4.9 IIFE

Q32. What is the module pattern? An IIFE that returns an object of public functions, while variables inside stay private through closure. It was the standard way to build modules before ES modules.

```
const bank = (() => { let balance = 0;
  return { deposit: (n) => (balance += n), get: () => balance }; })();
```

→ *More:* §4.9

Q33. ⋆ What is an execution context? What are the call stack and its phases? An execution context is the record the engine creates to run a piece of code: its variables (the **environment record**), the link to the outer scope, and the value of `this`. There is one global context, and a new one for each function call. Each has two phases: **creation** (declarations registered — this is hoisting — and `this` decided) and **execution** (code runs line by line). The **call stack** holds these contexts: a call pushes a context, a return pops it. A recursive function that never stops overflows it: `RangeError: Maximum call stack size exceeded`. → *More:* §4.6 · §12.1

Q34. ⋆ What is strict mode? How do you enable it? Strict mode is a stricter variant of JavaScript that turns silent mistakes into errors: assigning to an undeclared name throws instead of creating a global, `this` is `undefined` in a plain function call (not the global object), duplicate parameter names and `with` are forbidden, writing to read-only properties throws. Enable it with `'use strict';` at the top of a file or function. ES modules and class bodies are always strict. → *More:* §1.8 Strict mode · §17.4.2 Q22

Q35. What is `globalThis`? What problems do global variables cause? `globalThis` (ES2020) is the global object in every environment: `window` in a page, `self` in a worker, `global` in Node. Global variables are risky: any script can overwrite them, names collide between libraries, they stay in memory forever, and they make code hard to test. Keep state in modules, closures or passed-in parameters. → *More:* §4.5

Q36. What is wrong with `let a = b = 5;`? It is parsed as `let a = (b = 5)`. Only `a` is declared; `b` is assigned without a declaration, so in sloppy mode it becomes an accidental global variable, and in strict mode it throws `ReferenceError`. → *More:* §17.4.2 Q20

---

3. Functions and functional patterns

Q37. ⋆ How do arrow functions differ from regular functions? Arrow functions have no own `this`, `arguments`, `super` or `new.target`: they use the ones of the surrounding scope (lexical `this`). They cannot be called with `new` and have no `prototype`. They have a shorter syntax with implicit return for a single expression (wrap an object literal in parentheses: `() => ({ a: 1 })`). Use them for callbacks; use regular functions or methods for object methods that need `this`, and for constructors. → *More:* §4.3 Arrow functions · §17.4.3 Q24

Q38. What does "functions are first-class" mean? What is a higher-order function? First-class means functions are values: you can store them in variables, pass them as arguments, return them and put properties on them. A **higher-order function** takes a function as an argument and/or returns one: `map`, `filter`, `reduce`, `setTimeout`, `addEventListener`, and your own `debounce` or `memoize`. → *More:* §4.8 Higher-order functions and callbacks

Q39. What is a pure function? What are side effects? A pure function always returns the same output for the same input and has no side effects. A **side effect** is anything a function does besides returning a value: changing outside variables or its arguments, I/O (network, files, DOM, `console.log`), reading the clock or `Math.random()`. Pure functions are easy to test, cache (memoize) and run in parallel. **Referential transparency** means a call can be replaced by its result without changing the program — true for pure functions. → *More:* §4.11 Pure functions

Q40. What is a callback, and what is callback hell? A callback is a function passed to another function to be called later — when an event fires, a timer ends, or async work finishes. **Callback hell** is deeply nested callbacks for sequential async steps, each with its own error check, making code hard to read and error handling easy to forget. Promises and `async` / `await` flatten it. → *More:* §12.6 Callbacks and callback hell

Q41. ⋆ What is currying? Why is it useful? Currying turns a function of several arguments into a chain of functions taking one (or a few) arguments each: `add(1, 2, 3)` becomes `add(1)(2)(3)`. It lets you build specialised functions by fixing early arguments (`const tax = multiply(0.2)`), which suits `map` and composition. **Partial application** is related: fixing some arguments now (`fn.bind(null, a)`) and passing the rest later, without the one-at-a-time rule. → *More:* §17.3 Curry · Q255 below

Q42. What are `compose` and `pipe`? Both combine functions into one. `compose(f, g, h)(x)` is `f(g(h(x)))` — applies right to left, as in maths. `pipe(f, g, h)(x)` is `h(g(f(x)))` — left to right, the order you read. Implementation: `const pipe = (...fns) => (x) => fns.reduce((v, f) => f(v), x);` and `compose` uses `reduceRight`. → *More:* §17.3 Compose and pipe

Q43. ⋆ What is memoization? Memoization caches a function's results by its arguments so a repeat call returns the cached value instead of recomputing. It only makes sense for pure functions. The cache key is the tricky part (one primitive argument: use it directly; several: `JSON.stringify(args)` or a nested `Map`); unbounded caches leak memory, so cap them (LRU) or use a `WeakMap` for object keys. → *More:* §17.3 Memoize · Q257 below

Q44. ⋆ What is the difference between rest parameters and spread syntax? Both use `...`. **Rest** collects many values into one array: in a parameter list (`function f(...args)`) or a destructuring pattern (`const [first, ...others] = arr`). **Spread** expands one iterable or object into many: in a call (`Math.max(...nums)`), an array literal (`[...a, ...b]`) or an object literal (`{ ...defaults, ...opts }`). Rest must be last. → *More:* §4.4 Parameters

Q45. What is the `arguments` object, and how does it differ from rest parameters? `arguments` is an array-like object (indexed, with `length`, but no array methods) holding all arguments of a regular function call. Rest parameters give a real array, only of the arguments you didn't name, and also work in arrow functions (which have no `arguments` of their own). Prefer rest. → *More:* §4.4

Q46. How do default parameters work? When are they used? A default (`function f(x = 10)`) applies only when the argument is `undefined` (missing or passed as `undefined`); `null`, `0` and `''` do not trigger it. Defaults are evaluated at call time, left to right, and can refer to earlier parameters (`(a, b = a * 2)`). A parameter with a default (and everything after it) is not counted in `fn.length`. → *More:* §4.4

Q47. How do you find how many parameters a function expects? `fn.length` gives the number of parameters before the first one with a default or a rest parameter: `((a, b = 2, ...c) => 0).length` is `1`. The `curry` implementation uses it to know when enough arguments have arrived.

Q48. What is recursion? Does JavaScript have tail-call optimisation? Recursion is a function calling itself on a smaller input until a **base case** stops it. Each call uses a stack frame, so very deep recursion (around ten thousand frames, depending on the engine) throws `RangeError`. ES2015 specified **proper tail calls** (a call in return position reusing the frame), but only Safari's engine implements it; V8 (Chrome, Node) does not. For deep inputs, use a loop with an explicit stack. → *More:* §4.10 Recursion

Q49. Why should you avoid `eval` and `new Function`? Both run a string as code. `eval` runs it in the current scope, so it can read and change local variables; both are code-injection risks when the string contains user input, block engine optimisations, and are blocked by a strict Content Security Policy. Use `JSON.parse` for data and lookup tables or functions instead of building code from strings. → *More:* §14.12 CSP

Q50. What is a thunk? A thunk is a function that wraps a computation to delay it: `const thunk = () => expensive(a, b)`. Nothing runs until you call it. Async thunks take a callback (or return a promise) and are the idea behind lazy evaluation and the Redux `thunk` middleware (functions dispatched instead of plain action objects).

Q51. What is an anonymous function, and when do you use one? A function without a name, usually a function expression or arrow passed inline: `arr.map((x) => x * 2)`, `setTimeout(function () { … })`, an IIFE. Use it for short one-off callbacks. Name functions that are reused, recursive, or that you want readable in stack traces.

---

4. `this`, `call` / `apply` / `bind`, prototypes and classes

Q52. ⋆ How does `this` work in JavaScript? For regular functions, `this` is decided at **call time** by how the function is called, checked in this order: (1) `new Fn()` — the new object; (2) `fn.call/apply(obj)` or a function made by `bind(obj)` — `obj`; (3) `obj.method()` — `obj`, the object before the dot; (4) a plain call `fn()` — `undefined` in strict mode, the global object in sloppy mode. **Arrow functions** ignore all of this and use `this` from the scope where they were written. In event listeners (regular function), `this` is the element. → *More:* §9.1 `this` — the call-time context

Q53. ⋆ What is the difference between `call`, `apply` and `bind`? All three set `this` explicitly. `fn.call(obj, a, b)` calls immediately with arguments listed; `fn.apply(obj, [a, b])` calls immediately with arguments as an array; `fn.bind(obj, a)` does not call — it returns a new function with `this` (and optionally the first arguments) fixed permanently. Mnemonic: **a**pply = **a**rray; bind = later. → *More:* §9.2 · §17.4.3 Q26

Q54. ⋆ Why does a method lose `this` when passed as a callback? How do you fix it? `setTimeout(user.greet, 100)` passes only the function; when the timer calls it, it is a plain call, so `this` is `undefined` (or global). Fixes: `user.greet.bind(user)`, a wrapper arrow `() => user.greet()`, or define the method as an arrow class field (`greet = () => …`) so it captures the instance. → *More:* §17.4.3 Q23

Q55. What is `this` inside an arrow function defined as an object-literal property? It is **not** the object. An object literal does not create a scope, so the arrow takes `this` from the code around the literal — `undefined` at the top of a module, `window` in a classic script. Use method shorthand (`greet() { … }`) for methods. → *More:* §17.4.3 Q25 · Q226 below

Q56. What happens if you `bind` a function twice, or call a bound function with `new`? The first `bind` wins: `f.bind(a).bind(b)` still has `this === a` (the second bind can still add arguments). `call` / `apply` on a bound function cannot change its `this` either. But `new` beats `bind`: `new (Fn.bind(obj))()` creates a fresh object as `this`, ignoring `obj`. → *More:* §17.4.3 Q27

Q57. ⋆ Explain prototypal inheritance and the prototype chain. Every object has a hidden link, `[[Prototype]]`, to another object (or `null`). When you read a property the object doesn't have, the engine follows that link and looks there, then that object's prototype, and so on until `null`. That chain is how objects share methods: arrays get `map` from `Array.prototype`, which gets `hasOwnProperty` from `Object.prototype`. Writing a property always creates it on the object itself (shadowing the prototype's). Classes are built on exactly this mechanism. → *More:* §9.3 Prototypes and the prototype chain

Q58. ⋆ What is the difference between `prototype` and `__proto__`? `Fn.prototype` is a property of **functions** (constructors and classes): the object that becomes the prototype of instances created with `new Fn()`. `obj.__proto__` is the old accessor for an **object's own** prototype link; use `Object.getPrototypeOf(obj)` / `Object.setPrototypeOf` instead. So `new Fn().__proto__ === Fn.prototype`. → *More:* §17.4.4 Q29

Q59. ⋆ What does the `new` keyword do? `new Fn(args)` (1) creates an empty object whose prototype is `Fn.prototype`; (2) calls `Fn` with `this` set to that object; (3) returns that object — unless `Fn` explicitly returns another **object**, which then replaces it (returning a primitive is ignored). → *More:* §9.4 Constructor functions and `new` · §17.4.3 Q28 · Q265 below

Q60. Explain `function Person(){}`, `const p = Person()` and `const p = new Person()`. The first declares a function (meant as a constructor, by the capital letter convention). `Person()` is a plain call: it returns `undefined` (no `return`), so `p` is `undefined`, and any `this.name = …` inside writes to the global object (sloppy) or throws (strict). `new Person()` creates and returns a new object linked to `Person.prototype`.

Q61. What are the differences between ES6 classes and ES5 constructor functions? Classes are mostly cleaner syntax over the same prototype mechanism, with real differences: a class must be called with `new` (calling it plainly throws), class bodies are always strict mode, class declarations are in the TDZ until their line (not usable early like function declarations), methods are non-enumerable, and classes add `extends` / `super`, `static` members and blocks, fields and `#private` members. Subclassing built-ins like `Array` and `Error` works properly only with classes. → *More:* §9.5 Classes · §17.4.4 Q31

Q62. How does inheritance work with `extends` and `super`? How was it done before ES6? `class Dog extends Animal` links `Dog.prototype` to `Animal.prototype` (instance methods) and `Dog` to `Animal` (static methods). In the subclass constructor you must call `super(args)` before using `this`, because the parent creates the object. `super.method()` calls the parent's version. Before ES6: `Dog.prototype = Object.create(Animal.prototype); Dog.prototype.constructor = Dog;` and call `Animal.call(this, args)` in the `Dog` constructor. → *More:* §9.6 Inheritance with `extends` and `super` · §17.4.4 Q34

Q63. What are static members, private fields and static blocks? `static` members belong to the class itself, not instances: utility functions (`Math`-style helpers), factory methods (`User.fromJSON`), shared counters or caches. `#field` and `#method()` are **private**: accessible only inside the class body, a `SyntaxError` elsewhere; `#field in obj` checks whether an object has it. A `static { … }` block runs once when the class is defined, for complex static initialisation. → *More:* §9.5 Classes

Q64. What are getters and setters? Accessor properties that run a function when read (`get fullName() { … }`) or assigned (`set fullName(v) { … }`), while looking like plain properties to callers. Uses: computed values, validation on assignment, lazy initialisation, keeping a stable API while changing internals. A getter without a setter makes the property read-only (assignment throws in strict mode). → *More:* §5.7 Getters and setters

Q65. ⋆ How does `instanceof` work? `obj instanceof C` walks `obj`'s prototype chain and returns `true` if it meets `C.prototype`. It can be fooled: changing prototypes changes the answer, it fails across realms (an array from an iframe is not `instanceof Array` of your window — use `Array.isArray`), and a class can

override it with `static [Symbol.hasInstance]`. Primitives always give `false`. For errors, `Error.isError(v)` (ES2026, in Node 24) is the cross-realm check. → *More:* §9.7 · §17.4.4 Q30 · Q266 below

Q66. What are the ways to create an object? An object literal `{}`; `new` with a constructor function or class; `Object.create(proto)` (choose the prototype, or `null` for a bare dictionary); a factory function returning a literal; `Object.fromEntries(pairs)`; `structuredClone` or spread of an existing object. Literals and classes cover almost all real code. → *More:* §5.1

Q67. What is the difference between `Object.create()` and `Object.assign()`? `Object.create(proto, descriptors?)` makes a **new empty object whose prototype is `proto`** — it sets up inheritance, copying nothing. `Object.assign(target, ...sources)` **copies own enumerable properties** from sources into an existing target (a shallow copy, running setters on the target) and returns the target. `{ ...a, ...b }` is the modern form of `Object.assign({}, a, b)`. → *More:* §5.6 · §17.4.4 Q32

Q68. What is a mixin? Composition vs inheritance? A mixin is a set of methods copied into a class (or applied as a function `Base => class extends Base {…}`), adding behaviour without a deep inheritance chain. **Composition** builds objects from smaller parts they *have* ("has-a") rather than inheriting what they *are* ("is-a"); it avoids fragile deep hierarchies and is generally preferred. Use inheritance for a true, shallow "is-a" relationship. → *More:* §9.8 Mixins · §9.9 Composition vs inheritance

Q69. What are the pros and cons of extending built-in prototypes (e.g. adding `Array.prototype.last`)? Pro: convenient syntax everywhere. Cons: it changes every array in the program, including in libraries; names can clash with future standard methods (this is why the standard had to call it `flat` instead of `flatten` — a library had already added a different `flatten`); and `for…in` can expose enumerable additions. Only acceptable for spec-compliant polyfills; otherwise write a helper function. → *More:* Q292 below (the LeetCode version)

Q70. How do you implement method chaining? Return `this` from each method that changes state, so the next call works on the same object: `calc.add(5).multiply(2).getResult()`. For immutable chaining (like array `map().filter()`), return a **`new`** instance instead. → *More:* Q300 below

Q71. How do you implement a singleton in JavaScript? The simplest singleton is an ES module that exports one instance: modules are evaluated once and cached, so every importer gets the same object. With a class, keep a `static #instance` and return it from a `static getInstance()` (or from the constructor). Singletons are global state in disguise, so they make tests harder; prefer passing dependencies in. → *More:* §10.3 How ES modules load

Q72. What are `Proxy` and `Reflect`? A `Proxy` wraps an object and intercepts operations on it through **traps** — `get`, `set`, `has`, `deleteProperty`, `apply`, `construct` and more. Uses: validation, default values, logging, change tracking for reactivity (Vue 3 uses proxies), negative array indexes. `Reflect` provides the default behaviour of each operation as a function (`Reflect.get(target, key, receiver)`), so a trap can do its extra work and then forward correctly.

```
const safe = new Proxy({}, { get: (t, k) => (k in t ? t[k] : `no ${String(k)}`) });
console.log(safe.name); // 'no name'
```

Q73. Discuss using object-oriented principles in JavaScript. JavaScript supports the four pillars: **encapsulation** (`#private` fields, closures, modules), **abstraction** (expose a small public API), **inheritance** (prototypes / `extends`), and **polymorphism** (any object with the right method works — "duck typing" — and subclasses override methods). In practice JavaScript code mixes OOP with functional style: classes for things with identity and lifecycle, plain functions and data elsewhere, composition over deep inheritance. → *More:* §9.9 · LLD Handbook

---

5. Objects, arrays, strings and collections

Q74. ⋆ What is the difference between a shallow copy and a deep copy? How do you deep-copy an object? A **shallow copy** (`{ ...obj }`, `Object.assign({}, obj)`, `[...arr]`, `arr.slice()`) copies only the top level; nested objects are still shared, so changing `copy.address.city` changes the original. A **deep copy** copies every level. Use `structuredClone(obj)` (built in; handles `Date`, `Map`, `Set`, typed arrays and circular references, but throws on functions and DOM nodes and drops prototypes/class identity).

`JSON.parse(JSON.stringify(obj))` is lossy: it drops `undefined` and functions, turns dates into strings, and throws on cycles. → *More:* §5.4 Spread, rest and copying · §17.3 Deep clone · §17.4.6 Q43

Q75. Explain mutable vs immutable. How do you make an object immutable? A mutable value can be changed in place (objects, arrays); an immutable one cannot (primitives — string methods always return new strings). Immutability makes change detection cheap (compare references, as React does) and prevents spooky action at a distance; the cost is extra copying. Tools: `const` (prevents reassignment only), `Object.freeze` (shallow — freeze recursively for deep), non-mutating methods (`toSorted`, `with`, spread), or a library like Immer. → *More:* §5.6 · §17.4.6 Q44

Q76. What is the difference between `Object.freeze`, `Object.seal` and `Object.preventExtensions`? And `const` vs `freeze`? `preventExtensions`: no new properties. `seal`: also no deleting, and existing properties become non-configurable — values can still change. `freeze`: also no changing values; everything is read-only. All three are **shallow** (nested objects stay mutable), and violations fail silently in sloppy mode but throw `TypeError` in strict mode. `const` is about the **variable** (cannot be reassigned); `freeze` is about the **object** (cannot be modified). They are independent. → *More:* §5.6

Q77. What are property descriptors? Each property has attributes: a data property has `value`, `writable`, `enumerable` (shows in `for…in` / `Object.keys`), `configurable` (can be deleted or redefined); an accessor property has `get` / `set` instead of `value` / `writable`. Read them with `Object.getOwnPropertyDescriptor(s)`, set them with `Object.defineProperty`. Properties created by `defineProperty` default to all `false`; normal assignment makes them all `true`. → *More:* §5.6

Q78. ⋆ How do you check if a key exists in an object? `Object.hasOwn(obj, 'k')` (ES2022) — own property only, the recommended check. `'k' in obj` — own **or inherited** (so `'toString' in {}` is `true`). `obj.k !== undefined` fails when the value is legitimately `undefined`. `obj.hasOwnProperty('k')` breaks for `Object.create(null)` objects or an object with its own `hasOwnProperty` key — hence `Object.hasOwn`. → *More:* §5.2

Q79. How do you loop over an object's properties? How do you check whether an object is empty? `for (const [k, v] of Object.entries(obj))` is the usual way; `Object.keys` and `Object.values` give keys or values. All three include only **own enumerable string** keys. `for…in` also visits inherited enumerable keys. Empty check: `Object.keys(obj).length === 0` (for a plain object). Order: integer-like keys first in ascending order, then other string keys in insertion order. → *More:* §5.6 · Q290 below

Q80. ⋆ What is the difference between `for…in` and `for…of`? `for…in` loops over the **enumerable property names** (strings) of an object, including inherited ones — meant for plain objects. `for…of` loops over the **values** of an **iterable** (arrays, strings, `Map`, `Set`, generators, `NodeList`) — use it for arrays. Using `for…in` on an array gives string indexes and any extra properties; `for…of` on a plain object throws `TypeError: obj is not iterable`. → *More:* §3.9 · §3.10 · §17.4.6 Q50

Q81. Give examples of object and array destructuring. Destructuring unpacks values into variables. You can rename, set defaults, nest, and collect the rest.

```
const { name, age: years = 0, address: { city } = {}, ...rest } = user;
const [first, , third = 'n/a', ...others] = list;
[a, b] = [b, a];                          // swap without a temp variable
function draw({ x = 0, y = 0 } = {}) {}     // named options with defaults
```

→ *More:* §5.5 · §7.7

Q82. ⋆ What are `JSON.stringify` and `JSON.parse`? What does `stringify` drop? `JSON.stringify(value, replacer?, indent?)` turns a value into a JSON string; `JSON.parse(text, reviver?)` turns JSON text back into values. `stringify` skips properties whose value is `undefined`, a function or a symbol (in arrays they become `null`), turns `NaN` / `Infinity` into `null`, calls `toJSON` (so dates become ISO strings), throws on `BigInt` and on circular references. A **replacer** filters or transforms values on the way out; a **reviver** transforms them on the way in (e.g. turning date strings back into `Date`). → *More:* §5.12 JSON · §17.4.6 Q49

Q83. How do you handle circular references when serializing? `JSON.stringify` throws `TypeError: Converting circular structure to JSON`. Options: use `structuredClone` (if you only need a copy), pass a replacer that tracks seen objects in a `WeakSet` and replaces repeats with a marker (simple, but it also marks an object that is merely referenced twice, not only true cycles), or restructure the data to use IDs instead of back-references.

```
const seen = new WeakSet();
JSON.stringify(obj, (k, v) => (typeof v === 'object' && v !== null ? (seen.has(v) ? '[Circular]' : (seen.add(v), v)) : v));
```

Q84. ⋆ When do you use a `Map` instead of an object? And `Set`? Use a `Map` for a dictionary with keys of any type (objects, numbers kept as numbers), frequent adds and deletes, a reliable `size`, guaranteed insertion order and no inherited keys (`'constructor'` can't clash). Use an object for fixed-shape records and JSON. A `Set` stores unique values (by `SameValueZero`, so `NaN` works) with O(1) `has`; ES2025 added set algebra: `union`, `intersection`, `difference`, `symmetricDifference`, `isSubsetOf`, `isSupersetOf`, `isDisjointFrom`. → *More:* §8.1 `Map` · §8.2 `Set` · §17.4.6 Q48

Q85. ⋆ What are `WeakMap` and `WeakSet`? How do they differ from `Map` and `Set`? Their keys must be objects (or non-registered symbols), and they hold them **weakly**: if nothing else references a key, it can be garbage-collected and its entry disappears. Therefore they are not iterable and have no `size` or `clear`. Use them to attach data to objects you don't own (DOM nodes, cache per object, private data) without causing memory leaks. → *More:* §8.3 · §17.4.7 Q53

Q86. ⋆ What is the difference between `map` and `forEach`? `map` returns a **new array** of the callback's return values (same length); use it to transform. `forEach` returns `undefined` and is only for side effects (logging, writing to the DOM). Neither can be stopped early with `break` (use `for…of`, `some` or `find`), and neither waits for `async` callbacks. → *More:* §7.5 Iteration methods

Q87. ⋆ What is the difference between `slice` and `splice`? `slice(start, end)` returns a **copy** of a portion (end exclusive), leaving the original untouched; negative indexes count from the end. `splice(start, deleteCount, ...items)` **mutates** the array — removing and/or inserting in place — and returns the removed elements. ES2023's `toSpliced` is the non-mutating version of `splice`. → *More:* §7.4 The methods table

Q88. Which array methods mutate the array? What are the non-mutating alternatives? Mutating: `push`, `pop`, `shift`, `unshift`, `splice`, `sort`, `reverse`, `fill`, `copyWithin` (and setting `length`). ES2023 added copies: `toSorted`, `toReversed`, `toSpliced`, and `with(i, v)` (copy with one element replaced). Non-mutating methods also include `map`, `filter`, `slice`, `concat`, `flat`. This matters for React state and for functions that receive arrays they don't own. → *More:* §7.4

Q89. Explain `reduce` with examples. `arr.reduce((acc, item, i, arr) => newAcc, initial)` walks the array carrying an **accumulator**; each step returns the next accumulator, and the last one is the result. Always pass `initial` (without it, the first element is used, and an empty array throws `TypeError`).

```
const total = cart.reduce((sum, item) => sum + item.price * item.qty, 0);
const countBy = words.reduce((m, w) => ((m[w] = (m[w] ?? 0) + 1), m), {});
```

→ *More:* §7.5

Q90. ⋆ How do you sort numbers correctly? Is `sort` stable? Default `sort()` converts elements to strings and compares UTF-16 code units, so `[10, 1, 2].sort()` gives `[1, 10, 2]`. Pass a comparator: `(a, b) => a - b` ascending; for strings use `a.localeCompare(b)`; for objects compare a field. Since ES2019, `sort` is required to be **stable** (equal elements keep their order), so you can sort by a secondary key first, then a primary key. `sort` mutates; use `toSorted` for a copy. → *More:* §7.6 Sorting correctly · §17.4.1 Q11

Q91. ⋆ How do you remove duplicates from an array? Primitives: `[...new Set(arr)]` (or `Array.from(new Set(arr))`). Objects by a key: `[...new Map(arr.map((o) => [o.id, o])).values()]` keeps the **last** object per id (in the position where that id first appeared); to keep the **first**, filter with a `Set` of seen ids — `const seen = new Set(); arr.filter((o) => !seen.has(o.id) && seen.add(o.id))`. `arr.filter((x, i) => arr.indexOf(x) === i)` works but is O(n²). → *More:* §8.2

Q92. What are `find`, `findIndex`, `findLast`, `some`, `every`, `includes` and `indexOf` for? `find`/`findIndex` return the first matching element/index (`undefined`/`-1` if none); `findLast`/`findLastIndex` (ES2023) search from the end. `some` is `true` if any element matches, `every` if all do (an empty array gives `true` for `every`). `includes(x)` checks presence using SameValueZero, so it finds `NaN`; `indexOf` uses `===` and cannot. → *More:* §7.5

Q93. How do you empty an array? What do `delete arr[i]` and `arr.length = 0` do? `arr.length = 0` empties the array in place (every reference sees it empty). `arr = []` makes the variable point to a new array (others still see the old one; impossible with `const`). `arr.splice(0)` also empties in place and returns

the removed items. `delete arr[i]` leaves a **hole** (`length` unchanged); use `splice` to remove. → *More:* §17.4.6 Q46

Q94. What are sparse arrays? What is the difference between `Array(3)` and `Array.of(3)`? A sparse array has **holes** — indexes with no element at all (`[1, , 3]`, `Array(3)`, `delete arr[1]`, or writing past the end). `forEach` / `map` / `filter` skip holes; `for…of` and spread treat them as `undefined`. `Array(3)` makes an empty array of length 3; `Array(3, 4)` and `Array.of(3)` make arrays of those elements. To create filled arrays use `Array.from({ length: 3 }, (_, i) => i)` or `Array(3).fill(0)`. → *More:* §7.2 Creating arrays

Q95. What is an array-like object? How do you convert it to an array? An object with numeric keys and a `length`, but without array methods — `arguments`, a `NodeList` from older APIs, a `{ length: 2, 0: 'a', 1: 'b' }`. Convert with `Array.from(x)` (works for array-likes and iterables, and takes a map function) or `[...x]` (iterables only). → *More:* §7.2

Q96. What is an iterator? How do you make an object iterable? An **iterator** is an object with a `next()` method that returns `{ value, done }`. An **iterable** is an object with a `[Symbol.iterator]()` method returning an iterator; `for…of`, spread, destructuring, `Array.from`, `Promise.all` and `new Map()` all consume iterables. Easiest way to make one: a generator method.

```
class Range { constructor(a, b) { this.a = a; this.b = b; }
  *[Symbol.iterator]() { for (let i = this.a; i <= this.b; i++) yield i; } }
console.log([...new Range(1, 3)]); // [1, 2, 3]
```

→ *More:* §8.4

Q97. What are generators and when would you use them? A generator function (`function*`) returns an iterator; each `yield` pauses the function and hands out a value, and the next `next()` resumes it (and `next(x)` makes the paused `yield` evaluate to `x`). Uses: lazy sequences (including infinite ones), custom iteration, paging through results, and (historically) async control flow before `async` / `await`. **Async generators** (`async function*`) yield promises' results and are consumed with `for await…of`. → *More:* §8.5 Generators · §12.12

Q98. What are iterator helpers (ES2025)? Methods on iterators — `map`, `filter`, `take`, `drop`, `flatMap`, `reduce`, `toArray`, `forEach`, `some`, `every`, `find` — plus `Iterator.from`. Unlike array methods, they are **lazy**: they process one element at a time and stop early, so they work on huge or infinite sequences without building intermediate arrays: `naturals().filter(isPrime).take(5).toArray()`. → *More:* §8.6 Iterator helpers (ES2025)

Q99. What do `Object.groupBy` and `Map.groupBy` do? ES2024. `Object.groupBy(items, fn)` returns a null-prototype object whose keys are the callback's return values and whose values are arrays of the items in each group; `Map.groupBy` returns a `Map`, so keys can be any value (objects, numbers). They replace the classic `reduce` grouping. → *More:* §8.7 · Q293 below

Q100. Are strings mutable? `slice` vs `substring` vs `substr`? Strings are immutable: every method returns a new string and `s[0] = 'x'` does nothing. `slice(start, end)` accepts negative indexes (from the end). `substring(start, end)` treats negatives as 0 and swaps the arguments if `start > end`. `substr(start, length)` is a deprecated legacy method; avoid it. → *More:* §6.4 · §6.6 String methods

Q101. How do you reverse a string? What goes wrong with emoji? `s.split('').reverse().join('')` works for plain ASCII but splits emoji and other characters outside the Basic Multilingual Plane, because `split('')` works on UTF-16 **code units** and those characters use two (`'👍'.length` is 2). `[...s].reverse().join('')` splits by **code points** and handles them; for user-perceived characters (flags, combined emoji, accents) use `Intl.Segmenter`. → *More:* §6.5 Unicode

Q102. What are template literals and tagged templates? Template literals are backtick strings with `${expression}` interpolation and real line breaks. A **tagged template** `tag`text ${x}`` calls `tag(strings, ...values)` with the literal parts and the evaluated values separately, so the function can escape, translate or build something else — used for HTML escaping, SQL parameter binding, styled-components and `String.raw` (no escape processing). → *More:* §6.3 Template literals

Q103. What are the common pitfalls of `Date`? What is Temporal? `Date` months are 0-based (`new Date(2026, 0, 1)` is January), objects are mutable, parsing non-ISO strings is unreliable, and date-only ISO strings (`'2026-01-01'`) are read as UTC while date-time strings without an offset are local time. Time zones other than local/UTC are hard. **Temporal** is the replacement API with immutable types (`PlainDate`,

`ZonedDateTime`, `Instant`, `Duration`); it is finished (Stage 4) for ES2027 and ships in Firefox 139+, Chrome/Edge 144+ and Node 26, but not in Node 24 or Safari yet, so use a polyfill there. For formatting, use `Intl.DateTimeFormat`. → *More:* §5.11 `Date`

Q104. How do you generate a random integer between `min` and `max` (inclusive)? `Math.floor(Math.random() * (max - min + 1)) + min`. `Math.random()` returns a float in [0, 1). It is not cryptographically secure: for tokens or IDs use `crypto.getRandomValues()` or `crypto.randomUUID()`. → *More:* §5.10 `Math`

---

6. The event loop, promises and `async` / `await`

Q105. ⋆ JavaScript is single-threaded — so how does it do asynchronous work? Your JavaScript runs on one thread with one **call stack**: one thing at a time. But the **host** (browser or Node) does the slow work — timers, network, file I/O — outside that thread, and when it finishes it queues a callback. The **event loop** runs queued callbacks one by one whenever the call stack is empty. So JavaScript never blocks waiting for I/O, but a long synchronous computation does block everything (clicks, rendering, other requests). → *More:* §12.1 · §12.2 The event loop

Q106. ⋆ Explain the event loop. What are the call stack, the task queue and the microtask queue? The **call stack** holds the functions running right now. The **task (macrotask) queue** holds callbacks from timers, I/O, UI events and messages. The **microtask queue** holds promise reactions (`then` / `catch` / `finally` callbacks, code after `await`), `queueMicrotask` callbacks and `MutationObserver` callbacks. The loop: run one task to completion; then run **all** microtasks, including ones added meanwhile; then (in browsers) render if needed; repeat. So all pending promise callbacks run before the next timer. → *More:* §12.2 · §12.3 Ordering puzzles · §17.4.5 Q35

Q107. ⋆ What is the difference between microtasks and macrotasks? Give examples of each. Microtasks: promise callbacks, `await` continuations, `queueMicrotask`, `MutationObserver`; in Node, `process.nextTick` runs in an even earlier queue. Macrotasks (tasks): `setTimeout`, `setInterval`, `setImmediate` (Node), I/O callbacks, UI events, `MessageChannel` messages. After every task the microtask queue is drained completely — which is why an infinite chain of microtasks can freeze the page just like a `while (true)` loop, while timers yield. → *More:* §12.2

Q108. ⋆ What does `setTimeout(fn, 0)` really do? It queues `fn` as a task to run **at least** 0 ms later — after the current code and all microtasks finish, and after any earlier tasks. Browsers clamp nested timers (more than five levels deep) to at least 4 ms, and background tabs to 1 s or more. The delay is a minimum, never a guarantee. To run something after the current code but before rendering or timers, use `queueMicrotask`. → *More:* §12.5 Timers

Q109. ⋆ What is a promise? What are its states? A promise is an object that stands for the eventual result of an async operation. It is **pending**, then becomes **settled** exactly once: **fulfilled** with a value or **rejected** with a reason (an error); after that it never changes. You react with `then(onFulfilled, onRejected)`, `catch`, `finally`, or `await`. Callbacks attached after settlement still run (asynchronously), which fixes the "missed the event" problem of callbacks. → *More:* §12.7 Promises

Q110. What are the pros and cons of promises over callbacks? Pros: flat chaining instead of nesting, one `catch` for a whole chain, guaranteed async and at-most-once callbacks (no "called twice" or "called synchronously" bugs), composition with `Promise.all` and friends, and `async` / `await` syntax. Cons: one-shot (a stream of events needs callbacks, async iterators or observables), not cancellable by themselves (use `AbortController`), and an unhandled rejection is easy to miss. → *More:* §12.6 · §12.7

Q111. ⋆ How does promise chaining work? `then` always returns a **new** promise. Whatever the callback returns becomes that promise's value; if it returns a promise, the chain waits for it; if it throws, the new promise rejects and control skips to the next `catch`. A `catch` that returns normally **recovers** — the chain continues fulfilled. Forgetting `return` inside a `then` is the classic bug: the chain moves on without waiting. → *More:* §12.7 · Q222 below

Q112. ⋆ What is the difference between `Promise.all`, `allSettled`, `race` and `any`? `all`: fulfils with an array of all values (in input order) when all fulfil; rejects as soon as **one** rejects. `allSettled`: waits for all and gives `{ status, value | reason }` for each — never rejects. `race`: settles like the **first** promise to settle, success or failure. `any`: fulfils with the first **success**; rejects with an

`AggregateError` only if all reject. Typical uses: parallel requests (`all`), batch with partial failures (`allSettled`), timeout (`race`), fastest mirror (`any`). → *More:* §12.8 · §17.4.5 Q42

Q113. Does `Promise.all` cancel the other promises when one rejects? No. A promise is only a result holder; the operations behind the other promises keep running and their results are ignored. To actually stop them, pass one `AbortSignal` to all of them and call `controller.abort()` in the `catch`. → *More:* §12.11 Cancellation

Q114. ⋆ How do `async` and `await` work? An `async` function always returns a promise: its `return` value fulfils it and a `throw` rejects it. `await p` pauses **that function only** until `p` settles, then resumes with the value (or throws the rejection, so `try`/`catch` works). The rest of the function is scheduled as a microtask; the caller and the event loop keep running meanwhile. It is syntax over promises, not threads. → *More:* §12.9 `async` / `await` · §17.4.5 Q36

Q115. ⋆ How do you run async operations in parallel vs in sequence with `await`? `const a = await f(); const b = await g();` is **sequential**: `g` starts only after `f` finishes (total ≈ sum). For independent operations, start both, then await together: `const [a, b] = await Promise.all([f(), g()]);` (total ≈ the slower). For a list processed one by one, use `for…of` with `await`; for a list in parallel with a limit, use a concurrency pool. → *More:* §17.4.5 Q41 · Q264 below

Q116. ⋆ Why doesn't `await` work inside `forEach`? `forEach` calls the async callback and ignores the promise it returns, so the loop finishes immediately and the code after it runs before any callback completes; errors become unhandled rejections. Use `for…of` (sequential) or `await Promise.all(items.map(async (x) => …))` (parallel). → *More:* §17.4.5 Q40

Q117. ⋆ How do you handle errors with promises and `async`/`await`? What is an unhandled rejection? With promises, end each chain with `.catch()`; with `async`/`await`, wrap awaits in `try`/`catch` (and use `finally` for cleanup). A rejection no handler ever catches is an **unhandled rejection**: browsers fire an `unhandledrejection` event and log it; Node prints the error and **exits with code 1** (the default since Node 15). Common causes: a missing `return` inside `then`, an un-awaited async call, `forEach` with async callbacks. → *More:* §12.10 Error handling across async code · §11.5

Q118. How do you cancel a `fetch` request or add a timeout? Create an `AbortController`, pass `controller.signal` to `fetch(url, { signal })`, and call `controller.abort()`; the `fetch` promise rejects with an `AbortError` `DOMException`. For a timeout use `AbortSignal.timeout(5000)` (rejects with a `TimeoutError`), and combine signals with `AbortSignal.any([a, b])`. The same signal works for event listeners, many Node APIs and your own functions. → *More:* §12.11

Q119. What is `queueMicrotask`? How does `process.nextTick` compare with `setImmediate`? `queueMicrotask(fn)` schedules `fn` in the microtask queue, like a resolved promise's `then` but without creating a promise. In Node, `process.nextTick(fn)` runs even sooner — its queue is drained before the promise microtasks. `setImmediate(fn)` is a macrotask that runs in the "check" phase, after I/O polling. Inside an I/O callback, `setImmediate` always runs before `setTimeout(fn, 0)`; in the main module their order is not guaranteed. → *More:* §12.4 Node's event loop · §17.4.5 Q37

Q120. What are `Promise.withResolvers` and `Promise.try`? `Promise.withResolvers()` (ES2024) returns `{ promise, resolve, reject }`, so you can settle a promise from outside an executor — handy for wrapping event-based APIs or building queues. `Promise.try(fn)` (ES2025) calls `fn` immediately and always returns a promise: a sync return fulfils it and a sync throw rejects it, so sync and async errors go to the same `catch`. → *More:* §12.7

Q121. When does a promise executor run? What if `resolve` is called twice? The executor function passed to `new Promise` runs **synchronously**, immediately, during the constructor. Only the first call to `resolve` or `reject` counts; later calls are ignored silently. Throwing inside the executor rejects the promise. → *More:* §17.4.5 Q38

Q122. What is `for await…of`? A loop over an **async iterable** — something whose `next()` returns promises, like an async generator, a Node readable stream, or paginated API results. Each iteration awaits the next value. It also accepts an array of promises, awaiting them in order (but they already run in parallel). `Array.fromAsync` (ES2026, in Node 24) collects one into an array. → *More:* §12.12

Q123. What is the difference between promises and observables? A promise delivers **one** value, is **eager** (the work started before you got it) and cannot be cancelled by itself. An observable (RxJS) delivers **zero to many** values over time, is **lazy** (nothing runs until someone subscribes), can be cancelled by

unsubscribing, and has operators like `map`, `debounceTime`, `switchMap`. Observables suit event streams; promises suit single requests.

Q124. What is the difference between synchronous and asynchronous functions? A synchronous function does all its work before returning; the caller waits, and the thread is blocked meanwhile. An asynchronous function starts the work and returns at once (usually a promise); the result arrives later through a callback or promise, while the thread stays free for other events. → *More:* §12.1

---

7. Errors, modules and tooling

Q125. ⋆ How do `try` / `catch` / `finally` work? What if `finally` has a `return`? `try` runs code; if it throws, `catch (err)` runs (the binding is optional since ES2019); `finally` always runs afterwards — after a normal finish, a `return`, a `break` or an exception — for cleanup. A `return` (or `throw`) in `finally` **overrides** the one from `try` / `catch`, and swallows a pending exception, so never return from `finally`. `try` / `catch` only catches errors thrown synchronously inside it or from awaited promises. → *More:* §11.2 · Q238 below

Q126. What are the built-in error types? `Error` (base), `SyntaxError` (invalid code or `JSON.parse` input), `ReferenceError` (unknown name, TDZ), `TypeError` (wrong kind of value: calling a non-function, reading a property of `undefined`), `RangeError` (value out of range, stack overflow), `URIError`, `EvalError`, and `AggregateError` (several errors, from `Promise.any`). Every error has `name`, `message` and a non-standard but universal `stack`. → *More:* §11.1

Q127. How do you create a custom error class? Extend `Error`, call `super(message, { cause })`, set `this.name`, and add any extra fields. Callers can then use `instanceof` or check `name` / `code`.

```
class HttpError extends Error {
  constructor(status, message, options) { super(message, options); this.name = 'HttpError'; this.status = status;
}
}
throw new HttpError(404, 'User not found', { cause: originalError });
```

→ *More:* §11.3 Custom errors · §11.4 `cause`

Q128. Why doesn't `try` / `catch` catch an error thrown inside a `setTimeout` callback? By the time the callback runs, the `try` block has finished and its stack frame is gone; the callback runs later from the event loop with a fresh stack. Put the `try` / `catch` inside the callback, or wrap the timer in a promise and `await` it. → *More:* §17.4.5 Q39

Q129. How do you catch errors globally? Browser: `window.addEventListener('error', …)` for uncaught exceptions (and failed resource loads, with capture) and `window.addEventListener('unhandledrejection', …)` for promises. Node: `process.on('uncaughtException')` and `process.on('unhandledRejection')` — use them to log and then exit, because the process may be in an unknown state; restart via a process manager. Error monitoring services (like Sentry) hook into these. → *More:* §11.5 · §12.10

Q130. What tools and techniques do you use to debug JavaScript? `console.log` / `table` / `dir` / `trace` / `time`; the `debugger` statement; DevTools breakpoints (line, conditional, logpoints, DOM-change, XHR/fetch, exception breakpoints), the call stack and scope panes, the Network and Performance panels; source maps (files that map built code back to the original source) to debug bundled code; `node --inspect` with Chrome DevTools or VS Code for Node; and a failing test that reproduces the bug. → *More:* §11.7 · §11.9

Q131. What are the performance implications of `try` / `catch`? In modern engines a `try` / `catch` costs essentially nothing when no error is thrown. **Throwing** is expensive (an error object and stack trace are built), so don't use exceptions for normal control flow in hot loops — return a value instead.

Q132. What is `using` (explicit resource management)? `using x = resource` declares a variable whose `[Symbol.dispose]()` method is called automatically when the block ends, even after an exception; `await using` calls `[Symbol.asyncDispose]()`. It replaces `try` / `finally` cleanup for files, locks and connections. The proposal is finished for ES2027 and already works in Node 24. → *More:* §11.6 Guaranteed cleanup

Q133. ⋆ What is the difference between ES modules and CommonJS? CommonJS (Node's original system): `require()` / `module.exports`, loaded synchronously at run time, exports are a copied value, `require` can be called anywhere. **ES modules** (the standard): `import` / `export` declared statically at the top, so tools

know the dependency graph before running (tree shaking, browser support); loaded asynchronously; exports are **live bindings**; strict mode always; top-level `await`; `this` is `undefined` at top level. Node picks the system by extension (`.mjs` / `.cjs`) or the `"type"` field in `package.json`. ESM can import CJS; Node 22.12+ can also `require()` ESM unless the module uses top-level `await`. → *More:* §10.8 CommonJS vs ES modules in Node

Q134. Named exports vs default exports? A module can have many named exports (`export const a`, imported as `import { a } from`) and at most one default (`export default`, imported under any name). Named exports keep names consistent across the codebase, are easier for auto-import and refactoring tools, and catch typos at link time; many teams prefer them. Default exports are common for "one main thing per file" (React components, config). → *More:* §10.2

Q135. What are live bindings in ES modules? An import is a read-only view of the exporting module's variable, not a copy. If the exporter later changes `export let count`, every importer sees the new value. Importers cannot assign to it (`TypeError`). In CommonJS, `const { count } = require(…)` copies the value at that moment. → *More:* §10.3

Q136. What is dynamic `import()`, and what are its use cases? `import('./module.js')` loads a module at run time and returns a promise of its namespace object. It works in CommonJS and classic scripts too. Uses: **code splitting** (load a heavy chart library only when the chart tab opens), conditional or locale-based loading, lazy routes in SPAs (`React.lazy`), loading ESM from CommonJS. Non-JavaScript modules need an **import attribute** (ES2025): `import config from './config.json' with { type: 'json' };` or `import('./config.json', { with: { type: 'json' } })`. → *More:* §10.4

Q137. What is top-level `await`? In an ES module you can use `await` outside any function (ES2022). The module's evaluation pauses until it settles, and modules that import it wait too — so use it for setup (loading config, a DB connection, a dynamic import), not for slow work. Before it, the workaround was an async IIFE. A module using it cannot be loaded with `require()`. → *More:* §10.5 Top-level `await`

Q138. What happens with circular imports? ES modules allow cycles: the graph is linked first, then evaluated depth-first, so one module in the cycle runs before the other has finished; reading a not-yet-initialised `let` / `const` / `class` export throws a TDZ `ReferenceError` (function declarations are hoisted and work). In CommonJS, the second module gets the first one's **partially filled** `module.exports`. Fix by moving shared code into a third module or deferring access into functions. → *More:* §10.3

Q139. How do you get `__dirname` and `__filename` in an ES module? They don't exist in ESM. Use `import.meta.dirname` and `import.meta.filename` (Node 20.11+), or `fileURLToPath(import.meta.url)`. `import.meta.url` is the module's own `file://` URL; `new URL('./data.json', import.meta.url)` resolves a file next to the module. → *More:* §10.6 `import.meta` and JSON modules · §15.5

Q140. What is tree shaking? A bundler optimisation that removes exports nobody imports. It relies on ES modules' **static** `import` / `export` (the bundler sees exactly what is used) and on modules being free of side effects at import time (`"sideEffects": false` in `package.json` tells the bundler it may drop unused files). CommonJS, `import * as` used dynamically, and code with top-level side effects defeat it. → *More:* §10.11 Bundlers

Q141. What do bundlers, transpilers and polyfills do? Shim vs polyfill? A **bundler** (Vite, webpack, esbuild, Rollup) follows imports and produces a few optimised files: minified, split, tree-shaken, with source maps. A **transpiler** (Babel, TypeScript, SWC, esbuild) rewrites new **syntax** into older syntax (or TS into JS). A **polyfill** adds a missing **API** at run time using existing features (`Array.prototype.at` written in JS). A **shim** is the broader term: any library that fills in or adapts an API, possibly with a different interface. → *More:* §10.11 · §10.12 Transpilers and polyfills

Q142. What do `^` and `~` mean in `package.json`? Why commit the lockfile? Versions follow **semver**: `MAJOR.MINOR.PATCH` (breaking, feature, fix). `^1.4.2` allows `>=1.4.2 <2.0.0` (any non-breaking update); `~1.4.2` allows `>=1.4.2 <1.5.0` (patches only). (For `0.x`, `^0.4.2` means `<0.5.0`.) The lockfile (`package-lock.json`) records the exact resolved version of every package in the tree, so every install and CI build gets identical code; use `npm ci` in CI. → *More:* §10.10 Semantic versioning, ranges and lockfiles

Q143. What are minification and obfuscation? Minification removes whitespace and comments and shortens local names to make files smaller and faster to download; it is a normal build step (Terser, esbuild). **Obfuscation** deliberately makes code hard to read to deter copying; it is not security — anything shipped to the browser can be read — and it is not encryption, which needs a key to reverse. → *More:* §10.11

8. The DOM and events

Q144. What is the DOM? What is the difference between `window` and `document`? The DOM (Document Object Model) is the browser's live tree of objects representing the HTML page; JavaScript reads and changes the page through it. `window` is the global object for the browser tab: timers, `location`, `history`, `localStorage`, viewport size — these browser-level APIs are called the BOM (Browser Object Model). `document` (`window.document`) is the root of the DOM tree for the loaded page. → *More:* §13.1 The DOM tree

Q145. ⋆ Explain event bubbling and capturing (the event flow). A DOM event travels in three phases: **capture** — from `window` down through the ancestors to the target; **target** — at the element itself; **bubble** — back up from the target to `window`. Listeners run in the bubble phase by default; pass `{ capture: true }` to run in the capture phase. Most events bubble; some don't (`focus`, `blur`, `mouseenter`, `load` — use `focusin` / `focusout` to get bubbling focus events). → *More:* §13.6 Propagation

Q146. ⋆ What is event delegation, and why use it? Instead of a listener on every child, put one listener on a common ancestor and use `event.target.closest(selector)` to find which child was hit. It works because events bubble. Benefits: one listener instead of thousands (less memory), and it automatically works for children added later. Check the matched element is inside the container, and remember non-bubbling events need capture.

```
list.addEventListener('click', (e) => {
  const btn = e.target.closest('button.delete');
  if (btn && list.contains(btn)) removeItem(btn.dataset.id);
});
```

→ *More:* §13.7 Event delegation

Q147. ⋆ What is the difference between `event.target` and `event.currentTarget`? `target` is the element where the event originated (the innermost element clicked). `currentTarget` is the element whose listener is running now — the one you called `addEventListener` on. With delegation they differ; `currentTarget` is `null` after the handler returns (so log it inside the handler). → *More:* §13.6

Q148. ⋆ `preventDefault()` vs `stopPropagation()` vs `stopImmediatePropagation()` vs `return false`? `preventDefault()` cancels the browser's **default action** (following a link, submitting a form, checking a checkbox) but the event still propagates. `stopPropagation()` stops the event moving to further elements, but other listeners on the current element still run. `stopImmediatePropagation()` also stops those. `return false` does nothing in `addEventListener`; only in an inline `onclick` property handler does it act like `preventDefault()` (and in jQuery, like both). → *More:* §13.8 Default actions and `preventDefault` · §13.6

Q149. What is the difference between an attribute and a property? An **attribute** is written in the HTML and lives on the element as a string (`getAttribute('value')`). A **property** is a field of the DOM object (`input.value`), with a real type. Many are linked at first, but some diverge: for an `<input>`, the `value` attribute is the **initial** value while the `value` property is the **current** text the user typed. `data-*` attributes map to `element.dataset`. → *More:* §13.4

Q150. ⋆ What is the difference between `DOMContentLoaded` and `load`? Between `async` and `defer` scripts? `DOMContentLoaded` fires when the HTML is parsed and the DOM is built (deferred scripts have run); `load` fires later, when images, stylesheets and iframes have also finished. A plain `<script>` blocks parsing while it downloads and runs. `defer` downloads in parallel and runs after parsing, in document order — the right default for app scripts. `async` downloads in parallel and runs as soon as it arrives, in any order — for independent scripts like analytics. Module scripts are deferred by default. → *More:* §13.12 When scripts run

Q151. `querySelectorAll` vs `getElementsByClassName`: what is the difference? `querySelectorAll` takes any CSS selector and returns a **static** `NodeList` (a snapshot; later DOM changes don't affect it), which has `forEach`. `getElementsByClassName` / `getElementsByTagName` return a **live** `HTMLCollection` that updates as the DOM changes (so looping while removing elements can skip items) and has no `forEach`. `querySelector` returns the first match or `null`. → *More:* §13.2 Selecting and traversing

Q152. ⋆ `innerHTML` vs `textContent` vs `innerText`? `innerHTML` reads/writes the element's HTML markup — setting it parses HTML, so untrusted input causes XSS, and it destroys and recreates child nodes (losing listeners). `textContent` reads/writes raw text of all descendants, including hidden ones, and

never parses HTML — use it for user data. `innerText` returns the **rendered** text (respects CSS, skips hidden elements) and forces a layout to compute it, so it is slower. → *More:* §13.3 · §14.10 XSS

Q153. How do you add many elements to the page efficiently? Build them off-page and insert once: append to a `DocumentFragment` (a lightweight container whose children move into the DOM in one operation) or call `parent.append(...nodes)` / `replaceChildren(...nodes)` once. This causes one layout instead of one per element. For very long lists, render only the visible rows (virtualisation). → *More:* §13.3 · §13.11

Q154. ⋆ What is the difference between reflow and repaint? What is layout thrashing? Reflow (layout) recalculates the size and position of elements — triggered by changing geometry (width, font, adding nodes). **Repaint** redraws pixels without layout changes (colour, visibility). Reflow is more expensive and may cascade. **Layout thrashing** is interleaving DOM writes with reads of layout properties (`offsetHeight`, `getBoundingClientRect`) in a loop, which forces a synchronous reflow each time. Fix: batch all reads, then all writes; animate `transform` / `opacity` (often handled by the compositor without layout or paint); use `requestAnimationFrame`. → *More:* §13.11

Q155. Why remove event listeners? How? A listener keeps its callback — and everything the callback closes over — alive while the target exists, so listeners on long-lived targets (`window`, `document`) from removed components leak memory and keep running. Remove with `removeEventListener(type, sameFunctionReference, sameCaptureFlag)` (an inline arrow can't be removed), use `{ once: true }` for one-shot listeners, or pass `{ signal }` and call `controller.abort()` to remove many at once. → *More:* §13.5 Events and listeners

Q156. What are passive event listeners? `addEventListener('touchstart', fn, { passive: true })` promises the browser that `fn` will not call `preventDefault()`, so it can start scrolling immediately instead of waiting for the handler. Browsers make `touchstart` / `touchmove` / `wheel` listeners on `window`, `document` and `body` passive by default. → *More:* §13.5

Q157. How do you create and dispatch a custom event? `el.dispatchEvent(new CustomEvent('cart:add', { detail: { id: 7 }, bubbles: true }))`, and listen with `addEventListener('cart:add', (e) => e.detail.id)`. Set `bubbles: true` if ancestors should hear it (custom events don't bubble by default). Dispatch is **synchronous**: listeners run before `dispatchEvent` returns. Plain `EventTarget` can be subclassed for non-DOM event buses. → *More:* §13.5

Q158. What is `MutationObserver`? An API that calls your callback (as a microtask, batched) when a DOM subtree changes: children added/removed, attributes changed, text changed. It replaced the slow, synchronous mutation events. Uses: reacting to DOM changes made by third-party code, auto-initialising widgets inserted later. Call `disconnect()` when done. → *More:* §13.10

Q159. How do you handle forms with JavaScript (submit, validation)? Listen for `submit` on the form (not `click` on the button, so the Enter key works too), call `preventDefault()` to stop the page reload, read values with `new FormData(form)`, and send them with `fetch`. Use HTML constraints (`required`, `type="email"`, `pattern`, `min`) and the Constraint Validation API (`checkValidity()`, `reportValidity()`, `setCustomValidity()`, `input.validity`) before custom code. Always validate again on the server. → *More:* §13.9 Forms

Q160. What are Web Components and the Shadow DOM? Web Components are the browser's native component model: **custom elements** (`class MyCard extends HTMLElement`, registered with `customElements.define('my-card', MyCard)`), **Shadow DOM** (`this.attachShadow({ mode: 'open' })` — a hidden subtree whose styles and IDs are isolated from the page), and `<template>` / `<slot>` for content. They work with any framework or none.

---

9. Browser APIs, storage, networking and security

Q161. ⋆ What are the differences between cookies, `localStorage` and `sessionStorage`? Cookies (~4 KB each) are sent to the server with every matching request; they can be `HttpOnly` (invisible to JavaScript), `Secure`, `SameSite`, and expire when you choose — use them for sessions. **`localStorage`** (~5 MB per origin, strings only, synchronous) persists until cleared and is never sent automatically. **`sessionStorage`** is the same but per tab, cleared when the tab closes. JavaScript can read both storages, so never put tokens or secrets there if XSS is possible. For large or structured data use IndexedDB. → *More:* §14.3 Web Storage · §14.4 Cookies and IndexedDB

Q162. What is the `storage` event? When one tab changes `localStorage`, every **other** tab of the same origin receives a `storage` event on `window` with `key`, `oldValue`, `newValue`. It is a simple way to sync tabs (e.g. log out everywhere); `BroadcastChannel` is the more general tool. → *More:* §14.3

Q163. What is IndexedDB, and when would you use it? An asynchronous, transactional key-value database in the browser that stores structured data (objects, files, blobs) with indexes, and can hold far more than `localStorage` (a share of free disk space). Use it for offline apps, caching API data, and drafts. Its raw API is event-based and verbose, so most people use a small promise wrapper. → *More:* §14.4

Q164. What do the cookie attributes `HttpOnly`, `Secure` and `SameSite` do? `HttpOnly` hides the cookie from `document.cookie`, so an XSS script cannot steal it. `Secure` sends it only over HTTPS. `SameSite` controls sending on cross-site requests: `Strict` never, `Lax` (the browsers' default when the attribute is missing, in Chrome and Edge) only on top-level navigations with safe methods like a link click, `None` always (requires `Secure`). Also: `Max-Age` / `Expires`, `Path`, `Domain`; the `__Host-` prefix forces `Secure`, `Path=/` and no `Domain`. → *More:* §14.4

Q165. What is the same-origin policy? A browser rule: a page may only **read** responses and data from the same **origin** — the same scheme, host and port (`https://app.com:443`). Other origins can still be sent requests (forms, images, scripts, `fetch`) but scripts cannot read the response unless the server allows it with CORS. It stops a malicious site from reading your bank's pages using your cookies. → *More:* §14.2

Q166. ⋆ What is CORS? What is a preflight request? CORS (Cross-Origin Resource Sharing) lets a **server** opt in to cross-origin reads via response headers like `Access-Control-Allow-Origin: https://app.com`. For "non-simple" requests — methods other than GET/HEAD/POST, custom headers such as `Authorization`, or `Content-Type: application/json` — the browser first sends an `OPTIONS` **preflight** asking permission, and only sends the real request if the server answers with matching `Access-Control-Allow-*` headers. With cookies, the server must send `Access-Control-Allow-Credentials: true` and an exact origin (not `*`). CORS is enforced by the browser; it is not server security, and fixing a CORS error means changing the server (or using a same-origin proxy). → *More:* §14.2

Q167. ⋆ What is XSS, and how do you prevent it? Cross-site scripting: an attacker gets their script to run in your page — **stored** (saved in the DB and shown to others), **reflected** (in a URL echoed back) or **DOM-based** (your JS writes untrusted data into the page). The script can then do anything the user can, including reading tokens. Prevention: insert user data as text (`textContent`, framework escaping), never into `innerHTML` / `eval` / `href="javascript:"`; sanitise HTML you must render with DOMPurify; set a strict **Content Security Policy**; keep session cookies `HttpOnly`; consider Trusted Types. → *More:* §14.10

Q168. ⋆ What is CSRF, and how do you prevent it? Cross-site request forgery: a malicious site makes the victim's browser send a request (e.g. a hidden form POST) to your site, and the browser attaches the victim's cookies, so the action runs as them. Prevention: `SameSite=Lax` or `Strict` cookies; a **CSRF token** (a random value in the page that the server checks on every state-changing request); checking the `Origin` / `Sec-Fetch-Site` header; never changing state on GET. APIs authenticated with an `Authorization` header (not cookies) are not exposed to CSRF. → *More:* §14.11

Q169. What is a Content Security Policy (CSP)? An HTTP response header (`Content-Security-Policy`) that tells the browser which sources of scripts, styles, images and connections are allowed. A strict policy (scripts only from your origin or with a per-response **nonce**, no inline scripts, no `eval`) means that even if an attacker injects markup, their script won't run. It is defence in depth against XSS; also `frame-ancestors` blocks clickjacking. → *More:* §14.12

Q170. Where should you store a JWT or auth token in the browser? The safest common choice is an `HttpOnly; Secure; SameSite` cookie set by the server: JavaScript (and therefore XSS) cannot read it, and `SameSite` plus CSRF tokens handle CSRF. `localStorage` is easy but any XSS can steal the token. Another pattern keeps a short-lived access token **in memory** and a refresh token in an `HttpOnly` cookie. Either way, XSS prevention comes first. → *More:* §14.4

Q171. What is clickjacking? What is prototype pollution? Clickjacking: your page is loaded in an invisible iframe on an attacker's site so the user clicks your buttons unknowingly; block framing with `Content-Security-Policy: frame-ancestors 'self'` (or `X-Frame-Options: DENY`). **Prototype pollution**: code that deep-merges untrusted JSON writes a key like `__proto__` or `constructor.prototype`, adding properties to `Object.prototype` and so to every object. Prevent it by rejecting those keys, using `Object.create(null)` or `Map` for dictionaries, and validating input against a schema. → *More:* §14.12 · §17.4.4 Q32

Q172. ⋆ How does `fetch` work? Does it reject on a 404? `fetch(url, options)` returns a promise of a `Response`. It rejects **only on network failure** (or abort/CORS failure); HTTP errors like 404 or 500 still fulfil, so check `response.ok` (status 200–299) or `response.status`. Reading the body is a second async step: `await res.json()`, `text()`, `blob()`; the body can be read only once. Options: `method`, `headers`, `body` (stringify JSON yourself), `credentials`, `signal`. Unlike `XMLHttpRequest`, it is promise-based and supports streaming.

```
const res = await fetch('/api/users', { signal: AbortSignal.timeout(5000) });
if (!res.ok) throw new Error(`HTTP ${res.status}`);
const users = await res.json();
```

→ *More:* §14.1 `fetch` and HTTP

Q173. What are Web Workers? What can't they do? A Web Worker runs a script on a separate background thread, so heavy computation (parsing, image processing) doesn't freeze the UI. It communicates with the page by `postMessage` (data is copied using structured clone, or **transferred** for `ArrayBuffer`s). Workers have no access to the DOM, `window` or `localStorage`, but they can use `fetch`, timers, IndexedDB and WebSockets. → *More:* §14.8 Web Workers

Q174. What is a service worker? What is a PWA? A service worker is a background script that sits between your page and the network: it can intercept `fetch` requests and answer from a cache (offline support), receive push messages and do background sync. It has no DOM access, requires HTTPS, and has its own lifecycle (install, activate, then controls pages). A **PWA** (progressive web app) is a website with a service worker and a web app manifest, so it can work offline and be installed like an app. → *More:* §14.8

Q175. How does client-side routing work in an SPA? A single-page application changes the URL without a page reload using the **History API**: `history.pushState(state, '', '/path')` adds an entry and `replaceState` changes the current one; the app then renders the view for the new path. Back/forward fire a `popstate` event, which the app handles by rendering the matching view. The server must return the app's HTML for every route so direct links work. → *More:* §14.7 The History API

Q176. How do you lazy-load images or implement infinite scroll? Use `IntersectionObserver`: it calls you asynchronously when a target element enters or leaves the viewport (with a `rootMargin` to start early), without scroll listeners or layout reads. For images, `<img loading="lazy">` is built in. For infinite scroll, observe a sentinel element after the last item and load the next page when it becomes visible (with a guard so only one load runs at a time). → *More:* §14.9

Q177. What is `requestAnimationFrame`? `requestAnimationFrame(cb)` runs `cb` just before the browser's next repaint (usually 60 times a second, matching the display), with a high-resolution timestamp. Use it for animations and DOM writes tied to frames instead of `setTimeout`: it is synchronised with rendering and paused in background tabs. → *More:* §14.5

Q178. How do you read query-string parameters? `new URL(location.href).searchParams.get('q')` or `new URLSearchParams(location.search).get('q')`. `URLSearchParams` also builds query strings with correct encoding (`params.set('q', 'a b&c')`). Manually, use `encodeURIComponent` for a single value; `encodeURI` is for a whole URL and leaves `&`, `?`, `/` alone. → *More:* §14.6 `URL` and `URLSearchParams`

Q179. What is `postMessage`, and how do you use it safely? `otherWindow.postMessage(data, targetOrigin)` sends data between windows, iframes or workers of different origins. Always pass an exact `targetOrigin` (not `'*'`) so data isn't delivered to the wrong site, and in the `message` handler check `event.origin` against an allow-list and validate `event.data` before using it.

Q180. Polling vs long polling vs WebSockets vs Server-Sent Events? Polling asks the server every few seconds (simple, wasteful, delayed). **Long polling** keeps a request open until there is news, then reconnects. **Server-Sent Events** (`EventSource`) is a one-way server-to-browser stream over plain HTTP with automatic reconnect — good for notifications and feeds. **WebSockets** give a persistent two-way connection — good for chat, games, collaboration. → *More:* §14.1

Q181. What is feature detection, and why prefer it over user-agent sniffing? Feature detection checks whether the API exists before using it (`if ('IntersectionObserver' in window)`, `CSS.supports(…)`), then uses it or a fallback/polyfill. User-agent sniffing guesses features from the browser name string, which is easy to spoof, frozen or reduced in modern browsers, and wrong as soon as a browser adds the feature. → *More:* §10.12

10. Performance and memory

Q182. How does garbage collection work in JavaScript? Memory is freed automatically. Engines use **reachability**: starting from roots (globals, the current call stack, active closures), everything reachable through references is kept; everything else is garbage (**mark-and-sweep**). Cycles are not a problem — two objects pointing at each other but unreachable are collected. V8 is **generational**: new objects go into a small young area collected often and cheaply; survivors are promoted to the old area, collected less often. → *More:* §17.4.7 Q51

Q183. ⋆ What causes memory leaks in JavaScript? How do you find them? Something keeps references alive that you no longer need: forgotten event listeners and `setInterval`s, growing caches/maps with no limit, closures holding large data, detached DOM nodes still referenced from JS, accidental globals, unresolved promises holding resources. Find them with DevTools Memory: take heap snapshots before and after repeating an action and compare, look for detached elements, or use the allocation timeline. Fix by cleaning up (remove listeners, clear timers, `AbortController`), bounding caches (LRU), and using `WeakMap`. → *More:* §17.4.7 Q52

Q184. What are `WeakRef` and `FinalizationRegistry`? `WeakRef(obj)` holds a reference that doesn't stop garbage collection; `deref()` returns the object or `undefined` once it's gone — useful for caches of large objects. `FinalizationRegistry` lets you register a callback that may run after an object is collected — for cleaning external resources. Both are non-deterministic (GC timing is unpredictable, callbacks may never run), so don't base program logic on them. → *More:* §8.3 · §17.4.7 Q53

Q185. ⋆ What is the difference between debouncing and throttling? When do you use each? Debounce waits until calls stop for a quiet period, then runs once — for search-as-you-type, auto-save, resize-end. **Throttle** runs at most once per interval while calls continue — for scroll position, mouse move, rate-limited API calls, infinite-scroll checks. Both reduce how often an expensive handler runs. → *More:* §17.3 Debounce · §17.3 Throttle

Q186. ⋆ How would you improve the performance of a slow web page's JavaScript? Measure first (Lighthouse, Performance panel, Core Web Vitals). Then: ship less JS (code splitting, lazy loading with dynamic `import()`, tree shaking, drop heavy libraries); load scripts with `defer`; break **long tasks** (anything over 50 ms blocks input) into chunks with `scheduler.yield()` or `setTimeout`, or move them to a Web Worker; avoid layout thrashing; debounce/throttle frequent handlers; use event delegation; virtualise long lists; cache network results; memoise expensive pure computations. → *More:* §13.11 · §10.11

Q187. What are Core Web Vitals? Google's user-centred metrics: **LCP** (Largest Contentful Paint — loading; good ≤ 2.5 s), **INP** (Interaction to Next Paint — responsiveness to clicks and keys; good ≤ 200 ms; it replaced FID in 2024), and **CLS** (Cumulative Layout Shift — visual stability; good ≤ 0.1). JavaScript mostly hurts INP (long tasks on the main thread) and LCP (render-blocking or client-rendered content).

Q188. How do you measure the performance of code? `performance.now()` gives a high-resolution timestamp for timing a piece of code; `performance.mark`/`measure` create named entries visible in DevTools; `console.time`/`timeEnd` for quick checks. For pages, use the Performance panel and Lighthouse; for real users, `PerformanceObserver` (long tasks, paint and web-vitals entries). In Node, `node --cpu-prof` or `--inspect` profiles. Benchmark with realistic data and warm-up, since the JIT optimises hot code. → *More:* §11.7 · §11.9

Q189. How does V8 run your code fast? What are hidden classes and inline caching? V8 first interprets bytecode (Ignition), collects type feedback, then compiles hot functions to optimised machine code (TurboFan, with intermediate tiers) — **JIT** (just-in-time) compilation. Objects get **hidden classes** (shapes) describing their property layout; objects created with the same properties in the same order share a shape. **Inline caches** remember the shape seen at each property access, making repeat accesses fast. So: initialise all properties in the constructor in the same order, don't `delete` properties, keep functions monomorphic (called with one shape of argument). If assumptions break, V8 **deoptimises** back to slower code. → *More:* §1.3 JavaScript engines

Q190. How do you render a list of 100,000 items without freezing the page? Don't create 100,000 DOM nodes: **virtualise** — render only the rows in (and just around) the viewport and reposition them on scroll (libraries: TanStack Virtual, react-window), or paginate. If data processing is heavy, do it in

a Web Worker or in chunks that yield to the event loop. CSS `content-visibility: auto` lets the browser skip rendering off-screen sections.

Q191. A page freezes while the user types in a search box. How do you debug and fix it? Record the interaction in the Performance panel and look for long tasks on the main thread. Usual causes: filtering a huge list or calling an API on every keystroke, and re-rendering everything. Fixes: debounce the input; cancel stale requests with `AbortController` (and ignore out-of-order responses); memoise the filtering; virtualise the result list; move heavy matching to a worker. → *More:* §12.13 Real-world async patterns

---

11. Node.js

Q192. ⋆ What is Node.js? Is it single-threaded? Node is a JavaScript runtime outside the browser: Google's V8 engine plus **libuv** (a C library providing the event loop, async I/O and a thread pool) plus core modules (`fs`, `http`, `crypto`, `stream`…). Your JavaScript runs on one main thread, but libuv uses the OS's async facilities for network I/O and a **thread pool** (4 threads by default, `UV_THREADPOOL_SIZE`) for file system, DNS lookup, `crypto` and `zlib` work. With `worker_threads` you can run JavaScript on extra threads too. → *More:* §15.1 · §15.2

Q193. ⋆ Explain the phases of Node's event loop. Each turn goes through phases: **timers** (`setTimeout`/`setInterval` callbacks that are due), **pending callbacks** (some deferred system errors), **idle/prepare** (internal), **poll** (wait for and run I/O callbacks), **check** (`setImmediate` callbacks), **close callbacks** (`'close'` events). Between every callback, Node drains the `process.nextTick` queue, then the promise microtask queue. → *More:* §12.4 Node's event loop

Q194. ⋆ Why is blocking the event loop bad in Node? How do you handle CPU-heavy work? One thread serves every request; a 2-second synchronous loop (big JSON parse, image resize, `fs.readFileSync` in a request handler, a catastrophic regex) delays **all** other clients by 2 seconds. Fixes: use async APIs; split work into chunks; move CPU work to `worker_threads` or a separate service/queue; scale across cores with multiple processes (cluster, containers). → *More:* §15.2

Q195. `worker_threads` vs `child_process` vs `cluster`? `worker_threads`: extra JavaScript threads in the same process, each with its own V8 and event loop, communicating by messages or `SharedArrayBuffer` — for CPU-heavy JS. `child_process` (`spawn`, `exec`, `fork`): separate OS processes, for running other programs or isolating work. `cluster`: forks several Node processes that share one server port to use all CPU cores (often replaced by a process manager or container orchestration). → *More:* §15.2 · §15.11

Q196. ⋆ What are streams? What is backpressure? A stream processes data piece by piece (chunks) instead of loading it all into memory. Types: **Readable** (file read, HTTP request), **Writable** (file write, HTTP response), **Duplex** (TCP socket) and **Transform** (gzip). **Backpressure** happens when the reader is faster than the writer: `write()` returns `false` and you should wait for `'drain'`. `pipeline(src, transform, dest)` from `node:stream/promises` handles backpressure, errors and cleanup for you — use it instead of `.pipe()`.

```
import fs from 'node:fs';
import zlib from 'node:zlib';
import { pipeline } from 'node:stream/promises';
await pipeline(fs.createReadStream('big.log'), zlib.createGzip(), fs.createWriteStream('big.log.gz'));
```

→ *More:* §15.9 Streams

Q197. What is a `Buffer`? A `Buffer` is Node's fixed-size chunk of raw bytes (a subclass of `Uint8Array`), used for binary data: files, network packets, images, crypto. Convert with `Buffer.from('hi', 'utf8')` and `buf.toString('base64')`. `Buffer.alloc(n)` gives zero-filled memory; `allocUnsafe` is faster but may contain old data. → *More:* §15.7

Q198. What is `EventEmitter`? Node's built-in publish/subscribe class (`node:events`): `on(event, fn)` registers listeners, `emit(event, ...args)` calls them **synchronously** in registration order, `once` listens once, `off` removes. Streams, servers and sockets are emitters. Emitting `'error'` with no listener throws, so always handle `'error'`. Warns after 10 listeners per event (a common leak hint). `events.once(emitter, 'x')` gives a promise. → *More:* §15.8 Events: `EventEmitter` · Q258 below

Q199. How do you read command-line arguments and environment variables? What do exit codes mean? `process.argv` is the array `[nodePath, scriptPath, ...args]` (`util.parseArgs` parses flags). `process.env.NAME` reads environment variables (always strings); Node can load a `.env` file with `--env-file=.env`. Exit code `0` means success, anything else failure; set `process.exitCode = 1` and let the process end naturally, rather than `process.exit()`, which cuts off pending I/O. → *More:* §15.6 `process`

Q200. `fs.readFileSync` vs `fs.readFile` vs `fs/promises`? `readFileSync` blocks the thread until the file is read — fine in startup scripts and CLIs, bad inside a server request. `fs.readFile(path, cb)` is the callback async version; `import { readFile } from 'node:fs/promises'` is the promise version for `await` — the modern default. For large files use streams. → *More:* §15.4 Files with `node:fs/promises`

Q201. How do you create an HTTP server in Node without a framework? `http.createServer((req, res) => {...}).listen(3000)`. The handler reads `req.method`, `req.url` (parse with `new URL(req.url, 'http://localhost')`), headers and the body stream, then sets `res.statusCode`, headers and calls `res.end(body)`. Frameworks (Express, Fastify, Hono) add routing, middleware (functions that run in a chain before your handler), body parsing and error handling on top. → *More:* §15.10

Q202. What is middleware (as in Express)? A function `(req, res, next)` in a pipeline that runs for each request: it can read or modify the request/response, end the response, or call `next()` to pass control on (`next(err)` jumps to error-handling middleware, which has four parameters). Used for logging, auth, body parsing, CORS, rate limiting. Order of registration is the order of execution. → *More:* §15.10

Q203. What is the difference between `dependencies`, `devDependencies` and `peerDependencies`? `npm` vs `npx`? `dependencies` are needed at run time; `devDependencies` only for development and builds (test runners, bundlers, types); `peerDependencies` are packages a library expects the host app to provide (a React component library and `react`). `npm` installs and manages packages and runs scripts; `npx` runs a package's binary without a global install. → *More:* §10.9 npm and `package.json`

Q204. How does JavaScript in Node differ from JavaScript in the browser? Same language and engine family; different host APIs. Node has no DOM, `window` or `document`, but has file system, processes, sockets, `Buffer`, `require` and full OS access; the browser is sandboxed and has the DOM, `localStorage`, rendering. The global object is `global` vs `window` (`globalThis` in both). Many web APIs now exist in both: `fetch`, `URL`, `AbortController`, `structuredClone`, web streams, `crypto.subtle`. → *More:* §15.1

Q205. How do you scale a Node.js application? Keep each process non-blocking (async I/O, offload CPU work). Run several stateless processes per machine (one per core) behind a load balancer, and several machines horizontally; keep state (sessions, caches) in shared stores like Redis or a database. Use queues for slow background jobs, streams for big payloads, caching and connection pooling for the database. → *More:* §15.2 · System Design Fundamentals

Q206. What are Deno and Bun? Alternative JavaScript runtimes. **Deno** (by Node's creator) uses V8 and Rust, is secure by default (explicit permissions for files and network), runs TypeScript natively and bundles a formatter, linter and test runner. **Bun** uses JavaScriptCore (Safari's engine) and Zig, aims for speed, and bundles a package manager, bundler and test runner. Both are largely Node-compatible; Node remains the mainstream production choice. Node has closed some gaps: it runs `.ts` files by type stripping (Q216) and has an opt-in, now stable **permission model** (`node --permission --allow-fs-read=./data app.js`). → *More:* §15.12

---

12. TypeScript basics

Q207. ⋆ What is TypeScript, and why use it? TypeScript is JavaScript plus a static type system: you annotate (or let it infer) types, and the compiler `tsc` reports type errors **before** the code runs; the types are then erased and plain JavaScript runs. Benefits: catches typos and wrong-shape bugs early, much better editor autocomplete and refactoring, types document APIs. Costs: a build/check step and learning curve. It does not check data at run time (validate external input with a schema library). → *More:* §16.8

Q208. ⋆ What is the difference between `interface` and `type`? Both describe object shapes and are mostly interchangeable. `interface` can be **reopened** (declaration merging — two declarations with the same name combine, used to augment library types) and extended with `extends`. `type` aliases can name **any** type — unions (`'a' | 'b'`), tuples, primitives, mapped and conditional types — and combine with `&`. Common

rule: `interface` for public object shapes, `type` for everything else; be consistent. → *More:* §16.10 Everyday types

Q209. ⋆ `any` vs `unknown` vs `never`? `any` turns type checking off for that value — anything goes, errors spread silently. `unknown` means "could be anything, so you must check before using it" — the safe type for external input (`JSON.parse`, `catch` errors). `never` is the type with no values: a function that always throws or never returns, or a branch that should be impossible (used for exhaustive `switch` checks). → *More:* §16.10 · §16.11 Narrowing

Q210. What is type narrowing? What are type guards and discriminated unions? Narrowing is TypeScript refining a union type inside a branch after a check: `typeof x === 'string'`, `x instanceof Date`, `'key' in x`, `Array.isArray(x)`, truthiness, equality. A **type guard** is a function returning `x is Type`, so its callers get narrowing. A **discriminated union** is a union of object types sharing a literal field (`kind: 'circle' | 'square'`); switching on that field narrows to the right member, and a `never` default catches missing cases. → *More:* §16.11

Q211. ⋆ What are generics? Type parameters that let one function or type work with many types while keeping them linked: `function first<T>(arr: T[]): T | undefined` returns the same type the array holds. Constraints restrict them: `<T extends { id: string }>`. Used everywhere in library types: `Array<T>`, `Promise<T>`, `Map<K, V>`. → *More:* §16.12 Generics and utility types

Q212. Name some utility types. `Partial<T>` (all optional), `Required<T>`, `Readonly<T>`, `Pick<T, 'a' | 'b'>`, `Omit<T, 'password'>`, `Record<K, V>` (object with keys K), `ReturnType<typeof fn>`, `Parameters<typeof fn>`, `Awaited<P>` (unwrap a promise), `NonNullable<T>`, `Exclude` / `Extract` (filter union members). They are built from mapped and conditional types. → *More:* §16.12

Q213. What do `keyof` and `typeof` mean in TypeScript types? In type position, `typeof value` gives the type of a JavaScript value (`type Config = typeof defaultConfig`). `keyof T` gives the union of T's keys (`keyof { a: 1; b: 2 }` is `'a' | 'b'`). Together: `function get<T, K extends keyof T>(obj: T, key: K): T[K]` — type-safe property access, where `T[K]` is an indexed access type. → *More:* §16.12

Q214. Enums vs union of string literals vs `as const`? What does `satisfies` do? A string-literal union (`type Status = 'idle' | 'loading'`) costs nothing at run time and is usually preferred. `enum` generates a JavaScript object and is not supported by Node's type stripping. An `as const` object (`const Status = { Idle: 'idle' } as const`) gives a runtime object plus literal types — the common enum replacement. `value satisfies Type` checks that a value matches a type **without** widening it, so you keep the precise inferred type (e.g. which keys exist). → *More:* §16.13

Q215. What is structural typing? TypeScript compares types by **shape**, not by name: any object with the required properties is compatible, even if declared separately ("duck typing" checked at compile time). Object literals passed directly get an extra check for unknown properties (excess property checking). → *More:* §16.10

Q216. Can Node 24 run TypeScript directly? Yes: `node app.ts` works without flags by **stripping** type annotations (stable in Node 24.12). It does no type checking, so still run `tsc --noEmit`; it supports only erasable syntax (no `enum`, namespaces with code, or parameter properties — enable `erasableSyntaxOnly`), needs `import type` for type-only imports (enforce with `verbatimModuleSyntax`), requires real `.ts` extensions in imports, ignores `tsconfig.json`, and won't strip files in `node_modules`. → *More:* §16.9

Q217. What are the pros and cons of languages that compile to JavaScript? Pros: types (TypeScript) or other features, earlier error detection, better tooling. Cons: a build step, source maps needed for debugging, possible mismatch between the language and the JavaScript it produces, an extra dependency, and team learning cost. TypeScript is the exception that became mainstream because it is "JavaScript plus types", not a different language. → *More:* §16.8

---

13. Output-prediction puzzles ("What will this print?")

Say the answer out loud **before** reading it. Each answer was checked by running the code in Node; files are ES modules (strict mode) unless the question says otherwise. Many more, with real output, are in §17.4 and §12.3.

Q218. ⋆ Timers, promises and `queueMicrotask`.

```
console.log('1');
setTimeout(() => console.log('2'), 0);
Promise.resolve().then(() => console.log('3'));
queueMicrotask(() => console.log('4'));
console.log('5');
```

`1 5 3 4 2`. Synchronous code first (1, 5); then the microtask queue in order of queuing (3, 4); then the timer task (2). → *More:* §17.4.5 Q35

Q219. ⋆ Nested tasks and microtasks.

```
console.log('A');
setTimeout(() => console.log('B'), 0);
Promise.resolve()
  .then(() => { console.log('C'); setTimeout(() => console.log('D'), 0); })
  .then(() => console.log('E'));
(async () => { console.log('F'); await null; console.log('G'); })();
console.log('H');
```

`A F H C G E B D`. The async function runs synchronously up to its first `await` (F). Microtasks queued during sync code: C, then G. Running C queues E (the next `then`) and timer D. Microtasks drain completely (C, G, E) before any timer; timers then run in the order created (B, D).

Q220. ⋆ `async` / `await` ordering.

```
async function a() { console.log('a1'); await b(); console.log('a2'); }
async function b() { console.log('b'); }
console.log('start');
a();
Promise.resolve().then(() => console.log('p'));
console.log('end');
```

`start a1 b end a2 p`. Calling `b()` runs its body synchronously. `await` of an already-fulfilled promise queues the continuation (a2) as a microtask — before `p`, which is queued later. → *More:* §17.4.5 Q36

Q221. ⋆ The promise executor.

```
const p = new Promise((resolve) => { console.log('A'); resolve('B'); console.log('C'); });
p.then(console.log);
console.log('D');
```

`A C D B`. The executor runs synchronously, and `resolve` does not stop it (C still prints). `then` callbacks are always asynchronous.

Q222. ⋆ Errors flowing through a chain.

```
Promise.resolve(1)
  .then((x) => { throw new Error('boom ' + x); })
  .then(() => console.log('skipped'))
  .catch((e) => { console.log(e.message); return 2; })
  .then((x) => console.log('after', x))
  .finally(() => console.log('finally'));
```

`boom 1`, `after 2`, `finally`. A throw skips `then` handlers until a `catch`; a `catch` that returns a value recovers the chain; `finally` receives no value and passes the previous result through.

Q223. What does an `async` function return?

```
async function f() { return 1; }
console.log(f());
```

`Promise { 1 }` (in Node). An `async` function always returns a promise; use `await f()` or `f().then(…)` to get `1`.

Q224. ⋆ `var` hoisting inside a function.

```
var x = 1;
function f() { console.log(x); var x = 2; }
f();
```

`undefined`. The inner `var x` is hoisted to the top of `f` and shadows the outer `x`, so at the `console.log` the local `x` exists but is still `undefined`. With `let x = 2` it would throw `ReferenceError` (TDZ). → *More:* §17.4.2 Q16

Q225. Function vs variable hoisting (a classic script).

```
console.log(typeof foo);
var foo = 1;
function foo() {}
console.log(typeof foo);
```

`function`, then `number`. Both declarations are hoisted; the function declaration wins at the start. Then the assignment `foo = 1` runs and replaces it.

Q226. ⋆ `this` in methods, arrows and nested functions.

```
const obj = {
  name: 'obj',
  regular() { return this.name; },
  arrow: () => this?.name,
  nested() { return [function () { return this?.name; }(), (() => this.name)()]; },
};
console.log(obj.regular(), obj.arrow(), obj.nested());
```

`obj undefined [ undefined, 'obj' ]`. `regular` is called on `obj`. The arrow takes `this` from module top level, which is `undefined`. Inside `nested`, the inner regular function is a plain call (`this` is `undefined` in strict code), while the inner arrow inherits `nested`'s `this`. → *More:* §17.4.3 Q25

Q227. A detached method (a CommonJS file, sloppy mode).

```
const user = { name: 'Ann', greet() { return 'Hi ' + this?.name; } };
const g = user.greet;
console.log(user.greet(), g());
```

`Hi Ann Hi undefined`. `g()` is a plain call: in sloppy mode `this` is the global object, which has no `name`. In strict mode `this` would be `undefined` (`this?.name` avoids the crash here; `this.name` would throw).

Q228. Class field arrow vs prototype method.

```
class Counter { count = 0; inc() { this.count++; } incArrow = () => { this.count++; }; }
const c = new Counter();
const { inc, incArrow } = c;
incArrow();
try { inc(); } catch (e) { console.log(e.constructor.name); }
console.log(c.count);
```

`TypeError`, then `1`. The arrow field is created per instance and captures it; the prototype method loses `this` when extracted (class bodies are strict, so `this` is `undefined`).

Q229. Closures keep separate state.

```
function make() { let c = 0; return { inc: () => ++c, get: () => c }; }
const a = make(), b = make();
a.inc(); a.inc(); b.inc();
console.log(a.get(), b.get());
```

`2 1`. Each call to `make` creates a new `c`; the two functions returned by one call share it.

Q230. ⋆ Coercion with `+` and `-`.

```
console.log("5" + 3 - 3, "5" - 3 + 3);
console.log([1, 2, 3] + [4], null + 1, undefined + 1);
console.log(1 < 2 < 3, 3 > 2 > 1);
```

`50 5` — `"5" + 3` is `"53"`, minus 3 is `50`; `"5" - 3` is `2`, plus 3 is `5`. `1,2,34 1 NaN` — arrays become strings; `null` becomes `0`, `undefined` becomes `NaN`. `true false` — `1 < 2` is `true`, `true < 3` is `1 < 3`; `3 > 2` is `true`, `true > 1` is `1 > 1`. → *More:* §17.4.1 Q1

Q231. `Number()` vs `parseInt()` on odd inputs.

```
console.log(Number(''), Number(' 12 '), Number('12px'), Number(null), Number(undefined));
console.log(Number([]), Number([5]), Number([1, 2]), parseInt('12px'), parseInt(0.0000005));
```

`0 12 NaN 0 NaN`, then `0 5 NaN 12 5`. Arrays convert via their string form (`''`, `'5'`, `'1,2'`). `parseInt(0.0000005)` is `5` because the number is first turned into the string `'5e-7'`, and parsing stops at `e`.

Q232. An object that equals three different numbers.

```
let count = 0;
const obj = { valueOf() { return ++count; } };
console.log(obj == 1, obj == 2, obj == 3);
```

`true true true`. `==` with a number converts the object with `valueOf` each time, and this one returns a new number on every call. (`===` would give `false` three times.)

Q233. ⋆ Default `sort`.

```
console.log([3, 20, 100].sort(), ['b', 'a', 'C'].sort());
```

`[ 100, 20, 3 ] [ 'C', 'a', 'b' ]`. Elements are compared as strings by UTF-16 code unit: `'1' < '2' < '3'`, and uppercase letters come before lowercase. Use `(a, b) => a - b` and `localeCompare`.

Q234. `map` with a block body.

```
console.log([1, 2, 3].map((n) => { n * 2; }));
```

`[ undefined, undefined, undefined ]`. Braces make a block body, which needs an explicit `return`. Write `(n) => n * 2`.

Q235. Holes and `length`.

```
const arr = [1, 2, 3];
arr[10] = 11;
console.log(arr.length, arr.filter(() => true).length);
```

`11 4`. Writing index 10 extends `length` to 11 and leaves holes at 3–9; `filter` skips holes.

Q236. `includes` vs `indexOf` with `NaN`, and `indexOf` with types.

```
console.log([NaN].includes(NaN), [NaN].indexOf(NaN), [1, 2, 3].indexOf('2'));
```

`true -1 -1`. `includes` uses SameValueZero (treats `NaN` as equal to itself); `indexOf` uses `===`, so it finds neither `NaN` nor the string `'2'`.

Q237. Constructors that return values.

```
function Foo() { this.a = 1; return { b: 2 }; }
function Bar() { this.a = 1; return 5; }
console.log(new Foo().a, new Foo().b, new Bar().a);
```

`undefined 2 1`. Returning an object from a constructor replaces the new object; returning a primitive is ignored. → *More:* §17.4.3 Q28

Q238. `return` inside `finally`.

```
function f() { try { return 'try'; } finally { console.log('cleanup'); } }
function g() { try { return 'try'; } finally { return 'finally'; } }
console.log(f()); console.log(g());
```

`cleanup`, `try`, `finally`. `finally` runs before the function actually returns; a `return` in `finally` overrides the one in `try`.

Q239. Generators and `next(value)`.

```
function* gen() { const x = yield 1; console.log('got', x); yield x * 2; }
const it = gen();
console.log(it.next().value); console.log(it.next(5).value); console.log(it.next().done);
```

`1`, `got 5`, `10`, `true`. The first `next()` runs to the first `yield`. The value passed to the second `next(5)` becomes the result of that `yield` expression.

Q240. Object key order.

```
console.log(Object.keys({ b: 1, 2: 1, a: 1, 1: 1 }));
```

`[ '1', '2', 'b', 'a' ]`. Integer-like keys come first in ascending numeric order, then string keys in insertion order (and keys are always strings).

Q241. Strings are immutable.

```
const s = 'abc';
s[0] = 'z';
```

In an ES module (strict mode) this throws `TypeError: Cannot assign to read only property '0' of string 'abc'`. In sloppy mode the assignment is silently ignored and `s` stays `'abc'`.

Q242. Mixing `??` with `||`.

```
console.log(0 || null ?? 'x');
```

`SyntaxError`. The language forbids mixing `??` with `||` or `&&` without parentheses, because the intended precedence is unclear. Write `(0 || null) ?? 'x'` (gives `'x'`).

Q243. ⋆ Loop closures with `let`.

```
const fns = [];
for (let i = 0; i < 3; i++) fns.push(() => i);
console.log(fns.map((fn) => fn()));
```

`[ 0, 1, 2 ]`. `let` in a `for` header creates a new binding for each iteration; with `var` it prints `[ 3, 3, 3 ]`. → *More:* §17.4.2 Q18

Q244. Array-to-primitive comparisons.

```
console.log(String([null]), String([undefined, null]), [] == '', [0] == false);
```

An empty string, `,`, `true`, `true`. `join` turns `null` / `undefined` into empty strings; `[]` becomes `''`; `[0]` becomes `'0'`, then `0`, and `false` becomes `0`.

---

14. Coding tasks: implement-from-scratch classics

Every snippet below was run against tests. In the interview, first state the contract (inputs, outputs, edge cases), write the simple version, then mention extensions (cancel, `this`, errors, leading edge). Algorithm and data-structure problems (two pointers, sliding window, BFS, DP, heaps…) are in §17.2 Coding patterns.

Q245. ⋆ Implement `debounce`.

```
function debounce(fn, wait) {
  let timer;
  const debounced = function (...args) {
    clearTimeout(timer);                                  // every call restarts the wait
    timer = setTimeout(() => fn.apply(this, args), wait);   // keep the caller's this and latest args
  };
  debounced.cancel = () => clearTimeout(timer);
  return debounced;
}
```

Follow-ups: a `leading` option (run on the first call, then ignore until quiet), `flush()` (run the pending call now), and returning a promise of the result. → *More:* §17.3 Debounce

Q246. ⋆ Implement `throttle`.

```
function throttle(fn, wait) {
  let last = 0, timer = null, lastArgs;
  return function (...args) {
    const now = Date.now();
    lastArgs = args;
    if (now - last >= wait) { last = now; fn.apply(this, args); }          // leading call
    else if (!timer) timer = setTimeout(() => {                             // one trailing call
      last = Date.now(); timer = null; fn.apply(this, lastArgs);
    }, wait - (now - last));
  };
}
```

Runs at most once per `wait`, with a trailing call so the last value is not lost. → *More:* §17.3 Throttle

Q247. ⋆ Implement `Promise.all`.

```
function promiseAll(iterable) {
  return new Promise((resolve, reject) => {
    const items = [...iterable], results = new Array(items.length);
    let left = items.length;
    if (left === 0) return resolve(results);                // empty input resolves at once
    items.forEach((item, i) => {
      Promise.resolve(item).then((v) => {                    // also accepts plain values
        results[i] = v;                                      // keep input order
        if (--left === 0) resolve(results);
      }, reject);                                            // first rejection wins
    });
  });
}
```

→ *More:* §17.3 Promise polyfills

Q248. Implement `Promise.allSettled`.

```
const promiseAllSettled = (iterable) => promiseAll([...iterable].map((p) =>
  Promise.resolve(p).then((value) => ({ status: 'fulfilled', value }),
                          (reason) => ({ status: 'rejected', reason }))));
```

Each promise is turned into one that always fulfils with a status object, so `all` never rejects. → *More:* §17.3

Q249. ⋆ Implement `Promise.race` and `Promise.any`.

```
const promiseRace = (iterable) => new Promise((resolve, reject) => {
  for (const p of iterable) Promise.resolve(p).then(resolve, reject);     // first to settle wins
});
function promiseAny(iterable) {
  return new Promise((resolve, reject) => {
    const items = [...iterable], errors = new Array(items.length);
    let left = items.length;
    if (left === 0) return reject(new AggregateError([], 'All promises were rejected'));
    items.forEach((p, i) => Promise.resolve(p).then(resolve, (e) => {
      errors[i] = e;
      if (--left === 0) reject(new AggregateError(errors, 'All promises were rejected'));
    }));
  });
}
```

Extra calls to `resolve` / `reject` after the first are ignored, which is what makes `race` this short. `race([])` stays pending forever. → *More:* §17.3

Q250. ⋆ Implement `Function.prototype.bind`.

```
Function.prototype.myBind = function (ctx, ...preset) {
  const fn = this;
  return function bound(...later) {
    if (new.target) return new fn(...preset, ...later);   // `new` ignores the bound this
    return fn.apply(ctx, [...preset, ...later]);         // partial application
  };
};
```

→ *More:* §17.3 A `bind` polyfill

Q251. Implement `call` and `apply` without using them.

```
Function.prototype.myCall = function (ctx, ...args) {
  ctx = ctx == null ? globalThis : Object(ctx);    // box primitives
  const key = Symbol('fn');                        // unique key: no clash with ctx's own props
  ctx[key] = this;                                 // call it as a method so this === ctx
  try { return ctx[key](...args); } finally { delete ctx[key]; }
};
Function.prototype.myApply = function (ctx, args = []) { return this.myCall(ctx, ...args); };
```

This mimics sloppy-mode behaviour (`null` becomes the global object, primitives are boxed); the real `call` passes `ctx` unchanged to a strict-mode function. → *More:* §17.3

Q252. ⋆ Implement a deep clone (with circular references).

```
function deepClone(value, seen = new WeakMap()) {
  if (value === null || typeof value !== 'object') return value;    // primitives (and functions) as-is
  if (seen.has(value)) return seen.get(value);                        // cycle: reuse the copy
  if (value instanceof Date) return new Date(value);
  if (value instanceof Map) { const m = new Map(); seen.set(value, m);
    value.forEach((v, k) => m.set(deepClone(k, seen), deepClone(v, seen))); return m; }
  if (value instanceof Set) { const s = new Set(); seen.set(value, s);
    value.forEach((v) => s.add(deepClone(v, seen))); return s; }
  const copy = Array.isArray(value) ? [] : Object.create(Object.getPrototypeOf(value));
  seen.set(value, copy);
  for (const key of Reflect.ownKeys(value)) copy[key] = deepClone(value[key], seen);  // incl. symbols
  return copy;
}
```

Mention that `structuredClone` is the built-in answer, and what it cannot copy (functions, DOM nodes, prototypes). → *More:* §17.3 Deep clone

Q253. ⋆ Implement deep equality.

```
function deepEqual(a, b) {
  if (Object.is(a, b)) return true;
  if (typeof a !== 'object' || typeof b !== 'object' || a === null || b === null) return false;
  if (Array.isArray(a) !== Array.isArray(b)) return false;
  const ka = Object.keys(a), kb = Object.keys(b);
  return ka.length === kb.length && ka.every((k) => Object.hasOwn(b, k) && deepEqual(a[k], b[k]));
}
```

Extensions: `Date` / `Map` / `Set` support, and a `seen` map for cycles. → *More:* §17.3 Deep equal

Q254. ⋆ Flatten a nested array (with and without recursion).

```
const flatten = (arr, depth = Infinity) => depth < 1 ? arr.slice()
  : arr.reduce((acc, x) => (Array.isArray(x) ? acc.push(...flatten(x, depth - 1)) : acc.push(x), acc), []);

function flattenIterative(arr) {                    // no recursion: explicit stack
  const stack = [...arr], out = [];
  while (stack.length) { const x = stack.pop(); Array.isArray(x) ? stack.push(...x) : out.push(x); }
  return out.reverse();
}
```

The built-in is `arr.flat(depth)` (`flat(Infinity)` for all levels). Flattening an object to dotted keys (`{ 'a.b': 1 }`) is the same recursion over `Object.entries`. → *More:* §17.3 Flatten

Q255. ⋆ Implement `curry`.

```
function curry(fn) {
  return function curried(...args) {
    return args.length >= fn.length               // enough arguments: call the original
      ? fn.apply(this, args)
      : (...more) => curried.apply(this, [...args, ...more]);   // otherwise collect more
  };
}
const add3 = curry((a, b, c) => a + b + c);
add3(1)(2)(3); add3(1, 2)(3); add3(1)(2, 3);       // all 6
```

→ *More:* §17.3 Curry

Q256. ⋆ Implement `sum(1)(2)(3)()` (infinite currying).

```
const sum = (a) => (b) => (b === undefined ? a : sum(a + b));
sum(1)(2)(3)();   // 6
```

Variant without the final `()`: return a function with a custom `valueOf` (`f.valueOf = () => total`) so `+sum(1)(2)(3)` or `sum(1)(2) == 3` works.

Q257. ⋆ Implement `memoize`.

```
function memoize(fn, keyFn = (...args) => JSON.stringify(args)) {
  const cache = new Map();
  return function (...args) {
    const key = keyFn(...args);
    if (!cache.has(key)) cache.set(key, fn.apply(this, args));   // has(): caches undefined results too
    return cache.get(key);
  };
}
```

Discuss the key (objects as arguments → `WeakMap` or a key function), memory growth (LRU limit or TTL), and memoising async functions (cache the promise and delete it on rejection). → *More:* §17.3 Memoize

Q258. ⋆ Implement an `EventEmitter` (`on`, `off`, `once`, `emit`).

```
class EventEmitter {
  #events = new Map();                                  // name -> array of listeners
  on(name, fn) {
    if (!this.#events.has(name)) this.#events.set(name, []);
    this.#events.get(name).push(fn);
    return () => this.off(name, fn);                    // handy unsubscribe function
  }
  off(name, fn) {
    const list = this.#events.get(name) ?? [];
    this.#events.set(name, list.filter((f) => f !== fn && f.original !== fn));
    return this;
  }
  once(name, fn) {
    const wrapper = (...args) => { this.off(name, wrapper); fn.apply(this, args); };
    wrapper.original = fn;                              // so off(name, fn) also removes it
    return this.on(name, wrapper);
  }
  emit(name, ...args) {
    const list = this.#events.get(name);
    if (!list?.length) return false;
    [...list].forEach((f) => f.apply(this, args));      // copy: listeners may unsubscribe
    return true;
  }
}
```

→ *More:* §17.3 EventEmitter · §15.8

Q259. ⋆ Implement an LRU cache with O(1) `get` and `put`.

```
class LRUCache {
  #map = new Map();                          // Map remembers insertion order
  constructor(capacity) { this.capacity = capacity; }
  get(key) {
    if (!this.#map.has(key)) return -1;
    const v = this.#map.get(key);
    this.#map.delete(key); this.#map.set(key, v);          // move to "most recent" end
    return v;
  }
  put(key, value) {
    this.#map.delete(key); this.#map.set(key, value);
    if (this.#map.size > this.capacity) this.#map.delete(this.#map.keys().next().value); // evict oldest
  }
}
```

In other languages (or if asked to avoid `Map` ordering) use a hash map plus a doubly linked list. → *More:* §17.3 LRU cache

Q260. ⋆ Write polyfills for `map`, `filter` and `reduce`.

```
Array.prototype.myMap = function (cb, thisArg) {
  const out = new Array(this.length);
  for (let i = 0; i < this.length; i++) if (i in this) out[i] = cb.call(thisArg, this[i], i, this);
  return out;
};
Array.prototype.myFilter = function (cb, thisArg) {
  const out = [];
  for (let i = 0; i < this.length; i++) if (i in this && cb.call(thisArg, this[i], i, this)) out.push(this[i]);
  return out;
};
Array.prototype.myReduce = function (cb, ...init) {
  let i = 0, acc;
  if (init.length) acc = init[0];
  else { while (i < this.length && !(i in this)) i++;
    if (i >= this.length) throw new TypeError('Reduce of empty array with no initial value');
    acc = this[i++]; }
  for (; i < this.length; i++) if (i in this) acc = cb(acc, this[i], i, this);
  return acc;
};
```

Points interviewers check: skipping holes (`i in this`), passing `(value, index, array)`, `thisArg`, and `reduce` without an initial value. (Use regular functions, not arrows, so `this` is the array.)

Q261. Implement `once`.

```
function once(fn) {
  let done = false, result;
  return function (...args) {
    if (!done) { done = true; result = fn.apply(this, args); }
    return result;                      // later calls return the first result
  };
}
```

(The LeetCode version returns `undefined` on later calls — read the spec.)

Q262. Implement `sleep` and `promisify`.

```
const sleep = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

const promisify = (fn) => (...args) => new Promise((resolve, reject) =>
  fn(...args, (err, value) => (err ? reject(err) : resolve(value))));   // Node error-first callback
```

Node ships `util.promisify` and `node:timers/promises` (`await setTimeout(100)`).

Q263. ⋆ Implement retry with exponential backoff.

```
async function retry(fn, { retries = 3, delay = 100 } = {}) {
  for (let attempt = 0; ; attempt++) {
    try { return await fn(attempt); }
    catch (err) {
      if (attempt >= retries) throw err;              // out of attempts: give up
      await sleep(delay * 2 ** attempt);              // 100, 200, 400 ms…
    }
  }
}
```

Mention adding random **jitter** so many clients don't retry in lockstep, retrying only transient errors (timeouts, 503), and an `AbortSignal`.

Q264. ⋆ Run async tasks with a concurrency limit (a promise pool).

```
async function runWithLimit(tasks, limit) {          // tasks: functions returning promises
  const results = new Array(tasks.length);
  let next = 0;
  async function worker() {
    while (next < tasks.length) { const i = next++; results[i] = await tasks[i](); }
  }
  await Promise.all(Array.from({ length: Math.min(limit, tasks.length) }, worker));
  return results;
}
```

`limit` workers each pull the next task as soon as they finish one; `next++` is safe because JavaScript runs one piece of code at a time. → *More:* §12.13 Real-world async patterns

Q265. Implement the `new` operator and `Object.create`.

```
function myNew(Ctor, ...args) {
  const obj = Object.create(Ctor.prototype);                  // 1. link the prototype
  const result = Ctor.apply(obj, args);                       // 2. run the constructor
  const isObject = result !== null && (typeof result === 'object' || typeof result === 'function');
  return isObject ? result : obj;                             // 3. object return wins
}
function objectCreate(proto) { function F() {} F.prototype = proto; return new F(); }
```

→ *More:* §9.4

Q266. Implement `instanceof`.

```
function myInstanceOf(obj, Ctor) {
  if (obj === null || (typeof obj !== 'object' && typeof obj !== 'function')) return false;
  for (let p = Object.getPrototypeOf(obj); p; p = Object.getPrototypeOf(p)) {
    if (p === Ctor.prototype) return true;
  }
  return false;
}
```

→ *More:* §9.7

Q267. Implement lodash-style `get(obj, 'a.b[0].c', fallback)`.

```
function get(obj, path, fallback) {
  const keys = Array.isArray(path) ? path : path.replace(/\[(\w+)\]/g, '.$1').split('.').filter(Boolean);
  let cur = obj;
  for (const k of keys) { if (cur == null) return fallback; cur = cur[k]; }
  return cur === undefined ? fallback : cur;
}
```

For a fixed path, optional chaining does this: `obj?.a?.b?.[0]?.c ?? fallback`.

Q268. Implement `setInterval` using `setTimeout`. Why would you?

```
function mySetInterval(fn, ms) {
  let id, stopped = false;
  const tick = () => { id = setTimeout(() => { fn(); if (!stopped) tick(); }, ms); };  // schedule the next run
after this one
  tick();
  return () => { stopped = true; clearTimeout(id); };   // the flag also works when fn() itself cancels
}
```

Without the `stopped` flag, calling the cancel function from inside `fn` would not work: `tick()` would schedule a new timer right after it. Chained timeouts guarantee a gap between runs; `setInterval` fires on schedule even if the previous (async) run has not finished, so calls can pile up.

Q269. Implement a basic `Promise` class.

```
class MyPromise {
  #state = 'pending'; #value; #handlers = [];
  constructor(executor) {
    const settle = (state, value) => {
      if (this.#state !== 'pending') return;                    // settle only once
      this.#state = state; this.#value = value;
      this.#handlers.forEach((h) => h());
    };
    try { executor((v) => settle('fulfilled', v), (e) => settle('rejected', e)); }
    catch (e) { settle('rejected', e); }
  }
  then(onFul, onRej) {
    return new MyPromise((resolve, reject) => {
      const run = () => queueMicrotask(() => {                  // callbacks are always async
        const cb = this.#state === 'fulfilled' ? onFul : onRej;
        if (typeof cb !== 'function') return (this.#state === 'fulfilled' ? resolve : reject)(this.#value);
        try { resolve(cb(this.#value)); } catch (e) { reject(e); }
      });
      this.#state === 'pending' ? this.#handlers.push(run) : run();
    });
  }
  catch(onRej) { return this.then(undefined, onRej); }
}
```

This shows the core ideas: three states, settle once, async callbacks, `then` returns a new promise. A full Promises/A+ version also **adopts** a returned promise or thenable (waits for it instead of fulfilling with it). → *More:* §12.7

Q270. Small warm-ups: FizzBuzz, `duplicate([1, 2, 3])`, unique, chunk, palindrome.

```
for (let i = 1; i <= 100; i++) console.log((i % 3 ? '' : 'fizz') + (i % 5 ? '' : 'buzz') || i);
const duplicate = (arr) => [...arr, ...arr];                    // [1,2,3,1,2,3]
const unique = (arr) => [...new Set(arr)];
const chunk = (arr, n) => Array.from({ length: Math.ceil(arr.length / n) }, (_, i) => arr.slice(i * n, i * n +
n));
const isPalindrome = (s) => { const t = s.toLowerCase().replace(/[^a-z0-9]/g, ''); return t ===
[...t].reverse().join(''); };
```

→ *More:* §17.2 Coding patterns

---

15. LeetCode "30 Days of JavaScript"

LeetCode's **30 Days of JavaScript** study plan is the most common online practice set for JavaScript-specific interview tasks. Its sections are Closures, Basic Array Transformations, Function Transformations, Promises and Time, JSON, and Classes. The problems below follow the plan's order; extra JavaScript problems often practised alongside it come at the end. Read each LeetCode statement for exact edge cases; the answers here are compact versions that pass the stated examples.

Q271. Create Hello World Function. Return a function that always returns `"Hello World"`, whatever it is called with.

```
const createHelloWorld = () => (...args) => 'Hello World';
```

Tests that you know functions are values that can be returned (higher-order functions).

Q272. Counter. `createCounter(n)` returns a function that returns `n`, then `n + 1`, and so on.

```
const createCounter = (n) => () => n++;      // postfix: return the old value, then increment
```

The parameter `n` is captured by the closure and lives on between calls. → *More:* §4.7 Closures

Q273. To Be Or Not To Be. `expect(val)` returns `{ toBe, notToBe }`: `toBe(v)` returns `true` if `v === val`, otherwise throws `"Not Equal"`; `notToBe(v)` returns `true` if `v !== val`, otherwise throws `"Equal"`.

```
const expect = (val) => ({
  toBe: (v) => { if (v !== val) throw new Error('Not Equal'); return true; },
  notToBe: (v) => { if (v === val) throw new Error('Equal'); return true; },
});
```

This is the shape of Jest's `expect` — an object of closures.

Q274. Counter II. `createCounter(init)` returns `{ increment, decrement, reset }`, each returning the new value.

```
function createCounter(init) {
  let value = init;
  return { increment: () => ++value, decrement: () => --value, reset: () => (value = init) };
}
```

Q275. Apply Transform Over Each Element in Array. Implement `map(arr, fn)` without `Array.prototype.map`; `fn` receives `(arr[i], i)`.

```
function map(arr, fn) { const out = []; for (let i = 0; i < arr.length; i++) out.push(fn(arr[i], i)); return out;
}
```

Q276. Filter Elements from Array. Implement `filter(arr, fn)` keeping elements where `fn(arr[i], i)` is truthy, without `filter`.

```
function filter(arr, fn) { const out = []; for (let i = 0; i < arr.length; i++) if (fn(arr[i], i))
out.push(arr[i]); return out; }
```

Q277. Array Reduce Transformation. Implement `reduce(nums, fn, init)`; an empty array returns `init`.

```
function reduce(nums, fn, init) { let acc = init; for (const n of nums) acc = fn(acc, n); return acc; }
```

Q278. ⋆ Function Composition. `compose([f, g, h])` returns `x => f(g(h(x)))`; an empty list gives the identity function.

```
const compose = (functions) => (x) => functions.reduceRight((acc, fn) => fn(acc), x);
```

→ *More:* §17.3 Compose and pipe

Q279. Return Length of Arguments Passed.

```
const argumentsLength = (...args) => args.length;
```

Rest parameters collect any number of arguments into a real array.

Q280. ⋆ Allow One Function Call. `once(fn)` calls `fn` the first time and returns its result; later calls return `undefined`.

```
function once(fn) {
  let called = false;
  return (...args) => { if (called) return undefined; called = true; return fn(...args); };
}
```

Q281. Memoize. `memoize(fn)` returns a version that never calls `fn` twice with the same inputs (`sum(a, b)`, `fib(n)`, `factorial(n)` — note `sum(2, 3)` and `sum(3, 2)` are different inputs).

```
function memoize(fn) {
  const cache = new Map();
  return (...args) => {
    const key = args.join(',');              // inputs are numbers here; use JSON.stringify for general data
    if (!cache.has(key)) cache.set(key, fn(...args));
    return cache.get(key);
  };
}
```

→ *More:* Q257 above

Q282. Add Two Promises. Return a promise of the sum of two promises' numbers.

```
const addTwoPromises = async (p1, p2) => (await p1) + (await p2);
```

Both promises already run in parallel (they were created before the call); `Promise.all([p1, p2])` works too.

Q283. Sleep.

```
const sleep = (millis) => new Promise((resolve) => setTimeout(resolve, millis));
```

Q284. Timeout Cancellation. `cancellable(fn, args, t)` runs `fn(...args)` after `t` ms and returns `cancelFn`; calling `cancelFn` before then prevents the call.

```
function cancellable(fn, args, t) {
  const id = setTimeout(() => fn(...args), t);
  return () => clearTimeout(id);
}
```

Q285. Interval Cancellation. Call `fn(...args)` immediately and then every `t` ms until `cancelFn` is called.

```
function cancellable(fn, args, t) {
  fn(...args);
  const id = setInterval(() => fn(...args), t);
  return () => clearInterval(id);
}
```

Q286. ⋆ Promise Time Limit. `timeLimit(fn, t)` returns an async function that resolves like `fn`, or rejects with `"Time Limit Exceeded"` if `fn` takes longer than `t` ms.

```
const timeLimit = (fn, t) => (...args) => Promise.race([
  fn(...args),
  new Promise((_, reject) => setTimeout(() => reject('Time Limit Exceeded'), t)),
]);
```

In production, clear the losing timer and use `AbortSignal.timeout(t)` to actually stop the work.

Q287. ⋆ Cache With Time Limit. A class with `set(key, value, duration)` (returns `true` if an unexpired key already existed, and overwrites it), `get(key)` (value or `-1`) and `count()` (number of unexpired keys).

```
class TimeLimitedCache {
  #store = new Map();                                  // key -> { value, expires }
  set(key, value, duration) {
    const alive = this.get(key) !== -1;
    this.#store.set(key, { value, expires: Date.now() + duration });
    return alive;
  }
  get(key) { const e = this.#store.get(key); return e && e.expires > Date.now() ? e.value : -1; }
  count() { let n = 0; for (const e of this.#store.values()) if (e.expires > Date.now()) n++; return n; }
}
```

(Using `get` inside `set` assumes `-1` is never a stored value, which holds for this problem.) Alternative: a `setTimeout` per key that deletes it (clear the old timer on overwrite).

Q288. Debounce. Same as the classic: delay each call by `t` ms and cancel the previous pending call.

```
function debounce(fn, t) {
  let id;
  return (...args) => { clearTimeout(id); id = setTimeout(() => fn(...args), t); };
}
```

→ *More:* §17.3 Debounce

Q289. ⋆ Execute Asynchronous Functions in Parallel. Given an array of functions that return promises, implement `promiseAll(functions)` without `Promise.all`.

```
function promiseAll(functions) {
  return new Promise((resolve, reject) => {
    const results = new Array(functions.length);
    let left = functions.length;
    if (left === 0) return resolve(results);
    functions.forEach((fn, i) => fn().then((v) => {
      results[i] = v;
      if (--left === 0) resolve(results);
    }, reject));
  });
}
```

→ *More:* §17.3 Promise polyfills

Q290. Is Object Empty. Return `true` for `{}` or `[]`.

```
const isEmpty = (obj) => Object.keys(obj).length === 0;
```

A version that stops at the first key without building a key array is `for (const _ in obj) return false; return true;` — but it breaks if anyone has added enumerable properties to `Array.prototype` (as the "Array Prototype Last" solution below does with plain assignment).

Q291. Chunk Array. Split `arr` into sub-arrays of length `size` (the last may be shorter), without lodash's `_.chunk`.

```
function chunk(arr, size) {
  const out = [];
  for (let i = 0; i < arr.length; i += size) out.push(arr.slice(i, i + size));
  return out;
}
```

Q292. Array Prototype Last. Add `arr.last()` returning the last element, or `-1` for an empty array.

```
Array.prototype.last = function () { return this.length ? this[this.length - 1] : -1; };
```

Use a regular function so `this` is the array. In real code use the built-in `arr.at(-1)`, and avoid extending built-ins (Q69). If you must, use `Object.defineProperty` so the method is non-enumerable.

Q293. ⋆ Group By. Add `arr.groupBy(fn)` returning an object whose keys are `fn(item)` and values are arrays of items.

```
Array.prototype.groupBy = function (fn) {
  const out = {};
  for (const item of this) (out[fn(item)] ??= []).push(item);
  return out;
};
```

The standard version is `Object.groupBy(arr, fn)` (ES2024). → *More:* §8.7

Q294. Sort By. Return `arr` sorted ascending by the number `fn(item)`.

```
const sortBy = (arr, fn) => arr.toSorted((a, b) => fn(a) - fn(b));
```

Q295. Join Two Arrays by ID. Merge two arrays of objects by `id`; for the same `id`, properties from `arr2` override `arr1`; return the result sorted by `id`.

```
function join(arr1, arr2) {
  const byId = new Map();
  for (const o of arr1) byId.set(o.id, { ...o });
  for (const o of arr2) byId.set(o.id, { ...byId.get(o.id), ...o });   // later spread wins
  return [...byId.values()].sort((a, b) => a.id - b.id);
}
```

Q296. ⋆ Flatten Deeply Nested Array. Implement `flat(arr, n)` (flatten up to depth `n`) without `Array.prototype.flat`.

```
function flat(arr, n) {
  if (n === 0) return arr.slice();
  const out = [];
  for (const x of arr) Array.isArray(x) ? out.push(...flat(x, n - 1)) : out.push(x);
  return out;
}
```

Q297. Compact Object. Remove all falsy values from an object or array, recursively (`compactObject([null, 0, 5, [0], [false, 16]])` → `[5, [], [16]]`).

```
function compactObject(obj) {
  if (obj === null || typeof obj !== 'object') return obj;
  if (Array.isArray(obj)) return obj.filter(Boolean).map(compactObject);
  const out = {};
  for (const [k, v] of Object.entries(obj)) if (v) out[k] = compactObject(v);
  return out;
}
```

Q298. ⋆ Event Emitter. `subscribe(eventName, callback)` returns `{ unsubscribe() }`; `emit(eventName, args = [])` calls the callbacks in subscription order and returns an array of their results.

```
class EventEmitter {
  #events = new Map();                                  // name -> Set of entries
  subscribe(eventName, callback) {
    if (!this.#events.has(eventName)) this.#events.set(eventName, new Set());
    const subs = this.#events.get(eventName), entry = { callback };  // wrapper: same callback twice is fine
    subs.add(entry);
    return { unsubscribe: () => { subs.delete(entry); } };
  }
  emit(eventName, args = []) {
    return [...(this.#events.get(eventName) ?? [])].map(({ callback }) => callback(...args));
  }
}
```

→ *More:* Q258 above

Q299. Array Wrapper. Class `ArrayWrapper(nums)` where `obj1 + obj2` gives the sum of all elements and `String(obj)` gives `"[1,2,3]"`.

```
class ArrayWrapper {
  constructor(nums) { this.nums = nums; }
  valueOf() { return this.nums.reduce((a, b) => a + b, 0); }   // used by +
  toString() { return `[${this.nums.join(',')}]`; }            // used by String()
}
```

→ *More:* Q17 above (object-to-primitive conversion)

Q300. ⋆ Calculator with Method Chaining. `new Calculator(10).add(5).subtract(7).getResult()` → `8`; `divide(0)` throws `"Division by zero is not allowed"`.

```
class Calculator {
  constructor(value) { this.value = value; }
  add(v) { this.value += v; return this; }              // return this -> chainable
  subtract(v) { this.value -= v; return this; }
  multiply(v) { this.value *= v; return this; }
  divide(v) { if (v === 0) throw new Error('Division by zero is not allowed'); this.value /= v; return this; }
  power(v) { this.value **= v; return this; }
  getResult() { return this.value; }
}
```

Q301. Check if Object Instance of Class. Return whether a value is an instance of a class, including primitives (`5` is an instance of `Number`) and without throwing for odd inputs.

```
function checkIfInstanceOf(obj, classFunction) {
  if (obj == null || typeof classFunction !== 'function') return false;
  for (let p = Object.getPrototypeOf(obj); p !== null; p = Object.getPrototypeOf(p)) {
    if (p === classFunction.prototype) return true;      // getPrototypeOf boxes primitives
  }
  return false;
}
```

Unlike `instanceof`, this says `5` is a `Number`, because `Object.getPrototypeOf(5)` is `Number.prototype`.

Q302. Generate Fibonacci Sequence and Nested Array Generator. Write a generator yielding Fibonacci numbers forever, and one yielding the integers of a nested array in order.

```
function* fibGenerator() { let [a, b] = [0, 1]; while (true) { yield a; [a, b] = [b, a + b]; } }
function* inorderTraversal(arr) { for (const x of arr) Array.isArray(x) ? yield* inorderTraversal(x) : yield x; }
```

`yield*` delegates to another generator. Infinite generators are safe because values are produced only on demand. → *More:* §8.5 Generators

Q303. Throttle, Curry, JSON Deep Equal, Promise Pool, Call Function with Custom Context, Undefined to Null. These JavaScript problems are also commonly practised with the plan; each matches a classic above:

- **Throttle** and **Curry** → the throttle and curry answers in group 14.
- **JSON Deep Equal** → the deep-equality answer (JSON values only, so no `Date` / `Map` handling is needed).

- **Promise Pool** (`promisePool(functions, n)`) → the concurrency-limit answer, without collecting results.
- **Call Function with Custom Context** (`fn.callPolyfill(context, ...args)`) → the `call` polyfill.
- **Undefined to Null** → a recursive walk, or `JSON.parse(JSON.stringify(obj, (k, v) => (v === undefined ? null : v)))`.

Sources of the question lists

Question wording and frequency were taken from these lists and posts (titles only; all answers are this book's own):

- LeetCode — *30 Days of JavaScript* study plan (leetcode.com/studyplan/30-days-of-javascript); problem lists as mirrored in public solution repositories (e.g. github.com/Rizwan-Damani/leetcode-30-days-js, github.com/mahmoudabozied4/30-Days-of-JavaScript-LeetCode).
- sudheerj, *javascript-interview-questions* (github.com/sudheerj/javascript-interview-questions) — the 500+ question list.
- h5bp, *Front-end Developer Interview Questions* — JavaScript questions and coding questions (github.com/h5bp/Front-end-Developer-Interview-Questions).
- GreatFrontEnd — *Front End Interview Playbook: JavaScript coding* and *10 Must-Know JavaScript Coding Interview Questions* (greatfrontend.com).
- NamasteDev — "JavaScript interview questions by company" (namastedev.com).
- Facts were checked against MDN Web Docs, the ECMAScript specification and TC39 finished-proposals list, the Node.js documentation, and OWASP cheat sheets, as for the rest of this book.